Министерство цифрового развития, связи и массовых коммуникаций
Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

А. Н. Полетайкин, Д. А. Демчук

DevOps-инженерия

Учебное пособие

Новосибирск
2025

УДК 004(075.8)

Утверждено редакционно-издательским советом СибГУТИ

Рецензенты:
заведующий кафедрой прикладной математики и кибернетики СибГУТИ доктор техн. наук, доц. И.В. Нечта;
заведующий кафедрой информационных технологий ФГБОУ ВО «Кубанский государственный университет», кандидат физ.-мат. наук, доц. В.В. Подколзин

Полетайкин, А. Н. DevOps-инженерия : учебное пособие / А. Н. Полетайкин, Д. А. Демчук ; Сибирский государственный университет телекоммуникаций и информатики, кафедра математического моделирования и цифрового развития бизнес-систем. – Новосибирск, 2025. – 188 с.

Учебное пособие адресовано студентам магистратуры, обучающимся по направлениям подготовки 02.04.02 «Фундаментальная информатика и информационные технологии», 09.04.03 «Прикладная информатика». Учебное пособие содержит теорию и методологию инженерного DevOps и его применения для создания информационных и программных систем автоматизированным способом, а также календарный график, задания и методические рекомендации к выполнению учебной работы в форме лабораторных, практических и курсовой работ. Изложение материала выполнено с позиции системного подхода в стиле, доступном для студента, владеющего базовыми знаниями в области программной инженерии.

Учебное пособие может быть использовано в программах учебных курсов «Современные технологии разработки ПО», «Автоматизация программирования», «Методология и технология проектирования ИС», «Методологии DevOps» и других, связанных с современными технологиями разработки ПО и их применением для решения прикладных задач.

© Полетайкин А. Н., Демчук Д. А., 2025
© Сибирский государственный университет телекоммуникаций и информатики, 2025
© Кубанский государственный университет, 2025

Содержание

От авторов 6
Введение 7
1. Основы инженерного DevOps 9
1.1. Цикл DevOps 10
1.2. Ключевые ценности DevOps 13
1.3. Централизация в DevOps 15
1.4. Автоматизация в DevOps 16
1.5. Команда DevOps 17
1.6. Культура разработки по DevOps 18
1.7. Развитие принципов разработки ПО 19
1.8. Применение DevOps вне IT 20
1.9. Проблемы коммуникации в DevOps-процессах 20
1.10. Рациональный унифицированный процесс 22
1.11. Контрольные вопросы 25
2. Архитектура информационных систем 26
2.1. Многослойная архитектура 26
2.2. Многоуровневая архитектура 27
2.3. Сервис-ориентированная архитектура 30
2.4. Микросервисная архитектура 32
2.5. Ключевые свойства микросервисов 35
2.6. Недостатки микросервисов 43
2.7. Контрольные вопросы 51
3. Технологии коллективной разработки информационных систем 52
3.1. Модель авторской разработки 53
3.2. Модели коллективной разработки программного обеспечения 54
3.2.1. Иерархическая модель и ее недостатки 56
3.2.2. Модель группы и ее недостатки 58
3.2.3. Состав команды и распределение ролей 59
3.2.4. Модель команды главного программиста 60
3.3. Фреймворки и их использование при разработке ИС 61
3.4. Фреймворк Django 65
3.5. Система контроля версий Git 68
3.6. Системы контейнеризации 72
3.6.1. Возможности Docker 72
3.6.2. Отличие Docker от виртуальных машин 73
3.6.3. Структурный состав Docker 74
3.6.4. Dockerfile и Docker-Compose 76
3.7. Оркестрация контейнеризированных приложений 77
3.7.1. Система оркестрации Kubernetes 79
3.7.2. Объекты кластера Kubernetes 80
3.7.3. Преимущества и недостатки Kubernetes 86
3.8. Контрольные вопросы 87

4. Лабораторный практикум 88
4.1. Лабораторная работа 1 89
4.1.1. Описание используемых команд Linux 89
4.1.2. Настройка сети и установка дополнительного ПО 92
4.1.3. Задание 96
4.1.4. Содержание отчета 99
4.1.5. Работа с Git 99
4.1.6. Контрольные вопросы 99
4.2. Лабораторная работа 2 100
4.2.1. Установка СУБД PostgreSQL 100
4.2.2. Установка фреймворка Django 102
4.2.3. Создание приложения Django 106
4.2.4. Задание 110
4.2.5. Содержание отчета 110
4.2.6. Работа с Git 110
4.2.7. Контрольные вопросы 110
4.3. Лабораторная работа 3 111
4.3.1. Подготовка репозиториев Docker 111
4.3.2. Установка Docker и дополнительного ПО 112
4.3.3. Задание 112
4.3.4. Содержание отчета 112
4.3.5. Работа с Git 113
4.3.6. Контрольные вопросы 113
4.4. Лабораторная работа 4 113
4.4.1. Создание собственного образа для web-сервера apache 113
4.4.2. Запуск контейнера 115
4.4.3. Docker-compose 115
4.4.4. Запуск контейнера через Docker-compose 117
4.4.5. Задание 121
4.4.6. Содержание отчета 122
4.4.7. Работа с Git 122
4.4.8. Контрольные вопросы 122
4.5. Лабораторная работа 5 123
4.5.1. Описание основных действий 123
4.5.2. Врéменное хранение данных 123
4.5.3. Постоянное хранение данных 123
4.5.4. Тома Docker 124
4.5.5. Сети в контейнерах Docker 127
4.5.6. Сетевые драйверы в Docker 128
4.5.7. Создание сети с использованием Docker-compose 130
4.5.8. Использование переменных окружения Docker 131
4.5.9. Задание 133
4.5.10. Содержание отчета 133
4.5.11. Работа с Git 133
4.5.12. Контрольные вопросы 134

4.6. Лабораторная работа 6 134
4.6.1. Общие положения 134
4.6.2. Создание образа контейнеров 134
4.6.3. Создание пространства имен 138
4.6.4. Создание пода 138
4.6.5. Задание 139
4.6.6. Содержание отчета 140
4.6.7. Работа с Git 140
4.6.8. Контрольные вопросы 140
4.7. Лабораторная работа 7 140
4.7.1. Теоретические положения 140
4.7.2. Принципы CI/CD 141
4.7.3. Этапы CI/CD 141
4.7.4. Преимущества и недостатки CI/CD 142
4.7.5. Инструменты для CI/CD 142
4.7.6. Применение методологии с использованием Gitlab 143
4.7.7. Пример использования CI/CD 144
4.7.8. Задание 151
4.7.9. Содержание отчета 151
4.7.10. Работа с Git 152
4.7.11. Контрольные вопросы 152
5. Курсовое проектирование 153
5.1. Общие сведения 153
5.2. Цель и задачи выполнения курсовой работы 153
5.3. Содержание и этапы выполнения курсовой работы 154
5.4. Тематика курсового проектирования 157
5.5. Требования к структуре курсовой работы 158
5.6. Требования к оформлению пояснительной записки 163
5.7. Требования к демонстрационным материалам 167
5.8. Порядок сдачи на проверку и защиты курсовой работы 169
Заключение 172
Список рекомендуемой литературы 173
Приложение А. Образец титульного листа пояснительной записки к КР 179
Приложение Б. Образец бланка задания на КР 180
Приложение В. Пример реализации простейшего web-приложения с микросервисной архитектурой 181

От авторов

Данное учебное пособие является результатом многолетней адаптации реального DevOps-процесса к подготовке магистрантов по направлениям 02.04.02 «Фундаментальная информатика и информационные технологии», 09.04.03 «Прикладная информатика», осуществляемой авторами в Сибирском государственном университете телекоммуникаций и информатики и Кубанском государственном университете.

Теоретический блок разработан на основе материалов магистерских курсов «DevOps-инженерия», «Современные технологии разработки ПО», «Технологии автоматизации программирования», «Методология и технология проектирования ИС», прочитанных авторами в 2023–2025 годах в указанных выше вузах с использованием открытых источников информации по инженерному DevOps. Материал излагается в форме, доступной для студента, освоившего программу бакалавриата по направлениям подготовки цифровой экономики (УГСН 02 «Компьютерные и информационные науки», 09 «Информатика и вычислительная техника», 10 «Информационная безопасность», 11 «Электроника, радиотехника и системы связи»).

При подготовке практических заданий и методических материалов к ним расчет был сделан на уровень бакалаврской подготовки студентов «выше среднего». То есть такой, который предполагает самостоятельный подход к DevOps-процессу, начиная от способности домыслить краткие методические инструкции до рабочих проектных решений и самостоятельно что-то найти в доступной документации, в том числе размещенной в сети Интернет, и заканчивая выбором и применением индивидуального стека технологий и самостоятельной организацией индивидуализированного DevOps-процесса.

В практической части выделены лабораторный практикум и курсовая работа. Лабораторный практикум направлен преимущественно на освоение технологии DevOps-инженерии. Курсовая работа ориентирована на разработку информационной (программной) системы с микросервисной архитектурой с использованием освоенных (либо осваиваемых в ходе выполнения курсовой работы) технологий. Реализация курсовой работы является предпочтительной, так как в этом случае практический результат изучения курса представляет собой готовый рабочий проект информационной (программной) системы. В случае же реализации только лабораторного практикума полноценный рабочий проект может быть получен из материалов отчетов с проведением дополнительных работ по проектированию и документированию системы.

Заметим, что теоретический материал пособия не является исчерпывающим для выполнения практикума в силу значительной обширности предмета изучения. Базовые сведения о методах и моделях программной инженерии, лежащих в основе DevOps-инженерии, читатель сможет почерпнуть в авторском учебном пособии [1], а также в других источниках из списка рекомендуемой литературы, представленного в конце основной части пособия.

Введение

DevOps-инженерия – это методология и набор практик, направленных на объединение команд разработки (Dev) и эксплуатации (Ops) для ускорения, автоматизации и улучшения процессов создания, развертывания и поддержки программного обеспечения. Как и любая инженерия, она требует для эффективного практического применения формирования комплекса соответствующих профессиональных компетенций. Дидактическая цель предлагаемого учебного пособия – совершенствование и (или) получение новых компетенций, необходимых для профессиональной деятельности в области создания и эксплуатации качественного программного обеспечения информационных систем. Новизна заключается в использовании на всех этапах жизненного цикла информационной системы (ИС) систематических, дисциплинированных и измеримых практик в соответствии с методологией DevOps.

DevOps как методология представляет собой учение об организации деятельности по автоматизации технологических процессов проектирования, разработки, тестирования, сборки, настройки и развертывания программного обеспечения (ПО). Это прогрессивная информационная и программная технология, определяющая непрерывные процессы постановки задачи на создание ИС, формализации требований к ИС, проектирования, кодирования, сборки и тестирования – непрерывной интеграции ИС, а также непрерывного развертывания ИС с непрерывным мониторингом и получением обратной связи от пользователя. Все эти непрерывные процессы формируют максимально удобную и комфортную среду для всех субъектов деятельности: разработчиков и пользователей ИС. Все важные с этой точки зрения аспекты непрерывной автоматизированной разработки ИС в соответствии с методологией DevOps рассматриваются в разделе 1.

Одним из наиболее эффективных архитектурных подходов к построению масштабируемых и гибких ИС является микросервисная архитектура (МСА). Она позволяет разрабатывать, внедрять и масштабировать отдельные компоненты системы независимо друг от друга, что особенно важно для динамично развивающихся бизнес-процессов. Однако внедрение МСА влечет за собой новые вызовы, связанные с обеспечением целостности, согласованности и устойчивости функционирования всей системы.

В распределенных цифровых средах отказ одного из сервисов может привести к каскадным сбоям, деградации производительности и, как следствие, к снижению качества производимой продукции. Поэтому наряду с МСА рассматриваются монолитные модульные и сервисные архитектуры. Различные аспекты проектирования архитектурных решений при создании ИС в соответствии с методологией DevOps рассматриваются в разделе 2.

Раздел 3 посвящен технологиям коллективной разработки ИС. Рассматриваются все компоненты технологии: принципы, нормы, методы и модели организации коллективной разработки. Выполнен обзор фреймворков для разработки ИС. Подробно рассмотрен фреймворк Django для разработки

веб-приложений. Отдельное внимание уделено системе управления версиями Git и ее использованию при коллективном способе разработки в распределенном варианте организации работ. В связи с приоритетным использованием МСА также рассмотрены инструменты контейнеризации (Docker) и оркестрации контейнеризированных приложений (Kubernetes). Docker является важным инструментом в современном IT-ландшафте, особенно для разработки, развертывания и масштабирования микросервисных приложений. Kubernetes является одной из самых популярных систем оркестрации. Он поддерживает сложные кластеры и предоставляет широкие возможности для настройки и расширения микросервисов приложений. Все эти инструменты сегодня являются неотъемлемыми компонентами технологии разработки ПО, реализуемой в соответствии с методологией DevOps.

В разделе 4 представлен лабораторный практикум по курсу «DevOps-инженерия», который предполагает выполнение семи лабораторных работ. Практикум базируется на использовании операционной системы Linux, которая является приоритетной ввиду адаптации к ней подавляющего большинства инструментов DevOps-инженерии. Инструменты разработки (Development Circle) Python, Django, PostgreSQL, рассмотренные в разделе 3, являются рекомендуемыми. Средства развертывания (Deployment Circle) Docker и Kubernetes предлагаются к использованию как широко распространенные и хорошо документированные. Студент на свое усмотрение может предложить свой индивидуальный стек технологий для выполнения лабораторных работ.

Раздел 5 посвящен курсовой работе, которая может выполняться синхронно с лабораторным практикумом либо являться его логическим продолжением. В отличие от лабораторного практикума, курсовая работа ориентирована на полноценное рабочее проектирование ИС в соответствии с методологией и с использованием технологий DevOps-инженерии. В качестве базовых требований здесь к обучающимся предъявляются знания и умения в области системного анализа, программной инженерии и организации баз данных. Результатом курсового проектирования является хорошо документированная готовая к использованию и перманентному развитию ИС. Курсовая работа рассчитана на среднестатистического студента, ее сложность заметно ниже сложности лабораторного практикума. Поэтому в случае отсутствия в рабочей программе дисциплины курсовой работы она вполне может выполняться в формате расчетно-графической работы.

Учебное пособие может быть полезно не только студентам, но и специалистам разных областей, интересующимся методологией и технологиями DevOps-инженерии и намеревающимся использовать их в своей практике построения и применения информационных систем. Список рекомендуемой литературы существенно расширяет учебно-методическое пространство пособия, а рекомендуемая документация по инструментам DevOps-инженерии призвана устранить недостатки изложения в пособии необходимых для разработки качественных продуктов. При грамотном использовании рекомендуемой литературы и документации обучающиеся могут без особого труда освоить увлекательные технологии DevOps.

1. Основы инженерного DevOps

DevOps – это методология разработки ПО, сфокусированная на предельно активном взаимодействии и интеграции в одной команде программистов, тестировщиков и админов, синхронно обслуживающих общий для них сервис/продукт. Главная цель – создание единого цикла взаимозависимости разработки, эксплуатации и деплоя ПО, чтобы в конечном счете помогать организациям (сервисам, стартапам) быстрее и проще создавать и обновлять их программные продукты и сервисы, эксплуатируемые в режиме реального времени или «в продакшене».

В современной ИТ-среде специалист по DevOps представляет собой уникальный синтез технической экспертизы и междисциплинарной компетентности. Этот профессионал, которого на индустриальном жаргоне часто называют «девопсер», обычно вырастает из системных администраторов или разработчиков, приобретая со временем универсальную квалификацию. Конечно, человек для подобной роли нужен не только более технически квалифицированный, но также и неравнодушный к самому продукту, дотошный и умеющий договариваться/объяснять, который хочет и может вникать во все подозрительные проявления работы кода и непонятное поведение серверов, особенно если делать это придется в нерабочее время, ликвидируя результаты чужих багов.

Задачи DevOps:

1. Автоматическое развертывание.
2. Решения для обеспечения высокой доступности. Например, SQL Always On, Service Fabric.
3. Проектирование архитектуры, консультация программистов, тестировщиков, системных администраторов.

На рис. 1.1 демонстрируется системообразующая функция платформы Azure DevOps, выступающей центральным узлом взаимодействия различных подразделений организации. Платформа объединяет проектные команды разработчиков, занятых созданием и совершенствованием продукта; операционную среду, отвечающую за анализ и трансформацию бизнес-процессов; тестировщиков, обеспечивающих контроль качества; конечных пользователей, чьи потребности являются конечной целью всей системы.

Важно подчеркнуть, что DevOps представляет собой не просто методологический инструментарий, но прежде всего новую культуру организационного взаимодействия. Эта культура характеризуется преодолением традиционных барьеров между подразделениями, формированием общего концептуального пространства, ориентацией на сквозные процессы создания ценности.

Центральная идея DevOps-подхода заключается в приоритете социального эффекта – создания реальной пользы для конечных пользователей (клиентов, пациентов, потребителей услуг), качества производственных процессов – обеспечения бесперебойного создания качественного продукта,

экономической эффективности, которая возникает как естественное следствие первых двух факторов.

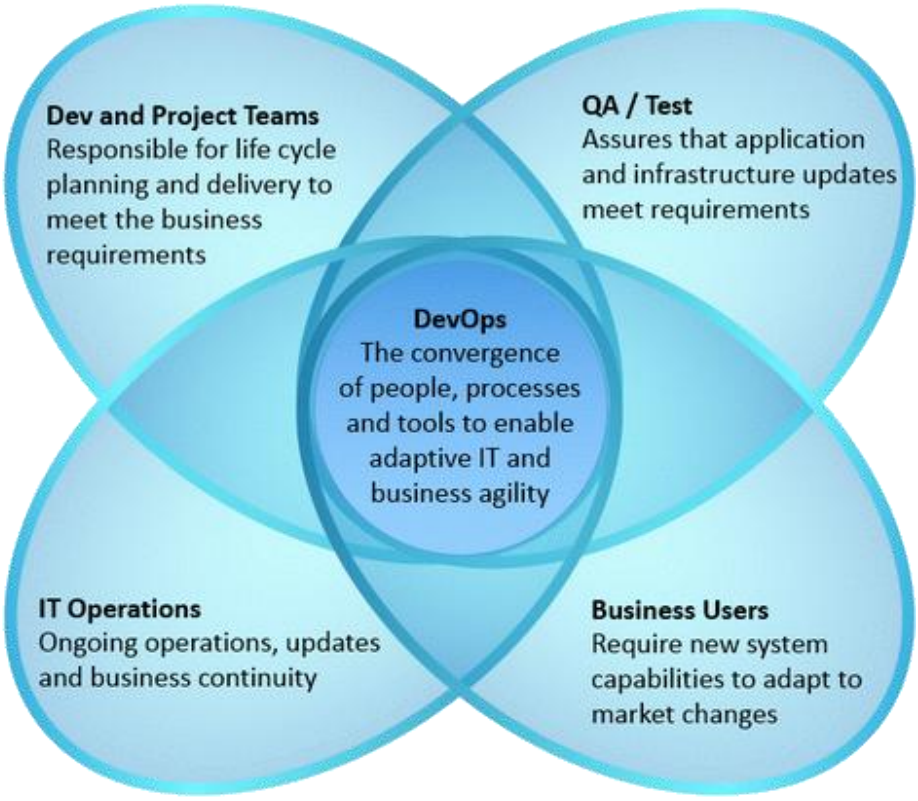


Рисунок 1.1 – Схема взаимодействий в методологии DevOps

1.1. Цикл DevOps

DevOps выступает не просто технологической платформой, а концептуальным каркасом, позволяющим организации перейти от разрозненных операций к целостной системе создания ценности, где технические решения подчинены гуманитарным целям удовлетворения реальных потребностей общества.

Иллюстрация этого «бесконечного» DevOps-процесса символически представляется в виде перевернутой восьмерки (рис. 1.2), что и отражает непрерывность, непрерывный процесс производства ПО. Подобно тому, как жизненный цикл по определению – это непрерывный процесс, DevOps – это тоже непрерывный процесс. На самом деле он, конечно, когда-то заканчивается.

DevOps представляет собой системный подход, объединяющий технические процессы и организационную культуру в единый цикл разработки программного обеспечения. Данная методология выходит за рамки набора практик, формируя целостную философию создания ПО, особенно востребованную в условиях цифровой трансформации и широкого распространения облачных технологий.

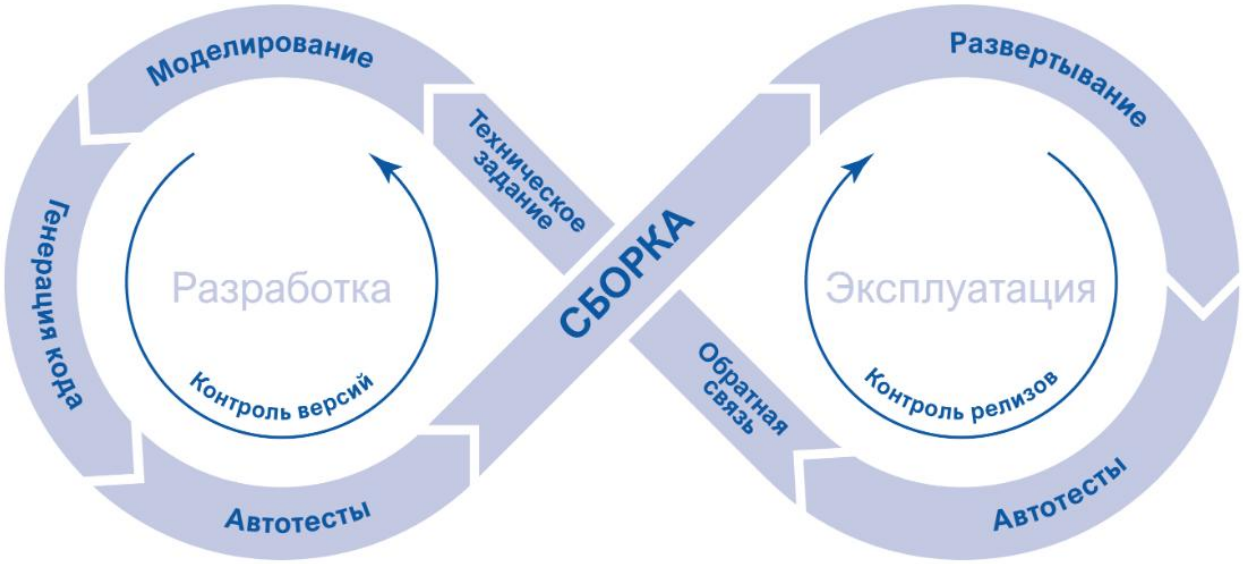


Рисунок 1.2 – Структура жизненного цикла DevOps-разработки ПО

Исходной точкой DevOps-цикла является этап формирования *технического задания* (ТЗ), где анализируются текущие потребности заказчика. Учитывая изменчивый характер этих потребностей, каждый новый цикл разработки может начинаться с пересмотра и актуализации требований. На основании проведенного анализа формулируются конкретные технические требования и формируется задание на разработку.

Этап *моделирования* в DevOps предполагает использование UML (Unified Modeling Language) в соответствии с методологией Rational Unified Process (см. подраздел 1.10). Графические модели, построенные на основе UML-нотации, сопровождаются детальными текстовыми описаниями, что обеспечивает комплексное представление проектных решений.

Процесс *генерации кода* сочетает автоматизированные методы с ручной доработкой. Современные инструменты, включая генеративный искусственный интеллект, позволяют автоматически создавать фрагменты кода, которые затем подвергаются тщательной доработке разработчиками. Параллельно с генерацией основного кода создаются тестовые сценарии – *автотесты*, часть которых также может быть автоматически сгенерирована на основе построенных моделей.

Непрерывная интеграция (Continuous Integration) обеспечивает регулярную *сборку* приложения в единый продукт с параллельным документированием. Результатом данного этапа является готовая к развертыванию версия программного обеспечения, передаваемая заказчику.

Процесс *развертывания* (Continuous Deployment/Delivery) осуществляется итеративно: первая версия системы устанавливается «с нуля», последующие обновления накладываются на работающую систему. Пользователи получают новые функциональные возможности без необходимости ручного вмешательства в процесс установки, наблюдая лишь

конечный результат в виде улучшенного функционала или устраненных ошибок.

Таким образом, весь цикл DevOps делится на две большие части:

1) Continuous Integration (CI), начиная с осмысливания потребностей и заканчивая тестированием и сборкой;

2) Continuous Deployment (CD), где продукт непрерывно разворачивается.

Функциональность продукта постепенно нарабатывается, новая функциональность интегрируется в систему. На каждой итерации добавляется новая порция функциональности.

Обратная связь от пользователей (Continuous Feedback) является важным элементом DevOps-цикла. Пользователи могут добровольно сообщать о результатах использования новых версий либо обязаны информировать разработчиков о выявленных ошибках, как это реализовано в механизме отправки отчетов об ошибках в Microsoft Windows.

Завершающим этапом цикла является *анализ полученной обратной связи* (на рис. 1.2 символически показана стрелка назад) и *оценка результатов* (Continuous Assessment). Весь процесс DevOps характеризуется непрерывностью всех составляющих: моделирования, генерации кода, тестирования, интеграции, развертывания, обратной связи и оценки.

Таким образом, в цикле DevOps все процессы непрерывные (Continuous) – Continuous Modeling, Continuous Generation Code, Continuous Testing, Continuous Integration, Continuous Deployment/Delivery, Continuous Testing, Continuous Feedback, Continuous Assessment.

В промышленных масштабах DevOps демонстрирует особую эффективность при разработке коммерческих программных продуктов. Модель Continuous Delivery обеспечивает регулярное обновление ПО непосредственно на рабочих местах пользователей. Большинство базовых обновлений распространяется бесплатно, в то время как расширенные функциональные возможности могут предлагаться за дополнительную плату.

Поскольку DevOps – это командная работа (между сотрудниками, занимающимися разработкой, операциями и тестированием), нет единого инструмента «DevOps»: это скорее набор (инструментальная цепочка), состоящий из нескольких инструментов. Как правило, инструменты DevOps вписываются в одну или несколько следующих категорий, что отражает ключевые аспекты разработки ПО.

1. Кодирование – разработка и анализ кода, инструменты контроля версий, слияние кода.

2. Сборка – инструменты непрерывной интеграции, статус сборки.

3. Тестирование – инструменты непрерывного тестирования, обеспечивающие быструю и своевременную оценку бизнес-рисков.

4. Упаковка – репозиторий артефактов, предварительная установка приложения.

5. Релиз – управление изменениями, официальное утверждение выпуска, автоматизация выпуска.

6. Настройка – конфигурация и управление инфраструктурой. Инфраструктура как инструменты кода.

7. Мониторинг – измерение производительности приложений, взаимодействие с конечным пользователем.

8. Непрерывная поставка.

9. Непрерывная интеграция.

Из этого перечня видно, что инструментальная цепочка DevOps типична для традиционного жизненного цикла ПО. Несмотря на то, что доступно множество инструментов, некоторые категории имеют особо важное значение в настройке инструментальных средств DevOps.

1.2. Ключевые ценности DevOps

DevOps как методология основывается на трех ключевых принципах: системном подходе, непрерывной обратной связи и обмене знаниями. Эти принципы формируют концептуальную основу эффективной организации процессов разработки и эксплуатации программного обеспечения.

Системный подход в DevOps является естественным продолжением системной природы самого программного обеспечения, которое представляет собой комплекс взаимосвязанных функций, задач и данных, управляющих определенными процессами. Применение системного мышления к процессу разработки позволяет рассматривать каждый этап жизненного цикла и каждую роль в команде как взаимосвязанные элементы единого организма. Такой подход обеспечивает создание целостной системы разработки, сочетающей технические инструменты, методологические принципы DevOps и организационную культуру, что в совокупности повышает корректность, гибкость и эффективность процесса создания ПО.

Принцип обратной связи является обязательным условием эффективной реализации DevOps-практик. В рамках данной методологии обратная связь реализуется через механизм непрерывного мониторинга (Continuous Feedback), который включает диагностику, отслеживание и измерение параметров системы в реальном времени. Современный мониторинг исключительно компьютеризирован, поскольку только цифровые методы обеспечивают необходимую точность и оперативность сбора данных. Получаемые в результате мониторинга структурированные данные подлежат автоматизированной обработке и анализу.

Обработанные данные мониторинга передаются в специализированные системы хранения и анализа, где преобразуются в формализованные знания. Применение методов интеллектуального анализа данных (Data Mining) позволяет извлекать из этих знаний ценную информацию для принятия управленческих решений, направленных на оптимизацию процессов разработки, совершенствование программного продукта и решение других производственных задач. Важно отметить, что принципы DevOps обладают достаточной универсальностью для применения не только в сфере разработки программного обеспечения, но и в других предметных областях.

Теоретической основой DevOps выступает синтез трех инженерных дисциплин: программной инженерии, отвечающей за процессы разработки и сопровождения ПО; системной инженерии, обеспечивающей управление сложными системами и инфраструктурой; бизнес-инжиниринга, ориентированного на оптимизацию процессов с учетом организационных и коммерческих требований. Взаимодействие этих дисциплин (рис. 1.3) формирует комплексный подход к управлению DevOps-процессами (рис. 1.4), объединяющий технические, системные и бизнес-аспекты разработки программного обеспечения.

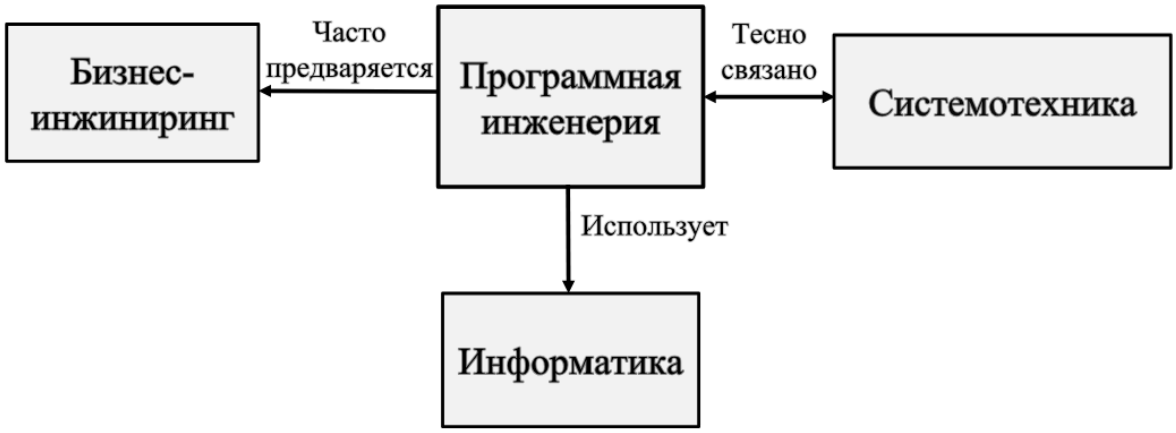


Рисунок 1.3 – Схема взаимодействия разных инженерий в программном процессе



Рисунок 1.4 – Триада управления программным процессом по DevOps

DevOps как методология опирается на три взаимосвязанные инженерные дисциплины, формирующие его концептуальную основу. Программная инженерия обеспечивает методологический базис для реализации и поддержки полного жизненного цикла разработки, материализуясь в DevOps через механизмы непрерывной интеграции и поставки (CI/CD).

Системная инженерия предоставляет необходимую инфраструктурную платформу, включающую вычислительные ресурсы, телекоммуникационные системы и сопутствующее аппаратное обеспечение, без которых невозможно функционирование DevOps-процессов.

Бизнес-инжиниринг выступает интеграционным звеном, обеспечивающим адаптацию технических решений к конкретным бизнес-контекстам. Данная дисциплина гарантирует соответствие DevOps-практик стратегическим целям организации, их встраивание в существующие бизнес-процессы и оценку экономической эффективности внедряемых решений.

Таким образом, эффективная реализация DevOps требует синхронизированного взаимодействия программной инженерии (разработка), системной инженерии (инфраструктура) и бизнес-инжиниринга (управление), что обеспечивает целостность подхода на всех уровнях – от технической реализации до бизнес-результатов.

1.3. Централизация в DevOps

Централизация представляет собой фундаментальный принцип организации DevOps-среды, обеспечивающий создание единой точки консолидации критически важных компонентов системы. Наиболее полная реализация данного принципа предполагает развертывание географически распределенных катастрофоустойчивых центров обработки данных (ЦОД) уровня Tier 4, обеспечивающих максимальную надежность хранения данных за счет их репликации между континентами. Такая архитектура гарантирует сохранность информации даже в случае масштабных катастроф регионального масштаба.

В практических реализациях чаще применяются решения уровня Tier 3, предусматривающие полное резервирование оборудования, круглосуточную техническую поддержку и высокий уровень защиты при приемлемых показателях надежности. Централизованное хранение предполагает организацию пяти ключевых репозиториев: исходного кода, тестовых сценариев, требований, доработок и технической документации.

Для поддержки полного цикла разработки требуется специализированная инфраструктура, включающая серверы сборки, тестовые стенды и вычислительные мощности. В экосистеме Microsoft Azure DevOps Services централизованное хранилище реализуется на базе СУБД Microsoft SQL Server, что обеспечивает оптимальную интеграцию с другими компонентами платформы.

Централизация обеспечивает прозрачность процессов разработки, позволяя оперативно принимать корректирующие решения и избегать

дублирования работ. Как показано на рис. 1.5, подцикл непрерывной поставки (CD) включает регулярное тестирование, автоматизированную сборку и планирование релизов. Реализация такого цикла возможна только при условии комплексной автоматизации, где программное обеспечение выступает одновременно как инструмент разработки и как конечный продукт.

Таким образом, централизация в DevOps представляет собой сбалансированную модель, сочетающую единое управление ключевыми компонентами с распределенным характером выполнения задач.

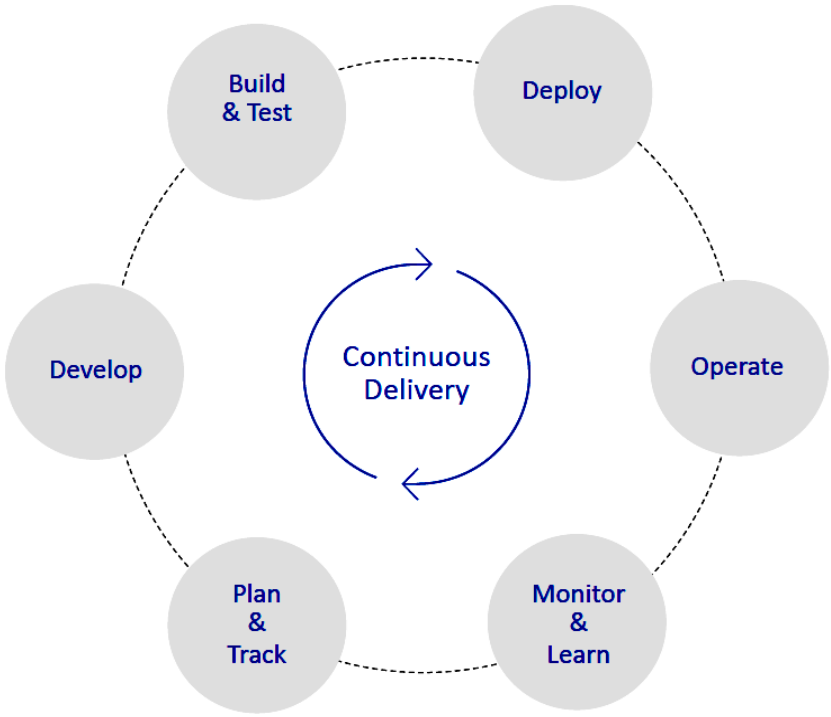


Рисунок 1.5 – Структура подцикла DevOps / CD

1.4. Автоматизация в DevOps

Современные подходы к обеспечению качества кода предполагают комплексную автоматизацию проверочных процедур. Хотя традиционные методы, такие как ручное тестирование и вычитка кода, сохраняют свою актуальность для особо критичных участков кода, основная масса проверок (до 97 %) выполняется автоматизированными средствами. Это позволяет существенно оптимизировать трудозатраты тестировщиков, сосредоточив их внимание на наиболее важных фрагментах кода.

Системы версионного контроля, такие как GitHub, обеспечивают автоматизацию процессов непрерывной интеграции (CI). GitHub как OpenSource-решение предоставляет комплексные возможности для организации командной разработки, включая создание репозиториев и управление виртуальной файловой структурой. Для эффективного использования системы требуется знание ее командного интерфейса.

Современная программная инженерия опирается на широкий спектр автоматизированных инструментов: языки программирования (Python), среды разработки (Visual Studio), платформы (.NET Framework), системы управления базами данных, средства контейнеризации и оркестрации микросервисов. Подробно об этих инструментах см. раздел 3. Большинство из них распространяются по OpenSource-модели, обеспечивая доступность для разработчиков.

Ключевым показателем успешности автоматизации является ценность конечного продукта для бизнеса. Эта ценность определяется следующими параметрами: соблюдение временных рамок разработки, соответствие бюджетным ограничениям, минимизация времени от разработки до релиза, сведение к минимуму проблем после выпуска.

Качественно организованный программный процесс позволяет минимизировать количество ошибок, однако полностью исключить их невозможно из-за постоянной эволюции требований. Особую сложность представляют изменения, поступающие на этапе выпуска релиза – такие требования переносятся в бэклог проекта для последующей реализации в рамках стандартного DevOps-цикла.

1.5. Команда DevOps

DevOps как методология предполагает формирование специализированной команды, расширяющей традиционный состав участников разработки программного обеспечения. Помимо стандартных ролей (руководитель проекта, аналитик, архитектор, разработчики, тестировщики), в DevOps-команде появляются специфические позиции, отражающие особенности данной методологии.

Инженер разработки представляет собой модифицированную роль разработчика, обладающего экспертизой в области DevOps и ответственного за реализацию подцикла непрерывной интеграции (CI). Инженер контроля качества расширяет традиционные функции тестировщика, включая в свою зону ответственности не только автоматизированное тестирование (Continuous Testing), но и другие методики обеспечения качества.

Особое значение приобретает роль инженера безопасности, обусловленная требованиями к защите облачной IT-инфраструктуры. Вопреки распространенному заблуждению о ненадежности облачных решений, практика демонстрирует их существенное преимущество перед локальными системами в вопросах безопасности. Статистика свидетельствует, что более 95 % инцидентов, связанных с утечкой или потерей данных в локальных инфраструктурах, вызваны человеческим фактором. В облачных средах этот риск минимизирован за счет высокой профессиональной подготовки обслуживающего персонала и строгих репутационных механизмов, побуждающих провайдеров обеспечивать максимальный уровень защиты.

Инженер инфраструктуры эволюционирует из традиционного системного администратора в специалиста по интеграции компании с облачными

сервисами. Его ключевая задача заключается в оптимальном управлении облачными ресурсами, их масштабировании и cost-оптимизации для нужд разработки.

Современная практика предполагает формирование тесного партнерства между IT-департаментами и бизнес-подразделениями при определении моделей потребления IT-ресурсов. Хотя IT-отделы сохраняют свою центральную роль, она трансформируется в сторону посредничества в сфере облачных услуг, управления гибридными облачными средами, координации IT-стратегии на уровне предприятия.

Однако это партнерство носит двусторонний характер. Бизнес-подразделения получают возможность прямого приобретения облачных сервисов для оперативного удовлетворения своих потребностей. При этом сохраняется ценность стратегического подхода IT-служб, проявляющаяся в управлении рисками, оптимизации затрат за счет эффекта масштаба, снижении совокупной стоимости владения, обеспечении единообразия политик.

Современные условия выдвигают три ключевых требования к DevOps-инженерам:

1. Способность согласовывать IT-стратегию с бизнес-приоритетами, адаптируя решения к меняющимся потребностям бизнес-подразделений.
2. Глубокое понимание перспективных технологий и лучших практик в IT-области, позволяющее оставаться на переднем крае технологического развития.
3. Умение проводить быстрые преобразования, стимулировать инновации и создавать пользовательские среды, положительно влияющие на финансовые показатели компании.

Неспособность DevOps-команд соответствовать этим требованиям может привести к ситуации, когда бизнес-подразделения начнут самостоятельно инвестировать в облачные технологии, что создаст риски усложнения IT-ландшафта, потери эффективности, нарушения единообразия IT-услуг.

В таких условиях IT-департамент рискует превратиться из катализатора роста в традиционный центр затрат, что демонстрирует потенциальный конфликт между устаревшими операционными моделями и растущими бизнес-ожиданиями.

1.6. Культура разработки по DevOps

Культура разработки, неоднократно обсуждавшаяся в рамках данного пособия, представляет собой систему идеологических принципов, этических норм и организационных правил, обеспечивающих эффективную реализацию непрерывных процессов. Эти принципы, подробно изложенные в кодексе этики программного инженера, в полной мере применимы к инженерному DevOps, где культура служит фундаментом для продуктивного взаимодействия между участниками процесса.

Следует подчеркнуть, что DevOps не сводится к отдельной роли или рутинной деятельности, формализованной через научный подход и

стандартизированные процедуры. Причем эта деятельность не ограничивается рамками конкретного департамента – масштабы DevOps-практик потенциально глобальны, охватывая распределенные команды по всему миру. По своей сути DevOps представляет собой культурную среду, обеспечивающую эффективные условия для создания продуктов, виртуальную структуру с практически неограниченными возможностями масштабирования.

Методологически DevOps нельзя рассматривать как простой набор инструментов. Это комплексный технологический и методологический базис, позволяющий интегрировать различные технические решения, методологические подходы и инструментальные средства для оптимизации процессов разработки. Примечательно, что сфера применения DevOps-принципов не ограничивается созданием программного обеспечения.

Важной характеристикой DevOps является его двунаправленная природа. Система непрерывно получает и обрабатывает мощные информационные потоки обратной связи (Continuous Feedback) от многочисленных пользователей и заказчиков. Эффективная обработка этих данных представляет собой отдельную методологическую задачу, требующую специальных подходов и решений.

1.7. Развитие принципов разработки ПО

DevOps представляет собой эволюционное развитие методологии разработки программного обеспечения, воплощающее современные технологические подходы и инновационные практики. В рамках гибких методик DevOps ключевым аспектом становится реализация **частых релизов**, стремящихся к непрерывному потоку обновлений. Хотя абсолютная непрерывность является идеализированной концепцией, методология предполагает максимально возможное сокращение интервалов между релизами до состояния их практического слияния в единый поток.

Фундаментальной инновацией DevOps выступает принцип **модульности кода**, обусловленный требованиями коллективной разработки и разделения труда. Исходный код организуется в виде атомарных компонентов, хранящихся в системе контроля версий. Идеальная модель предполагает генерацию нового релиза после каждого внесенного изменения, хотя на практике частота релизов определяется ресурсными ограничениями.

DevOps создает методологическую основу для внедрения современных архитектурных решений, в частности **микросервисной архитектуры и контейнеризации**. Микросервисный подход, являясь альтернативой монолитным архитектурам, обеспечивает существенно более высокие показатели масштабируемости и гибкости системы, несмотря на повышенные требования к квалификации разработчиков. Контейнеризация выступает естественным технологическим дополнением микросервисной архитектуры, предоставляя изолированную среду для каждого сервиса, что повышает надежность, безопасность и эффективность всей системы в целом.

Данные принципы представляют собой закономерное развитие методологии разработки программного обеспечения, отвечающее современным требованиям к скорости, надежности и масштабируемости IT-решений.

1.8. Применение DevOps вне IT

Как было продемонстрировано ранее, принципы DevOps обладают значительным потенциалом для применения за пределами IT-сферы. Ключевые концепции DevOps-инжиниринга, включая распределенную ответственность, междисциплинарную коммуникацию, непрерывную обратную связь, обмен знаниями и автоматизацию процессов, демонстрируют свою эффективность в различных областях человеческой деятельности.

Методологический аппарат DevOps находит практическое применение в таких сферах, как маркетинг, проектное управление, производственные процессы, медицинская практика, банковские операции и образовательная деятельность. Универсальность подхода обусловлена его фундаментальной ориентированностью на оптимизацию взаимодействия между участниками процессов, обеспечение прозрачности операций и повышение эффективности за счет автоматизации рутинных задач.

Способность DevOps-принципов к трансдисциплинарной адаптации подтверждает их концептуальную ценность как современной методологии управления сложными процессами, независимо от конкретной предметной области. Это свидетельствует о том, что DevOps представляет собой не просто набор технических практик, а универсальную систему организации эффективной деятельности.

1.9. Проблемы коммуникации в DevOps-процессах

В процессе работы над проектом возникают проблемы коммуникации в осуществлении программного процесса, характерные для иерархической модели группы (см. подраздел 1.5). На рис. 1.6 эти проблемы представлены стенами. Они могут возникать на разных этапах жизненного цикла.

На этапе планирования возникает принципиальная проблема взаимодействия между заказчиком и поставщиком. Эффективная координация требует от поставщика не только точного восприятия исходных требований заказчика, но и активного участия в их уточнении и дополнении. При выявлении технических или ресурсных ограничений поставщик обязан своевременно информировать заказчика и совместно искать пути их преодоления.

Формулировка требований представляет собой критически важный этап, определяющий весь последующий процесс разработки. Включение аналитика в работу на этой стадии позволяет провести профессиональный анализ требований на соответствие критериям полноты, непротиворечивости и реализуемости, что создает основу для стабильного развития проекта.

Статистически подтверждено, что в проектах с численностью участников 10-15 человек вероятность возникновения проблемных ситуаций достигает 50 %, причем основной причиной выступает дефицит коммуникации как внутри команды, так и с заказчиком. Это подчеркивает необходимость выстраивания прозрачных и эффективных каналов взаимодействия.

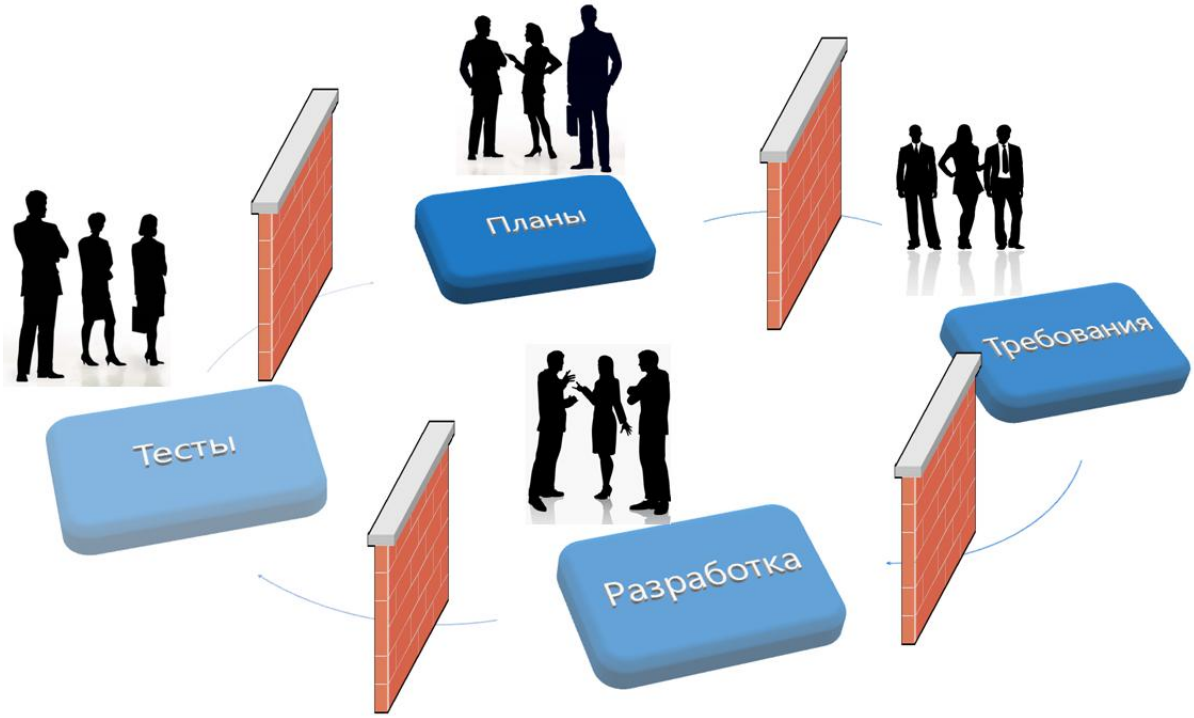


Рисунок 1.6 – Проблемы коммуникации в программном процессе

Процесс разработки включает два взаимосвязанных этапа: проектирование и непосредственную реализацию. Для крупных проектов особое значение приобретает четкое распределение ролей и налаживание системы коммуникации, охватывающей не только основную команду разработчиков, но и смежные подразделения, участвующие в связанных бизнес-процессах.

На этапе тестирования возникает необходимость трехсторонней коммуникации между тестировщиками, разработчиками и заказчиками, что обусловлено постоянной эволюцией требований. Распространенное заблуждение о возможности спонтанного разрешения проблем требует внедрения формализованных практик взаимодействия, включая регулярные совещания, системы обмена сообщениями и другие инструменты синхронной и асинхронной коммуникации.

Методология DevOps предлагает комплексный подход к организации коммуникационных процессов на протяжении всего жизненного цикла проекта, обеспечивает единую среду для взаимодействия всех участников процесса, поддержку различных форматов коммуникации (онлайн и оффлайн), безопасное хранение и регламентированный доступ к проектным данным, интеграцию с разнообразными технологическими решениями.

Архитектурная реализация включает специализированные серверные компоненты, предназначенные для управления большими массивами данных и обеспечения контролируемого доступа к ним, что создает технологическую основу для эффективной командной работы.

1.10. Рациональный унифицированный процесс

Процесс построения диаграмм UML имеет свои особенности, тесно связанные с семантикой элементов этих диаграмм. Язык UML используется в ходе объектно-ориентированного проектирования (ООП) ПО. При этом сам процесс ООП в контексте языка UML получил специальное название – рациональный унифицированный процесс (Rational Unified Process, RUP). Концепция RUP и основные ее элементы разработаны А. Джекобсоном в ходе его работы над языком UML.

Унифицированный процесс – это процесс, управляемый прецедентами, которые отражают сценарии взаимодействия пользователей с ИС и взгляд пользователей на систему снаружи.

Суть концепции RUP заключается в последовательной декомпозиции процесса ООП на отдельные этапы. На каждом этапе осуществляется разработка соответствующих типов канонических диаграмм модели системы.

Этап 1. Строятся логические представления статической модели структуры системы (Кто? Что? Сколько?).

Этап 2. Строятся логические представления динамической модели поведения системы (Как? Когда?).

Этап 3. Строятся физические представления модели системы (Что? Где? Куда?).

Таким образом, процесс ООП можно представить как поуровневый спуск от наиболее общих моделей и представлений концептуального уровня к более частным и детальным представлениям логического и физического уровней (рис. 1.7). При этом на каждом из этапов ООП данные модели последовательно дополняются все большим количеством деталей, что позволяет более адекватно отражать различные аспекты конкретной реализации сложной системы.

В результате реализации RUP должны быть построены канонические диаграммы на языке UML. При этом последовательность их разработки в основном совпадает с их последовательной нумерацией. Более детально основные этапы RUP показаны на рис. 1.8.

Одним из важнейших этапов разработки согласно концепции RUP является этап определения требований, который заключается в сборе всех возможных пожеланий заказчика к работе системы. Позднее эти данные должны быть систематизированы и структурированы. На данном же этапе в ходе интервью с будущими пользователями и изучения документов аналитики должны собрать как можно больше требований к будущей системе. При этом, как уже неоднократно отмечалось ранее, пользователи часто сами не представляют, что они должны получить в итоге.



Рисунок 1.7 – Общая схема взаимосвязей моделей и представлений сложной системы в процессе объектно-ориентированного анализа и проектирования

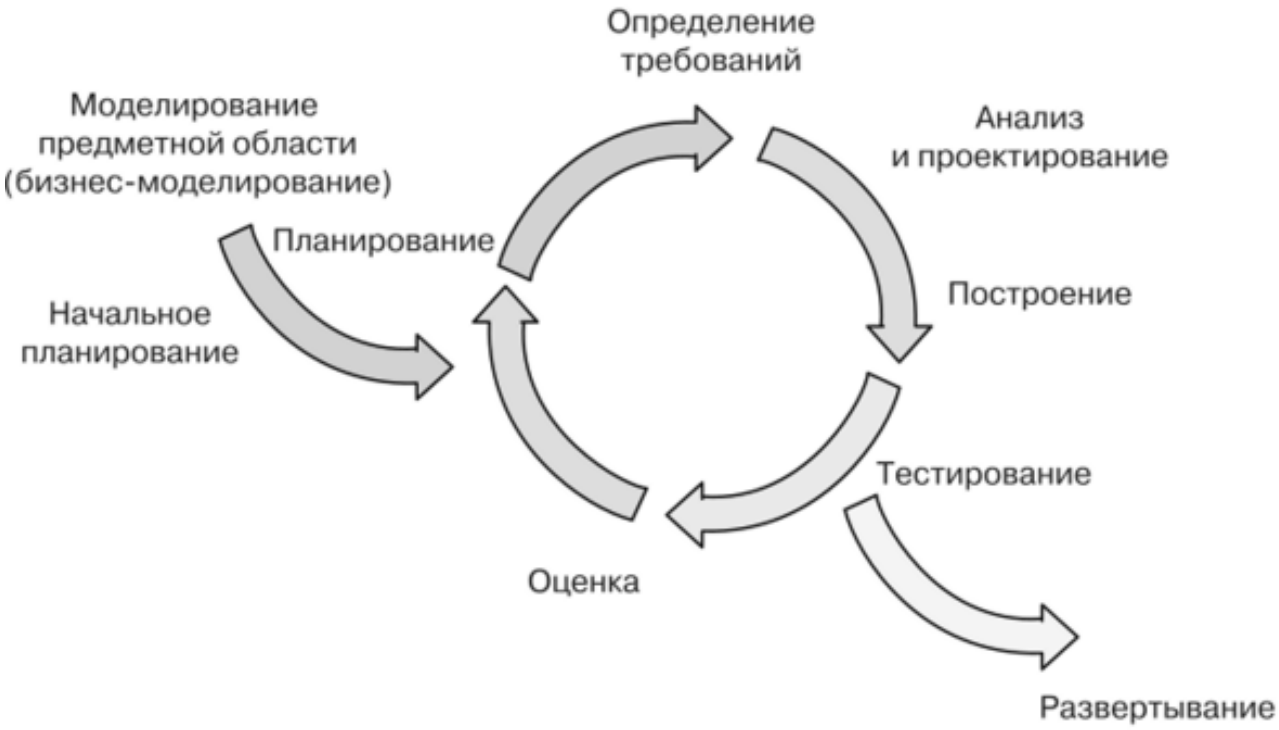


Рисунок 1.8 – Схема применения концепции RUP при разработке ИС

Для облегчения этого процесса аналитики используют диаграммы прецедентов и диаграммы классов. Именно эти диаграммы строятся в первую очередь. Логично вначале построить диаграмму вариантов использования, а затем для каждого варианта или для ИС в целом строить диаграмму классов. Логика RUP в плане построения диаграмм UML показана на рис. 1.9.

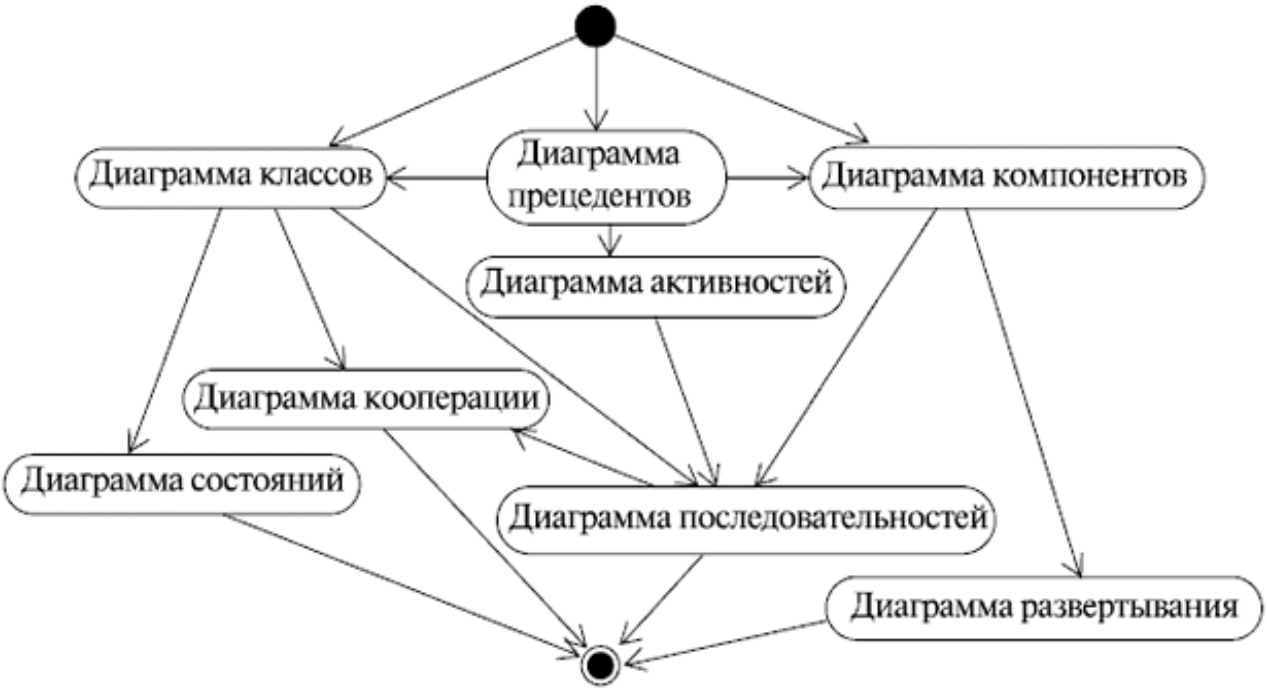


Рисунок 1.9 – Стандартный процесс RUP при построении модели ИС

Далее для каждого варианта или для ИС в целом строится диаграмма активностей. Классы в их взаимодействии моделируются диаграммами взаимодействия, а классы с наиболее нетривиальным поведением моделируются диаграммами состояний. Последними строятся диаграммы физической реализации ИС. Логическая структура построения модели ИС на этапах ее жизненного цикла показана на рис. 1.10.



Рисунок 1.10 – Логическая структура построения модели ИС на этапах ее жизненного цикла

Процесс RUP обширен и содержит рекомендации по ведению различных программных проектов – от создания программ группой разработчиков в несколько человек до распределенных программных проектов, объединяющих тысячи человек на разных континентах. Однако, несмотря на колоссальную разницу между ними, методы применения моделей, создаваемых при помощи UML, будут одни и те же. Диаграммы UML, создаваемые на различных этапах разработки, неотделимы от остальных артефактов программного проекта и часто являются связующим звеном между отдельными процессами (этапами) RUP.

Решение о применении конкретных диаграмм зависит от поставленного в компании процесса разработки, который, хотя и называется унифицированным, однако не есть нечто статичное. Компания «Rational» не только предлагает его улучшать и дорабатывать, но и предоставляет специальные средства внесения изменений в базу данных RUP.

1.11. Контрольные вопросы

1. Что такое DevOps? Каковы его цели и задачи?
2. Какие непрерывные процессы включает в себя цикл DevOps?
3. В каких сферах применяется DevOps?
4. Перечислите основные проблемы реализации цикла DevOps и способы их преодоления.
5. Пересилите ключевые ценности DevOps и поясните их суть.
6. Опишите модель сложной системы в контексте реализации цикла DevOps.
7. Перечислите и охарактеризуйте основные функции DevOps-инженера.
8. Опишите модель сложной системы в контексте процесса RUP.
9. Каково типовое содержание процесса RUP?

2. Архитектура информационных систем

Архитектура ИС – это ее внутренняя структура (компоненты и их связи), основы пользовательского интерфейса продукта, а также средства разработки и управления проектом.

2.1. Многослойная архитектура

Многослойная архитектура (Layered Architecture) представлена на рис. 2.1. Здесь выделяется три слоя, каждый из которых отвечает за специфику и этапы работы системы. Нижний слой отвечает за хранение данных. Верхний слой – за интерфейс взаимодействия с пользователем. Средний слой реализует бизнес-логику, уровень приложения, который и реализует основные расчетные вычислительные операции обработки данных.



Рисунок 2.1 – Многослойная архитектура

Пользователи управляют системой через интерфейс, бизнес-логика обеспечивает решение их задач, для чего требуется обмен данными. Иногда сквозные транзиты данных приходится выполнять напрямую между интерфейсом и базой данных (БД), минуя бизнес-логику. Это делает данную архитектуру избыточной. В таком случае более предпочтительна двухуровневая архитектура.

Преимущества многослойной архитектуры:

– более простая реализация по сравнению с другими подходами;

– предлагает абстракцию благодаря разделению ответственностей между уровнями;

– изолирование защищает одни слои от изменений других;

– повышает управляемость программного обеспечения за счет слабой связанности.

Недостатки многослойной архитектуры рассмотрим более подробно.

– Не предлагает большой масштабируемости. Масштабируемость возможна только в пределах слоев. Можно нарастить интерфейс, бизнес-логику и данные, но какие-то комплексные решения по наращиванию функционала реализовать затруднительно, потому что приходится вносить изменения в каждый слой, что не совсем удобно.

– ПО, созданное с таким подходом, будет иметь монолитную структуру, усложняющую внесение модификаций. Гораздо проще вносить изменения в атомарные компоненты. В многослойной архитектуре приходится модифицировать все три слоя во всех реализациях модифицируемых функций. А это несет риски внесения ошибок во все слои, что чревато последствиями, в первую очередь временными задержками на поиск и устранение ошибок.

– Данные должны проходить по каждому слою, даже если нет необходимости передавать их с определенных слоев. Необходимо транзитировать эти данные через бизнес-логику, обеспечивать корректность перетоков данных, что создает некоторые трудности, в первую очередь логические и временны́е.

2.2. Многоуровневая архитектура

Многоуровневая архитектура (Tiered Architecture) строится примерно так же, как и многослойная (см. подраздел 2.1), но секционирована по отдельным уровням. Это более универсальное решение. В общем случае многоуровневая архитектура включает N уровней, на каждом из которых своя иерархия и функции управления. Самый простой вариант – двухуровневая клиент-серверная архитектура. Более прогрессивная – трехуровневая архитектура, она реализуется широко для относительно простых решений. Общее представление многоуровневой архитектуры показано на рис. 2.2. Есть клиент и сервер. Клиент – пассивный узел, сервер – активный узел. В то же время сервер нуждается в услугах клиента. Клиент требует определенных вычислений, выдает задание (request) на вычисление, сервер отвечает (response). Обычно клиентов много, а сервер один. Хотя бывает, что серверов тоже много.

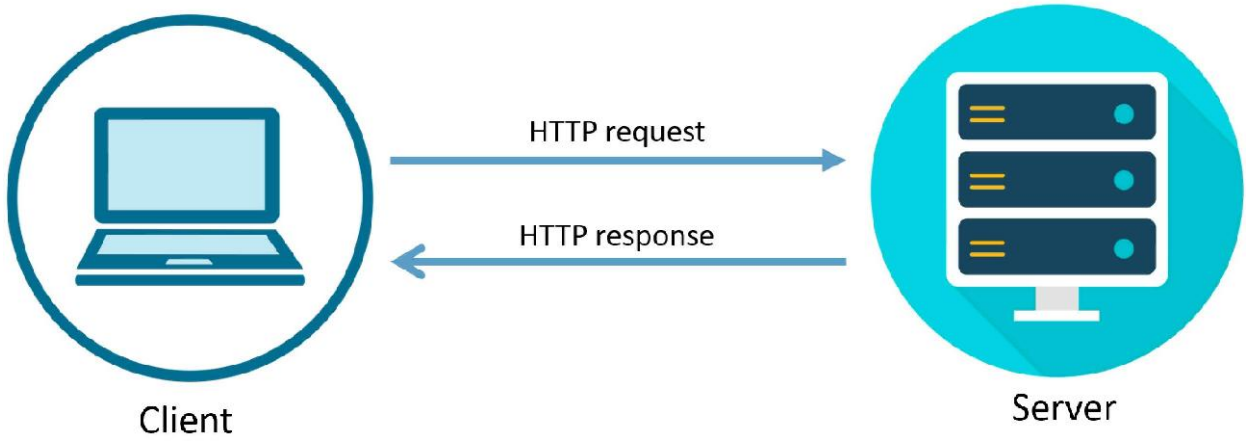


Рисунок 2.2 – Многоуровневая архитектура

Двухуровневая клиент-серверная архитектура показана на рис. 2.3. Эта архитектура самая простая, намного проще, чем многослойная. Здесь есть рабочие станции с приложениями – это интерфейс и частично бизнес-логика, а также имеется сервер, который также реализует серверную бизнес-логику и централизовано хранит и перерабатывает данные.



Рисунок 2.3 – Двухуровневая архитектура

Если архитектура отображает веб-приложение, то говорят, что рабочие станции представляют фронтэнд-часть приложения, а на сервере работает бэкенд-часть. Возможен вариант, когда на сервере работает только БД (сервер БД), а вся бизнес-логика реализована на клиентских машинах. Эта неоднозначность вызывает недостаток надежности, который частично преодолевается в трехуровневой архитектуре.

Трехуровневая архитектура (рис. 2.4) предполагает раздвоение сервера на сервер приложений, реализующий бизнес-логику, и сервер БД. Возникает промежуточный уровень по аналогии с трехслойной архитектурой. Такая архитектура с разделенными серверами отличается более высокой надежностью и безопасностью функционирования ИС. На серверных уровнях может быть множество серверов (рис. 2.5): web-сервер, почтовый сервер, интернет-сервер, файл-сервер, сервер сборок (при разработке приложений) и т. д. Данная многосерверность определяется сложностью, динамичностью и другими особенностями предметной области.



Рисунок 2.4 – Трехуровневая архитектура

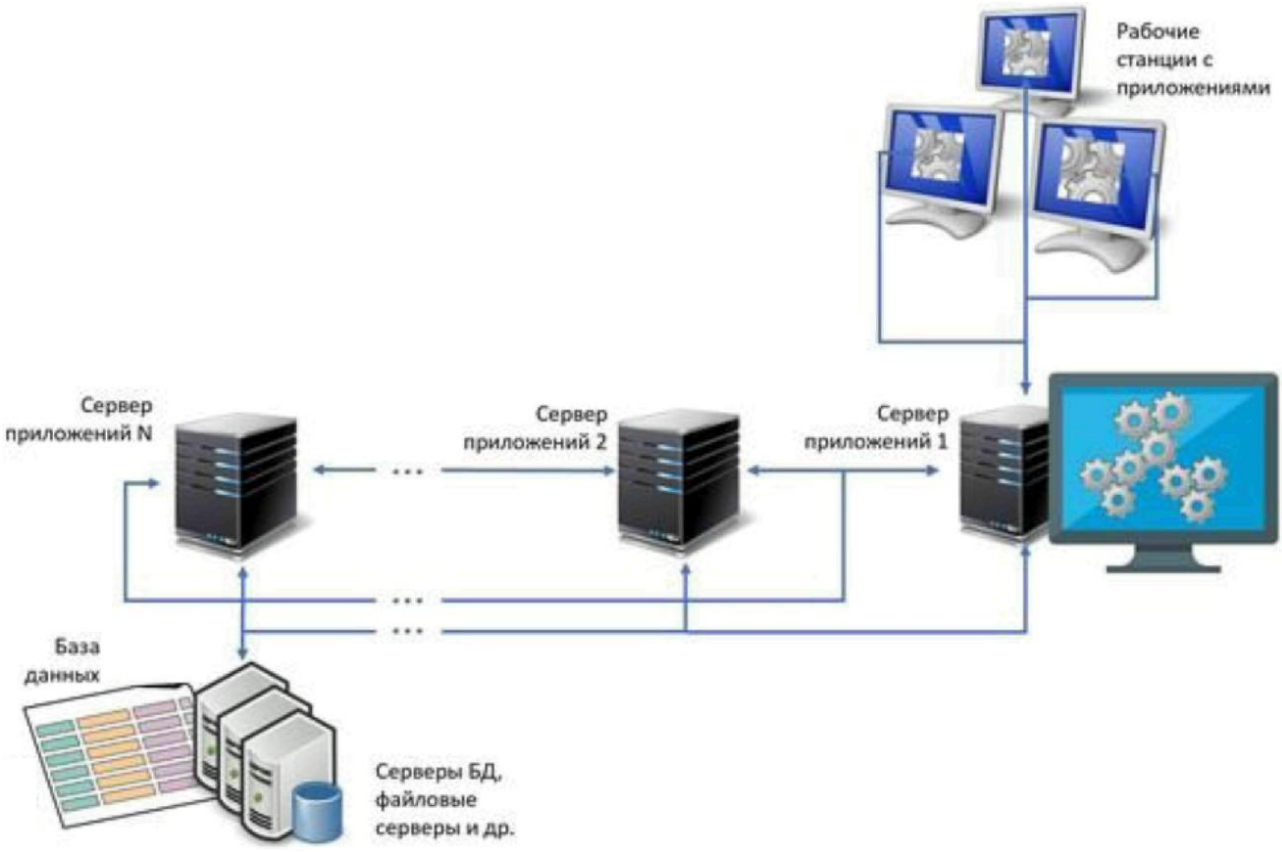


Рисунок 2.5 – N-уровневая архитектура

2.3. Сервис-ориентированная архитектура

Сервис-ориентированная архитектура (Service Oriented Architecture – SOA) – это более продвинутое и современное архитектурное решение. Она ориентирована на реализацию сервисов, программного обеспечения, информационных систем в виде сервисов в облаках, как правило, реализуя тем самым отдельные автоматизированные функции или комплексы функций (подсистемы) на базе отдельных сервисов. Тем не менее, это все еще целостная монолитная система – так называемый *монолит*, который реализует функции ИС и ориентирован на определенную структуру предметной области.

Основные элементы SOA:

- сервисы (Services);
- сервисная шина (Service Bus);
- сервисный репозиторий (Service Repository catalogue of services);
- безопасность SOA (SOA Security);
- управление SOA (SOA Governance).

Сервис – это компонент (модуль) ИС, непосредственно доступный пользователю. Например, сервис для получения прогноза работы через Интернет и пользовательский web-интерфейс. Также сервисы можно определить как стандартные интерфейсы, взаимодействующие через сервисную шину (Service Bus) с ядром системы посредством протокола HTTP (рис. 2.6). Стандартные интерфейсы могут быть уже реализованы в виде библиотек и ориентированы на разные задачи и категории пользователей (клиентов).

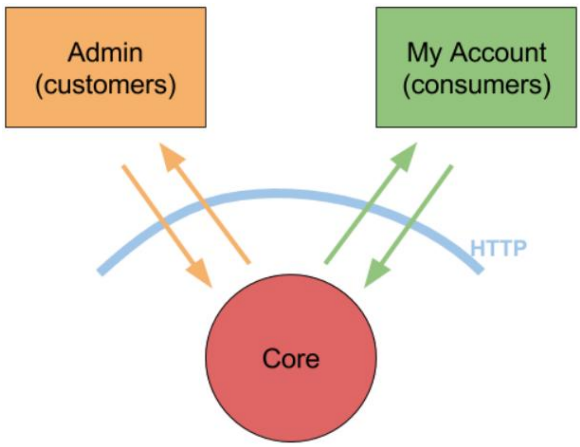


Рисунок 2.6 – Принцип действия SOA-архитектуры

Приложение таким образом собирается по частям из нескольких модулей, комплектующих. Это называется компонентно-ориентированное программирование, где есть уже готовые сервисы. Основные цели, к достижению которых стремится SOA-архитектура, – повысить масштабируемость. Здесь масштабируемость высокая, что позволяет существенно упростить процессы контроля и управление, сократить (оптимизировать) издержки, связанные с разработкой приложения. Можно увеличить долю повторного использования и независимость от платформ, так

как каждый сервис может быть реализован на своем определенном языке. Более подробная схема SOA показана на рис. 2.7. В левой части показана область клиентских приложений. В центре схемы – сервисная шина, обеспечивающая взаимодействие. Веб-сервис обеспечивает взаимодействие по сети с центральным сетевым хранилищем. В правой части схемы показаны службы, отвечающие за безопасность и управление. Также здесь показана бизнес-логика, реализованная на веб-сервере. Эти компоненты, как правило, находятся в облаке и доступны через веб-сервис.

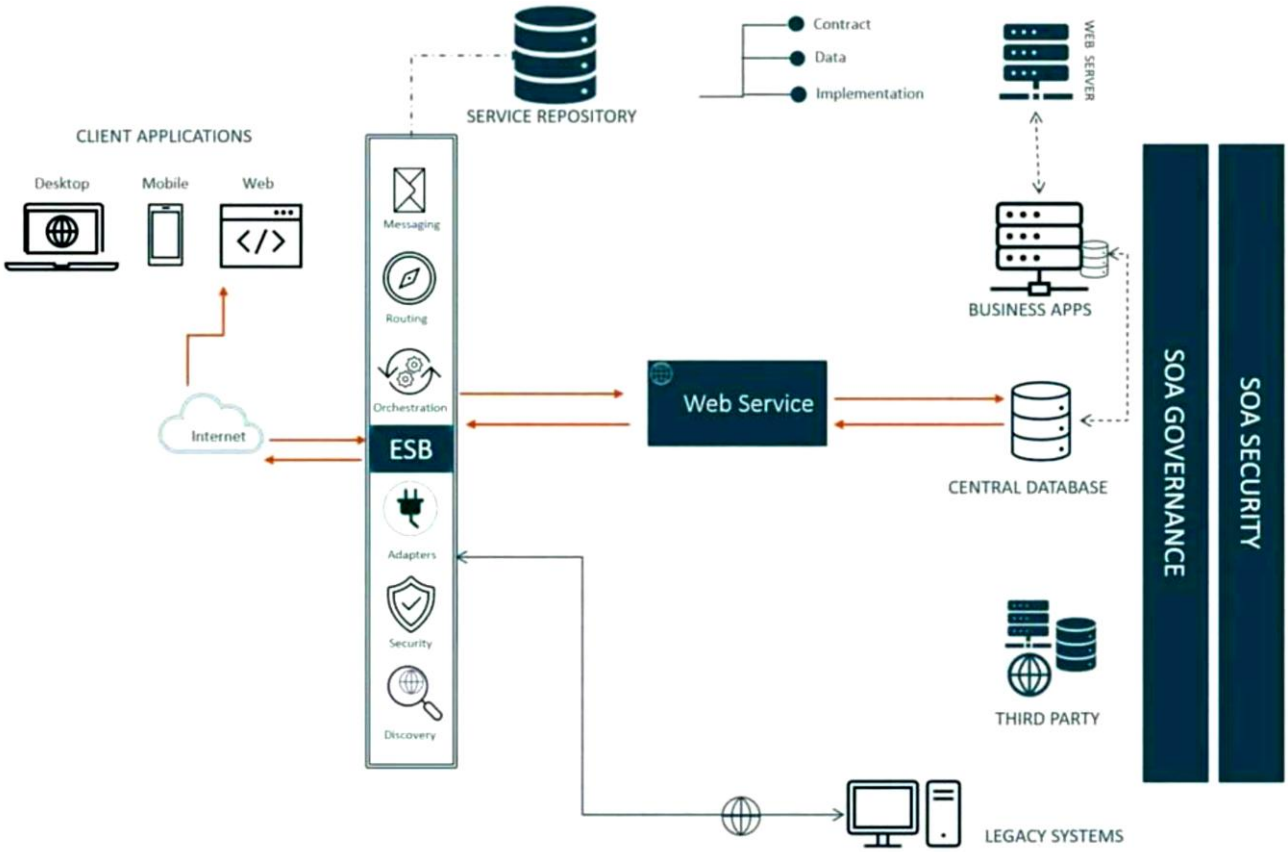


Рисунок 2.7 – Сервис-ориентированная архитектура

Выделяют следующие типы сервисов:

– организационные сервисы (Entity Service), отвечающие за организационные функции;

– доменные сервисы (Domain Service) отвечающие за организацию, сетевой доступ, защищенность и безопасность ИС;

– вспомогательные сервисы (Utility Service), служебные программы, утилиты (например, балансеры, планировщики задач и пр.);

– интегрированный сервис (Integrated Service), определяющий всю логику работы ИС;

– сервис приложений (Application Service), представляющий базовые компоненты, которые реализуют отдельные функции ИС;

– сервис безопасности (Security Service), обеспечивающий безопасность ИС.

SOA фактически соединяет в себе многослойную и многоуровневую архитектуры и обладает более высокой степенью масштабируемости и надежности. Функционал поставляется клиентам как сервис по подписке и, возможно, за абонентскую плату (рис. 2.8). SOA позволяет локализовать функционал в функциональных блоках – сервисах. Это дает возможность без критичного влияния на всю систему поставлять сервисы клиентам или, наоборот, в случае необходимости отказаться от них и встроить другое решение. Такая гибкость достигается путем использования API или других транспортных инструментов.

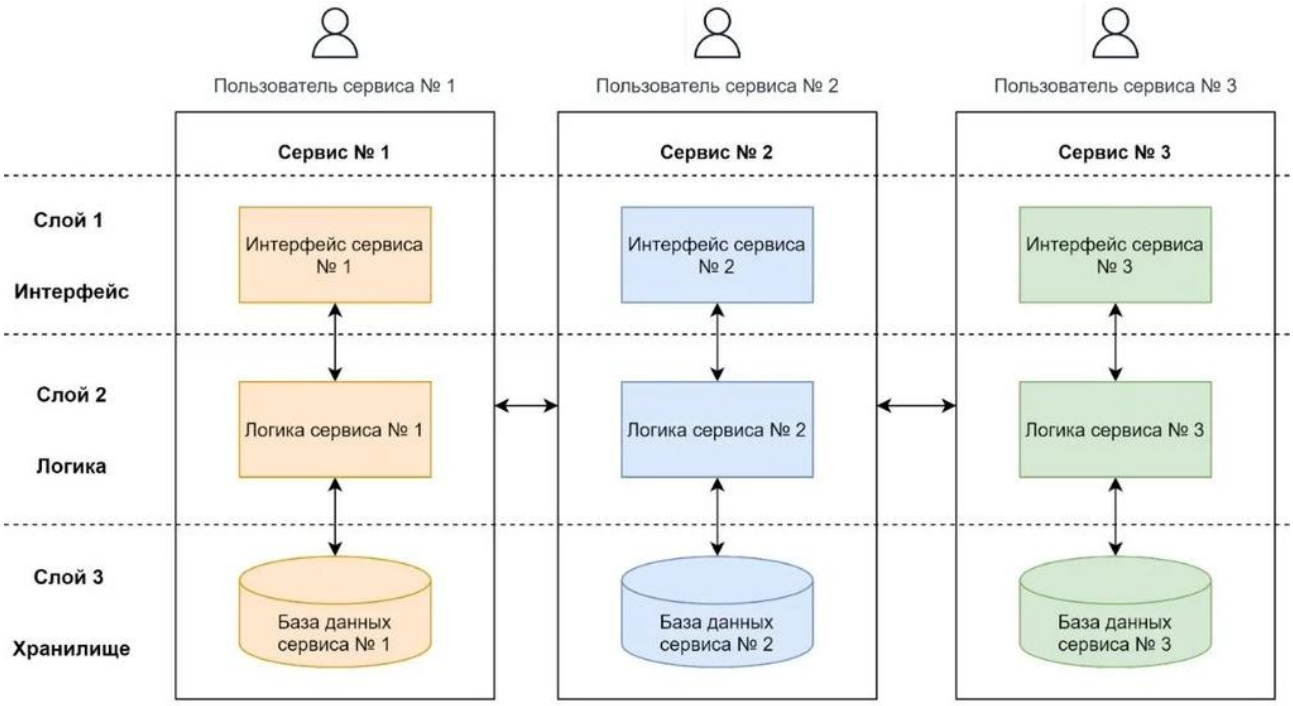
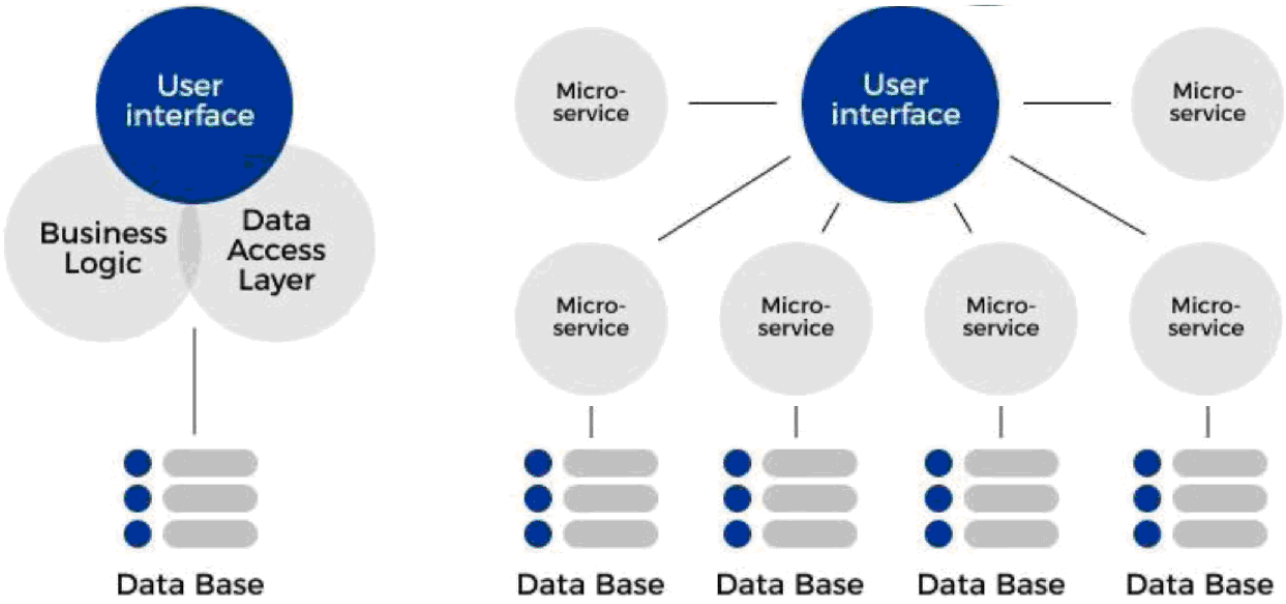


Рисунок 2.8 – Упрощенное представление SOA-архитектуры

Однако в связи с тем, что все нововведения осуществляются через доработку приложения, разработчики сталкиваются с проблемой сильной связанности: первичные связи между данными могут использоваться несколькими сервисами, которые могут использовать одни и те же структуры БД, и непоправимые изменения могут «сломать» бизнес-логику. Также могут возникать ситуации, когда появляется нерегулируемая нагрузка, справиться с которой позволит только очень сильное масштабирование. Эффективное решение таких проблем возможно на основе микросервисной архитектуры.

2.4. Микросервисная архитектура

Микросервисная архитектура (Microservice Architecture, рус. МСА) – это более современная архитектура. Принципиальное отличие от рассмотренных выше так называемых монолитных архитектур показано на рис. 2.9. Так, монолит предполагает «сращивание» трех базовых слоев. Ярким примером такого монолита является многослойная архитектура (рис. 2.1).



а б

Рисунок 2.9 – Принципиальное отличие архитектур ИС:
а – монолитная; б – микросервисная

Аналогично устроены многоуровневая и сервис-ориентированная архитектуры с разницей в концепции и способах разделения функциональности. В монолите бизнес-логика взаимодействует с интерфейсом и хранилищем данных напрямую.

Микросервисная архитектура предполагает разведение отдельных атомарных функций по отдельным микросервисам. Каждый из них обосабливается в отдельном контейнере со своей базой данных и там автономно функционирует, взаимодействуя с другими микросервисами через API (Application Programming Interface). Также имеется пользовательский интерфейс, который позволяет взаимодействовать со всеми микросервисами. Подробная схема устройства МСА показана на рис. 2.10.

МСА доводит SOA-архитектуру практически до совершенства, декомпозируя функционал ИС на отдельные микросервисы. При этом каждая отдельная функция ИС реализуется в виде отдельного сервиса. Тем самым реализуется что-то вроде принципа единственной ответственности для класса, когда класс реализует только одну функцию, и это правильно. Также здесь выполняется правило единичности функционального требования к ИС. Подобно тому, как класс обеспечивает инкапсуляцию данных и обрабатывающих их функций, функции ИС так же упаковываются в контейнере, и образуют микросервис. Микросервисы взаимодействуют между собой подобно объектам, которые обмениваются сообщениями.

На техническом уровне микросервисы взаимодействуют через API. Как правило, такое взаимодействие не является интенсивным. Если требуется интенсивный инфообмен между микросервисами, то лучше отказаться от МСА в пользу SOA или многоуровневой архитектуры.

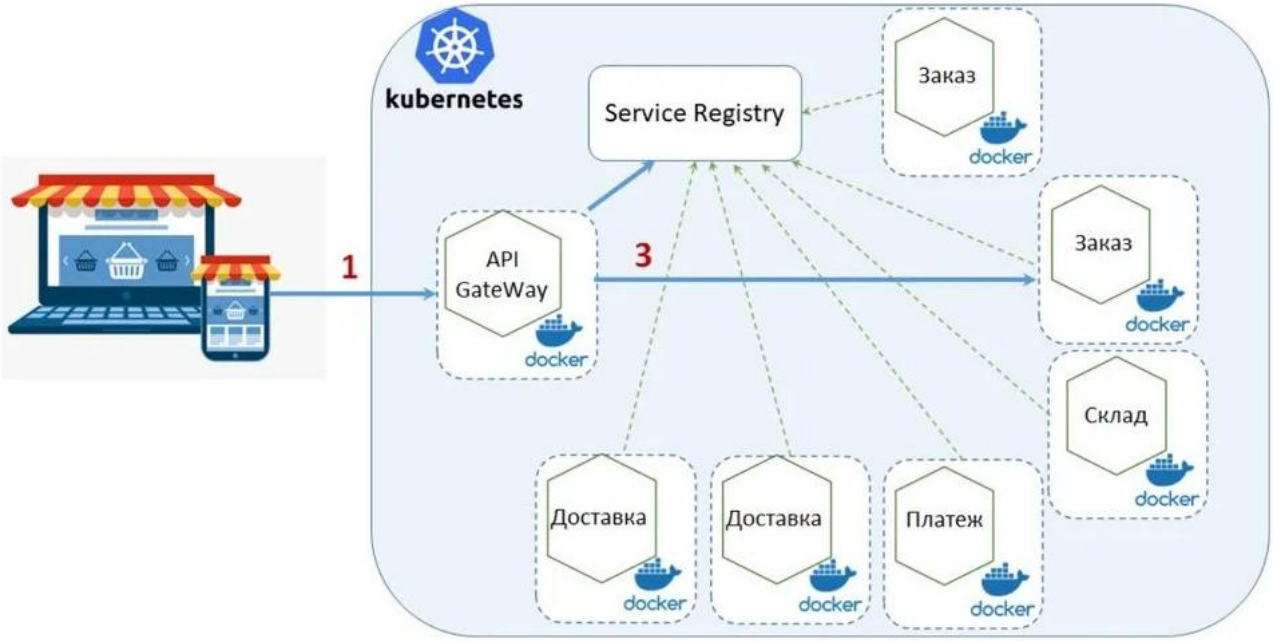


Рисунок 2.10 – Схема контейнеризации в МСА

Элементами МСА являются:

– сервисы (Services) со своими данными и другими особенностями, которые в данном случае называются микросервисами, потому что они более-менее атомарны;

– сервисная шина (Service Bus), которая в контексте МСА обеспечивает взаимодействие между микросервисами, а также между микросервисами и внешними системами, выполняет функции маршрутизации, преобразования данных, оркестрации сервисов и обеспечения надежного обмена сообщениями;

– внешняя конфигурация (External Configuration), которая фиксирует архитектурные особенности МСА – настройки, которые управляют поведением микросервисов и хранятся отдельно (в БД, файловой системе, системе управления конфигурацией или облачном сервисе) и загружаются во время выполнения, это позволяет изменять поведение сервисов без перекомпиляции и переразвертывания, что особенно важно в МСА, где сервисы часто меняются и развертываются в разных средах;

– шлюз API (API Gateway), который обеспечивает единую точку входа в ИС и взаимодействие клиентов с соответствующими микросервисами, выполняя такие функции, как маршрутизация, аутентификация, авторизация, ограничение скорости, агрегация и преобразование запросов и ответов;

– контейнеры (Containers), которые инкапсулируют микросервисы со всеми их функциями данных, обеспечивая независимость, масштабируемость и упрощенное развертывание каждого сервиса. На рис. 2.10 показаны микросервисы, упакованные в контейнеры Docker, в том числе и шлюз API. Оркестрацию осуществляет Kubernetes. Более подробно об этих процессах можно прочесть в подразделах 3.6 и 3.7.

2.5. Ключевые свойства микросервисов

Микросервисы – это независимо выпускаемые сервисы, которые моделируются вокруг предметной области бизнеса. Они характеризуются четкими и стабильными границами, которые не меняются при изменении внутренней реализации. Такой подход приводит к созданию систем с более слабой *связанностью* (*coupling*) и более сильной *связностью* (*cohesion*).

Далее представлены основные свойства микросервисов.

Независимое развертывание – это концепция, которая позволяет вносить изменения в один микросервис, разворачивать его и предоставлять эти изменения пользователям без необходимости изменять другие микросервисы. Это гарантирует высокий уровень качества и низкие риски внесения ошибок в другие части системы. Подход к независимому развертыванию должен быть обязательной практикой. Это один из принципов DevOps – непрерывное и в данном случае независимое, то есть еще более гибкое, развертывание. Все это повышает управляемость процессом развертывания, включая отсутствие необходимости остановки системы при развертывании отдельного микросервиса (непрерывность по DevOps). Вместе с тем реализация этой концепции довольно сложна, требуется достаточная квалификация разработчиков (высокий порог вхождения).

Слабая связанность. Для возможности независимого развертывания необходимо обеспечить слабую связанность между микросервисами – то есть иметь возможность изменять один сервис, не затрагивая другие. *Между сервисами должны существовать явные, четко определенные и стабильные контракты.* Контракты – это ответственности микросервисов, проявляющиеся в их функционале.

Слабая связанность позволяет очертить явные границы между микросервисами. Вместе с тем некоторые подходы, например совместное использование баз данных, могут затруднить достижение этой цели. Если вдруг требуется совместное использование БД, относящихся к разным микросервисам, то тогда лучше не прибегать к этой архитектуре, а ограничиться SOA либо другими монолитными архитектурами.

Моделирование вокруг предметной области. Это крайне важный аспект использования микросервисов. Принцип моделирования – один из пяти принципов кибернетики, положенных в основу проектирования ИС. Использование технологий проектирования помогает структурировать код так, чтобы он лучше отражал предметную область. К примеру, технология RUP (см. подраздел 1.10) позволяет построить комплексную модель ИС на языке UML и затем сгенерировать шаблонный код.

В случае использования MCA микросервисы разрабатываются как сквозные срезы бизнес-функциональности предметной области. При этом следует отдавать приоритет сильной связности бизнес-функциональности, а не технической функциональности. К тому же здесь есть возможность конструировать отдельные микросервисы, загружать их в библиотеки, экспортировать в другие проекты, равно как и импортировать из других

проектов. И в этом плане микросервисы тоже удобны, потому что спроектировать их по частям гораздо проще, чем проектировать всю систему целиком.

Изменения чрез моделирование также дает возможность быстро адаптировать функциональность к изменяющимся реалиям предметной области. МСА гарантирует, что внесенные изменения будут максимально эффективными.

Использование МСА диктует требования к командообразованию. Здесь характерен отказ от горизонтальной многоуровневой архитектуры и горизонтального формирования команд. Приоритет отдается формированию вертикальных команд по области внесения изменений в отдельные микросервисы. Можно создать вертикально фрагментированную команду небольшой численности, которая прекрасно справляется с небольшим подпроектом отдельного микросервиса, работая, например, по методике Scrum. Отдельная команда должна нести полную сквозную ответственность за выделенную бизнес-функциональность – от пользовательского интерфейса и бизнес-логики до записи в БД и отправки данных в Интернет.

Границы ответственности. Проводятся главным образом по хранимым и перерабатываемым данным. Общих БД следует максимально избегать и конструировать нужные данные вместе с функциями в микросервисах. Если микросервису требуется доступ к данным, хранящимся в другом микросервисе, он должен их запросить у соответствующего сервиса посредством API. Но это взаимодействие должно носить скорее эпизодический характер, чем систематический. Выше уже отмечалась такая особенность МСА.

Такой подход позволяет микросервисам реализовать эффективную инкапсуляцию данных и четко разграничить функциональность на такую:

а) которая может быть свободно изменена (внутренняя относительно независимая реализация);

б) которую необходимо редко изменять (внешний контракт, используемый потребителями информации).

Микросервисы должны рассматриваться как сквозные срезы бизнес-функциональности, которые инкапсулируют пользовательский интерфейс (UI), бизнес-логику и данные. Таким образом можно избежать ненужной сложности кода. Так как микросервисы относительно простые, трудоемкость изменений бизнес-функциональности минимальна.

Инкапсуляция данных и подобное поведение обеспечивают сильную связность бизнес-функций. Сокрытие же поддерживающей базы данных способствует снижению технической связанности между сервисами. Все это повышает надежность и безопасность ИС.

Размер. Цель микросервисов – получить как можно меньший интерфейс взаимодействия между самими микросервисами. Поскольку микросервис должен быть максимально доступен и прост для изменения, он должен быть такого размера, при котором его можно легко понять. Идеальный размер микросервиса позволяет одной команде разработчиков эффективно им управлять и обслуживать. При этом команда не должна владеть больше чем

одним-двумя сервисами. Написать огромный монолит таким составом крайне проблематично, поэтому размер команды неявно определяет размер микросервиса.

С ростом размера микросервиса возникает проблема неодинаковости его понимания членами команды. Здесь в первую очередь интересен взгляд со стороны разработчика. Поскольку довольно часто применяется разделение труда, то отдельные микросервисы разрабатывают отдельные разработчики. При этом у них может быть разная точка зрения на какой-то определенный микросервис. Например, микросервис, отвечающий за миграцию данных, разработчиком этого микросервис так и понимается – как мост. А разработчик связанного микросервиса, из которого данные извлекаются, может понимать этот микросервис как потребитель данных. Это порождает проблему непонимания. Чтобы ее нивелировать, разработчикам надо взаимодействовать.

Тем не менее, если члены команды все больше и больше понимают, что не осознают сущности и размеров сервиса, который они развивают, это также является той гранью, когда сервис достиг своего максимума и его необходимо декомпозировать.

Последнее, в свою очередь, порождает проблему выбора оптимального размера микросервиса. Оптимальный размер микросервиса – это размер, при котором он достаточно мал для независимой разработки, развертывания и масштабирования, но достаточно велик, чтобы избежать чрезмерной сложности управления множеством мелких сервисов. На рис. 2.11 видно, как продуктивность работы монолита начинает падать с ростом его сложности. Для микросервисов изначальная продуктивность ниже, однако по мере роста сложности деградация эффективности для них не так заметна.



Рисунок 2.11 – График соотношения продуктивности приложений, реализованных в разных архитектурах

Поэтому проблема выбора оптимального размера микросервиса – это более серьезная проблема. Тут, собственно, надо балансировать сложность, как показано на этом графике. Размер не должен быть сильно большим, чтобы обеспечить понимаемость и не нарушить принцип слабой связанности, потому что если заложить туда много функций, повышается вероятность того, что ему придется обмениваться данными с другими микросервисами, что нежелательно. С другой стороны, если размер будет очень мелкий, примитивный, то это лишняя трудоемкость по его созданию и поддержке. Тогда неизбежно возникает проблема роста числа микросервисов, что требует решения проблемы управления, оркестрации. Поэтому важно выполнять оптимизацию и найти баланс по размеру и сложности микросервисов.

Гибкость микросервиса трактуется в различных аспектах: организационном, техническом, масштабируемости и надежности. Все эти аспекты уже рассмотрены выше. Возможность внесения изменений в отдельные части ИС (микросервисы) без необходимости переписывать или переделывать всю систему целиком – наиболее выраженное свойство МСА. Это позволяет командам разработчиков быстрее адаптироваться к изменяющимся требованиям, вносить улучшения и добавлять новые функции, не затрагивая другие части приложения.

Важно отметить, что МСА имеет высокую стоимость, поэтому экономическая эффективность может достигаться только за счет комплексного эффекта от других ее преимуществ. В случае необходимости жесткой экономии, оптимизации преимущественно по экономическим показателям от МСА следует отказаться. В остальных случаях важно найти баланс между сохранением открытых возможностей и затратами на такую архитектуру. Это нелегко и может стать настоящим искусством.

Технологическая неоднородность. Часто бывает так, что ИС содержит множество микросервисов, каждый из которых разрабатывается на своем языке программирования с помощью разных технологий. В системе, состоящей из нескольких взаимодействующих микросервисов, могут быть использованы разные технологии для каждого отдельного микросервиса.

Технологическая неоднородность (heterogeneity) – это понятие, отражающее различия в свойствах или характеристиках чего-либо, в данном случае в технологическом процессе или объекте.

К примеру, на платформе .NET может быть разработано приложение, в котором разные классы могут быть написаны на разных языках. В МСА это доводится практически до совершенства. Каждый микросервис может разрабатываться при помощи своего собственного стека технологий, а для повышения производительности части системы можно использовать другой, более подходящий, технологический стек. Все определяется необходимостью, особенностями решаемой задачи.

Надежность. Это одно из преимуществ микросервисов. Главная идея для повышения надежности ИС с МСА – это принцип переборки (bulkhead). Границы сервисов становятся потенциальными переборками. Если в системе обнаруживается компонент, который вышел из строя, его можно изолировать,

чтобы остальная часть системы продолжала функционировать. Соответственно, потеря одного компонента на работоспособности ИС существенно не отразится, просто немного просядет эффективность системы, какая-то функция откажет, но система в целом продолжит работать.

Таким образом, имеет место предотвращение полного отказа определенных сервисов за счет снижения общей производительности системы. В монолитной архитектуре сбой порой даже одного сервиса может вывести из строя систему целиком, особенно если это какой-то центральный сервис, сервис хранения данных. МСА не имеет общего хранилища, поэтому такой вариант невозможен.

Масштабирование отдельных сервисов может проходить независимо друг от друга, что упрощает обработку высоких нагрузок и обеспечивает эффективное потребление ресурсов. Также это позволяет запускать другие части системы на менее мощном аппаратном обеспечении. С помощью микросервисов можно масштабировать только ту часть программы, для которой это необходимо, остальное можно не трогать, и тогда там не будет ошибок. Ошибки при масштабировании возможны на стыках компонентов – это проблема интеграции, которая решается посредством интеграционного тестирования. В условиях МСА такие риски минимальны. Поскольку микросервисы относительно легки, то на них можно не тратить ресурсы.

В случае монолитной архитектуры приходится масштабировать все целиком. Это всегда трудоемко и несет высокие риски внесения ошибок в другие части системы.

Масштабирование благоприятно влияет на производительность. Из-за слабой связи производительность правильно масштабируемой ИС всегда выше, чем немасштабированной ИС.

Гибкая масштабируемость достигается путем применения соответствующих технологий, а именно – контейнеризации, например, в docker-контейнер. Соответствующие решения схематично показаны на рис. 2.10. При использовании данной технологии микросервисы упаковываются в docker-контейнеры, которыми управляет специальная служба (например, Kubernetes или Consul).

Полнота развертывания. Микросервисы позволяют внести изменения в один сервис и развернуть его независимо от остальной части системы. Если проблема все же возникает, ее можно за короткое время изолировать, откатить систему к предыдущему состоянию. Такие возможности помогают быстрее внедрить новые функции в действующее приложение. Это нас возвращает к методологии DevOps (см. раздел 1), где реализуется непрерывность развертывания. Реализация МСА посредством DevOps – распространенная практика. Схематично полный цикл реализации DevOps показан на рис. 2.12.

Непрерывная интеграция состоит из следующих этапов:

1. Планирование и проектирование. На этом этапе осуществляется аналитика и разбиение функционала на микросервисы.

2. Разработка. На этом этапе осуществляется создание микросервисов. Разработанные сервисы попадают в систему управления версиями, например GitHub.

3. Сборка. На данном этапе с помощью специального сервиса, например Travis CI, осуществляется сборка.

4. Тестирование. На данном этапе также с помощью сервиса сборки осуществляется первичное тестирование и проверка различных зависимостей в исходном коде.

Непрерывная поставка состоит из следующих этапов:

1. Выпуск. На данном этапе образы сервисов упаковываются в контейнеры и размещаются в репозитории продукта.

2. Установка. Репозиторий, например Docker Hub, на данном этапе осуществляет перенос сервисов на стенды.

3. Операционная работа. На данном этапе осуществляется UI-тестирование, развертывание системы в staging-окружении и production-окружении.

Согласованность рабочих процессов в организации при использовании МСА тесно связана с DevOps-практиками, такими как автоматизация развертывания, тестирование и мониторинг. Микросервисы помогают достичь оптимального соотношения размера команды и эффективности коллективной разработки. С учетом того, что отдельные микросервисы создаются разными специалистами с использованием разных технологий, можно переопределять ответственность за сервисы по мере изменения организации коллективной разработки. Так, если работник вышел из строя, то можно относительно безболезненно его заменить, потому что микросервис небольшой и объем работы у другого работника будет небольшой. В этом случае можно немного реструктурировать его нагрузку.

В целом согласованность рабочих процессов в микросервисной архитектуре – это сложная задача, требующая тщательного планирования, выбора подходящих инструментов и стратегий, а также постоянного мониторинга и совершенствования.

Повторное использование функциональности микросервисов для разных целей обеспечивает переносимость и кроссплатформенность приложений на основе МСА. Подобно библиотечным классам, микросервисы также можно использовать повторно. Каждый микросервис упаковывается в отдельный контейнер, что обеспечивает высокую переносимость и рассмотренную выше полноту развертывания по DevOps.

Это особенно важно при рассмотрении использования системы со стороны пользователей. Ведь именно пользователь является основным генератором идей видоизменения и повторного использования готовой функциональности. МСА позволяет это делать легко и просто. Это напрямую повышает экономическую эффективность проектов и снижает затраты на разработку. Также это позволяет сократить время разработки (CI-подцикл не требуется), улучшить качество кода и упростить поддержку. Кроме того, правильная масштабируемость повышает удобство использования (usability).

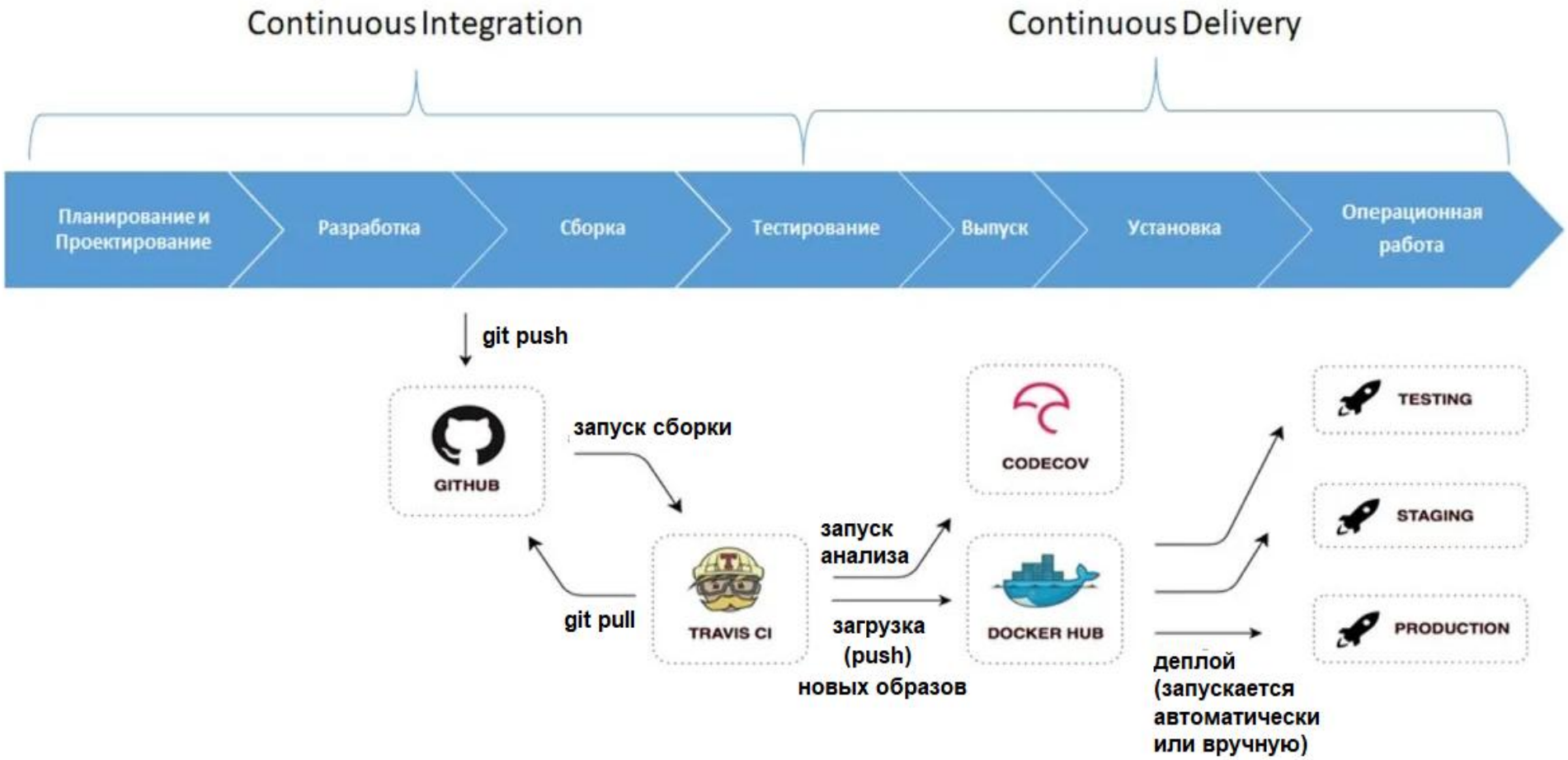


Рисунок 2.12 – Полный цикл реализации МСА посредством DevOps

Технологическая нагрузка при реализации МСА связана с увеличением сложности управления инфраструктурой, необходимостью использования различных инструментов для разработки, развертывания и мониторинга, а также с повышенными требованиями к коммуникации между сервисами и сложностью тестирования и отладки. Архитектура микросервисов поддерживает кодирование каждого микросервиса на отдельном языке программирования, выполнение его в своей среде или использование отдельной БД. Важен поиск баланса масштабности и сложности используемой технологии с издержками, которые она может повлечь. Важно внедрять новые сервисы МСА по мере необходимости. Это освежает архитектурное решение системы, а также позволяет не перегружаться сложностью инструментов, можно получить больше вариантов решения задач за счет множественности микросервисов. Нет нужды в кластере Kubernetes, когда в системе всего три сервиса. По крайней мере на первых порах архитектура может быть простой. Потом, по мере разработки системы, могут присоединяться новые микросервисы. Очень легко таким образом по DevOps, итерациями, наращивать функциональность, добавляя новые микросервисы и дорабатывая существующие.

Наглядно технологическая нагрузка может быть представлена на примере контейнеризации. На рис. 2.13 представлен микросервис, упакованный в контейнер вместе с необходимыми технологиями, структурированными по отдельным слоям.



Рисунок 2.13 – Слои микросервиса в контейнерном образе

Контейнеризация – это современная технология, которая обеспечивает абстракцию всех операционных систем (ОС) и внешней коммуникации не такой высокой ценой, как в случае аппаратной виртуализации. Она позволяет запускать ПО в изолированных на уровне ОС пространствах.

Контейнеры (контейнерные образы) являются наиболее распространенной формой виртуализации на уровне ОС. Компоненты ПО в контейнерном образе укладываются в слои, что повышает эффективность физического развертывания микросервиса. При обновлении верхних слоев можно повторно использовать кэшированные нижние слои. Таким образом, двоичные файлы из кэшированных слоев не перемещаются, и в других предварительных действиях тоже нет необходимости. Задача разработчиков – конфигурировать все эти компоненты так, чтобы они взаимодействовали друг с другом, исправно работали в ОС сервера виртуальных машин и осуществляли коммуникацию с внешними системами. При необходимости могут быть использованы средства оркестрации, такие как Kubernetes и Marathon.

Стоимость. Микросервисы могут принести больше прибыли, если использовать их:

– для привлечения новых клиентов, благодаря высокой переносимости, надежности и полноты развертывания микросервисов и пр.;

– для наработки авторитета, репутации на рынке программных систем, в силу тех же причин;

– для параллельной разработки дополнительных функциональных возможностей.

При разработке сложных систем МСА экономически более оправдана, чем монолит. Правильное масштабирование снижает стоимость обслуживания микросервисов. Однако это справедливо, если не ставится задача обеспечить приоритетную экономическую эффективность, о чем уже упоминалось ранее.

2.6. Недостатки микросервисов

Большинство проблем, связанных с микросервисами, можно отнести к распределенным системам. Поэтому они с такой же вероятностью проявятся как в распределенном монолите, так и в микросервисной архитектуре.

Коллективная разработка. Наиболее чувствительным недостатком МСА являются ограничения, накладываемые на коллективную разработку. Чем больше в ИС микросервисов, тем сложнее запускать их на одной машине для отладки и тестирования. Это требует распределения работы в команде и также приводит к увеличению ее размера. Кроме того, с ростом числа микросервисов становится сложнее разобраться в межсервисном взаимодействии. В обоих случаях наилучшим решением здесь будет ограничить объем фрагментов, над которыми будет трудиться отдельный разработчик.

При разработке ИС с МСА критически важна роль DevOps-инженера, который берет на себя ответственность за автоматизацию и оптимизацию процессов разработки, тестирования и развертывания ИС.

Кроме того, даже в хорошо отлаженной и оптимизированной командной работе при использовании МСА возникают проблемы с реализацией модели коллективного владения кодом. Такая модель практикуется в технологии командной разработки. Это она из практик Agile – гибкой разработки ПО. Она подразумевает, что любой разработчик может работать над любой частью системы. Для микросервисной архитектуры это неприемлемо, поскольку здесь свои автономные микросервисы, которые предполагают авторскую разработку либо парное программирование (одна из практик Agile). Для разработки требуется рабочая подгруппа, организованная по модели группы равных. Принцип же коллективного владения кодом в чистом виде в МСА неприемлем и противоречит самой идее МСА.

Таким образом, наиболее приемлемые технологии коллективной разработки ИС с МСА – Scrum of Scrum с гибридной моделью команды разработки с центральной моделью главного программиста (в которую входит DevOps-инженер) и множеством иерархически с ней связанных моделей группы равных (рис. 2.14), работающих над одним или несколькими микросервисами, объединенными по функциональному признаку.

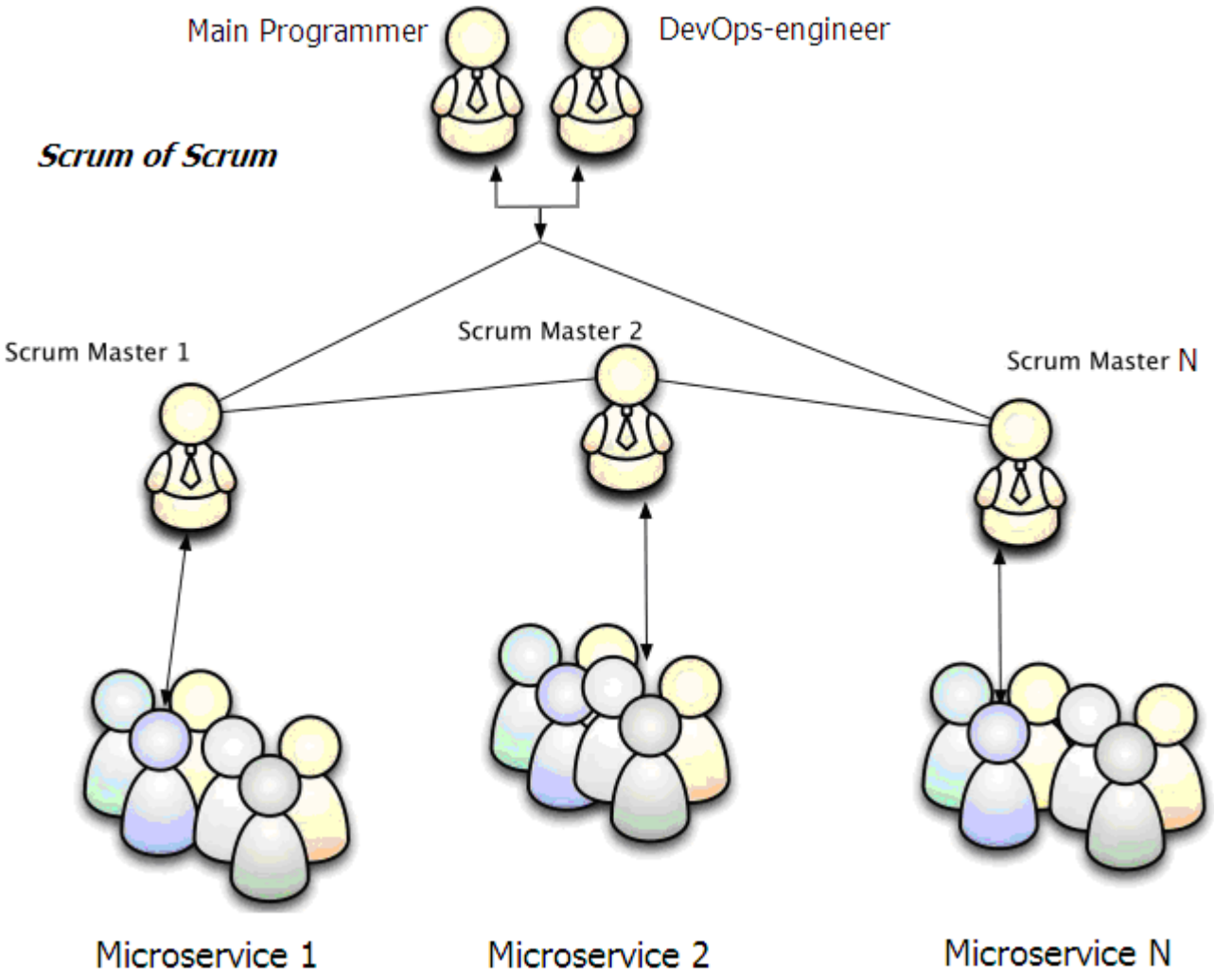


Рисунок 2.14 – Гибридная модель команды разработки ИС с микросервисной архитектурой

Стоимость. Микросервисная архитектура может быть дорогой, особенно на начальном этапе. Необходимо тщательно проанализировать требования к проекту, оценить риски и затраты, а также учитывать возможность масштабирования и развития системы в будущем. Микросервисы – плохой выбор для организации, ориентированной на снижение расходов в ИТ. Когда сфера ИТ рассматривается как центр расходов, а не доходов, максимальную отдачу от микросервисной архитектуры получить не удастся. Переход на микросервисы для снижения затрат на ИТ маловероятен. В этом случае лучше перейти на SOA или многоуровневую архитектуру. Там все можно оптимизировать до предела, особенно если обратиться к многослойной архитектуре, где все предельно четко, понятно и минималистично. Это будет в ущерб надежности, масштабируемости и безопасности, но зато дешевле.

Важно отметить, что с точки зрения экономической целесообразности МСА становится выгодной лишь в перспективе поступательного развития и усложнения системы. Так, на рис. 2.15 показано примерное отношение стоимости кода по мере развития проекта: чем больше кода придется дописать в монолите, тем дороже окажется решение. В случае же доработки микросервисов (при прочих благоприятных условиях) имеет место экспоненциальное удешевление разработки и сопровождения ИС.

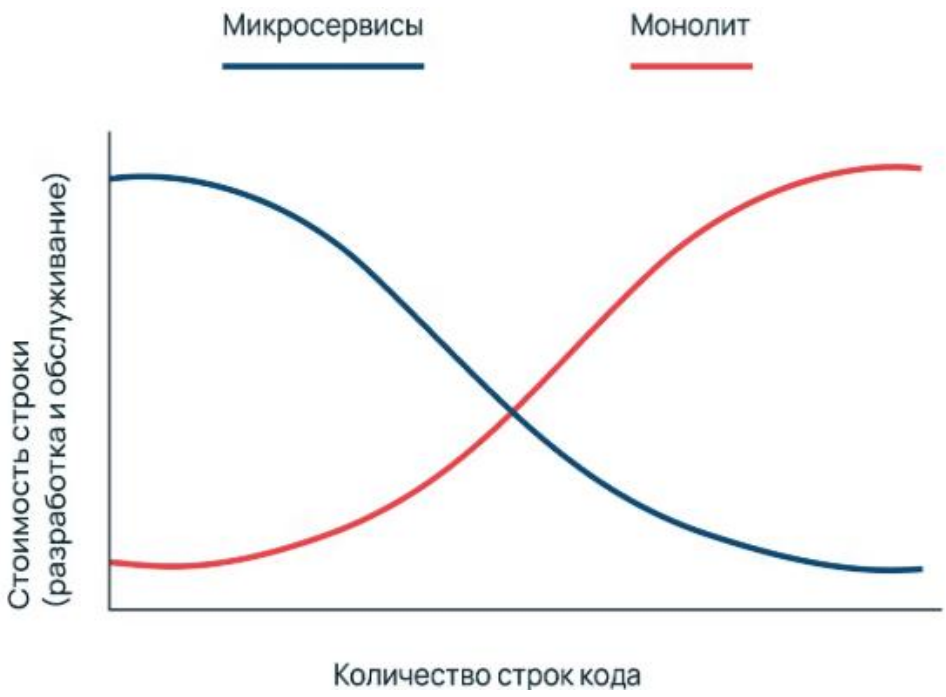


Рисунок 2.15 – Соотношение стоимостей разработки ИС с монолитной и микросервисной архитектурой

Также важно, что по ходу проекта с ростом его функциональности стоимость поддержки ИС с модульной архитектурой (многослойной и многоуровневой) растет по экспоненте. При использовании же прогрессивных архитектур на основе сервисов с ростом функциональности и при масштабировании наблюдается линейный рост стоимости (рис. 2.16).

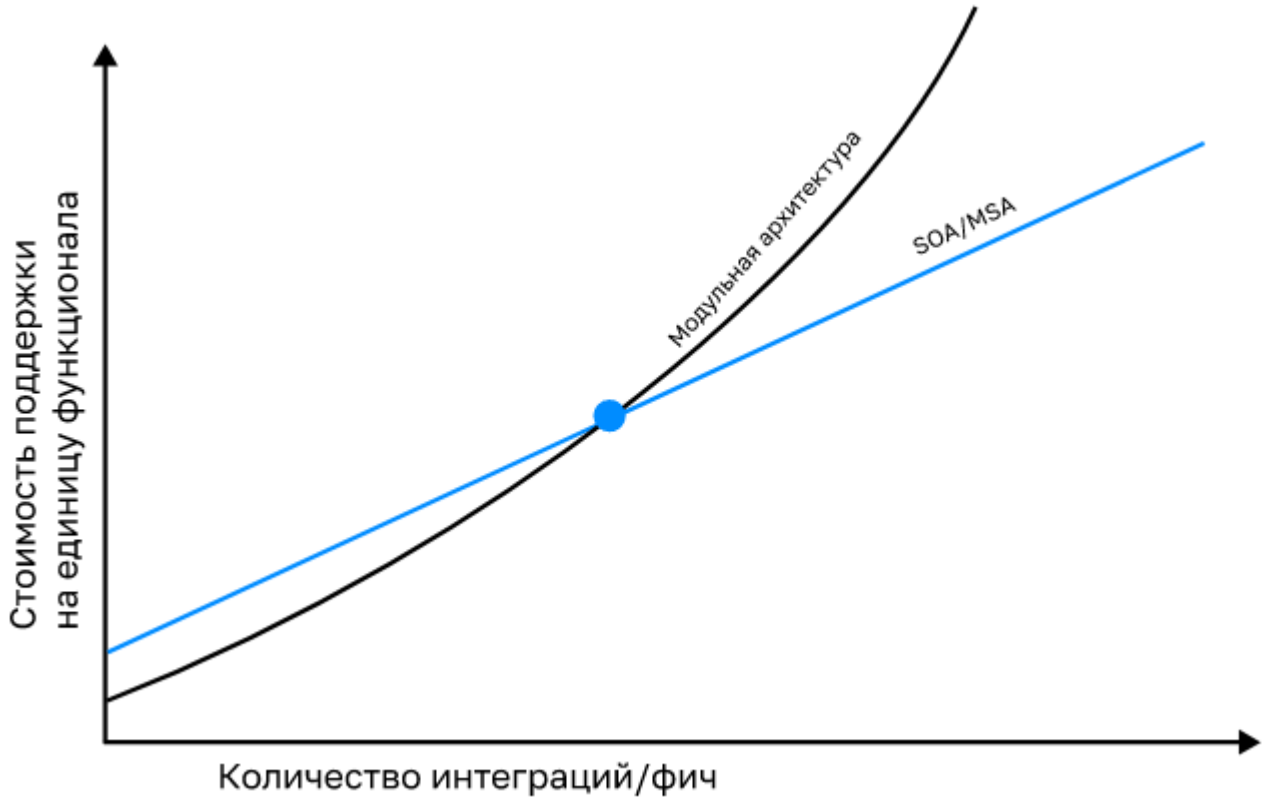


Рисунок 2.16 – Соотношение стоимостей поддержки ИС с модульной и сервисной архитектурой

Отчетность. С отказом от использования общих БД значительно усложняется выполнение запросов к БД и формирование отчетности. Если в системе надо сформировать какие-либо глобальные, комплексные отчеты, то МСА не подходит, поскольку приходится создавать интенсивные потоки между микросервисами. В любой организации бывают критические моменты, когда идет глобальный (например, квартальный) отчет или проводится какая-то проверка и между подразделениями протекают очень интенсивные информационные потоки. Микросервисная архитектура может просто отказать из-за такого инфообмена. Либо в самый критический момент существенно просядет эффективность ИС. Это недопустимо. Поэтому если такие задачи стоят, лучше отказаться от микросервисной архитектуры.

В качестве альтернативы можно публиковать нужные для отчетности данные из микросервисов в отдельных репозиториях, централизованных хранилищах. Однако это также создает дополнительные инфопотоки и нагрузку на ESB. Лучше этого избегать. Более современным подходом к построению отчетности является потоковая передача данных в режиме реального времени.

Мониторинг и устранение неполадок. В микросервисной архитектуре затруднительно диагностировать неполадки. Любая система должна мониториться для обнаружения некорректных ситуаций. Любые нештатные ситуации и инциденты информационной безопасности должны предупреждаться, а не упреждаться и не устраняться в условиях аварий. В МСА это затруднительно делать, потому что много микросервисов, которые

приходится диагностировать по отдельности. Это непрерывный процесс, который должен быть интегрирован в жизненный цикл каждого микросервиса. При этом получение достоверной характеристики работоспособности ИС с МСА оказывается также весьма затруднительным.

К примеру, в монолите, если процессор находится под нагрузкой и долго работает на 100 %, это является явным признаком серьезной проблемы в системе. Утверждать то же самое о микросервисной архитектуре с десятками и сотнями процессов однозначно нельзя. К тому же если в монолите диагностику несложно провести для системы в целом, то в МСА так не получится сделать ввиду слабой связанности микросервисов, что затрудняет диагностику либо делает ее невозможной. Для того, чтобы проводить диагностику системы целиком, важно, чтобы компоненты системы обладали сильной связанностью. Это хорошо реализовано в модульных архитектурах. Несколько меньше это реализовано в SOA, так как загружает каналы передачи данных потоками диагностической информации. Поэтому для диагностики, для повышения эффективности диагностики лучше всего подходят монолитные архитектуры, преимущественно модульного типа. Таким образом, если имеется какая-то критическая система реального времени, где надо диагностировать все узлы системы постоянно, измеряя сотни параметров за доли секунды, то МСА никак не подходит, несмотря на широту функционала, это должен быть модульный монолит.

Для полноценного же мониторинга микросервисов необходимы специализированные инструменты, такие как Zabbix, Prometheus, PLG, ElasticSearch, Fluent Bit и др., для сбора данных по установленным метрикам, а также локального и удаленного логирования. Однако это потребует существенных вычислительных и финансовых затрат. Читатель без особого труда может ознакомиться со структурами перечисленных систем и убедиться, что все они по сложности не уступают сложной структуре ИС с микросервисной архитектурой. Рисунок 2.11 дает вполне адекватное представление о трудоемкости процесса мониторинга в монолитной и микросервисной архитектурах.

Безопасность. В микросервисных архитектурах большая часть данных передается по сети между сервисами. Это может привести к риску утечки данных в процессе передачи, а также открыть возможности для кибератак, таких как «человек посередине». Так, хакер может перехватить данные между двумя API микросервисов, и никто даже не узнает об этом, потому что микросервисы взаимодействуют автономно. По этой причине не должно быть интенсивных потоков между микросервисами. Тогда риски информационной безопасности будут минимальными.

Также важно уделить большее внимание защите передаваемых данных, чтобы только авторизованные пользователи имели доступ к ним. Возможно, потребуется принять дополнительные меры для обеспечения безопасности, чтобы предотвратить утечки и несанкционированный доступ к данным. Опять же, важно не нагружать микросервисы обработкой интенсивных потоков данных и не нарушать принципы слабой связанности.

Таким образом, основные проблемы связаны с распределенной природой системы, сложностью управления доступом и повышенными требованиями к мониторингу и логированию (см. предыдущий пункт). Функции безопасности обычно встраиваются в монолит или в отдельную службу аутентификации. В микросервисах эта функциональность реализуется на уровне инфраструктуры, чтобы создать единую точку аутентификации для каждого микросервиса. Благодаря этому система безопасности становится более гибкой, а приложение в целом – более устойчивым к DDoS-атакам. Однако для реализации такого решения требуются значительные ресурсы, которые по аналогии с вопросами стоимости и устранения неполадок оправданы при значительной сложности системы с микросервисной архитектурой.

В целом безопасность МСА требует комплексного подхода и тщательного планирования на всех этапах разработки, развертывания и эксплуатации, желательно с применением специальных инструментов, таких как JWT, OAuth 2.0 и mTLS для обеспечения надежной аутентификации и предотвращения утечек, а также систем управления доступом, таких как RBAC и ABAC, для повышения уровня защиты системы. Важным аспектом является обеспечение изоляции сервисов. В случае компрометации одного микросервиса необходимо минимизировать возможность распространения угрозы на другие части системы. Этого можно достичь с помощью строгого ограничения сетевых прав доступа между микросервисами, применения принципов минимальных привилегий и контейнеризации с использованием, например, Kubernetes. Кроме того, необходимо также учитывать риски, связанные с распределенной природой системы, и принимать соответствующие меры для их минимизации.

Тестирование микросервисов так же трудоемко, как и разработка. Монолит тестировать гораздо проще. В микросервисной архитектуре объем сквозных тестов становится весьма значительным. Причем особую роль приобретают интеграционные тесты для тестирования взаимодействия микросервисов. Написание и поддержка интеграционных тестов являются более сложным заданием по сравнению с разработкой модульных тестов. Однако такие тесты максимально приближены к пользовательскому опыту.

Также в МСА требуется запускать тесты в нескольких процессах, а каждый из них должен быть развернут и настроен для тестовых сценариев. Необходимо быть готовыми к ложным срабатываниям, связанным, например, с сетевыми тайм-аутами неудачных развертываний.

Наряду с обычными видами функционального тестирования для МСА нужно проводить некоторые другие виды тестирования, такие как нагрузочное и стрессовое тестирование, тестирование балансировки нагрузки, тестирование репликации данных, тестирование доступности, тестирование безопасности, мутационное тестирование т. д. А в пирамиду тестирования микросервисов добавляются два новых типа: компонентные и контрактные тесты (рис. 2.17). Все это заметно утяжеляет процесс тестирования.

Поскольку для работы модулей микросервисов в МСА часто требуется сетевой вызов, наряду с классическими (одиночными, Solitary) unit-тестами возникает необходимость в использовании общительных (Sociable) unit-тестов.

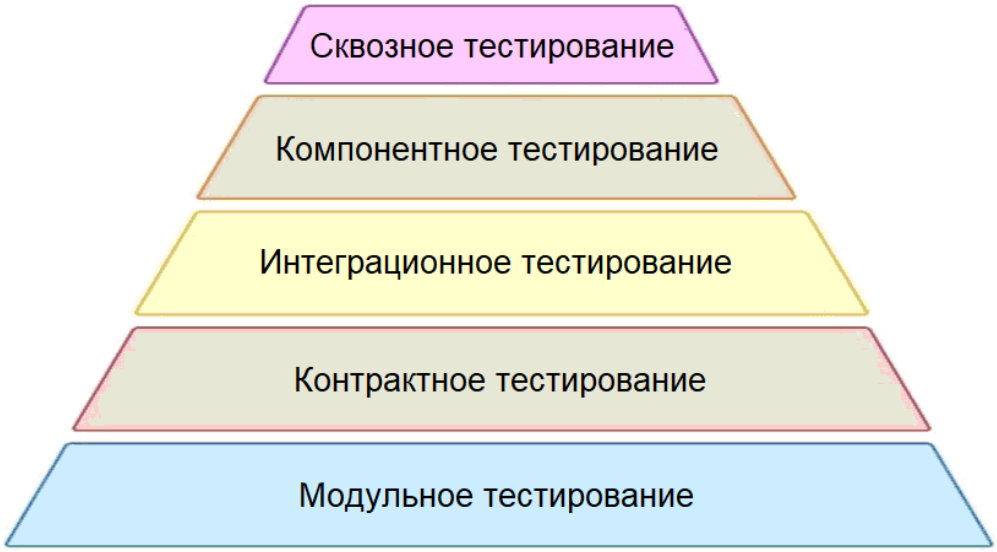


Рисунок 2.17 – Пирамида тестирования микросервисов

Контрактное тестирование микросервисов относится к методу черного ящика. Оно связано с понятием *контракта*, фиксирующего все возможные входы и выходы микросервиса с их структурами данных и возможными побочными эффектами. Контракт формируется, когда два сервиса связываются через интерфейс. Тесты направлены на проверку соблюдения контракта. Контрактные тесты всегда выполняются в процессе непрерывной интеграции по DevOps, чтобы обнаружить несовместимости до развертывания.

Назначение интеграционных тестов в МСА – выявить дефекты интерфейсов, заставив микросервисы взаимодействовать. В отличие от контрактных тестов, интеграционные тесты используют реальные сервисы и реализуются на уровне их интерфейсов.

Компонентные тесты направлены на тестирование компонентов, под которыми в МСА понимаются микросервисы. Часто компонентное тестирование выполняет сквозное тестирование группы микросервисов. Сервисы, не входящие в область действия компонента, имитируются. Существуют разные типы компонентного тестирования. Более продуктивно их выполнять методом белого ящика.

Больше информации о тестировании микросервисов можно найти в источнике [51].

Время ожидания отклика в системе с МСА может быть больше, чем в монолитной, из-за распределенной природы системы и необходимости взаимодействия между сервисами. Вместо того, чтобы передавать информацию только в пределах одного процесса, как в монолите, в МСА необходимо сериализовать данные, передавать их по сети и затем десериализовать на другой стороне. Значительное влияние на время ожидания в микросервисной архитектуре оказывает сложность системы, а также процессы синхронизации данных между сервисами. Оптимизировать этот параметр можно за счет оптимизации сетевых вызовов, протоколов передачи данных и использования кэширования, а также оптимизации кода микросервисов. Мониторинг времени

ожидания в каждом сервисе и трассировка запросов между сервисами позволяют выявить узкие места и принять меры по их устранению.

В целом МСА может приводить к увеличению времени ожидания по сравнению с монолитной, однако при правильном подходе можно значительно сократить задержки и обеспечить высокую производительность системы.

Согласованность данных в микросервисах – это проблема обеспечения целостности и непротиворечивости данных между различными сервисами, каждый из которых может иметь свою базу данных. Актуально для распределенных систем, где разные процессы управляют состоянием в разных БД. При этом могут решаться совершенно разные задачи. Возникают потенциальные проблемы с согласованностью данных. Обычно они устраняются на уровне API микросервисов. API согласовывают перетоки данных, и если они грамотно организованы, то проблем согласованности данных можно избежать.

В монолитной архитектуре обычно полагаются на транзакции в БД посредством SQL-запросов для обеспечения согласованности изменений состояния. В микросервисной же архитектуре использование распределенных транзакций становится проблематичным.

Существуют различные подходы, позволяющие эффективно управлять данными и обеспечивать приемлемую согласованность данных в МСА:

– транзакционная согласованность: использование распределенных транзакций с двухфазным коммитом хотя и обеспечивает строгую согласованность, но может сильно сказаться на производительности и доступности системы;

– использование паттерна «Согласованность на основе событий», при котором изменения в одном микросервисе порождают события, которые обрабатываются другими микросервисами, что позволяет достичь согласованности асинхронно;

– компенсирующие транзакции: в случае ошибок или несогласованности данных применяются операции, отменяющие или корректирующие предыдущие транзакции;

– архитектура, управляемая событиями: микросервисы публикуют и подписываются на события, чтобы уведомлять друг друга об изменениях;

– CQRS (Command Query Responsibility Segregation): разделение операций чтения и записи для оптимизации производительности и повышения согласованности.

Выбор подхода зависит от требований к системе: для систем, требующих строгой согласованности, могут потребоваться более сложные подходы, такие как двухфазный коммит или CQRS. Для систем, где допустима временная несогласованность, может быть достаточно согласованности в конечном итоге.

В целом согласованность данных в МСА – это сложная проблема, требующая тщательного анализа и выбора подходящих подходов в зависимости от конкретных потребностей системы. В особо сложных случаях может быть применено оптимизационное математическое моделирование.

2.7. Контрольные вопросы

1. Что такое архитектура ИС?
2. Какие основные архитектуры ИС используются при их создании?
3. В чем принципиальные отличия между многослойной и многоуровневой архитектурой ИС?
4. Что такое сервис-ориентированная архитектура?
5. Что такое микросервисная архитектура?
6. Дайте определение сервиса в сервис-ориентированной и микросервисной архитектурах.
7. Какие архитектурные решения и в каких случаях применяются при создании ИС?
8. Каковы особенности командообразования при разработке ИС с микросервисной архитектурой?
9. Чем объясняется указанное на рис. 2.15 соотношение стоимостей разработки ИС с монолитной и микросервисной архитектурой?
10. Какие архитектурные решения являются наиболее подходящими при организации DevOps-процесса?
11. Каковы особенности тестирования микросервисов?
12. Какая архитектура ИС является наиболее надежной и безопасной?

3. Технологии коллективной разработки информационных систем

Технология DevOps предполагает, как правило, коллективную разработку информационных систем. Коллективная разработка, в свою очередь, предполагает разделение труда и специализацию ролей в соответствии с определенной моделью. Существует несколько видов моделей разработки, которые определяются моделями коллективов разработчиков. В зависимости от количества участников и типов взаимоотношений между ними это может быть авторская или коллективная разработка, или же общинная разработка как один из частных случаев коллективной разработки. Авторская разработка проще и интереснее других, поскольку автор в одном лице занимается разработкой всей системы целиком, осуществляя при этом все виды деятельности и проходя все фазы жизненного цикла. Таким образом он отрабатывает универсальные навыки реализации программного процесса.

Коллективная разработка тоже интересна. Предполагается, что в ее рамках осуществляется профессиональная коммуникация, работа в команде, выработка мягких навыков, так называемых Soft Skills – в этом ее ценность. С другой стороны, она зачастую сводится к конвейерной работе, где каждый член команды выполняет отдельную возможную для него атомарную функцию, являясь просто «винтиком» в машине производства ПО. Такой способ разработки нивелирует творческую часть программного процесса и превращает производство ПО в рутину. Чтобы этого не произошло, необходимо наладить качественный программный процесс и обеспечить его эффективную организацию.

Современная наука об управлении (управление в организационных системах, кибернетика 2.0) пришла к заключению, что прогрессивное руководство должно быть коллегиальным, а не авторитарным. Это не означает, что руководителя нет вообще. Коллегиальное управление – это не лживая демократия, это очень жесткая власть принципов нравственности, доверия и взаимопомощи. В настоящей команде лидеров не так-то просто сразу понять, кто ее руководитель, из-за того, что он общается и действует в духе равенства и служения. Как же тогда понять, кто является руководителем? Тот, кто в наибольшей степени олицетворяет командные принципы нравственности, доверия и взаимопомощи. Людям обычно нужно, чтобы им формально указали на главного, но в случае производства нематериальной продукции, каковой и является ПО, данная материальная потребность контрпродуктивна. Квалифицированный руководитель не стремится управлять всем, предпочитая разделять эту ответственность с другими. Служение руководителю заключается именно в приверженности нравственным и этическим принципам команды: именно это удовлетворяет его больше всего. Коллегиальное управление важно, поскольку вовлекает каждого в сознательное (творческое) отношение к своим обязанностям, без чего не возникает атмосферы доверия и взаимопомощи, стопорятся DevOps-процессы производства ПО, а также невозможно эффективно руководить сложными и многопрофильными социальными и экономическими системами.

В то же время вовсе необязательно превращать разработку ПО в рутину. Собственно, DevOps-инженерия и призвана упредить эту рутину, максимально ее автоматизируя. А поскольку полностью исключить человеческий фактор невозможно, да и не нужно, разрабатываются технологии коллективной разработки. Существуют определенные конфигурации модели коллективной разработки, которые позволяют осуществлять довольно увлекательную разработку ПО коллективным способом, так называемыми «малыми группами». Однако для того, чтобы наглядно показать преимущества коллективной разработки, рассмотрим сначала более подробно авторскую разработку ПО.

3.1. Модель авторской разработки

Авторская разработка – это создание программ одним человеком.

Эта модель в основном предназначена для простых проектов. Простой проект можно легко сделать в одно лицо. Уважаемый читатель может вспомнить свои лабораторные работы по программированию на первом курсе, где он осуществлял типовую авторскую разработку, реализуя проект единолично и даже в один проход. Это были простые задачи, которые вполне возможно решать единолично.

Данная модель была распространена и преобладала в начале становления программного процесса (1960-1970 гг.). Сейчас она применяется только в простых проектах. Сложные проекты со временем оказались настолько сложны, что разрабатывать их авторским способом нереально. Особенно это касается крупных проектов, где для нормального оперативного решения задач требуется задействовать десятки разработчиков. Для этого существует коллективная методика.

Авторская методика выигрывает за счет трех организационных факторов:

1. Исключение межличностных коммуникаций, связанных с необходимостью погружения в информационное поле программного процесса. Межличностные коммуникации отвлекают внимание от работы, и если неграмотно поставить коммуникации в команде, то это может притормаживать проект. Кроме того, есть такие факторы, как взаимопонимание и совместимость характеров, психологические аспекты, когда люди не сходятся характерами и не понимают друг друга. Справедливости ради следует отметить, что бывает наоборот, когда члены команды хорошо друг друга понимают с полуслова. Это характерно для профессионалов, развитых специалистов, сформированных личностей, которые способны профессионально взаимодействовать с другими себе подобными специалистами. В этом случае коммуникации существенно не влияют на качество работы, а, наоборот, его повышают и стимулируют. Таким образом, при прочих неблагоприятных обстоятельствах коммуникации могут ухудшать программный процесс. Авторская разработка не имеет такого недостатка.

2. Исключение работ по разбиению проекта на составляющие компоненты – так, что автор сам отвечает за все процессы и сам видит всю

концепцию целиком, не разделяя ее на части. Разбиению обычно подвергают крупные проекты, которые дробят на подпроекты. При этом каждый проект курирует свой руководитель группы. Эти работы по разбиению на составляющие надо грамотно провести и организовать это разбиение. Это дополнительные временны́е затраты, весьма ощутимые для простого проекта. Проще взять и самому все разработать, чем разводить командную суету.

3. Координация действий между исполнителями и контроль их работы тоже требуют ресурсов: временны́х, кадровых, материальных. Несложные проекты, как правило, не обеспечены каким-то существенным временем: обычно они разрабатываются за недели, максимум несколько месяцев при таком же скромном прочем ресурсном обеспечении.

Таким образом, имеет место распыление ресурсов на организацию деятельности – коммуникацию, декомпозицию и координацию действий (рис. 3.1), что неизбежно связано с необходимостью порождения и изучения большого количества документации, может существенно утяжелить использование временны́х, финансовых и кадровых ресурсов – трех важнейших ресурсов программного процесса.



Рисунок 3.1 – Три фактора организации программного процесса

Таковы недостатки, характерные для коллективной разработки, однако у нее есть и множество преимуществ, которые рассмотрим далее.

3.2. Модели коллективной разработки программного обеспечения

Коллективная (командная) разработка – это создание программного продукта коллективом разработчиков. Одним из основных аспектов коллективной разработки является разделение труда.

Существуют две основные модели организации коллектива при разработке ПО:

1) иерархическая модель (начальник → подчиненные);

2) модель группы (без определенной структуры коллектива).

Иерархическая модель – это иерархия подчинения. Здесь есть начальник, его заместители и их подчиненные. Такая древовидная структура имеет степенной рост. В итоге крупные компании насчитывают тысячи человек. Обычно это два-три уровня иерархии. Ученые доказали, что начальник может эффективно руководить коллективом, если в подчинение у него находится не более 10-12 человек. Когда численность коллектива достигает 15 человек, начинают проявляться проблемы управления и тогда целесообразно выделять средний уровень подчинения и разбивать структуру на департаменты. Выделенные подразделения возглавляют руководители среднего звена, которые находятся в подчинении главного начальника. У них, в свою очередь, находятся в подчинении все остальные работники, разбитые на группы (рис. 3.2).

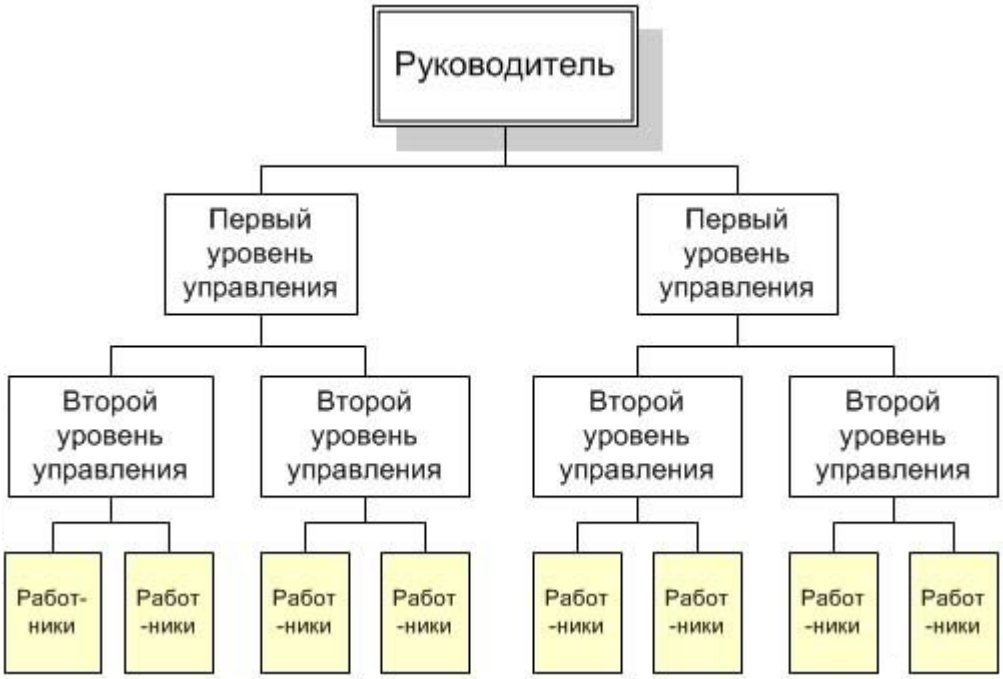


Рисунок 3.2 – Линейная организационная структура управления

Чем больше коллектив, тем масштабнее его иерархическая структура, где очень много ветвей передачи управленческих данных. При этом очевидно, что на нижнем уровне этой иерархия находятся работники, которые почти никогда не общаются с главным начальником. Вся информация передается вверх через цепочку иерархии подчинения. Это значит, что могут возникать проблемы с коммуникацией и координацией.

Малая группа более живучая, потому что у нее нет строгой и иерархической структуры. Малая группа, будучи сформированной на основе качественных высококвалифицированных специалистов, способна самоорганизоваться. То есть малая группа вполне может обладать свойством самоорганизации, самоорганизоваться без какого-либо явно выраженного лидера (в смысле назначенного отделом кадров). Это может быть просто спонтанно возникший лидер, выделяющийся в коллективе своими внутренними

организаторскими качествами, который берет на себя организационные задачи, ибо даже в модели малой группы как-то надо организовать работу группы.

В частности, кто-то должен отвечать за обеспечение ресурсами, коммуницировать с заказчиком и т. д. Тогда малые проекты, которые в принципе можно реализовать авторским способом, эффективнее реализовывать по модели малой группы при условии, что процесс будет обеспечен качественными ресурсами, в первую очередь трудовыми, то есть будет состоять из качественных специалистов. Однако когда мы говорим о качестве, имеются в виду не только профессиональные качества, но в первую очередь качества личностные, характер, потому что именно личностные качества во многом определяют качество программного процесса, которое, в свою очередь, определяет качество конечного продукта.

Основной моделью в наше время является иерархическая структура, поскольку она представляет собой традиционную парадигму разработки и на практике доказала свою живучесть. Насколько качественно она работает – это уже другой вопрос. Если кто-то кому-то не подчиняется, его увольняют и ставят на его место другого специалиста. Эта структура работает. Другое дело, что здесь есть свои недостатки, часть из которых рассмотрена в предыдущем подразделе. Эти недостатки тем актуальнее, чем крупнее проект реализуется в коллективе. Тогда энтропия начинает возрастать по экспоненте: чем больше в группе людей, тем интенсивнее растет энтропия. Чтобы она не росла, большую группу делят на несколько малых и выстраивают иерархию управления, простейшей из которых является линейная структура, показанная на рис. 3.2.

3.2.1. Иерархическая модель и ее недостатки

Иерархическая модель грешит множеством недостатков. Некоторые из них уже рассмотрены выше в связи с тремя факторами организации программного процесса (рис. 3.1). Наиболее очевидный недостаток – это *длительный процесс* и витиеватое прохождение инструкций сверху вниз и обратной связи снизу вверх.

Нехватка информации образуется в силу проблем коммуникации. Сверху вниз информация передается директивно и иногда просто не доходит до работников. Более того, люди склонны к совершению ошибок, забывчивы, медлительны, случайно или намеренно искажают информацию, в связи с чем возникает эффект «испорченного телефона». Внизу иерархии поступающая информация может восприниматься совсем не так, как ее передавало руководство. Обратные потоки тоже искажаются, причем бывает, что намеренно, в угоду руководству предоставляется идеализированная картина и отчеты. В малой группе такого нет и быть не может, но она ограничена по численности и по возможностям.

Невозможность учесть все особенности проекта проистекает из предыдущего недостатка, поскольку все особенности сложного проекта (проект сложный, так как потребовал иерархической модели) также сложно учесть. К тому же эти особенности надо как-то распределить по подпроектам, на

каждый из которых поставлена отдельная группа разработки. Это приводит к невозможности устранения всех ошибок в конечном продукте, из-за чего конечный продукт и проект в целом оказывается как минимум проблемным (который не укладывается в определенные временны́е, финансовые рамки либо реализован с неполной функциональностью), а может и вовсе оказаться проваленным (незавершенным) при прочих неблагоприятных обстоятельствах. Успешным такой проект будет с меньшей вероятностью.

Отсутствие полноценной связи между всеми участниками проекта – это проблема координации, которая, очевидно, негативно влияет на проект. Иерархическая модель в принципе не предполагает какие-либо коммуникации напрямую между работниками отдельных подразделений. Они могут где-то встречаться случайно, но какие-либо специальные встречи данной моделью не предусмотрены. Проводятся совещания руководителя проекта и руководителей проектных групп, а также по отдельным ветвям иерархической структуры внутри групп.

Отметим, что это все может быть нивелировано наличием эффективной инструментальной среды для управления жизненным циклом продукта, которая позволяет полностью или частично автоматизировать процессы жизненного цикла ИС, сделать максимально прозрачными все работы, все задачи в рамках реализации Backlog продукта. Такие средства делают прозрачным задел работы по продукту и области ответственности каждого работника, позволяя каждому члену команды понимать, кто чем занимается, или даже непосредственно и принудительно транслировать эту информацию работникам, если это критически важно для хода проекта.

Трудности освоения новых технологий – это проблема многочисленности команды и разделения ответственности между ее членами, если четко не поставлен процесс разделения ответственности. В этом случае возникают сложности расстановки приоритетов. Здесь действительно необходимо быть профессионалом, чтобы сориентироваться во множестве задач, распределенных по множеству разработчиков. К тому же нередко бывает так, что некоторые задачи пересекаются, например, связаны по данным или по каким-то отдельным подфункциям. В этом случае рабочим группам приходится взаимодействовать между собой и интегрировать свои решения. Здесь тоже есть проблема расстановки приоритетов – так, чтобы четко согласовать, синхронизировать свою деятельность с деятельностью других членов команды и членами другой команды, что возвращает нас к проблеме *координации*.

Структура равноправности соисполнителей на одном уровне иерархии, следствием чего является доверительность – каждый отвечает за свой фронт работы. При этом может случиться, как в басне о лебеде, раке и щуке, когда каждый тянет какие-то ресурсы на себя либо выполняет одну и ту же работу параллельно, не договорившись толком между собой. Эти проблемы чаще всего возникают в иерархической структуре, которая исключает или делает затруднительной коммуникацию с членами коллектива одного уровня, главным образом на нижнем уровне.

3.2.2. Модель группы и ее недостатки

Модель группы не определяет структуру коллектива с точки зрения отдела кадров. Она не предполагает жесткой структуры и поэтому лишена недостатков иерархической модели. Это гибкая модель. Ее задача – определить цели проекта и распределить обязанности внутри группы. Поскольку группа немногочисленная, члены группы распределяют обязанности самостоятельно. При этом в группе все-таки выделяется неформальный лидер, который хорошо знает команду (кто на что способен) и может сам предложить оптимальное распределение ролей. Руководители каждого направления с помощью выделенных им ресурсов выполняют возложенную на них часть работы. Все остальные члены группы его поддерживают или вносят какие-то коррективы и предложения, которые обсуждаются и принимаются или отклоняются. Кто-то отвечает за постановку задачи, кто-то за анализ, кто-то за кодирование, кто-то за тестирование, кто-то за сборку. Во времени эти задачи разделены, поэтому иногда кто-то кого-то где-то замещает, выполняет работу другого члена команды. По модели группы чаще всего работают универсальные специалисты, которые могут выполнять любую работу как авторскую, а также заменить другого члена группы на любой должности. Поэтому обязанности ролей и определяются в ходе работы над проектом и могут меняться динамически по ходу проекта, а не зафиксированы штатным расписанием. Возможна фиксация основной роли работника, который при этом может выполнять любые другие работы. В этих замещениях есть определенные несовместимости, которые будут рассмотрены отдельно.

Однако и у модели группы есть недостатки, например *разрозненная связь с внешними источниками информации*, просто потому, что члены команды равно отвечают за проект, несут коллективную ответственность и поэтому считают себя обязанными осуществлять внешние коммуникации, особенно если им это нравится делать. Как и в иерархической модели, если четко не поставлена процедура согласования всех действий, то могут возникать *проблемы коммуникации*. Но поскольку в малой группе чаще всего коммуникации поставлены нормально и не вызывают больших трудностей и недоразумений, то этот недостаток может быть сведен к нулю.

Несогласованное представление о разных сторонах проекта преодолевается эффективными коммуникациями и ежедневными совещаниями. Так, например, в методике Scrum практикуются каждодневные 15-минутные совещания. Они как раз призваны устранить два указанных выше недостатка.

То же касается *несогласованности личных планов членов группы*, с той лишь разницей, что личные планы касаются личной жизни, но и она может как-то влиять на процесс, поскольку нет четкой организации. К примеру, у членов команды возможен ненормированный рабочий день, если они не все успевают выполнить в срок. В этом случае возможны какие-либо дополнительные мотивирующие мероприятия в плане организации работ. Проблема возникает тогда, когда члены группы начинают злоупотреблять этой свободой. Свобода полезна для высокоорганизованных и качественных людей. Для

неорганизованных и некачественных людей свобода вредна и быстро приводит к распущенности. Поэтому этот недостаток нивелируется профессионализмом членов группы.

Отсутствие опыта, снижающее эффективность коллективной работы. Коллектив малой группы резко чувствителен к опыту его членов, поэтому коллективы малых групп обычно формируют из опытных и качественных людей. Хотя бывает так, что новичок попадает в команду малой группы. В относительной численности всего коллектива таких новичков должно быть не более 10-15 %, иначе будут проблемы. Поскольку обычно коллектив малой группы включает до 15 человек, это означает, что будет максимум один-два новичка, реже два, чаще один или ноль. Но даже если один или два так называемых стажера попадает в такую группу, то его прикрепляют к опытному работнику, он работает с ним под его курированием и неусыпным контролем – так развивается, зреет качественный специалист. Через некоторое время он наберется опыта и станет полноценным членом группы, который будет эффективно работать в коллективе.

3.2.3. Состав команды и распределение ролей

В состав команды входят специалисты по программной инженерии, которые владеют технологией разработки ПО, то есть всеми необходимыми методами, подходами, приемами, инструментами моделирования, разработки, отладки, версионного контроля, тестирования. В первую очередь это программисты, которые заняты непосредственно написанием кода и разработкой модульных тестов. Также нужны специалисты по качеству. Это тестировщики, профилировщики, аудиторы.

Иногда нужны технические писатели для создания полной документации по проекту. У малых групп обычно на это нет времени. Один из постулатов гибкой разработки, изложенных в agile-манифесте[1], гласит, что работоспособное ПО важнее всеобъемлющей документации. Поэтому если всеобъемлющая документация все-таки требуется, а качественная документация определяет качество программного продукта, то этой задачей занимаются технические писатели.

Также нужны специалисты по внедрению, развертыванию ПО на объекте и его сопровождению. Сопровождение уже внедренного продукта требует знания его специфики. Малые задачи по модификации ПО возникают на объекте периодически, поэтому работа ведется уже на готовом продукте, который подвергается систематическим изменениям.

Специалисты по продаже нужны там, где коробочные продукты выводятся на рынок, продвигаются с применением маркетинговых методов и средств поддержки пользователей. Сопровождение ПО этого профиля несколько отличается от сопровождения заказного продукта.

[1] www.agilemanifesto.org/iso/ru

Приведенный состав команды дает общее представление о процессе командной разработки ПО. В иерархической модели состав команды может быть примерно такой же, только там гораздо больше людей и появляются еще руководитель проекта, руководители групп, но только за исключением специалистов по продаже и специалистов по сопровождению. Специалисты по сопровождению и по продажам обычно работают с продуктом целиком, когда он уже готов.

Распределение ролей в команде по типам работы.

1. Разработка приложений требует:
   - программистов;
   - специалистов по инженерии и программированию;
   - специалистов по инженерии и знаниям.

Это специалисты, которые отвечают за архитектуру системы и за ее информационное обеспечение.

2. Работа с приложениями – эксплуатация:
   - специалист по приложениям;
   - администратор данных;
   - администратор баз данных.
3. Техническая поддержка:
   - системный администратор;
   - сетевой администратор;
   - администратор коммуникации.

Техническая поддержка направлена на поддержание действующей ИТ-инфраструктуры с программным обеспечением.

4. Обеспечение качества программы:
   - технический писатель;
   - специалист по тестированию;
   - специалист по качеству.

Это специалисты, составляющие документацию, обеспечивающие тестирование и обеспечение качества.

5. Маркетинг:
   - специалист по сопровождению;
   - специалист по продаже продукта.
6. Системное интегрирование:

– системный интегратор (основные задачи: предложить заказчику варианты решения проблемы, выбрать приемлемый вариант и реализовать).

Здесь системный интегратор занимается решением проблем, связанных с недоработками в интегрируемых частях системы, обеспечивает сборку крупного проекта в единую систему.

3.2.4. Модель команды главного программиста

Данная модель является специфической разновидностью модели группы и предназначена для проектов средней сложности с числом функциональных

требований от 50 до 100. Выделяется главный программист, что характерно все-таки для иерархической модели, когда есть главный и подчиненные программисты. Главный программист занимается анализом, проектированием, а также созданием документации, имеет большой опыт работы и существенные знания предметной области. Также он активно коммуницирует с заказчиком и решает с ним все вопросы. У главного программиста может быть дублер, которому он делегирует свои обязанности.

Кроме того, выделяются следующие роли в команде.

Администратор или менеджер, который занимается ресурсами, в частности финансовыми, трудовыми, материально-техническими. Он распределяет машинные ресурсы, координирует контакты с другими группами и руководством, налаживает коммуникации.

Технический редактор, который выполняет роль технического писателя, занимается документацией. Главный программист также работает над документацией, но он может выстраивать концепцию, разрабатывать какие-то ее части, проверять, перерабатывать и осмысливать с точки зрения заказчика.

Языковеды занимаются лингвистическим обеспечением. В проекте может использоваться множество языков программирования, языков представления данных и знаний, языков коммуникации со специалистами, профессионализмы. Языковед должен владеть терминологией. В частности, на этапе постановки задачи он задействован в анализе предметной области, поскольку хорошо владеет языковыми средствами. Впоследствии его задача состоит в нахождении эффективных способов программирования в зависимости от поставленных задач. Языковед обычно работает с несколькими бригадами программистов и консультирует их по поводу используемых тех или иных языковых средств.

Инструментальщик ответственен за инструментальное ПО, оказывает поддержку программистам. Данную роль может осуществлять администратор.

Отладчик занимается отладкой ПО. В его сфере ответственности находится организация тестирования и сборки продукта. Отладчик по завершении тестирования собирает продукт и, если ошибок в нем нет и все тесты успешно выполнены, размещает его на сервере развертывания.

Делопроизводитель – вспомогательный работник, который отвечает за регистрацию всех данных в команде. В настоящее время функции делопроизводителя автоматизированы и исключены из проекта.

3.3. Фреймворки и их использование при разработке ИС

Автоматизированное производство ПО и ИС требует наличия инструментов (прикладного инструментального ПО), обеспечивающих реализацию процессов жизненного цикла (ЖЦ) ИС. На протяжении всего существования ПО такие инструменты появлялись регулярно и на текущем уровне технологического развития обеспечивали решение задач по созданию ПО. С усложнением задач возникла необходимость в создании комплексных, платформенных решений, покрывающих если не все, то большую часть этапов

ЖЦ ИС. Соответственно, появляется множество инструментальных средств, средств для разработки ИС, так называемых *фреймворков*.

Фреймворк – программная платформа, определяющая структуру разрабатываемого программного решения.

Эта платформа обладает рядом структурных особенностей, таких как встроенная среда разработки, средства отладки, компиляции, тестирования, поддержки версионного контроля, поддержки процессов проектирования, включая сопряженность с различными CASE-средствами. Именно фреймворк является той основой технологии разработки, которая используется для создания приложений в том числе для проектного определения структуры разрабатываемой ИС. Причем эту структуру можно разработать и на уровне проектных решений, и на уровне рабочих решений. Например, может быть построена модель классов UML, отражающая концептуальную структуру ИС. Далее на ее основе могут быть сгенерированы рабочие решения в виде структур БД и объектов ПО, вплоть до шаблонной кодогенерации. Таким образом, фреймворк облегчает обработку компонентов ИС и тем самым позволяет эффективно осуществлять программный процесс и автоматизировать большую часть рутинной работы членов команды разработчика.

Фреймворк влияет на выбор того или иного архитектурного паттерна (шаблона) проектирования. Шаблон – это образец, определяющий структуру и поведение разрабатываемого решения либо его отдельных компонентов. Есть разные шаблоны (паттерны) проектирования ИС. Фреймворк поддерживает выбор шаблона и его применение в программном процессе. Есть MVC-паттерн (Model View Controller), например, определяющий схему разделения данных. Есть и другие паттерны, которые применяются при проектировании и разработке программных решений.

Мы рассмотрим фреймворки, которые базируются на языке программирования Python и используются его для разработки. Python – это один из топовых языков программирования. Данный язык уверенно лидирует среди современных языков программирования. Широта его применения и популярность объясняются очень просто. Прежде всего, это его свободная распространяемость, которая обеспечивает фреймворку обширное сообщество пользователей-разработчиков. Это характерно вообще для любого продукта класса.

Open Source – это программное обеспечение, которое поставляется для конечного пользователя с открытым исходным кодом.

Философия Open Source рассматривает проблемы разработки и использования ПО с точки зрения того, как сделать программы лучше. Данный сугубо практический аспект и составляет преимущество программ с открытым исходным текстом. Open Source не гарантирует свободы выполнять, копировать, распространять, изменять и улучшать программу.

Тем не менее при выходе продукта Open Source на рынок у него сразу же появляется очень много поклонников, потребителей, пользователей, и, естественно, формируется обширное сообщество в сети Интернет, появляются различные форумы для обмена опытом. Это позволяет, во-первых, эффективно

использовать данное средство, во-вторых, эффективно его развивать, формируя коллектив развития данного фреймворка в масштабах всей планеты в полном соответствии с принципом «человеческие знания принадлежат всему человечеству».

Второе преимущество Python – это его чрезвычайно обширная база библиотек, насчитывающая более 100 тысяч библиотек с миллионами классов внутри. Они охватывают широкий спектр задач, включая машинное обучение, анализ данных, веб-разработку, обработку изображений, текста и многое другое. По сути, Python обладает обширной и постоянно растущей экосистемой библиотек. По всему миру ученые проводят исследования и разрабатывают новые подходы к решению сложных задач, которые затем программируются и зашиваются в библиотеки как готовые стандартные компоненты. Многие из этих библиотек также являются открытым исходным кодом, что позволяет разработчикам использовать и адаптировать их под свои нужды. Так возникает парадигма компонентного программирования, главным образом опирающаяся на понятие программного компонента. Это заметно облегчает работу программистов. К тому же компонентное программирование гораздо проще, чем любое другое, особенно принимая во внимание тот факт, что эти компоненты уже отлажены, протестированы и, значит, гарантированно обладают высоким уровнем качества. Суть разработки заключается в компоновке, интеграции компонентов в единую систему, подобно деталям конструктора. Квинтэссенцией компонентного программирования является визуальное программирование, где компоненты интегрируются в систему посредством манипулятора «мышь». В компонентном программировании нет необходимости в модульном тестировании – во главу угла ставится интеграционное и сквозное тестирование. Интеграционное тестирование, в частности, направлено как раз на тестирование взаимодействия компонентов на стыках. Если все нормально с этими взаимодействиями, то система собирается и тестируется «насквозь» по принципу «end-to-end».

Вполне естественно, что фреймворки, использующие Python, в основном так же являются Open Source. Наиболее распространенные фреймворки: Django, Pyramid, TurboGears, Flask, Tornado. Основные характеристики этих фреймворков представлены в табл. 3.1.

Pyramid поддерживает статику разработки. Некоторые области не настолько динамичны, что делает нецелесообразным применение сложных решений для разработки. Соответственно, есть решения относительно простые, которые позволяют работать более эффективно. Функция генерации URL (Uniform Resource Locator, унифицированный указатель ресурса, более известный как адрес веб-ресурса в сети Интернет) актуализировалась с началом бурного развития Интернета. Она необходима для разработки веб-приложений. Предикатная логика лежит в основе любых логических операций, а также может быть использована для верификации программного обеспечения методом Model Checking.

Таблица 3.1 – Фреймворки для разработки приложений на языке Python

Фреймворк	Особенности	Достоинства	Недостатки
Pyramid	Инструментарий для работы со статичными ассетами. Генерация URL. Предикаты	Гибкость. Работа с Ajax-запросами. Поддержка SQLAlchemy	Сложность подготовки к работе Избыточность для простых приложений
TurboGears	Интеграция с библиотеками JS (MochiKit) для работы Одновременная поддержка нескольких БД. Поддерживает SQLAlchemy, Genshi, WebOb и Repoze	Гибкость и расширяемость. OpenSource-проект	Сложность освоения и использования
Flask	Встроенный дебаггер. Шаблоны Jinja2. Безопасность работы с куки на клиенте. Поддержка юнит-тестов	Быстрое прототипирование. Множество возможностей за счет низкоуровневой работы	Сложность работы. Высокий порог вхождения
Tornado	Асинхронность работы. Работа в реальном времени. Возможности аутентификации. Может выдерживать проблемы 10 000 соединений	Поддержка множественных пользовательских соединений. Работа в реальном времени. Поддержка переводов и локализации	Сложность работы Высокий порог вхождения
Django	Контроль версий для БД (использование миграций). Маршрутизация URL. Поддержка веб-серверов, аутентификации, интернационализации	Множество библиотек. Обширное сообщество и документация. Масштабируемость	Проблемы при работе с WebSockets. Готовые библиотеки могут снижать гибкость

TurboGears базируется на библиотеках Java Script, обеспечивает гибкую и эффективную обработку данных, хранящихся в нескольких БД. Тем самым обеспечиваются распространенные практики отделения данных от программы и разделения баз данных. Поэтому данный фреймворк удобно использовать для разработки ИС с микросервисной архитектурой. TurboGears поддерживает такие архитектурные решения на основе MCA. Имеется поддержка множества инструментов для различных аспектов веб-разработки. Кроме того, TurboGears поддерживает библиотеку Genshi, которая предоставляет интегрированный набор компонентов для разработки, подготовки и обработки веб-ресурсов (HTML либо XML разметки контента). Все это ускоряет, оптимизирует обработку данных в сети, парсинг данных, web-mining, шаблонную обработку данных в сети.

Flask имеет встроенные средства разработки и отладки, в частности шаблонизатор Jinja, обеспечивающий гибкую разработку приложений. Имеет

развитые средства информационной безопасности. Поддерживает модульное тестирование программ и быстрое прототипирование, что позволяет обеспечить качество предложения и минимизировать затраты на его разработку и последующее сопровождение. Сложность использования Flask определяется его технологической мощью поэтому для эффективного его использования необходим высокий уровень компетенций.

Tornado поддерживает асинхронную работу и параллелизм, что позволяет разрабатывать системы, работающие в режиме реального времени, где необходимо быстро и своевременно реагировать на непредсказуемые события, осуществлять параллельную потоковую обработку данных. Фреймворк позволяет разрабатывать сложные высоконагруженные системы с множеством пользовательских соединений.

Django рассмотрим более подробно, так как это рекомендуемая платформа для разработки веб-приложений в рамках лабораторного практикума, который рассматривается в разделе 4.

3.4. Фреймворк Django

Прежде всего, Django заметно упрощает работу с БД через миграции, а также маршрутизацию URL, что позволяет настроить синхронизацию и обеспечить гибкость разработки ИС на платформе. Множество поддерживаемых библиотек, обширное сообщество и документация делают Django очень эффективной, универсальной, масштабируемой и гибкой платформой. Один из характерных недостатков заключается в использовании стандартных библиотек, обширное использование которых может снижать гибкость. Если все делать на библиотеках, гибкость понижается, так как все опирается на готовые, конструктивно оформленные, завершенные компоненты. Стандартными компонентами нельзя эффективно решить сложные задачи. Django позволяет разрабатывать универсальные решения, но их гибкость зависит от баланса программирования новых и использования готовых компонентов.

Django использует шаблон проектирования MVC (Model View Controller, способ организации кода). Его компоненты:

– Model – бизнес-логика, то есть совокупность методов, правил и ограничений работы с данными;

– View – компонент, отображающий пользователю данные в зависимости от изменения модели;

– Controller – программный посредник, обрабатывающий действия пользователя и сообщающий модели, как она должна измениться.

В Django уровень Controller реализуется уровнем шаблонов (англ. Template). поэтому архитектуру Django часто называют «модель – шаблон – представление» (MTV). Здесь шаблоны определяют, как будет отображаться информация, а представление описывает, какие именно данные будут

показываться пользователю. В целом принцип MTV повторяет концепцию MVC, поэтому Django относится к MVC-совместимым платформам.

Базовая структура фреймворка включает следующие компоненты:

– URL-маршрутизаторы, перенаправляющие HTTP-запрос от браузера или любого иного веб-клиента в представления;

– представление, которое обрабатывает запрос, обращается к модели и сообщает ей, какие именно данные из БД нужно задействовать, чтобы удовлетворить запрос;

– модель (менеджер базы данных, ORM), «вытаскивающую» нужную информацию из БД и передающую ее представлению;

– HTML-шаблоны, которые используются представлением для демонстрации пользователю полученных от модели данных.

Примерная схема работы приложения на Django показана на рис. 3.3.

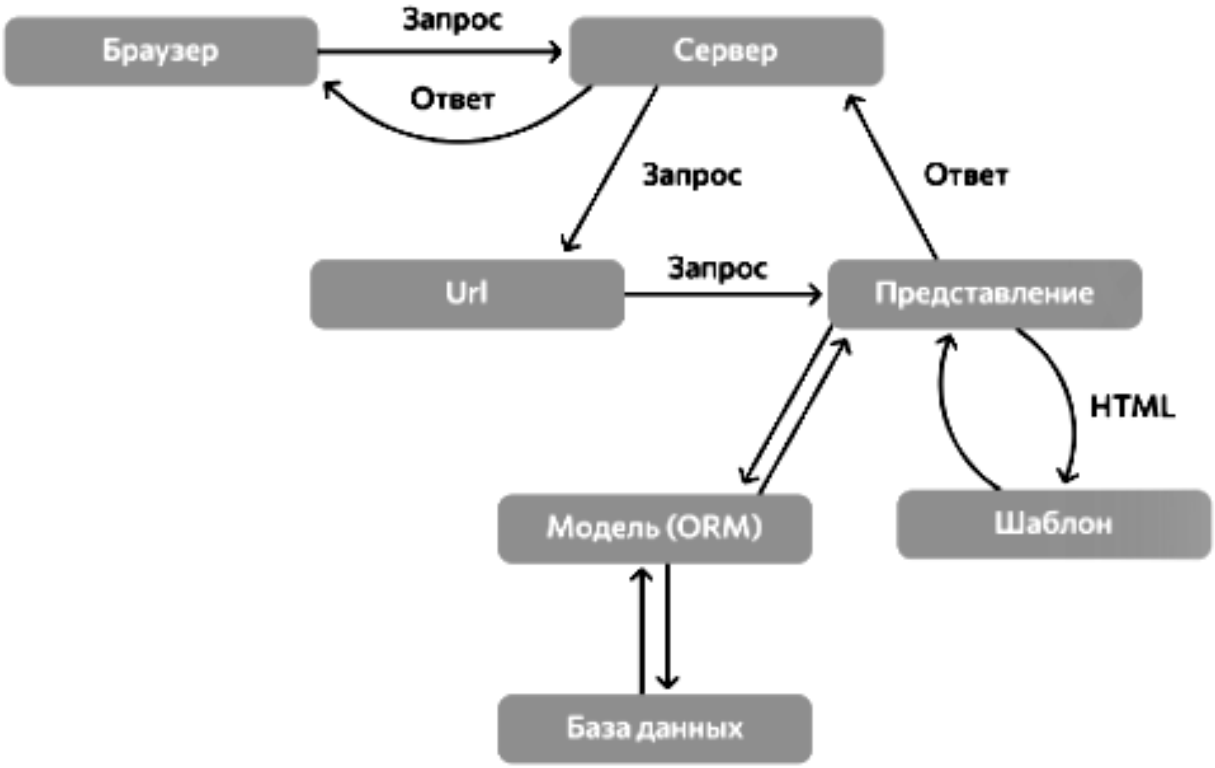


Рисунок 3.3 – Структурная схема работы приложения на Django

Создание проекта Django выполняется командой `startproject` с указанием имени проекта. Для запуска команд используется системный модуль `django-admin.py`. Полный синтаксис команды создания проекта с именем "first": `django-admin.py startproject first`.

В результате на диске создается структура каталога проекта, показанная на рис. 3.4. Корневой каталог проекта first, имя которого совпадает с именем проекта, содержит вложенный каталог first с глобальными файлами проекта и главный файл проекта manage.py – скрипт управления проектом (добавление новых приложений, сборка файлов и пр.).

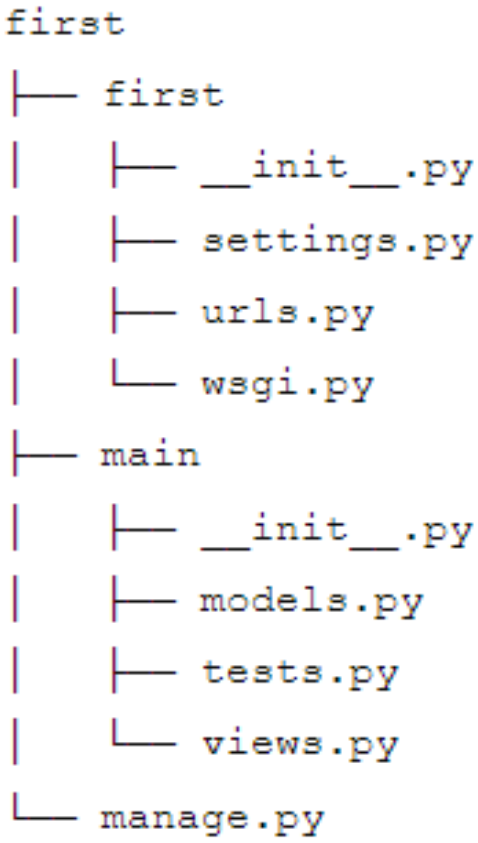


Рисунок 3.4 – Структура каталога проекта Django

Во вложенной директории приложения находятся различные модули:

– __init__.py – конструктор данного проекта, необходим для определения директории/пакета;

– settings.py – модуль глобальных настроек проекта (пути, БД, подключения и пр.);

– urls.py – модуль привязок URL (по какому адресу вызывается тот или иной скрипт), определяет все пути в приложениях проекта;

– wsgi.py – WSGI-приложение для работы с web-сервером из проекта.

Выполнив команду startapp (`./manage.py startapp main`), можно создать приложение с заданным именем "main". В результате в корневом каталоге проекта создается каталог main с файлами приложения:

– __init__.py – конструктор приложения, необходим для определения директории приложения;

– models.py – модели данных приложения;

– tests.py – шаблон для автоматических тестов приложения;

– views.py – представления приложения.

Порядок работы согласно схеме на рис. 3.3:

1. Браузер запрашивает URL, сервер передает запрос в Django.
2. Django формирует объект HttpRequest.
3. Запускаются методы process_request Middleware-классов.
4. Поиск соответствия в файле urls.py.

5. Запускаются методы process_request Middleware-классов.

6. Передается Request с параметрами (при их определении в файле urls.py) в найденное представление.

7. Выполняется функция представления, возвращается объект HttpResponse.

8. Запускаются методы process_request Middleware-классов.

9. Django возвращает ответ на web-сервер, а затем – в браузер.

Упомянутые в пунктах 3, 5 и 8 Middleware-классы выполняют работу по модификации запросов и ответов.

3.5. Система контроля версий Git

Система контроля версий (СКВ) – это программная система для облегчения работы с изменяющейся информацией, записывающая изменения в файл или набор файлов в течение времени и позволяющая вернуться позже к определенной версии, а также определять, кто и когда сделал то или иное изменение, и многое другое.

СКВ наиболее широко используются при разработке ПО для хранения исходных кодов разрабатываемой программы. Однако они могут с успехом применяться и в других областях, в которых ведется работа с большим количеством непрерывно изменяющихся электронных документов.

Среди СКВ ведущее место занимает Git – распределенная программная система управления версиями, используемая для отслеживания изменений в файлах, в первую очередь в программном коде. Разработанная Линусом Торвальдсом для использования в управлении разработкой ядра Linux, первая версия выпущена 7 апреля 2005 года. Лидирующие позиции Git также обусловлены свободной распространяемостью данной технологии[2]. Кроме того, Git позволяет разработчикам совместно работать над проектами, отслеживать историю изменений и легко возвращаться к предыдущим версиям. Git обладает необычайной легкостью в использовании не только как распределенная система контроля версий, но и в работе с локальными проектами.

Git сохраняет файлы разных версий как «снимки». Если в новой версии какой-либо файл не менялся, то сохраняется не «снимок», а ссылка на предыдущую версию файла (рис. 3.5). Git также позволяет выполнить откат к предыдущей версии компонента и тем самым устранить некорректные изменения в проекте.

Git хранит информацию в структуре данных, называемой репозиторием.

Репозиторий (repository) – специальное хранилище файлов («снимков») и папок проекта, а также истории изменений, ссылок на файлы, комментарии, изменения в которых также отслеживаются.

В распоряжении разработчика имеется так называемая рабочая копия (working copy) проекта, с которой он непосредственно работает. Рабочую копию необходимо периодически синхронизировать с репозиторием. Эта

[2] Бесплатный хостинг репозиториев для OpenSource-проектов: github.com.

операция предполагает отправку в него изменений, которые пользователь внес в свою рабочую копию (такая операция называется commit), и актуализацию рабочей копии, в процессе которой к пользователю загружается последняя версия из репозитория (этот процесс носит название update). Данные взаимно противоположные процессы структурно показаны на рис. 3.6.

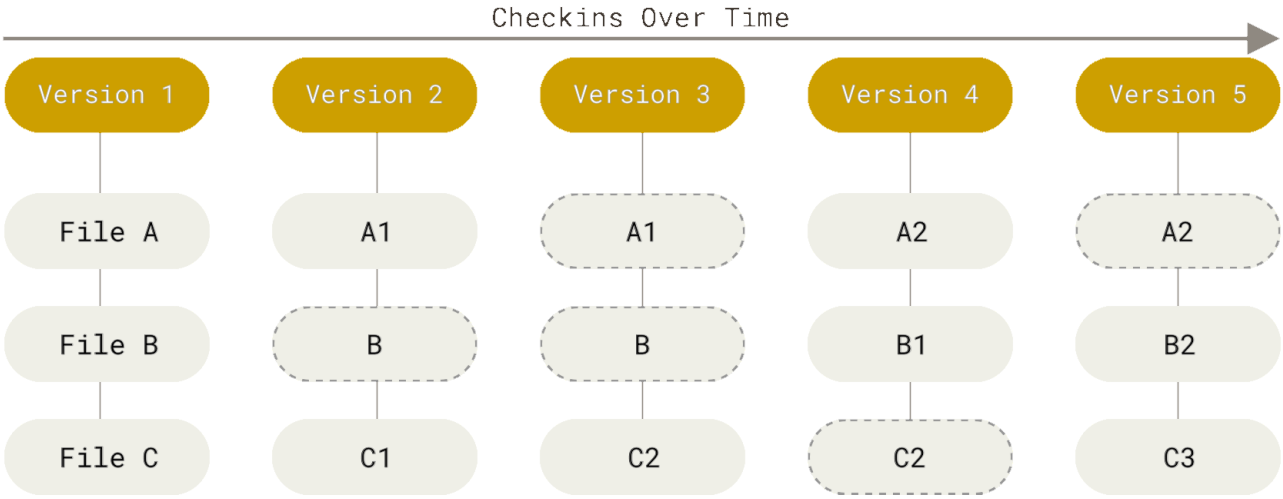


Рисунок 3.5 – Примерная структура изменения версий файлов в Git

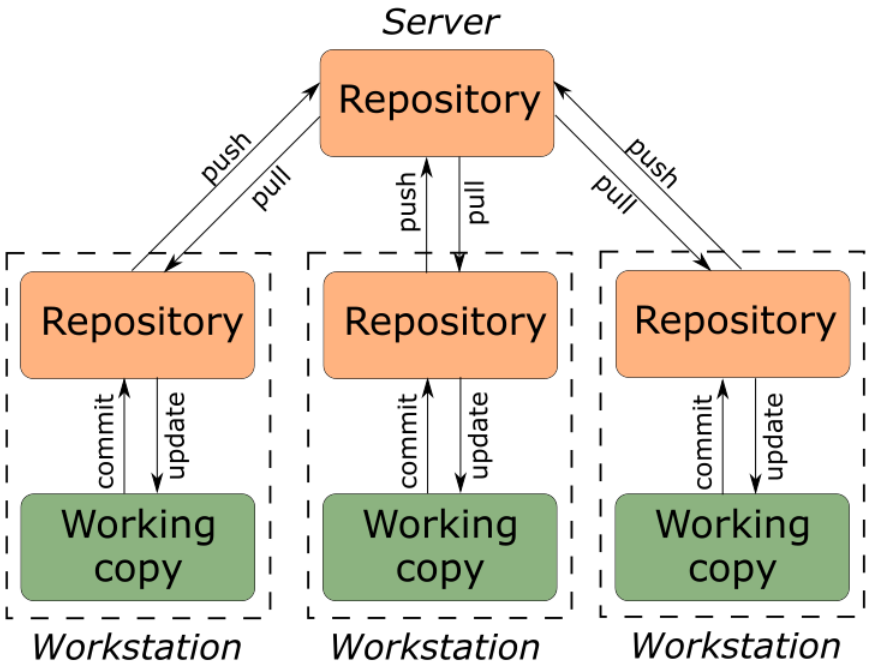


Рисунок 3.6 – Общая структура транзакций в Git

Репозиторий хранит:

– набор commit objects;
– набор ссылок на commit objects, называемых «heads».

Репозиторий хранится в той же директории, что и сам проект в поддиректории .git.

При этом структура хранилища файлов не отражает реальную структуру хранящегося в репозитории файлового дерева, она ориентирована на повышение скорости выполнения операций с репозиторием.

Основные отличия от систем с центральным репозиторием (например, CVS, SVN):

– существует только одна директория .git в корневой директории проекта;

– репозиторий хранится в файлах рядом с проектом;

– не существует центрального репозитория.

Каждая копия (клон) репозитория является независимой от всех прочих и может обмениваться с другими репозиториями историей изменений файлов.

Операции над репозиториями:

– Создание. Создать репозиторий можно в любой папке, не находящейся под версионным контролем;

– Клонирование. Можно клонировать любой репозиторий. При этом данные, находящиеся в исходном репозитории, будут скопированы в новый репозиторий.

Команды работы с удаленными репозиториями (рис. 3.6):

– Fetch – забирает данные в локальный репозиторий, но не сливает их с текущими наработками и не модифицирует то, над чем сейчас ведется работа;

– Pull – забирает изменения из удаленной ветки и автоматически сливает их с текущей веткой;

– Push – отправка данных на сервер и автоматическое слияние с выбранной веткой.

Ядро Git представляет собой набор утилит командной строки с параметрами. Все настройки хранятся в текстовых файлах конфигурации. Такая реализация делает Git легко портируемым на любую платформу и дает возможность легко интегрировать Git в другие системы (в частности, создавать графические Git-клиенты с любым желаемым интерфейсом).

Все операции в Git осуществляются с помощью команд из командной строки. Также имеются оболочки с графическим интерфейсом, которые облегчают работу:

– веб-интерфейсы GitHub, GitLab, BitBucket, GitList, Klaus, Gitea и др.;

– GUI для Git: GitHub Desktop, Sourcetree, GitKraken, Fork, TortoiseGit, Tower, Working Copy, SmartGit, Gitnuro (перечислены в порядке убывания популярности).

На рис. 3.7 показан один из возможных вариантов схемы работы с репозиториями командного проекта. В главный репозиторий загружается готовый функционал, при этом в него попадают и промежуточные версии ПО. Также есть репозиторий разработчика, отдельный для каждого разработчика, возможно локальные. Кроме того, у одного разработчика может быть несколько репозиториев на разных устройствах, как показано в левой части рис. 3.7. Репозитории разработчиков синхронизируются с главным репозиторием, который всегда содержит актуальный функционал последней версии ПО.

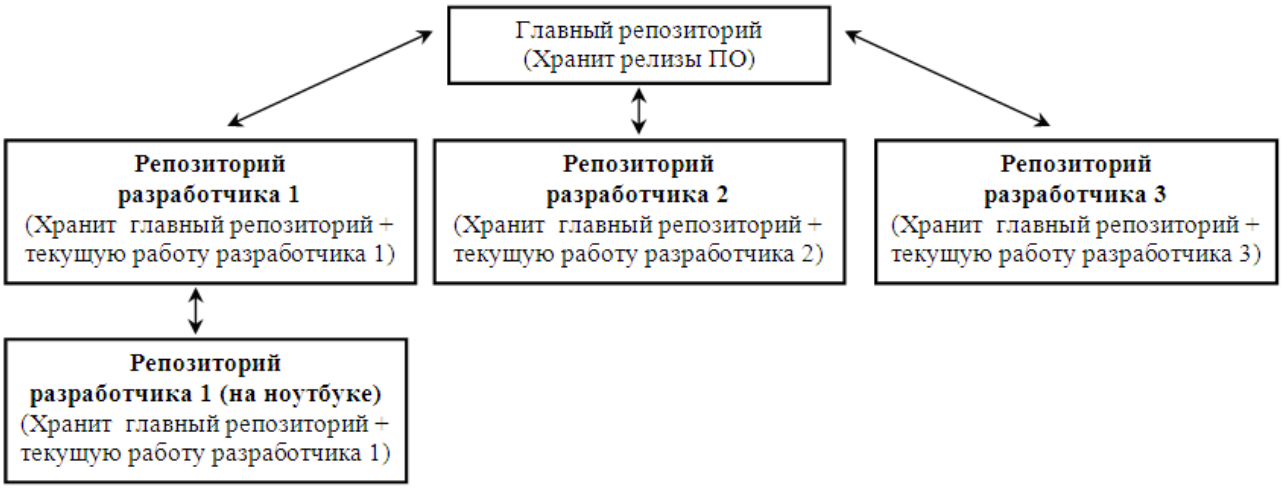


Рисунок 3.7 – Построение работы с репозиториями (вариант)

Файлы с позиции Git могут быть в трех основных состояниях (рис. 3.8):

– Изменен (modified). К измененным относятся файлы, которые поменялись, но еще не были зафиксированы;

– Индексирован (staged) – это измененный файл в его текущей версии, отмеченный для включения в следующий коммит;

– Зафиксирован (Committed) – значит, что файл уже сохранен в локальном репозитории.

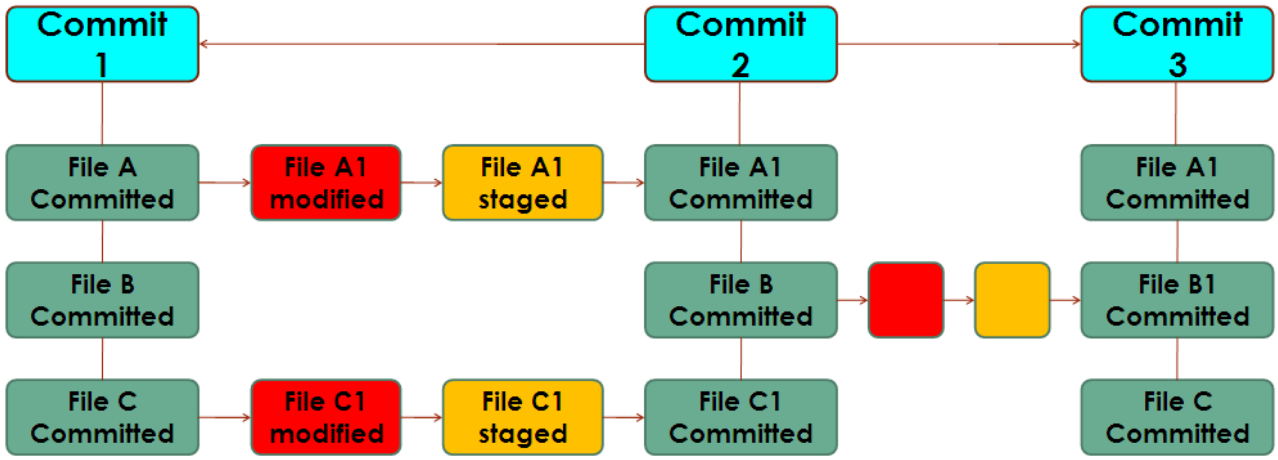


Рисунок 3.8 – Состояния файлов с позиции Git

Коммит слияния (Push) сливает изменения из разных веток. При таком подходе могут возникать конфликты, когда одни и те же строки изменяются разными авторами. При возникновении конфликтов процесс слияния останавливается, выдается предупреждение и список файлов, в которых есть конфликты. После разрешения конфликта необходимо произвести коммит.

Командная работа с серверным репозиторием состоит из следующих шагов:

1) локальный коммит инкремента (нового функционала);

2) запрос изменений с сервера (Fetch);
3) забор изменений, если они есть, и слияние с локальной веткой (Pull);
4) разбирательство с конфликтами, если они есть;
5) тестирование функционала версии с результатами слияния;
6) после прохождения тестов отправление на сервер (Push);

В результате новый функционал находится в общем репозитории.

3.6. Системы контейнеризации

Docker – это программная платформа класса Open Source (распространяется под открытой лицензией Apache 2.0) для автоматизации развертывания, масштабирования и управления приложениями в контейнерах.

Фактически Docker позволяет осуществлять непрерывное развертывание приложения по DevOps. Контейнеры, в свою очередь, представляют собой стандартные единицы ПО, которые включают в себя код приложения и все его зависимости, чтобы оно могло работать независимо от среды (более подробная информация о контейнерах приведена в подразделах 2.4 и 2.5). Docker позволяет «упаковать» приложение и его окружение в контейнер, который затем можно легко развернуть на любой инфраструктуре, поддерживающей Docker. Ключевое преимущество Docker состоит в том, что он позволяет пользователям упаковать приложение со всеми его зависимостями в стандартизированный модуль для разработки.

3.6.1. Возможности Docker

Docker обеспечивает следующие возможности DevOps-инженерии:

1. *Изолированный запуск приложений в контейнерах*. Это прямая ссылка на микросервисную архитектуру. В каждом контейнере можно развернуть свою автономную инструментальную среду и в ней формировать функциональную подсистему либо некоторый атомарный функционал приложения как отдельный микросервис.

2. *Упрощение разработки, тестирования и развертывания приложений*. Упрощение разработки заключается в том, что отдельные и относительно простые части приложения разрабатывать легче (принцип «разделяй и властвуй»). Декомпозиция сложной системы на более простые подсистемы является распространенным способом упрощения разработки (принцип декомпозиции), позволяет автономно с ними по отдельности работать и таким образом эффективно управлять разработкой. То же самое касается тестирования и развертывания. Тестировать небольшую часть функционала гораздо эффективнее, чем весь комплекс функций целиком. Комплексное (системное, сквозное) тестирование также необходимо на заключительном этапе. Но отдельные части приложения все-таки приходится тестировать значительно чаще, в режиме разделения по контейнерам, где они изолированно хранятся, работают относительно автономно. В частности, поэтому они подвергаются тестированию более осмысленно, эффективно по времени и

другим ресурсам. Так же и развертывание отдельных контейнеров гораздо проще, чем приложения целиком. Если функциональность изменилась только в одном контейнере, то развертывать все приложение заново смысла нет. В рамках методологии DevOps разворачивается только то, что претерпело изменения, а это все хранится в конкретных контейнерах, которые и разворачиваются у заказчика, обновляя его функционал «на ходу». Docker предоставляет такие возможности в рамках контейнеризации. Подробнее см. подраздел 2.6 и рис. 2.17.

3. *Отсутствие необходимости конфигурировать среду для запуска* – она поставляется вместе с приложением – в контейнере. При необходимости внесения очередной модификации в приложение с МСА Docker позволяет атомарно модифицировать содержимое контейнера. Конфигурация контейнера если и претерпевает изменения, то не часто. При необходимости сделать модификацию несложно, после чего нужно провести дополнительный сеанс интеграционного тестирования. Важно, что рабочая среда разворачивается вместе с приложением прямо в этом же контейнере и прямо в нем все конфигурируется. В итоге для каждого микросервиса все сконфигурировано, отлажено, оттестировано и может разворачиваться на стороне заказчика.

4. *Упрощение масштабируемости приложений и управление их работой с помощью систем оркестрации контейнеров*. Это также напрямую связано с МСА. Оркестрация контейнеров (автоматизация объединения контейнеров в сеть, подробно будет рассмотрена в подразделе 3.7) актуальна, когда их много. И чем больше контейнеров, тем актуальнее оркестрация. Это позволяет разворачивать приложение в нужном масштабе. Docker напрямую позволяет реализовывать оркестрацию. Также имеется более эффективный инструмент Kubernetes – специальный высокоэффективный оркестратор. Docker в этом плане поддерживает только базовые функция оркестрации – это так называемый Docker plain, который подразумевает работу с Docker на базовом уровне без использования дополнительных инструментов для управления сложными системами.

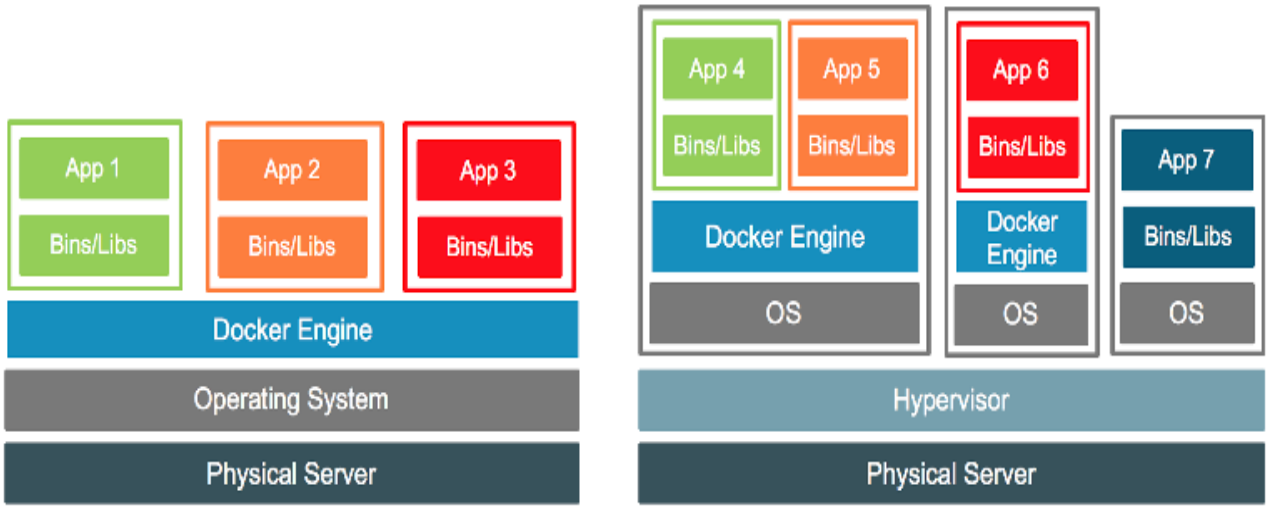
3.6.2. Отличие Docker от виртуальных машин

Современный подход в DevOps-инженерии – использование виртуальных машин (ВМ) для запуска приложений. Виртуальные машины запускают приложения внутри гостевой операционной системы, которая работает на виртуальной аппаратуре основной операционной системы сервера.

Виртуальные машины отлично подходят для полной изоляции процесса для приложения: никакие проблемы гостевой ОС не могут повлиять на ПО основной операционной системы. Контейнеры, по идее, схожи с ВМ, однако используют иной подход, благодаря которому имеют меньшую нагрузку, необходимую для виртуализации аппаратуры гостевой ОС.

Основное отличие Docker от ВМ наглядно показано на рис. 3.9. Контейнеризация позволяет разные приложения в разных средах разработки разворачивать на базе одной операционной системы (рис. 3.9, а). Разные

приложения со своими контейнерами, библиотеками возможно развернуть посредством Docker Engine на одной операционной системе, тогда как виртуальная машина требует для каждого отдельного контейнера, отдельного приложения развертывания отдельной операционной системы (рис. 3.9, б). Такой подход актуален, когда, например, для разных приложений требуется разная операционная система. В этом случае необходимо разворачивать виртуальные машины и на них отстраивать (возможно, так же на базе Docker) среды разработки и приложения. В остальных случаях на базе Docker реализуется более компактная архитектура, тогда как на ВМ получается более громоздкая архитектура. Каждая ВМ требует некоторой ресурсной базы, которая может исчисляться десятками и сотнями Гб. Развертывание же на базе Docker требует на порядок меньше ресурсов, поэтому налицо существенная экономия ресурсов.



а *б*

Рисунок 3.9 – Схема развертывания приложения на базе: *а* – Docker; *б* – ВМ

3.6.3. Структурный состав Docker

Основу программной платформы Docker составляет ее движок Docker Engine – клиент-серверное приложение, содержащее следующие основные компоненты (см. рис. 3.10):

– сервер, по сути являющийся долго работающим приложением, называемым «демоном»;

– REST API, определяющее интерфейсы для взаимодействия, которые могут использовать другие программы;

– консольный (CLI) клиент, использует Docker REST API для управления или взаимодействия с демоном Docker при помощи скриптов или непосредственных консольных команд.

Демон создает и управляет объектами Docker, такими как образы, контейнеры, сети и тома данных (на рис. 3.10 показаны в окружении движка). Рассмотрим их более подробно.

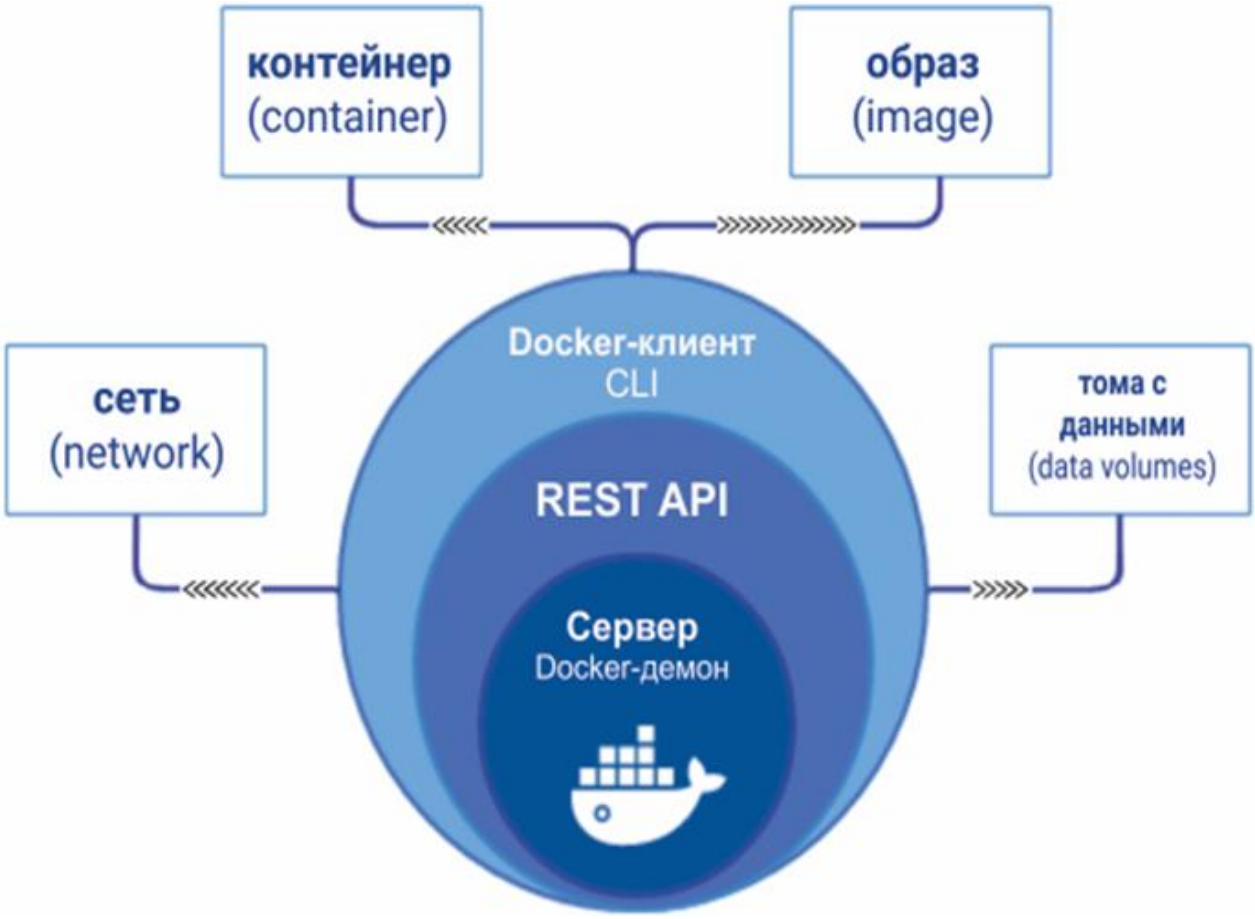


Рисунок 3.10 – Компонентный состав Docker Engine и его взаимодействие с объектами Docker

Образ представляет собой Read-only шаблон с набором инструкций, предназначенных для создания контейнера. Контейнер создается и далее в нем начинается работа с соответствующим атомарным функционалом приложения.

Образ состоит из неизменяемых слоев, каждый из которых добавляет/удаляет/изменяет файлы из предыдущего слоя. Неизменяемость слоев позволяет их использовать совместно в разных образах. На рис. 3.11 показана структура образа для создания четырех контейнеров для развертывания соответственно четырех микросервисов App1–App4.

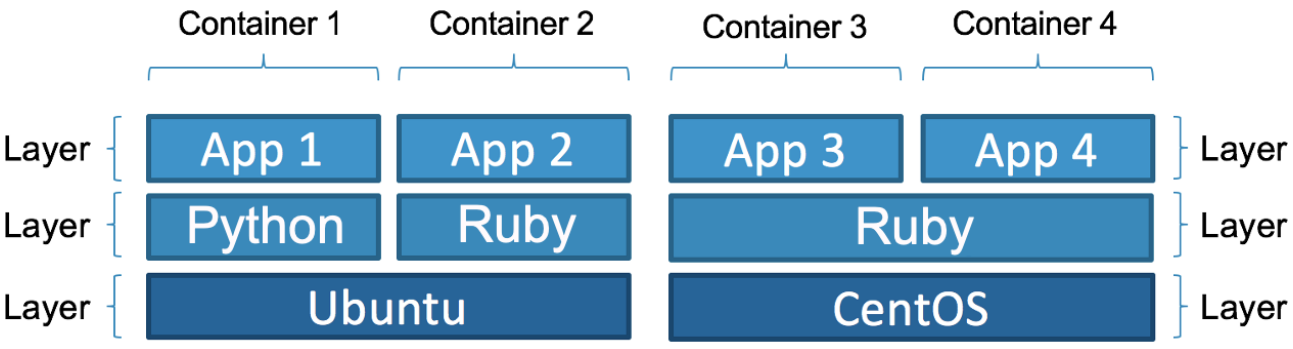


Рисунок 3.11 – Конфигурация образа для развертывания четырех микросервисов на базе разных инструментальных сред

Лежащие ниже слои формируют неизменяемую среду разработки, характерную для каждого контейнера. Поскольку слои неизменяемые, отлажены, оттестированы и готовы к применению, то они могут быть использованы для создания контейнеров. Это одна из причин, почему Docker настолько легковесный. При изменении образа Docker, например, в процессе обновления приложения на новую версию, собирается и заменяет только тот слой, который обновляется. Остальные слои остаются нетронутыми, поэтому для распространения обновления необходимо лишь передать обновленный слой. Механизм работы со слоями увеличивает скорость распространения образов Docker. Docker сам определяет, какой слой должен быть обновлен.

Контейнер Docker – это запускаемый экземпляр Docker-образа, рабочий компонент Docker. Образ статичен, а контейнер динамичен. То же самое можно сказать про виртуальные машины: если образ ВМ представляет собой набор файлов, то при его запуске на базе менеджера ВМ этот образ развернется в рабочую среду и будет динамичен. Однако Docker Engine очень выгодно отличается от ВМ рациональностью используемых ресурсов, так как общее использование слоев уменьшает количество использованного дискового пространства и ускоряет запуск и работу контейнеров.

При запуске образа с заданным именем image1 Docker выполняет следующие действия:

– скачивает образ: Docker проверяет наличие образа image1 на локальной машине, и если его нет, то скачивает его с Docker Hub. Если образ есть, то использует его;

– создает контейнер: когда образ получен, Docker использует его для создания контейнера;

– инициализирует файловую систему и монтирует read-only-образ;

– инициализирует сеть/мост: создает сетевой интерфейс, который позволяет Docker общаться с хост-машиной, устанавливает IP-адрес;

– запускает указанный в образе процесс, при этом выполняется бизнес-логика, которая там запрограммирована – это есть уже работа приложения;

– обрабатывает и выдает вывод приложения: подключается и логирует стандартный ввод-вывод и поток ошибок приложения для отслеживания работы приложения.

Каждый контейнер – это изолированная и безопасная платформа для приложений, однако ему может быть предоставлен доступ к ресурсам, работающим на другом хосте или контейнере – например, к централизованному хранилищу данных (persistent storage) или базе данных.

3.6.4. Dockerfile и Docker-Compose

Каждый образ определяется в Dockerfile (название должно быть с большой буквы).

Dockerfile – это файл инструкций для сборки образа, из которого в дальнейшем запускаются контейнеры.

Это простой текстовый файл, в котором содержится список команд Docker-клиента (CLI). В нем описывается, что должно находиться в образе, какие команды, зависимости и процессы он будет содержать. Это простой способ автоматизировать процесс создания образа.

При создании и запуске контейнера возможно указать сеть, пробрасываемые порты, примонтированные папки, тома данных (data volumes). Однако удобнее это делать с помощью специального файла docker-compose.yml – в формате YAML. Для этого служит Docker-Compose – специальное инструментальное средство, предназначенное для решения задач, связанных с развертыванием проектов.

Docker-Compose – это инструментальное средство для определения и запуска приложений Docker с несколькими контейнерами.

При работе в Docker-Compose используется файл YAML для настройки служб приложения. Затем создаются и запускаются все службы из конфигурации путем выполнения одной команды. При этом Docker-Compose удобно использовать для запуска из одного приложения (контейнера).

При запуске Docker-Compose создает все зависимости (сеть, volumes) для запуска сервисов, скачивает или собирает образы и запускает все сервисы Docker-Compose. Он может запускать или останавливать сервисы, создавать и удалять контейнеры, запускать отдельные команды в контейнере. Для этого в Docker-Compose предусмотрена система команд.

В общем можно сказать, что Dockerfile и Docker-Compose – это средства реализации Deployment Circle цикла DevOps, показанного на его изображении на рис. 1.2 в правой его части.

Для удобства и автоматизации работы Deployment Circle также существуют специальные инструменты, так называемые системы оркестрации. Именно они помогают автоматизировать все операции: развертывание, балансировку нагрузки, распределение ресурсов и прочее. Наибольшей популярностью сейчас пользуется Kubernetes. Однако есть и альтернативы: Docker Swarm, Apache Mesos и т. д.

3.7. Оркестрация контейнеризированных приложений

В классическом варианте контейнеризованных приложений управление жизненным циклом и контроль состояния производятся вручную. В таком варианте DevOps-инженер должен всегда быть начеку – вовремя отслеживать и мониторить состояние контейнера, управлять его жизненным циклом. Да, системы контейнеризации предоставляют фоновые процессы, способные выполнять перезапуск контейнеров в случае их сбоя, а также ряд других дополнительных функций. Однако этого не достаточно – такие решения довольно тривиальны и плохо масштабируются. В серьезных комплексах приложений необходимо управлять сотнями (если не тысячами) контейнеров, что без специальной автоматизированной поддержки сделать невозможно. Для этого предназначены системы оркестрации.

Оркестрация – это процесс автоматизации объединения контейнеров в сеть и управления ими, позволяющий развертывать приложения в нужном масштабе.

Исследования IBM показывают, что более 72 % разработчиков, использующих контейнеризацию, также используют и системы оркестрации. Особенно это касается DevOps-команд.

Для внесения ясности стоит отметить, что *контейнеризация* – это технология, которая позволяет упаковывать приложение и все его зависимости в единый контейнер. Этот контейнер можно запускать на любой системе, где установлен контейнерный движок, например, Docker Engine. Контейнеры обеспечивают изоляцию процессов и ресурсов, что позволяет избежать конфликтов между приложениями и их зависимостями. *Оркестрация* также оперирует с контейнеризованными приложениями, однако предоставляет более расширенный функционал для управления жизненным циклом контейнеров, автоматизации их развертывания и управления ими. В структуре системы оркестрации также используют контейнерные движки, однако такая система обладает значительно более широким функционалом. Именно в системах оркестрации появляется такое понятие, как *кластер* – упрощенно говоря, это группа серверов, работающих как единое целое, предназначенная для запуска приложений и распределения нагрузки.

Задачи оркестраторов контейнеров:

1. Подготовка инфраструктуры и развертывание.
2. Конфигурация и планирование.
3. Распределение ресурсов.
4. Масштабирование контейнеров на основе рабочих нагрузок.
5. Балансировка нагрузки.
6. Маршрутизация трафика.
7. Мониторинг состояния контейнеров.
8. Обеспечение безопасного взаимодействия между контейнерами.

Как и в случае с контейнеризацией, оркестрация использует декларативный подход к конфигурации – DevOps-инженер описывает файл конфигурации в формате YAML (см. подраздел 3.6.4). Далее инструмент оркестрации выполняет самостоятельное развертывание контейнеров, пытаясь привести приложение и его окружение к описанному состоянию.

Основные часто используемые системы оркестрации:

Kubernetes. Разработанный Google, Kubernetes является одной из самых популярных систем оркестрации. Он поддерживает сложные кластеры и предоставляет широкие возможности для настройки и расширения.

Docker Swarm. Встроенное решение для оркестрации от Docker, более простое в использовании, чем Kubernetes, но подходящее для небольших и средних проектов.

Apache Mesos. Мощная система оркестрации, которая может управлять масштабными ресурсами на уровне дата-центра. Она сложнее в освоении, чем Kubernetes и Docker Swarm.

Рассмотрим далее подробно чаще всего используемую систему оркестрации Kubernetes.

3.7.1. Система оркестрации Kubernetes

Kubernetes (K8s) – это программная платформа для автоматизации развертывания и масштабирования контейнеризированных приложений и управления ими. Она предлагает базовые механизмы для их развертывания, масштабирования и поддержки.

Система имеет открытый исходный код и быстро растущую экосистему. Kubernetes разработала компания Google на основе более ранней системы управления кластерами Borg. Первое название – Project Seven, в честь героини сериала Star Trek. Аллюзией на нее также являются семь ручек на штурвале логотипа проекта (рис. 3.12). Современное название отсылает к греческому слову «рулевой». Исходный код системы был представлен в 2014 году. После выхода версии 1.0 в 2015 году Google в сотрудничестве с Linux Foundation основала фонд Cloud Native Computing Foundation (CNCF) и передала ему Kubernetes в рамках начального технологического вклада.



Рисунок 3.12 – Логотип системы Kubernetes

Kubernetes автоматически распределяет контейнеры по серверам, следит за их состоянием, перезапускает отказавшие экземпляры, балансирует нагрузку и помогает обновлять приложения без простоя.

Чтобы понять, как Kubernetes работает, нужно разобрать его основной принцип – декларативное управление. Вместо того чтобы говорить системе, как что-то делать (например, «запусти этот контейнер на этом сервере»), пользователь просто описывает желаемое состояние. Например: «я хочу, чтобы у меня было три экземпляра этого контейнера». Kubernetes сам решает, как достичь этого состояния, и постоянно проверяет, чтобы реальность соответствовала заявленным требованиям.

Основные задачи Kubernetes:

1. Оркестрация контейнеров. Kubernetes автоматически распределяет контейнеры по доступным серверам (называемым узлами, нодами, англ. nodes).

Если один сервер выходит из строя, система переносит контейнеры на другие узлы, чтобы приложение продолжало работать.

2. Масштабирование. Если нагрузка на сервис увеличивается, Kubernetes может автоматически запустить дополнительные экземпляры контейнеров. Когда нагрузка снижается, лишние экземпляры отключаются, что помогает экономить ресурсы.

3. Балансировка нагрузки. Kubernetes распределяет входящие запросы между всеми работающими экземплярами контейнеров, чтобы ни один из них не был перегружен.

4. Автоматическое восстановление. Если контейнер отказывает, Kubernetes автоматически перезапускает контейнер или переносит его на другой узел.

5. Обновления без простоя. Kubernetes позволяет обновлять приложения постепенно, запуская новые версии контейнеров и останавливая старые. Если что-то пойдет не так, система может автоматически откатить изменения.

Kubernetes – это сложная система, но ее можно сравнить с оркестром (рис. 3.13). У каждого музыканта (узла) есть своя роль, а дирижер (мастер-узел) координирует их действия, чтобы получилась гармоничная мелодия.



Рисунок 3.13 – Большой симфонический оркестр

3.7.2. Объекты кластера Kubernetes

Чтобы понять, как Kubernetes работает, нужно разобраться с его архитектурой и ключевыми компонентами. Кластер Kubernetes – это группа машин (физических или виртуальных), которые работают вместе для запуска контейнеризованных приложений. Каждая машина в кластере называется узлом (см. рис. 3.14).

Kubernetes оперирует несколькими ключевыми объектами, которые помогают пользователю описать желаемое состояние системы. Рассмотрим подробнее эту архитектуру, начиная с ее объектов.

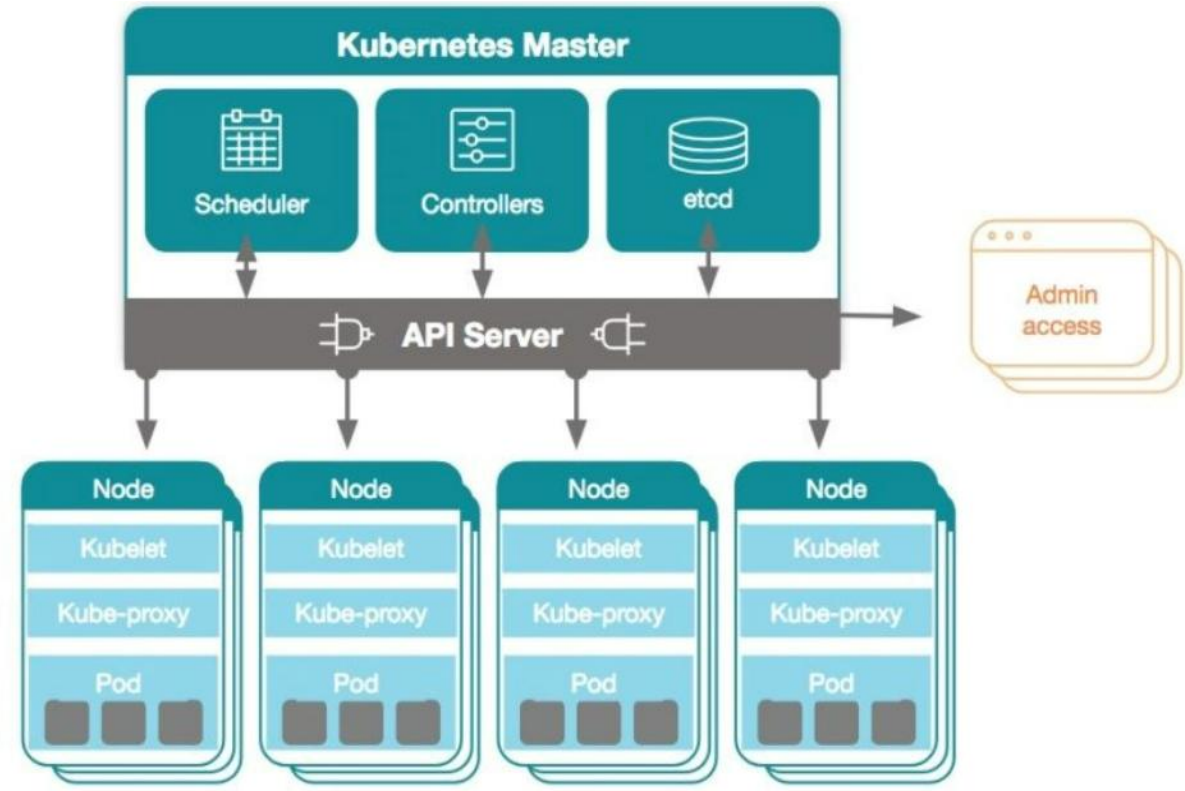


Рисунок 3.14 – Архитектура Kubernetes

Объекты Kubernetes – сущности, которые в архитектуре Kubernetes используются для представления состояния кластера (рис. 3.15).

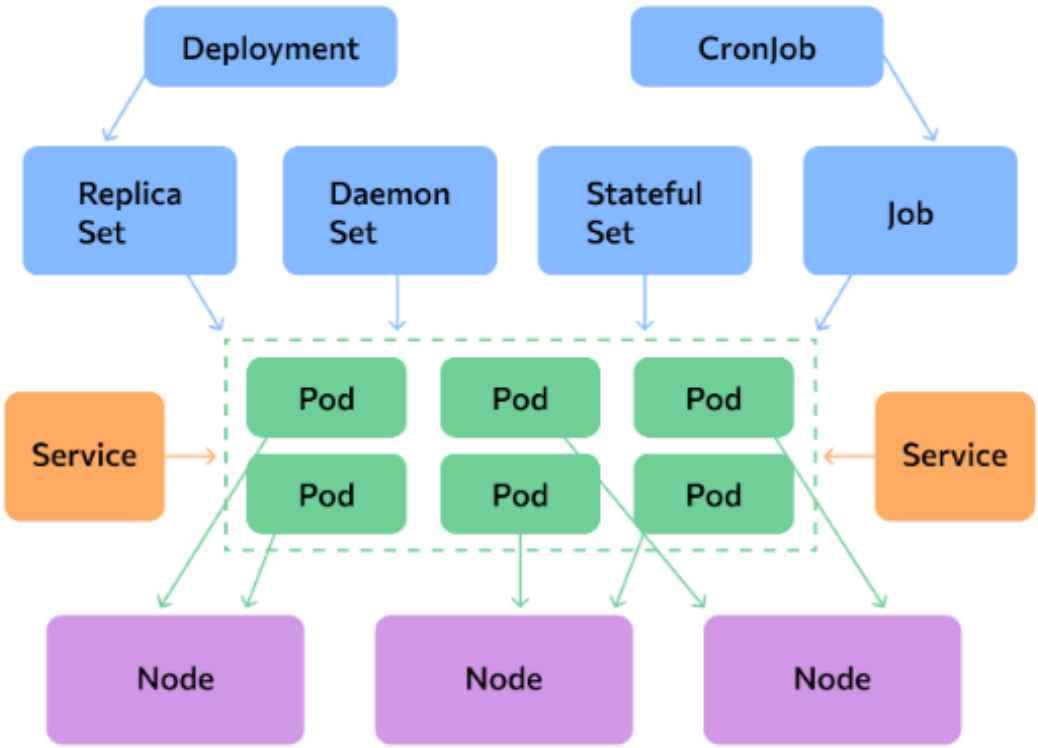


Рисунок 3.15 – Структура кластера Kubernetes

Nodes (ноды, узлы) – это фактически серверы кластера, на которых развертываются и запускаются контейнеры с приложениями. Узлы бывают двух типов:

1. Master (мастер-узлы) – узел управления кластером. Он следит за состоянием узлов кластера и распределяет нагрузку. Чаще всего мастер-узел занимается только управлением и не берет на себя рабочие нагрузки, хотя это не запрещено и возможно скофигурировать кластер таким образом, что мастер будет брать на себя рабочие нагрузки. Для повышения отказоустойчивости существует несколько мастер-нод.
2. Worker (рабочие узлы) – узлы, на которых запущена рабочая нагрузка (контейнеризированные приложения). Эти узлы выполняют реальную работу: они запускают контейнеры и обрабатывают запросы пользователей. Чем больше рабочих узлов, тем больше приложений можно запустить. Количество влияет на отказоустойчивость кластера, потому что при выходе из строя одного узла нагрузка переносится на другие. Рабочие узлы работают под управлением мастер-нод и образуют иерархические структуры (рис. 3.16).

Kubernetes не оперирует отдельным контейнером на уровне абстракции кластера, он оперирует некой оберткой над контейнерами – подами.

Pods (поды) – это объекты Kubernetes, включающие в себя один или несколько контейнеров. Контейнеры в поде запускаются и работают вместе, имеют общие сетевые ресурсы и хранилище. Например, IP-адрес назначается на под, порты также сопоставляются с подом. Kubernetes не управляет контейнерами напрямую, он собирает их в поды и работает с ними.

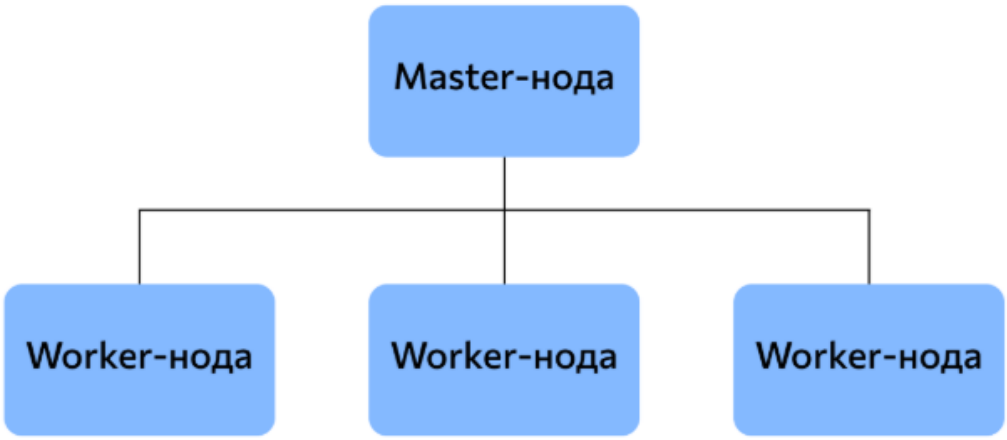


Рисунок 3.16 – Иерархия узлов кластера Kubernetes

Под и входящие в него контейнеры функционируют на одном узле (см. рис. 3.17), оставаясь на нем в соответствии с жизненным циклом. После пересоздания пода он может быть запущен на другом узле. Чаще всего в поде только один контейнер (приложение). Поды, состоящие из нескольких контейнеров, встречаются реже, например, когда несколько контейнеров представляют собой целостное приложение (само приложение и база данных, которая требуется только для функционирования данного приложения), и запускать их отдельно бессмысленно.

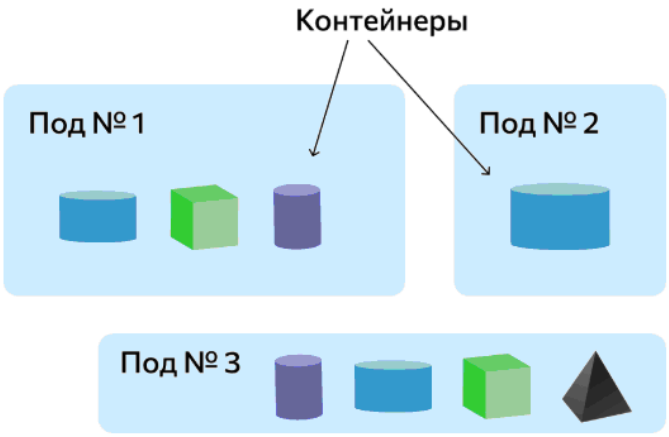


Рисунок 3.17 – Структура узла и работающие на нем контейнеры

Controllers (контроллеры) – это общее название средств управления, следящих за кластером и поддерживающих желаемое его состояние. На рис. 3.15 они показаны в верхней части схемы. Существует несколько типов контроллеров:

1. Deployment (развертывание). Набор алгоритмов и директив, описывающих поды, число их экземпляров, порядок их замены при изменении характеристик. С его помощью разработчик декларирует изменения подов и наборов реплик. Это самый популярный способ разместить приложение в Kubernetes.
2. ReplicaSet (набор реплик). Описывает и управляет работой нескольких подов на кластере. Чем больше таких реплик, тем выше устойчивость системы к отказам и лучше масштабируются приложения.
3. StatefulSet (набор состояния). Контроллер применяется для управления приложениями с отслеживанием состояния (Stateful-приложениями) с использованием постоянного хранилища.
4. DaemonSet (набор «демонов»). Представляет собой совокупность «демонов» (фоновых программ, работающих без взаимодействия с пользователем) и отвечает за запуск одной реплики выбранного пода на каждом отдельном узле. То есть по мере добавления узлов в кластер добавляются и поды.
5. Job (задание). Контроллер, отвечающий за однократный запуск выбранного пода и завершающий его работу. CronJob – его разновидность, запускающая группу заданий по заданному расписанию.

Services (сервисы) – объединяют поды в логическую группу и определяют политику доступа к ним. По функционалу они напоминают маршрутизатор и балансировщик нагрузки между подами.

Поды могут пересоздаваться, уничтожаться, мигрировать на другие узлы кластера и изменять IP-адресацию. Если к поду требуется обращение извне, то обратившиеся приложения (или пользователь) должны знать состояние пода, его местонахождение, адрес и т. д., быть «осведомленными» о жизненном цикле пода, а это крайне неудобно. Поэтому в Kubernetes есть сервисы, которые «знают» количество подов, узлы, на которых они работают, и иные параметры

и отслеживают их жизненные циклы. Сервис получает запрос от внешних систем или сервисов, определяет, какому именно поду его адресовать, и направляет запрос к поду (рис. 3.18).



Рисунок 3.18 – Принцип работы сервисов Kubernetes

Persistent Volumes (постоянные тома) – это способ постоянного хранения данных контейнеров, который по сути своей схож с томами в Docker. Контейнеры в подах могут в любой момент быть уничтожены или перезапущены в соответствии с жизненным циклом, поэтому все постоянные данные должны храниться вне контейнера. Кроме того, необходим обмен данными между приложениями, нужна также синхронизация данных между репликами.

Для решения этих проблем в Kubernetes есть Persistent Volumes (PV). Это постоянные тома, жизненный цикл которых не связан с подами и контейнерами. Самостоятельные ресурсы создаются и управляются отдельно. PV не могут располагаться непосредственно на файловой системе конкретного узла – данные должны быть актуальны даже в случае перемещения пода с

контейнерами на другой узел. Чаще всего они предоставляются через сетевые файловые системы (NFS) или распределенные файловые системы (glusterFS, Luster и т. д.), а также сервисы объектного хранилища S3. Могут предоставляться через различные платформы контейнеризации, являющиеся обертками над системой оркестрации, которые предоставляют удобный пользовательский интерфейс управления системой оркестрации.

Namespaces (пространства имен) – это возможность разделить кластер Kubernetes на виртуальные пространства, каждое из которых изолировано от других. Обычно пространства имен создаются для того, чтобы разделить команды, разные проекты или среды развертывания.

Ресурсы в них скрыты друг от друга, но не изолированы полностью. Сервис из одного пространства может взаимодействовать с сервисом из другого. При необходимости можно разграничить доступ к пространствам имен либо полностью их изолировать.

В каждом Namespaces есть свои ресурсы (рис. 3.19): сервисы, поды, развертывания, контроллеры.

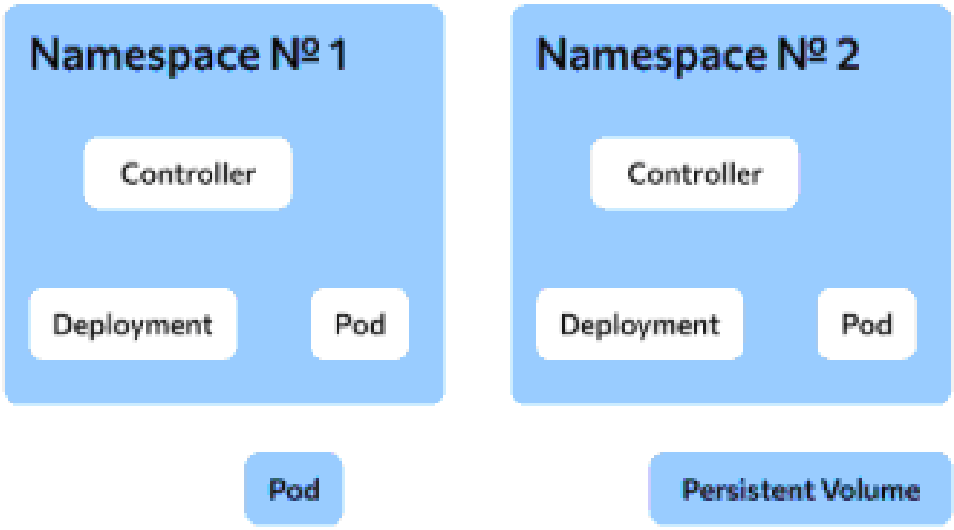


Рисунок 3.19 – Суть пространства имен Kubernetes

В одном пространстве имен они должны иметь уникальные названия, но эти же названия допустимо использовать в других пространствах. В Namespaces входят не все ресурсы: например, Persistent Volumes находятся вне пространств имен. Также допускается, что некоторые поды могут быть вне пространства имен.

Кроме вышеописанных основных сущностей, существуют еще различные дополнительные, которые сильно упрощают управление приложениями, например, ingress-контроллеры, операторы и т. д. С ними можно ознакомиться в официальной документации. Кроме того, кластер Kubernetes использует распределенное хранилище в формате ключ-значение – **etcd**, в котором хранит состояние кластера. Etcd отличается высокой надежностью и согласованностью и используется для хранения и управления критически важной информацией в Kubernetes.

Главная задача etcd – обеспечить отказоустойчивость кластера и консистентность данных. Etcd – самостоятельный проект. Он развивается отдельно от Kubernetes и применяется в разных продуктах, работает на master-нодах. Также зачастую вместе с кластером Kubernetes применяются такие программные решения, как система обнаружения сервисов (Service Discovery) на основе DNS и проверки их доступности – Consul и система безопасного хранения секретов Vault от компании Hashicorp.

Кроме того, существует более легкий аналог k8s – k3s, представляющий собой реализацию функционала Kubernetes в однонодовом режиме. В целом данное ПО вполне функционально и обладает всеми возможностями API Kubernetes. Также рекомендуется к использованию k9s – терминальный пользовательский интерфейс с открытым исходным кодом для управления и мониторинга кластеров Kubernetes, который подходит и для k3s.

3.7.3. Преимущества и недостатки Kubernetes

К преимуществам Kubernetes можно отнести его разнообразные возможности, такие как масштабируемость, отказоустойчивость, ускорение вывода продуктов на рынок, снижение затрат благодаря эффективному использованию ресурсов и автоматизации, а также портативность и мультиоблачность, что делает его универсальным решением для управления контейнеризированными приложениями. Отдельно стоит отметить активное сообщество и экосистему. Kubernetes поддерживается огромным сообществом разработчиков и компаний. Это означает, что всегда можно найти готовые решения, документацию и поддержку. Kubernetes также упрощает работу DevOps-команд, обеспечивает безопасность, предоставляет возможности для мониторинга и облегчает разработку благодаря унифицированной среде. Экосистема Kubernetes включает множество инструментов для мониторинга, CI/CD, поддержания безопасности и решения других задач.

Недостатки Kubernetes:

1. Сложность настройки и управления. Kubernetes – это мощная система, но ее освоение требует времени и усилий. Настройка кластера, конфигурирование объектов и управление ими может быть сложным, особенно для новичков (высокий порог вхождения и высокая стоимость специалистов, владеющих достаточным опытом). Для небольших проектов Kubernetes может оказаться избыточным решением. Работа с платформой даже сейчас, с обилием вспомогательных инструментов, остается достаточно сложной. Многие процессы необходимо настраивать вручную, сложно организована миграция между облаками и серверами. От команды требуется большой опыт даже для решения повседневных задач.
2. Высокие требования к инфраструктуре. Kubernetes требует значительных ресурсов для работы. Например, мастер-узлы (Control Planes) потребляют CPU и RAM даже в простое. Это может быть проблемой для маленьких команд или стартапов с ограниченным бюджетом.

3. Проблемы с отладкой и мониторингом. Хотя Kubernetes предоставляет множество инструментов для мониторинга, отладка проблем в кластере может быть сложной. Например, если Pod не запускается, нужно проверить логи, конфигурации, сетевые настройки и многое другое. Это требует глубокого понимания системы.

4. Риск переоценки технологии. Иногда компании внедряют Kubernetes только потому, что это модно, даже если их проект не требует такой сложной системы. Это может привести к лишним затратам времени и ресурсов. Например, для небольшого приложения с одним сервером использование Kubernetes может быть избыточным.

Kubernetes целесообразно использовать в следующих случаях:

– если есть сложное приложение с множеством микросервисов;

– когда нужно автоматизировать развертывание и масштабирование приложений и управление ими;

– когда инфраструктура распределена между несколькими серверами или облачными провайдерами;

– если приложение будет активно масштабироваться и должно быть отказоустойчивым.

От использования Kubernetes лучше воздержаться в следующих случаях:

1. Небольшие проекты. Для одного приложения, которое работает на одном сервере, Kubernetes может быть избыточным. Вместо этого можно использовать более простые инструменты, такие как Docker Swarm или даже Docker Compose.

2. Ограниченные ресурсы. Если нет достаточно мощных серверов или опыта работы с Kubernetes, внедрение этой технологии может стать головной болью. Лучше начать с чего-то более простого.

3. Короткие сроки разработки. Если нужно быстро запустить MVP (минимально жизнеспособный продукт), Kubernetes может замедлить процесс в силу своей сложности. В таких случаях лучше выбрать более простые решения.

3.8. Контрольные вопросы

1. Что такое командный проект?
2. В чем заключается задача планирования командного проекта?
3. Кто в проектной группе проводит оценку сложности реализации задач или трудозатрат по работам командного проекта?
4. Кто в проектной группе распределяет задачи между исполнителями командного проекта?
5. Перечислите и опишите основные инструменты разработки ИС.
6. Какие инструменты уместно использовать для разработки ИС с МСА?
7. Что такое контейнеризация и в чем ее назначение?
8. Какие инструменты используются для управления контейнеризацией?
9. В каких случаях целесообразно применение Kubernetes?
10. Как соотносятся ноды и поды в кластере Kubernetes?

4. Лабораторный практикум

Лабораторный практикум по курсу «DevOps-инженерия» предполагает выполнение семи лабораторных работ. Краткое содержание и график выполнения лабораторных работ представлены в табл. 4.1.

Таблица 4.1 – График выполнения лабораторных работ

№ л.р.	Тема и содержание работы	Неделя выполнения
1	Установка и настройка Linux (Debian 12). Настройка сети в Linux. Установка файлового менеджера MC. Работа в терминале через SSH. Работа с файлами и каталогами. Права доступа. Процессы. Bash.	1–3
2	Настройка среды разработки программного приложения: примерный стек: Python+Django+PostgreSQL.	4, 5
3	Работа с Docker Debian. Установка Docker и docker-compose	6, 7
4	Работа с Dockerfile и docker-compose. Создание образа и на его основе контейнера с использованием простого стартового образа Python: добавление файлов приложения Django в создаваемый образ.	8, 9
5	Тома, сети и переменные окружения Docker. Использование volumes. Получение данных темплейтов Django в Docker-контейнере из каталога на хост-машине.	10, 11
6	Оркестрация контейнеризированного приложения: установка и настройка системы оркестрации Kubernetes k3s, запуск приложения.	12, 13
7	Применение методологии CI/CD: автоматизированная сборка, упаковка и тестирование приложения, автоматизация развертывания в разных окружениях.	14–16

Использование операционной системы Linux является приоритетным ввиду адаптации к ней подавляющего большинства инструментов DevOps-инженерии (см. раздел 3). Остальные инструменты, используемые в лабораторных работах 2–7, являются рекомендуемыми. Это также относится к инструментам разработки приложений, рекомендуемых в лабораторной работе 2 (Python, Django, PostgreSQL). Что касается средств развертывания Docker и Kubernetes, то их рекомендация обусловлена широтой их использования и хорошим качеством документации. В любом случае студент может предложить свой сугубо индивидуальный стек технологий, обосновав использование каждого инструмента в отчете о выполнении соответствующей лабораторной работы. Успешная защита отчетов о выполнении всех работ дает основание для допуска к промежуточному контролю по дисциплине.

4.1. Лабораторная работа 1

Тема: Установка и настройка Linux.

Цель: Освоение техники установки ОС Linux и ее компонентов, изучение системных команд и команд работы с файлами и каталогами, распределение прав доступа в Linux и основы написание bash-скриптов. Освоение работы с текстовыми файлами, процессами.

4.1.1. Описание используемых команд Linux

Системные команды

su – изменить пользователя или стать суперюзером (root);

man – справка по командам Linux.

Пример использования: `man uname`

cat/etc/os-release – позволяет получить информацию об операционной системе.

Пример использования: простой скрипт – получить в переменную e имя ОС, в переменную f – версию ОС и вывести:

```
$e = cat /etc/os-release | grep "Name"
$f = cat /etc/os-release | grep "Version"
echo $e - $f
```

uname – утилита Unix Name, позволяет получить информацию о ядре операционной системы, версии ядра.

Пример использования: uname -r

free – просмотр свободной и используемой памяти в системе, по умолчанию значение в килобайтах.

Пример использования команды показан на рис. 4.1. Free – h в мегабайтах/гигабайтах – удобнее.

```
root@debian:~# free
               total        used        free      shared  buff/cache   available
Mem:         1020440       38792      908196        1736       73452      872648
Swap:        1046524           0     1046524
```

Рисунок 4.1 – Пример использования команды free

uptime – показывает время работы системы.

Пример использования:

```
root@debian:~# uptime
 07:46:06 up 20 min, 2 users,  load average: 0.00, 0.00, 0.00
```

Команды работы с файлами и каталогами

`ls` – просмотр содержимого каталога

Пример использования: вывод списка файлов в каталоге /var

```
root@debian:~# ls -l /var
total 36
drwxr-xr-x  2 root root  4096 Feb 13 07:24 backups
drwxr-xr-x  7 root root  4096 Feb 13 07:11 cache
drwxr-xr-x 25 root root  4096 Feb 13 07:12 lib
drwxrwsr-x  2 root staff 4096 Nov 19 10:25 local
lrwxrwxrwx  1 root root     9 Feb 13 07:02 lock -> /run/lock
drwxr-xr-x  5 root root  4096 Feb 13 07:13 log
drwxrwsr-x  2 root mail  4096 Feb 13 07:02 mail
drwxr-xr-x  2 root root  4096 Feb 13 07:02 opt
lrwxrwxrwx  1 root root     4 Feb 13 07:02 run -> /run
drwxr-xr-x  6 root root  4096 Feb 13 07:11 spool
drwxrwxrwt  3 root root  4096 Feb 13 07:26 tmp
```

`mkdir` – создание каталога;

`rmdir` – удаление пустого каталога;

`rm` – удаление файла или каталога (данная команда не предусматривает корзины. Под root позволено удалять ВСЕ вплоть до системных и конфигурационных файлов);

`touch` – создание файла;

`cd` – смена текущего каталога;

`find` – поиск файлов.

Пример использования: найти все файлы, содержащие «log», в каталоге /srv:

```
root@debian:/srv# find /srv -type f -name '*log*'
/srv/domains/mysite.com/logs/apache.log
/srv/domains/mysite.com/logs/kern.log
/srv/domains/mysite.com/logs/alternatives.log
/srv/domains/mysite.com/logs/syslog
/srv/domains/mysite.com/logs/auth.log
/srv/domains/mysite.com/logs/daemon.log
/srv/domains/mysite.com/logs/lastlog
/srv/domains/mysite.com/logs/dpkg.log
/srv/domains/mysite.com/logs/faillog
```

`cp` – копирование файлов;

`mv` – перемещение файлов.

Системные команды для настройки Linux

`useradd` – добавление пользователя;

`deluser` – удаление пользователя;

`chmod` – изменение прав;

`chown` – изменение владельца.

Команды работы с текстовыми файлами

`cat` – вывод текстового файла;

`tac` – обратный вывод текстового файла;

`more` – просмотр текстового файла;

`less` – просмотр текстового файла;

`head` – вывод первых строк файла;

`tail` – вывод последних строк файла;

`wc` – подсчет количества символов, строк, слов;

`grep` – фильтрация текста.

Команды работы с процессами

`kill` – передача сигнала процессу;

`nice` – запуск процесса с выставлением приоритета;

`renice` – позволяет изменить приоритет уже запущенному процессу;

`ps` – список процессов.

Операторы перенаправления

Операторы перенаправления способны изменять направление вывода и ввода информации. Так, оператор:

> – перенаправляет стандартный поток в файл (другой поток). При этом если файл существует, то он перезаписывается, если не существует – создается;

>> – перенаправляет стандартный поток в файл. При этом если файл существует, то информация добавляется в конец, если не существует – файл создается;

< – перенаправляет содержимое указанного файла на стандартный ввод программе;

>& – перенаправляет стандартные потоки вывода и ошибок друг в друга.

Программные каналы

Конвейер (Труба, Pipe, Канал) – программный интерфейс, позволяющий процессам обмениваться данными (односторонний поток).

Организацией канала занимается shell. Для управления каналом существует оператор “|”.

Например:

```
root@debian:~/lesson2# cat test2.txt | tail -n 2 | less
```

Скрипты

Скрипт fileexists.sh проверяет существование файла, переданного в параметре, выводит результат на экран и логирует в файл Logone.txt.

Для запуска необходимо дать права на исполнение скрипта, и вызвать его можно, например, так:

```
./fileexists.sh /var/log/auth.log
```

```
#!/bin/bash

#set filename to parameter 1
filename=$1

#log filename
log_file="Logone.txt"

#check file exists
if [[ -f "$filename" ]]; then
    result="File $filename exists"
else
    result="argument error. File $filename not found"
fi

# result output
echo $result

# result output to log file
echo $result >> ${log_file}
```

4.1.2. Настройка сети и установка дополнительного ПО

Сеть

По умолчанию для настройки сети в ОС Дебиан используется *ifupdown*, конфигурационный файл которого находится в /etc/network/interfaces. Примерная структура файла:

```
# This file describes the network interfaces available on
your system
# and how to activate them. For more information, see
interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
auto ens192
iface ens192 inet static
address 10.0.16.67/26
     gateway 10.0.16.65
```

Как видно из приведенного фрагмента, описываются два сетевых интерфейса – интерфейс локальной петли (lo) и основной сетевой интерфейс (ens192). При необходимости настройки адреса основного интерфейса с использованием DHCP необходимо описать настройки следующим образом:

```
# This file describes the network interfaces available on
your system
# and how to activate them. For more information, see
interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
auto ens192
iface ens192 inet dhcp
```

Выполнить настройку сети в соответствии с вашей конфигурацией. Проверить используемый адрес командой ip a

Для настройки указания используемых ДНС-серверов используется файл /etc/resolv.conf – примерное содержание файла следующее:

```
nameserver 77.8.8.8
nameserver 8.8.8.8
```

В данном примере производятся настройки использования двух ДНС-серверов – яндекс и гугл. Необходимо настроить свою систему на использование ДНС-серверов. После настройки проверить работоспособность командой пинг:

```
# ping www.ya.ru
```

В случае успешной настройки будут получены ответы (прервать – сочетание клавиш CTRL+C):

```
64 bytes from ya.ru (77.88.44.242): icmp_seq=1 ttl=56 time=4.54 ms
64 bytes from ya.ru (77.88.44.242): icmp_seq=2 ttl=56 time=4.24 ms
64 bytes from ya.ru (77.88.44.242): icmp_seq=3 ttl=56 time=4.24 ms
64 bytes from ya.ru (77.88.44.242): icmp_seq=4 ttl=56 time=4.27 ms
64 bytes from ya.ru (77.88.44.242): icmp_seq=5 ttl=56 time=4.27 ms
64 bytes from ya.ru (77.88.44.242): icmp_seq=6 ttl=56 time=4.25 ms
```

После этого можно командами exit (два раза) выйти из терминала и подключиться через SSH. SSH, или Secure Shell, это сетевой протокол, предназначенный для безопасного удаленного доступа к компьютерам и серверам. Он обеспечивает защищенное соединение для выполнения команд, передачи файлов и других операций, обычно используемых в администрировании. SSH шифрует весь трафик, включая логины и пароли, что делает его безопасной альтернативой менее защищенным протоколам, таким как Telnet. Для использования SSH необходима программа-клиент, для Windows-систем самый распространенный клиент – putty[3].

После скачивания и установки запустите клиент ssh и введите в поля формы (рис. 4.2) назначенный вами ip-адрес, нажмите open.

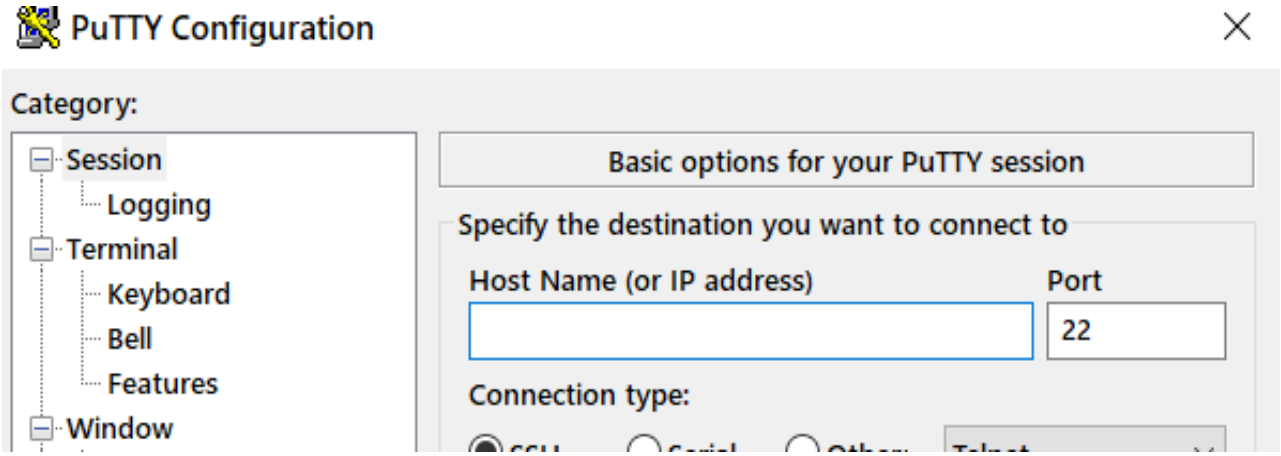


Рисунок 4.2 – Графический интерфейс putty

Войдите в сеанс, используя указанные при установке учетные данные, командой su – переключитесь в режим админа.

Можно приступать к установке ПО.

Работа с пакетами

`apt (advanced packaging tool)` – программа для установки, обновления и удаления программных пакетов в операционных системах Debian и основанных на них (Ubuntu, Linux Mint и т. п.), иногда также используется в дистрибутивах, основанных на Mandrake, например Mandriva, ALTLinux и PCLinuxOS

`apt install имя пакета` – установка пакета
`apt update` – обновление списка пакетов
`apt-cache search запрос` – поиск по пакетам

[3] Скачать дистрибутив данной свободно распространяемой утилиты можно бесплатно по ссылке: https://www.chiark.greenend.org.uk/~sgtatham/putty/latest.html

Для того, чтобы скачивать и устанавливать пакеты, необходимо указать используемые репозитории. Для Debian 12 приведите файл /etc/apt/sources.list к указанному ниже виду:

```
deb https://ftp.debian.org/debian/ bookworm contrib main non-free non-free-firmware
deb https://ftp.debian.org/debian/ bookworm-updates contrib main non-free non-free-firmware
deb https://ftp.debian.org/debian/ bookworm-backports contrib main non-free non-free-firmware
deb https://security.debian.org/debian-security/ bookworm-security contrib main non-free non-free-firmware
```

Далее для обновления кэша пакетов выполнить команду `apt update`. Для обновления всех пакетов в системе можно выполнить команду `apt full-upgreade`. Установить вспомогательное ПО:

mc – Midnight Commander – это консольный файловый менеджер для Unix-подобных операционных систем, таких как Linux. Он предоставляет пользователям удобный интерфейс для управления файлами и каталогами, выполняя основные операции, такие как копирование, перемещение, переименование, удаление, а также просмотр и редактирование файлов. MC также поддерживает работу с удаленными системами через FTP и SSH.

sudo – позволяет пользователю выполнять команды от имени другого пользователя, обычно суперпользователя (root), с использованием своих собственных учетных данных. Она предоставляет временное повышение привилегий для выполнения конкретной команды, в отличие от команды su, которая переключает пользователя на весь сеанс работы.

git – консольная утилита для управления системой контроля версий.

curl (client URL) – это утилита командной строки, предназначенная для передачи данных по различным протоколам (HTTP, HTTPS, FTP, SMTP и другим). Проще говоря, это инструмент, который позволяет отправлять запросы к серверам и получать ответы, как если бы это делал браузер, но без графического интерфейса.

Выполнить команду и дождаться установки пакетов:

```
apt install mc sudo git curl.
```

После установки следует внести пользователя, указанного при установке, в sudoers:

```
usermod –aG  sudo <имя пользователя>.
```

После этого можно выполнять команды, требующие повышения прав от обычного пользователя следующим образом:

```
$ sudo <команда>.
```

4.1.3. Задание

1. Установить в VirtualBox дистрибутив Debian 12[4]. Тип виртуального сетевого адаптера выбрать «Сетевой мост» (см. рис. 4.3).

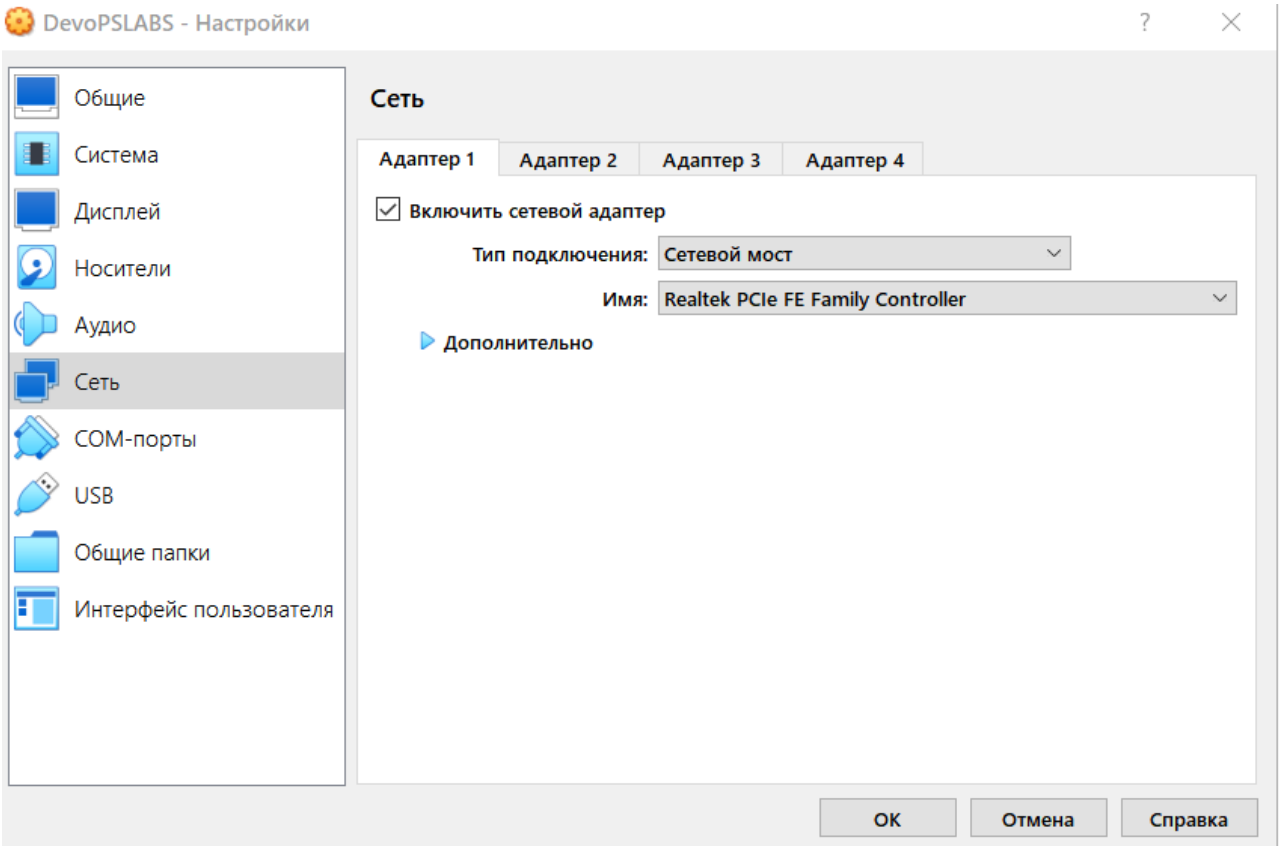


Рисунок 4.3 – Настройка параметров сети

При установке выбрать Install. Настройки сети выбрать по DHCP. Разметить диск автоматически с использованием LVM. Включить внешние источники установки (репозитории). Из программного обеспечения установить только SSH Server и Standard system utilities (см. рис. 4.4).

После установки ОС войдите в терминал под указанным в процессе установке пользователем, перейдите в режим администратора командой `su -`.

2. Выполнить следующие действия:
2.1. Авторизоваться в системе с использованием ssh.
2.2. Выполнить установку пакета логирования rsyslog
`apt install rsyslog`.
2.3. Вывести тип операционной системы, версию ядра.
2.4. Вывести объем свободной памяти в системе.
2.5. Вывести время работы системы.
2.6. Создать следующую структуру каталогов и файлов в /srv:

[4] Скачать с https://cdimage.debian.org/debian-cd/current/amd64/iso-dvd/

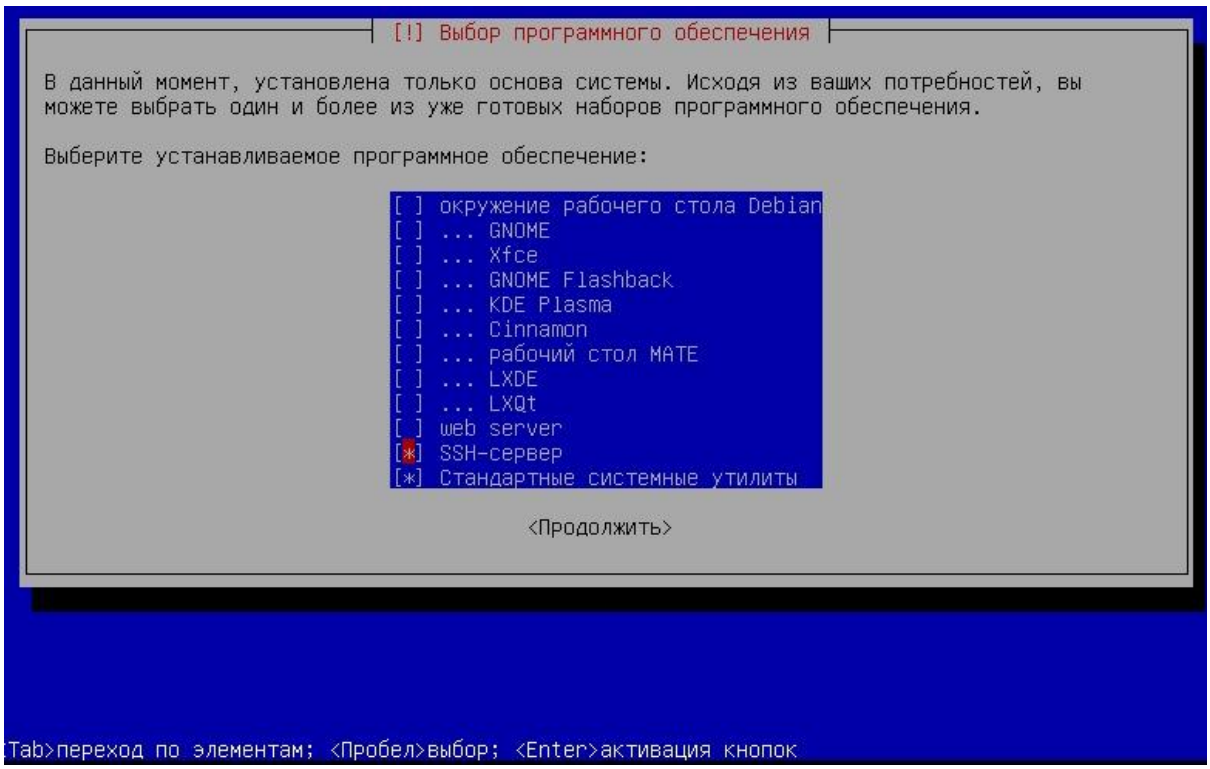


Рисунок 4.4 – Установка ПО для ВМ

```
/srv
/srv/domains
/srv/domains/mysite.com
/srv/domains/mysite.com/html
/srv/domains/mysite.com/html/site.html
/srv/domains/mysite.com/logs
/srv/domains/mysite.com/logs/apache.log
```

2.7. Скопировать все файла из каталога /var/log/ в каталог /srv/domains/mysite.com/logs/.

3. Написать команду, которая удалит все файлы из каталога /srv, не трогая структуру каталога.

4. Создать пользователя [username], где username – фамилия студента в латинской транскрипции, добавить в sudoers.

5. Вывести на экран содержимое файлов /etc/passwd , /etc/group , /etc/shadow, /etc/gshadow.

6. Выполнить следующие задания:

6.1. Создать в домашней папке каталог lab<Nвар> (например, для варианта №2: lab2).

6.2. Изменить владельца на созданного пользователя.

6.3. Установить права:

- владелец – чтение, запись, выполнение;
- группа – чтение и запись;
- остальные – только чтение.

7. Выполнить следующие задания:

7.1. Создать в домашней папке пользователя файл lab<Nвар>/lab<Nвар>.txt (например, для варианта №2: lab2/lab2.txt).

7.2. Изменить владельца на созданного пользователя.

7.3. Установить права:

- владелец – чтение, запись, выполнение;
- группа – чтение и запись;
- остальные – запретить доступ.

8. Вывести последние [Nвар * 2] строк файла /var/log/auth.log.

9. Вывести первые [Nвар * 3] строк файла /var/log/syslog.

10. В домашнем каталоге пользователя с помощью программных каналов, перенаправления и команды grep создать:

10.1. файл auth.log, содержащий все строки файла /var/log/auth.log, где встречается слово pam;

10.2. файл syslog, содержащий все строки файла /var/log/syslog, где встречается слово cron.

11. С помощью wc посчитать количество строк файла /var/log/auth.log, содержащих слово root.

12. Вывести все процессы, запущенные в системе, командой ps aux и написать, что означает каждое из выводимых полей.

13. Запустить в фоновом режиме поиск всех файлов в корневой директории, содержащих в своем имени «log». Результат направить в домашней папке в файл find_output.txt.

14. Написать скрипт для выполнения действий с файлами. Если значение параметров задано неверно, сообщить об ошибке.

Задания на написание скрипта (номер задания вычислить как остаток от деления Nвар на 10, увеличенный на 1):

1. Написать скрипт, который находит в каталоге, указанном в параметре, все файлы, в тексте которых есть подстрока «Bot».

2. Написать скрипт, уничтожающий процессы, заданные в параметре скрипта, и убедиться, что они удалились.

3. Написать скрипт, который ищет в каталоге /var/log все файлы, содержащие строку, переданную в параметре.

4. Написать скрипт, который выводит количество строк в файле, переданном в параметре 1, не содержащих значение, переданное в параметре 2.

5. Написать скрипт, который выводит количество строк в файле, переданном в параметре 2, содержащих значение, переданное в параметре 1.

6. Написать скрипт, который ищет в каталоге, переданном в параметре 1, все файлы, содержащие строку, переданную в параметре 2.

7. Написать скрипт, который считает количество процессов, содержащих в имени команды значение, переданное в параметре.

8. Написать скрипт, который выводит все процессы, содержащие в имени команды значение, переданное в параметре.

9. Написать скрипт, который создает файл, переданный в параметре 1, в каталоге, указанном в параметре 2, и ставит на него права: 740.

10. Написать скрипт, изменяющий приоритет, процесса идентификатор которого передан в параметре 1, на значение, переданное в параметре 2.

4.1.4. Содержание отчета

- тема, цель, задание (без рисунков);
- описание выполненных действий;
- выполнение контрольного задания (по номеру варианта);
- скриншоты результатов вывода команд;
- скриншот сетевых настроек;
- скриншоты вывода команд:

```
apt policy git
apt policy mc
apt policy sudo
apt policy curl
```

- выводы по работе.

4.1.5. Работа с Git

1. В GitLab (https://pa.k-lab.su) запросить создание персональной учетной записи.
2. После одобрения администратора создать ПУБЛИЧНЫЙ репозиторий с названием labs.
3. В локальной системе создать папку LABS. В ней создать инициирующий коммит в репозиторий (использовать ПО для работы с Git, например, GitHub Desktop).
4. Создать папку LAB1, в которую выложить отчет о выполнении лабораторной работы.
5. Выполнить коммит изменений в локальном репозитории, выполнить push в удаленный репозиторий.

ВНИМАНИЕ! ВСЕ КОММИТЫ В РЕПОЗИТОРИЙ ЗДЕСЬ И В ДАЛЬНЕЙШЕМ ПРОИЗВОДЯТСЯ В ВЕТКУ MAIN.

В ЭИОС вместе с отчетом в качестве комментария загрузить ссылку на репозиторий в формате https://[адрес]/[имя пользователя]/labs.git.

4.1.6. Контрольные вопросы

1. Что хранится в каталоге /etc?
2. Какой командой можно удалить не пустой каталог?
3. Найти в системе все файлы, имя которых начинается на «begin»?
4. Какой параметр uname позволяет вывести всю информацию о системе?
5. Как с помощью free посмотреть информацию о памяти в мегабайтах?

4.2. Лабораторная работа 2

Тема: Настройка среды для создания web-приложения.

Цель: Освоение навыков установки и настройки СУБД и средств разработки приложения на примере СУБД PostgreSQL и фреймворка Django.

4.2.1. Установка СУБД PostgreSQL

```
apt update && apt upgrade
apt install postgresql-17 # - на момент написания актуальная версия 17
```

После установки Postgresql выполнить настройки:

1. Задать нового пользователя СУБД:

Перейти в оболочку пользователя postgres

```
su – postgres.
```

Запустить psql, как показано на рис. 4.5.

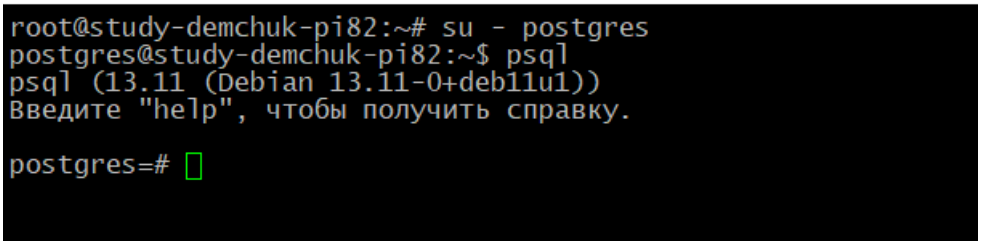


Рисунок 4.5 – Запуск утилиты psql

Далее создать пользователя:

```
CREATE USER userdb WITH PASSWORD 'password';.
```

Вместо password указать свой пароль. Далее создать новую базу данных:

```
CREATE DATABASE study;.
```

Дать полные привилегии пользователю:

```
GRANT ALL PRIVILEGES ON DATABASE "study" to userdb;.
```

Задать владельца:

```
ALTER DATABASE study OWNER TO userdb;.
```

Отработка указанных команд показана на рис. 4.6.

Выход из psql - \q, выход из оболочки пользователя postgres – `exit`.

Далее необходимо настроить доступы и разрешения в файле pg_hba.conf

```
cd /etc/postgres ql/номер версии/main
nano pg_hba.conf.
```

```
postgres=# CREATE USER userdb WITH PASSWORD '12345678';
CREATE ROLE
postgres=# CREATE DATABASE study;
CREATE DATABASE
postgres=# GRANT ALL PRIVILEGES ON DATABASE "study" to userdb;
GRANT
postgres=#
```

Рисунок 4.6 – Создание и настройка БД

Привести файл к следующему виду:

```
# Database administrative login by Unix domain socket
local   all             postgres                                peer
local   study            userdb                             scram-sha-
256
#  TYPE    DATABASE               USER                        ADDRESS
METHOD

# "local" is for Unix domain socket connections only
local          all                                              all
peer
# IPv4 local connections:
host    all             all             127.0.0.1/32    scram-sha-
256
host    study            userdb          127.0.0.1/32    scram-sha-
256
# IPv6 local connections:
host    all             all             ::1/128         scram-sha-
256
# Allow replication connections from localhost, by a user with the
# replication privilege.
local                  replication                              all
peer
host    replication     all             127.0.0.1/32    scram-sha-
256
host    replication     all              ::1/128        scram-sha-
256
```

Это позволит подключаться к СУБД созданному пользователю.

Сохранить файл и перезапустить postgtres:

```
systemctl restart postgresql.
```

Проверить прослушиваемые порты (рис. 4.7). Если прослушивается порт 5432, то все сделано успешно.

```
root@study-demchuk-pi82:/etc/postgresql/13/main# ss -tlpn
State              Recv-Q             Send-Q                        Local Address:Port
LISTEN             0                  128                                 0.0.0.0:22
LISTEN             0                  244                           127.0.0.1:5432
LISTEN             0                  128                                 [::]:22
LISTEN             0                  244                               [::1]:5432
root@study-demchuk-pi82:/etc/postgresql/13/main#
```

Рисунок 4.7 – Проверка портов

Далее можно подключиться в psql с помощью вновь созданного пользователя к разрешенной ему БД:
`psql -U userdb study`.

Отклик должен быть таким, как показано на рис. 4.8.

```
root@study-demchuk-pi82:/etc/postgresql/13/main# psql -U userdb study
Пароль пользователя userdb:
psql (13.11 (Debian 13.11-0+deb11u1))
Введите "help", чтобы получить справку.

study=>
```

Рисунок 4.8 – Подключение к psql

4.2.2. Установка фреймворка Django

Установить фреймворк Django можно непосредственно из настраиваемой среды:

1. Установить пакет PIP Python3:
`apt install python3-pip – будет устновлен PIP и зависимости`.

2. Обновить pip:
`pip3 install --upgrade pip`.

3. Установить виртуальную среду.

Виртуальная среда, или venv, не является неотъемлемой частью разработки на Django. Однако ее рекомендуется использовать, так как она позволяет создать множество виртуальных сред Python в одной операционной системе. Благодаря виртуальной среде приложение может запускаться независимо от других приложений на Python.

Можно запускать приложения на Django и без виртуальной среды. В этом случае все пакеты Django устанавливаются глобально. Однако что если после создания первого приложения выйдет новая версия Django? Если для второго проекта возникнет необходимость использовать новую версию Django, то из-за глобальной установки пакетов придется обновлять первый проект, который использует старую версию. Это потребует некоторой дополнительной работы по обновлению, так как не всегда соблюдается обратная совместимость между пакетами. Если же использовать для второго проекта старую версию, то это лишает его потенциальных преимуществ новой версии Django. Использование виртуальной среды как раз позволяет разграничить пакеты для каждого проекта.

```
cd /
mkdir django-apps
```

```
cd  / django-apps
apt install python3-venv
```

Создать виртуальную среду в каталоге *django-apps* и присвоить ей общее имя проекта:
`virtualenv env`.

Здесь *env* – общее имя среды. Можно использовать другое имя, имеющее значение для проекта. В результате в текущем каталоге django-apps появится соответствующая папка (рис. 4.9).

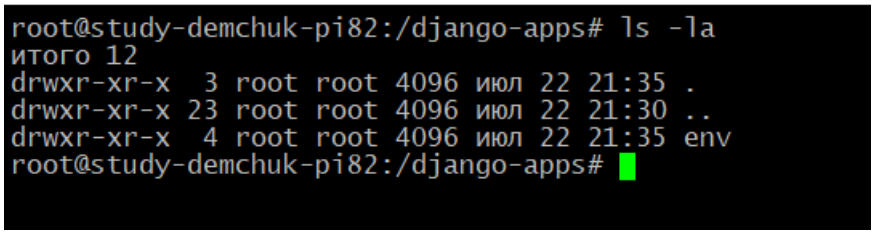


Рисунок 4.9 – Результат создания виртуальной среды

Далее следует активировать виртуальную среду с помощью следующей команды:
`. env/bin/activate`.

Точка перед командой означает – в текущем каталоге. Данную команду выполнять каждый раз после выхода из виртуальной среды при повторном запуске приложения. После активации префикс изменится на (env), что будет выглядеть примерно так, как показано на рис. 4.10, в зависимости от текущего каталога:

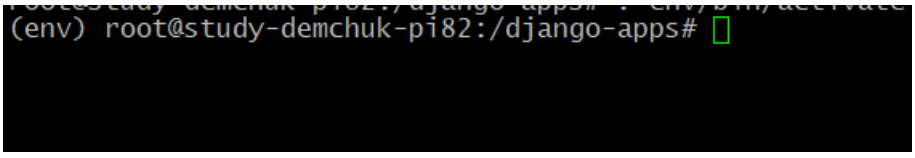


Рисунок 4.10 – Результат активации Django

Все дальнейшие действия выполнять в каталоге с активированной виртуальной средой!

4. Установить пакет Django в этой среде с помощью pip. Установка Django позволит создавать и запускать приложения Django, находясь в виртуальной среде:
`pip3 install django`.

После установки проверить версию продукта, как показано на рис. 4.11.

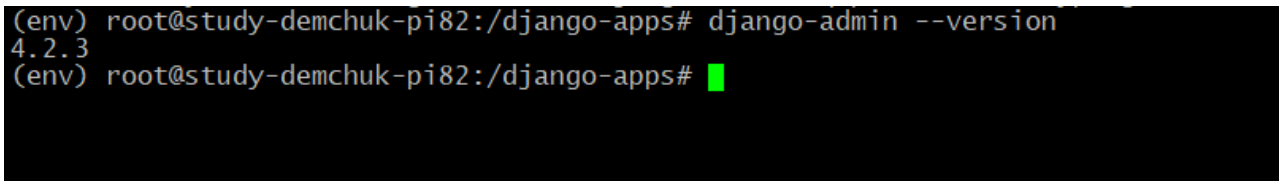


Рисунок 4.11 – Проверка версии Django

Установить библиотеку для работы с СУБД (в данном случае – PosgreSQL):

```
pip3 install psycopg2-binary.
```

Установить библиотеку запросов:

```
pip3 install requests.
```

Теперь можно сгенерировать приложение, используя утилиту командной строки *django-admin*, предназначенную для выполнения административных задач Python. Затем с использованием команды `startproject` создать структуру каталогов проекта для тестового сайта. Запустить следующую команду в каталоге *django-apps*:

```
django-admin startproject myfirstproj.
```

Появится папка с названием проекта (рис. 4.12).

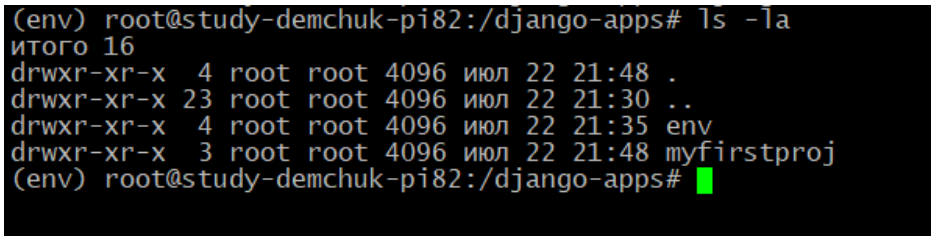


Рисунок 4.12 – Каталожная структура проекта Django

Выполнить некоторые настройки[5]:

```
nano myfirstproj/myfirstproj/settings.py
```

`ALLOWED_HOSTS = ['*']` – привести к такому виду, это позволит получить доступ к web-серверу фреймворка с любых хостов.

Словарь `DATABASE` привести к следующему виду:

```
DATABASES = {
    'default': {
        'ENGINE':
'django.db.backends.postgresql_psycopg2',
        'NAME': 'study',
```

[5] Все действия выполнять под пользователем root.

```
        'USER': 'userdb',
        'PASSWORD': '12345678',
        'HOST': '127.0.0.1',
        'PORT': '5432',
    }
}
```

где указать свои параметры подключения к БД. Выполнить миграции в БД (так как миграции подготовлены, то выполнять makemigrations не требуется): `python3 ./myfirstproj/manage.py migrate`.

Если все настроено верно, то в БД будут добавлены новые таблицы (рис. 4.13).

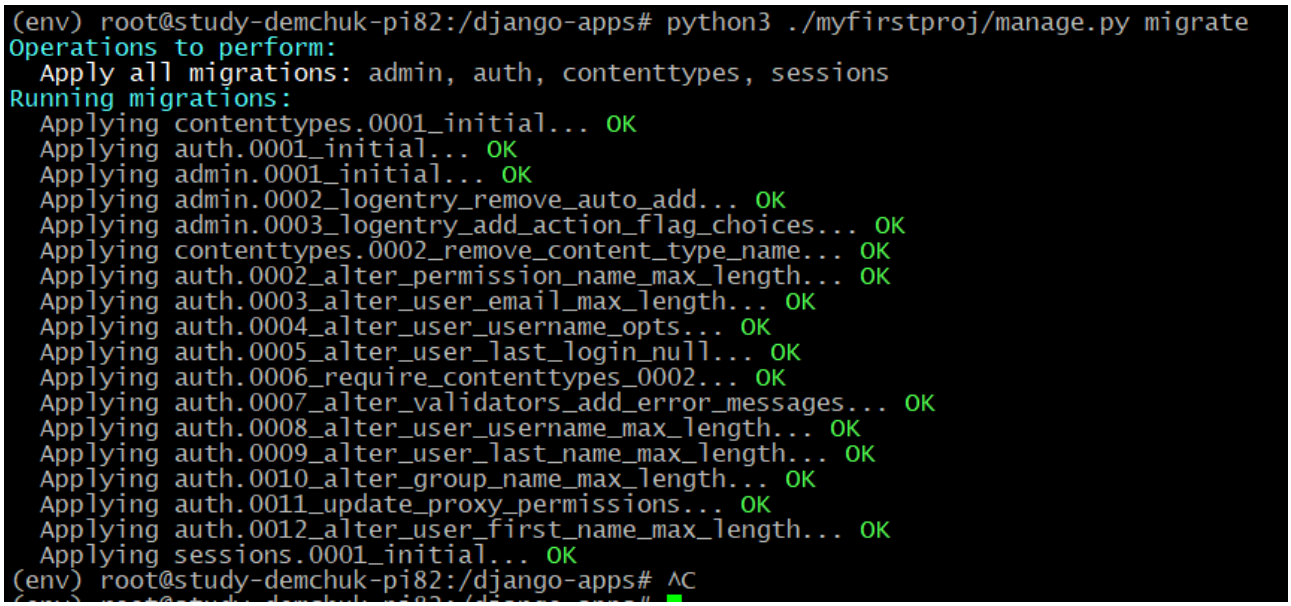


Рисунок 4.13 – Результат добавления таблиц в БД

Создать суперпользователя для администрирования:
`python3 ./myfirstproj/manage.py createsuperuser`

Будет запрошена информация и создан пароль. После этого можно запустить WEB-сервер фреймворка и получить доступ в админпанель Django `python3 ./myfirstproj/manage.py runserver 0.0.0.0:8000`.

Веб-сервер продолжит работу до нажатия сочетания Ctrl+C.

С рабочего места, с которого доступна ВМ по IP адресу, ввести в браузере:
`http://[ip-адрес ВМ]:8000/admin`.

Результат должен быть таким, как показано на рис. 4.14.

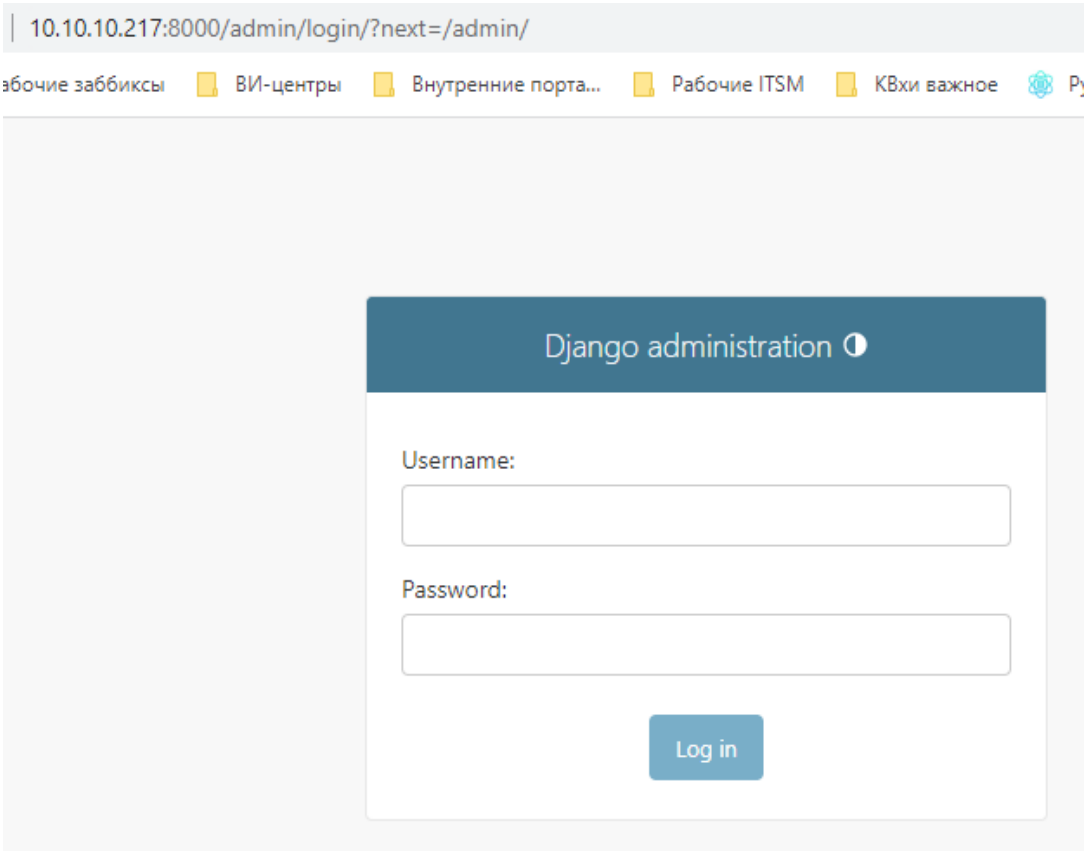


Рисунок 4.14 – Запущенный веб-сервер

Ввести логин и пароль ранее созданного пользователя и войти в админпанель (рис. 4.15).

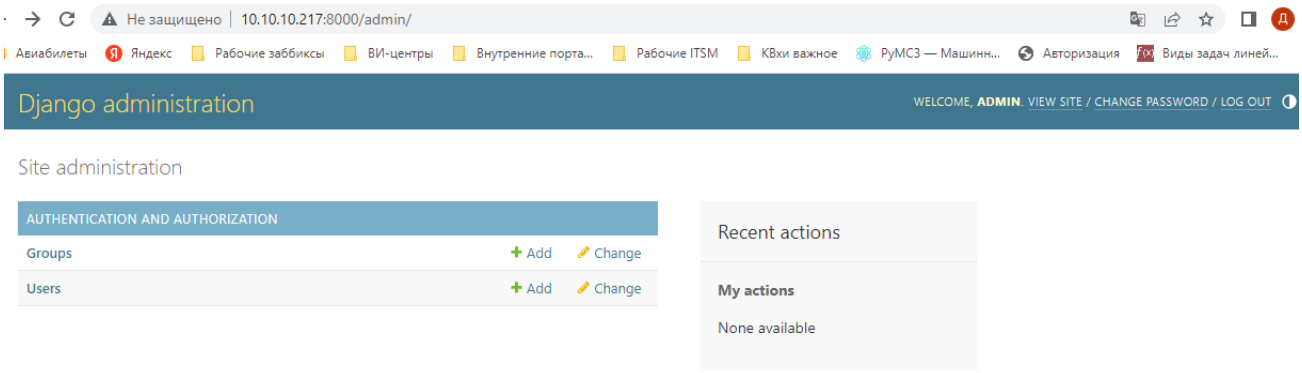


Рисунок 4.15 – Админпанель Django

Остановить WEB-сервер.

4.2.3. Создание приложения Django

Для создания приложения необходимо:

1. Перейти в той же папке с виртуальной средой в папку myfirstproj(`cd myfirstproj`) и создать приложение:

```
python3 manage.py startapp myapp.
```

При этом в папке проекта появится папка приложения (рис. 4.16).

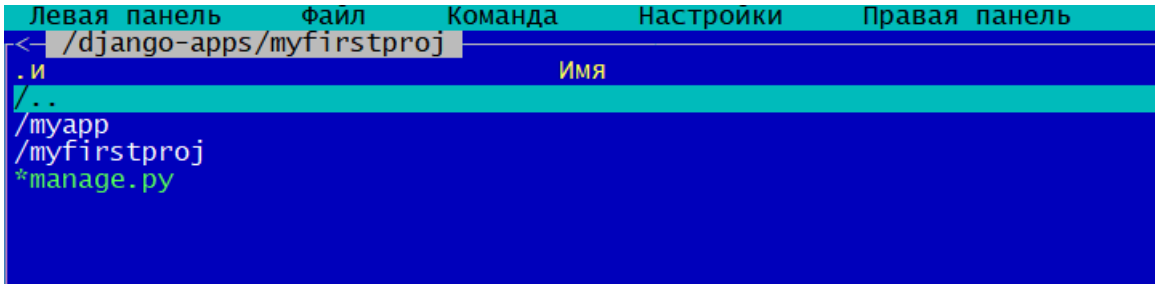


Рисунок 4.16 – Созданная директория приложения Django

2. В папке с приложением создать файл urls.py. Он будет определять все пути в приложении. Пока создать только основной индексный путь:

```
from django.contrib import admin
from django.urls import path, include, re_path
from . import views

urlpatterns = [
    path(r'', views.index, name='index'),
]
```

Строка означает, что url по умолчанию будет обработан представлением (функцией *index*).

3. В глобальном файле urls.py проекта создать путь к приложению:

```
nano ./myfirstproj/myfirstproj/urls.py.
```

4. Добавить указание на приложение и импортировать модули. Привести файл к следующему виду[6]:

```
from django.contrib import admin
from django.urls import path, include, re_path

urlpatterns = [
    path('admin/', admin.site.urls),
    path(r'myapp/',include('myapp.urls')),
]
```

Как результат, добавлена ссылка на приложение админ-панель.

Строка `path(r'myapp/',include('myapp.urls'))` указывает, что все url после `fqdn`, начинающиеся с `/myapp`, ведут в файл *urls.py* приложения *myapp*.

[6] Запись в файл возможна только под root.

5. Далее поправить настройки в *settings.py*:

```
nano ./myfirstproj/myfirstproj/settings.py.
```

Внести в списки приложений созданное приложение:

```
INSTALLED_APPS = [
    'myapp',
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
]
```

6. Перейти в папку с новым приложением:

```
cd ./myfirstproj/myapp/.
```

7. Создать в ней папку для шаблонов (название именно такое):

```
mkdir templates.
```

8. Написать первое представление. Открыть файл views.py.

Приведем файл к следующему виду (**все отступы ставить ТОЛЬКО пробелами, рекомендуется использовать 4 пробела на каждый уровень отступа. Python 3 запрещает смешивание табуляции и пробелов в отступах**):

```
from django.shortcuts import render
from django.http import HttpResponse
import requests

def index(request):
    a=9**0.5
    x_forwarded_for                                    =
request.META.get('HTTP_X_FORWARDED_FOR')
    if x_forwarded_for:
        ip = x_forwarded_for.split(',')[0]
    else:
        ip = request.META.get('REMOTE_ADDR')
    bc={}
    bc['sqr']=a
    bc['ipp']=ip
    return render(request, 'index.html',bc)
```

Итак, создана функция, в которую передается http-запрос (в дальнейшем из него возможно получить данные).

Для начала определяется IP-адрес посетителя.

Получается корень из 9 и добавляется в словарь.

Добавляется в словарь IP-адрес.

Далее передается рендеру реквест, имя шаблона, в который попадут данные и словарь.

9. Теперь в папке templates создать шаблон index.html:

```
<center><H1> The sqrt is {{sqr}} </H1></center></br>
<center><H1> Your IP is:  {{ipp}} </H1></center></br>,
```

где `{{sqr}} {{ipp}}` – передаваемые параметры.

Находясь в папке *myfirstproj* (там где файл *manage.py*), запустить сервер фреймворка:

```
python3 manage.py runserver 0.0.0.0:8000.
```

Если ошибки не допущены, то вывод будет таким, как показано на рис. 4.17.

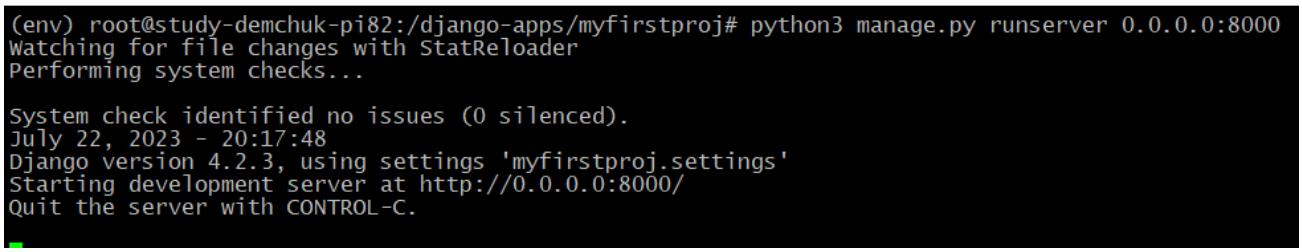


Рисунок 4.17 – Результат запуска приложения Django в командной строке

Проверка:

```
http://[ip-адрес ВМ]:8000/myapp.
```

Результат запуска приложения показан на рис. 4.18.

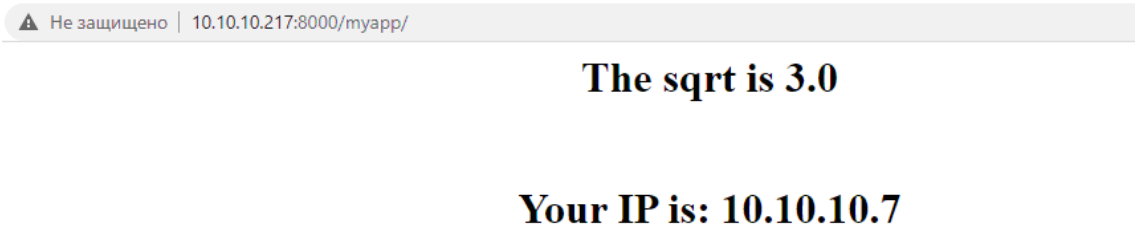


Рисунок 4.18 – Результат запуска приложения Django в браузере

В дальнейшем предлагается самостоятельно разобраться с созданием таблиц БД через описание классов в файле models.py, подготовками и выполнением миграций, а также с добавлением таблиц в админ-панель через запись в файле admin.py. Предлагаемый материал: https://metanit.com/python/django/5.1.php.

4.2.4. Задание

1. Согласно индивидуальному заданию на разработку ИС сформулировать ее название, разработать функциональные требования и требования к обеспечению ИС. Ограничений к инструментам разработки, СУБД не предъявляется, фреймворк Django и СУБД PostgreSQL приведены в качестве примера.

2. Согласно потребностям индивидуальной работы установить СУБД и фреймворк (или WEB-сервер с PHP). Виртуальную среду создавать с уникальным именем проекта.

3. Создать приложение, реализующее 1-2 базовых функциональных требований из числа заявленных ранее. Это может быть, например, разработка страницы для парсинга сайта или для пользовательского внесения справочных данных в БД. Главное требование: страницы должны быть динамические.

4. В gitlab создать инициализирующий коммит. Разместить в репозитории созданный проект приложения.

```
git clone URL репозитория (.git)
git add .
git commit -m "kakoy-to commit" && git branch -M main
git push -u origin main
```

4.2.5. Содержание отчета

- спецификация требований к ИС;
- краткие комментарии, отражающие ход разработки приложения (не более 2000 слов);
- ссылка на репозиторий с проектом ИС;
- скриншот развернутого репозитория.

4.2.6. Работа с Git

1. В ранее созданном локальном репозитории LABS создать папку LAB2, в которую выложить отчет и все используемые в процессе работы файлы.

2. Выполнить коммит изменений в локальном репозитории, выполнить push в удаленный репозиторий[7].

В ЭИОС вместе с отчетом в качестве комментария загрузить ссылку на репозиторий в формате https://[адрес]/[имя пользователя]/labs.git.

4.2.7. Контрольные вопросы

1. Что такое Django?
2. Как настроить Django из командной строки?

[7] Для копирования файлов на ВМ можно использовать программу WinSCP

3. Где расположена директория приложения Django?
4. Как запустить и остановить сервер Django?
5. Как выполнить проверку работы приложения на Django?

4.3. Лабораторная работа 3

Тема: Установка Docker Debian.

Цель: Освоение техники установки Docker Debian и работы с репозиториями Docker.

4.3.1. Подготовка репозиториев Docker

Установка необходимых пакетов:

```
apt update
apt install ca-certificates curl gnupg
```

Результат:

```
root@debian:~# apt update
root@debian:~# apt install ca-certificates curl gnupg
Чтение списков пакетов… Готово
Построение дерева зависимостей… Готово
Чтение информации о состоянии… Готово
```

На данном этапе установлен пакет ca-certificates самой новой версии (20210119). Будут установлены следующие дополнительные пакеты:

```
  dirmngr gnupg-l10n gnupg-utils gpg gpg-agent gpg-wks-
client gpg-wks-server gpgconf gpgsm libassuan0 libcurl4
libksba8 libnpth0 pinentry-curses
Предлагаемые пакеты:
  dbus-user-session   pinentry-gnome3   tor   parcimonie
xloadimage scdaemon pinentry-doc
Следующие НОВЫЕ пакеты будут установлены:
  curl dirmngr gnupg gnupg-l10n gnupg-utils gpg gpg-agent
gpg-wks-client gpg-wks-server gpgconf gpgsm libassuan0
libcurl4 libksba8 libnpth0 pinentry-curses
Обновлено 0 пакетов, установлено 16 новых пакетов, для
удаления отмечено 0 пакетов, и 0 пакетов не обновлено.
Необходимо скачать 8 282 kB архивов.
После данной операции объем занятого дискового
пространства возрастет на 16,9 MB.
Хотите продолжить? [Д/н] Y
```

Продолжится установка. Далее необходимо установить GPG-ключи официального репозитория Docker.

```
install -m 0755 -d /etc/apt/keyrings
$ curl -fsSL https://download.docker.com/linux/debian/gpg
| sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
chmod a+r /etc/apt/keyrings/docker.gpg
```

Данные команды не имеют никакого вывода в консоль при отсутствии ошибок.

Далее следует добавить репозиторий Docker в списки репозиториев apt:

```
echo \
  "deb [arch="$(dpkg --print-architecture)" signed-
by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/debian \
  "$(. /etc/os-release && echo "$VERSION_CODENAME")"
stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list >
/dev/null
```

Выполнить обновление кэша apt:

```
apt update.
```

4.3.2. Установка Docker и дополнительного ПО

```
apt install docker-ce docker-ce-cli containerd.io docker-
buildx-plugin docker-compose-plugin
```

Установка займет некоторое время. После установки проверить работоспособность Docker. Данная команда выводит список контейнеров. На данном этапе их нет.

```
root@debian:~# docker ps -a
CONTAINER ID IMAGE COMMAND CREATED STATUS
PORTS NAMES
root@debian:~#
```

4.3.3. Задание

Следуя инструкции выше, провести установку Docker Debian.

4.3.4. Содержание отчета

В отчете представить скриншоты выполнения команд:

```
docker -v
hostname
docker ps -a
```

4.3.5. Работа с Git

1. В ранее созданном локальном репозитории LABS создать папку LAB3, в которую выложить отчет.
2. Выполнить коммит изменений в локальном репозитории, выполнить push в удаленный репозиторий.

В ЭИОС вместе с отчетом в качестве комментария загрузить ссылку на репозиторий в формате https://[адрес]/[имя пользователя]/labs.git.

4.3.6. Контрольные вопросы

1. Что такое Docker?
2. Как установить и настроить Docker?
3. Как создать и подготовить репозитории Docker?
4. Как проверить работоспособность Docker?
5. Каково назначение команды `docker`?

4.4. Лабораторная работа 4

Тема: Запуск контейнера с использованием Dockerfile.

Цель: Изучение средств создания образа для контейнера с приложением, освоение техники запуска контейнеров Docker с использованием Dockerfile.

4.4.1. Создание собственного образа для web-сервера apache

Создание собственного образа для web-сервера Apache и запуск контейнера:

- создать в системе новую директорию;
- перейти в каталог с помощью команды `cd`;
- выполнить клонирование каталога, в котором размещен пример *Dockerfile*, с публичного репозитория GIT:

`git clone https://pa.k-lab.su/dad/dockerfile.git`.

В директории появится папка *dockerfile*, внутри которой находится каталог *.git* и *Dockerfile*. Приведем содержимое файла *Dockerfile* в соответствие с описанием:

```
#Задаем исходный образ
FROM debian
#Устанавливаем необходимое ПО
```

```
RUN apt update
RUN apt install -y apache2
RUN apt install -y apache2-utils
RUN apt clean
#Задаем порт
EXPOSE 80
#Даем команду на запуск. Внимание! Команда передается списком
CMD ["apache2ctl", "-D", "FOREGROUND"]
```

Находясь в той же папке, в которой находится Dockerfile, запустить сборку образа (точка в конце указывает на текущую директорию):

```
docker build -t apache:1.0 .
```

Вывод команды покажет этапы сборки и скачивания исходного образа с docker-hub (рис. 4.19).

```
root@debian:/test/dockerfile# docker build -t apache:1.0 .
[+] Building 64.6s (5/8)
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 456B
 => [internal] load metadata for docker.io/library/debian:latest
 => [1/5] FROM docker.io/library/debian@sha256:3d868b5eb908155f3784317b3dda2941df87bbbbaa4608f84
 => => resolve docker.io/library/debian@sha256:3d868b5eb908155f3784317b3dda2941df87bbbbaa4608f84
 => => sha256:3d868b5eb908155f3784317b3dda2941df87bbbbaa4608f84881de66d9bb297b 1.85kB / 1.85kB
 => => sha256:60774985572749dc3c39147d43089d53e7ce17b844eebcf619d84467160217ab 529B / 529B
 => => sha256:3676c78a12ad77e170916695d2f192797ddd3eb653d7a1b1ad4c20ab5471891a 1.46kB / 1.46kB
 => => sha256:d52e4f012db158bb7c0fe215b98af1facaddcbaee530efd69b1bae07d597b711 49.55MB / 49.55MB
 => => extracting sha256:d52e4f012db158bb7c0fe215b98af1facaddcbaee530efd69b1bae07d597b711
 => [2/5] RUN apt update
 => [3/5] RUN apt install -y apache2
 => => # Enabling module mpm_event.
 => => # Enabling module authz_core.
 => => # Enabling module authz_host.
 => => # Enabling module authn_core.
 => => # Enabling module auth_basic.
 => => # Enabling module access_compat.
```

Рисунок 4.19 – Результат запуска сборки образа Docker

В результате создалось 5 слоев, что достаточно много. Это связано с тем, что большинство команд в Dockerfile создают новый слой. Для того, чтобы уменьшить образ, можно, например, RUN apt install -y apache2 RUN apt install -y apache2-utils объединить в одну команду с RUN apt install -y apache2 apache2-utils.

После сборки образа следует проверить его наличие:

```
root@debian:/test/dockerfile# docker images
REPOSITORY     TAG            IMAGE ID            CREATED
SIZE
apache         1.0        29fd6a175703   About a minute ago
252MB
root@debian:/test/dockerfile#
```

4.4.2. Запуск контейнера

```
docker run --name firstapache -d -p 8080:80 apache:1.0
```

где
`--name firstapache` – имя запущенного контейнера;
`-p 8080:80` – сопоставление портов 8080 – порт хост-системы ВМ, 80 – порт в контейнере.

Вывод команды (показывает, что создался новый слой):

```
root@debian:/test/dockerfile# docker run --name firstapache -d -p 8080:80 apache:1.0
6006130ef383d40a6673878384a99b2fb92e832f3b5b493f77b9a5b54cdadbe8.
```

Просмотрим запущенный контейнер:
`docker ps -a`.

В результате будет выдана таблица, показанная на рис. 4.20.

```
CONTAINER ID   IMAGE          COMMAND                  CREATED
6006130ef383   apache:1.0     "apache2ctl -D FOREG…"   About a minute ago
```

а

```
STATUS              PORTS                                   NAMES
Up About a minute   0.0.0.0:8080->80/tcp, :::8080->80/tcp   firstapache
```

б

Рисунок 4.20 – Результаты запуска контейнера: а – начало; б – окончание

Далее проверить результат, введя в браузере на рабочем месте, с которого доступна ВМ по сети, ссылку http://[IP адрес ВМ с доекером]:8080. Появится страница Apache по умолчанию (рис. 4.21).

В результате запускается первый контейнер *Docker*. При этом не потребовалось установки на хостовую ВМ никакого ПО, кроме *Docker*. Все необходимые зависимости и само ПО установлены и работают внутри контейнера.

Далее необходимо остановить контейнер: docker stop [ID или имя работающего контейнера].

Затем выполнить команду docker system prune –all, согласиться при ответе на вопрос. Данная команда удалит все неработающие контейнеры и образы, с ними связанные.

4.4.3. Docker-compose

Это очень мощный инструмент, плагин для Docker, использующий YAML, который позволяет упростить работы, связанные с запуском контейнеров.

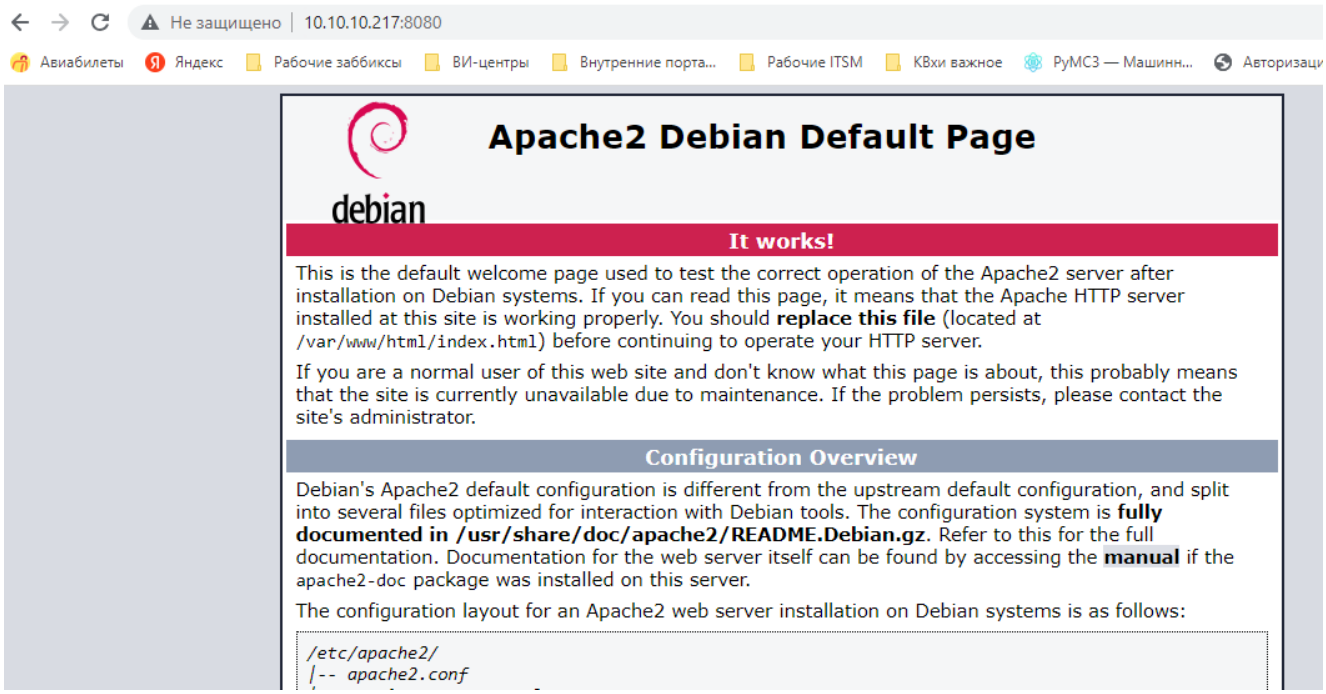


Рисунок 4.21 – Страница по умолчанию сервера Apache

Например, можно задать несколько контейнеров, их очередность запуска, зависимость и т. д. Полное описание см. на официальной странице документации (https://docs.docker.com/compose/compose-file/03-compose-file/).

Следует иметь в виду, что формат YAML чувствителен к отступам. Дочерний элемент располагается со сдвигом в два символа относительно родительского.

Многие производители ПО выкладывают для установки просто файл *docker-compose.yml*, где прописывают инструкции по скачиванию их образов, настройке контейнеров, примонтированных папок и зависимостей. Конечному потребителю необходимо просто создать файл *docker-compose.yml*, запустить его с параметрами, и необходимые образы будут скачаны с *docker-hub* в локальный репозиторий, а из них запущены контейнеры. Например, тот же web-сервер Apache, который был запущен ранее, может быть запущен посредством файла Docker-compose следующего содержания:

```
services:
   apache_my:
      image: httpd:2.4
    ports:
        - 8073:80
```

где `apache_my` – имя будущего контейнера, `image: httpd:2.4` – официальный образ Apache, размещенный на docker-hub, далее указываются порты. В тот же файл можно добавить переменные окружения, указывающие на расположение корневой директории сервера, а также иные параметры,

необходимые для настройки web-сервиса. Полное описание к официальному образу (https://hub.docker.com/_/httpd).

Далее, находясь в директории с файлом *docker-compose.yaml*, выполнить команду:

```
docker compose -f docker-compose.yml up -d.
```

В выводе будет показано извлечение образа и старт контейнера (рис. 4.22).

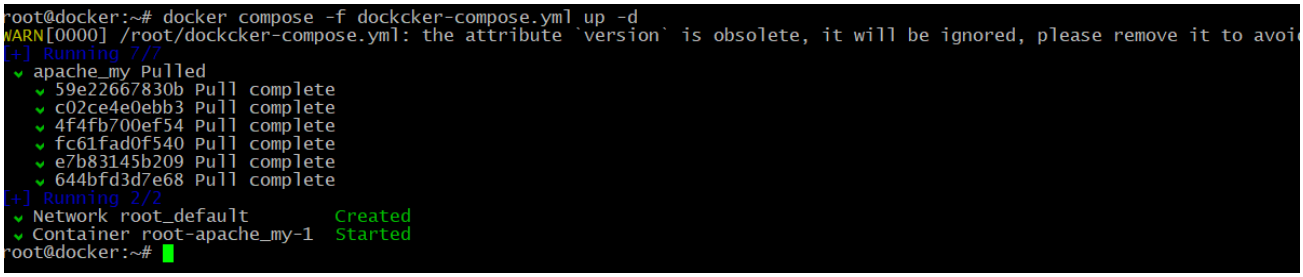


Рисунок 4.22 – Процесс запуска docker-compose

Результат запуска показан на рис. 4.23.

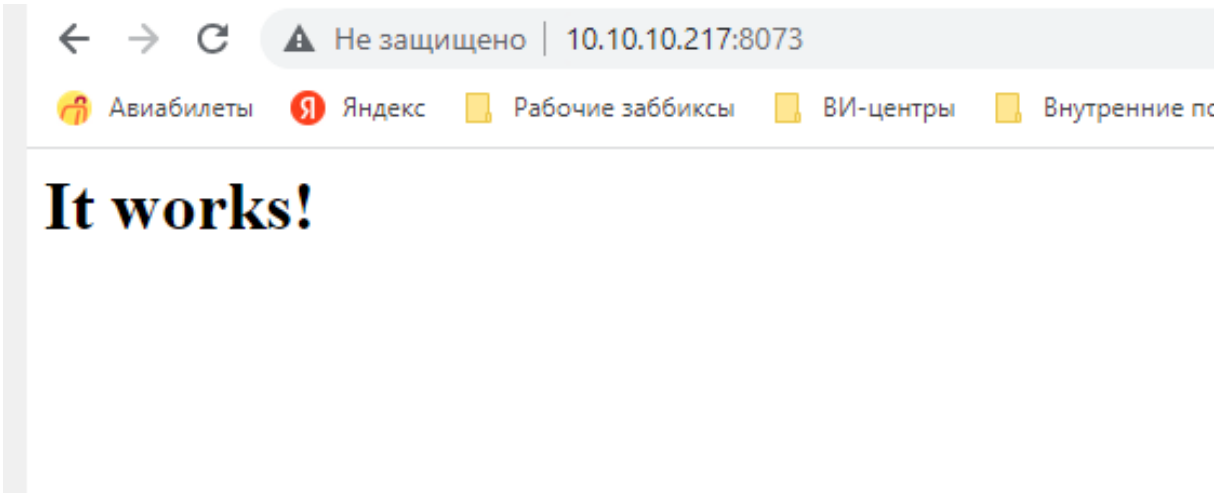


Рисунок 4.23 – Результат запуска docker-compose

Итак, официальный образ на Docker-hub содержит в себе страницу по умолчанию более лаконичную, чем тот, что был создан на основе изначального образа Debian, в который установлен web-сервер Apache с репозиториев Debian.

4.4.4. Запуск контейнера через Docker-compose

Для тренировки проведем запуск через Docker-compose. Для этого необходимо выполнить следующие действия:

1. Зарегистрироваться на Docker-hub (там все достаточно понятно и прозрачно). Необходимо подтвердить адрес e-mail, ответив на письмо.

2. Сделать новый образ web-сервера Apache, с того Dockerfile, который был создан ранее: `docker build -t dad1981/apache:1.0` [8]

3. Проверить наличие образа в локальном репозитории.

4. Далее выполнить команду `docker login`, ввести свой логин Docker-hub и пароль. **При вводе пароля символы не отображаются**! Дождаться авторизации (рис. 4.24).

```
root@debian:/test/dockerfile# docker login
Login with your Docker ID to push and pull images from Docker Hub. If you don't have
Username: dad1981
Password:
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
```

Рисунок 4.24 – Вход в Docker-hub

5. Далее выполнить push (рис. 4.25):
   `docker push dad1981/apache:1.0` [1]

```
root@debian:/test/dockerfile# docker push dad1981/apache:1.0
The push refers to repository [docker.io/dad1981/apache]
fc98637293a2: Pushed
48bdbccd828e: Pushed
ddda2cdc6344: Pushing [==================>        ]  44.79MB/116.8MB
a496db8703eb: Pushed
61581d479298: Mounted from library/debian
```

Рисунок 4.25 – Процесс загрузки образа в репозиторий

6. Проверить образ в репозитории на Docker-hub.

Важно! Все создаваемые репозитории публичны, не изменять это свойство.

На рис. 4.26–4.28 показаны результаты проверки.

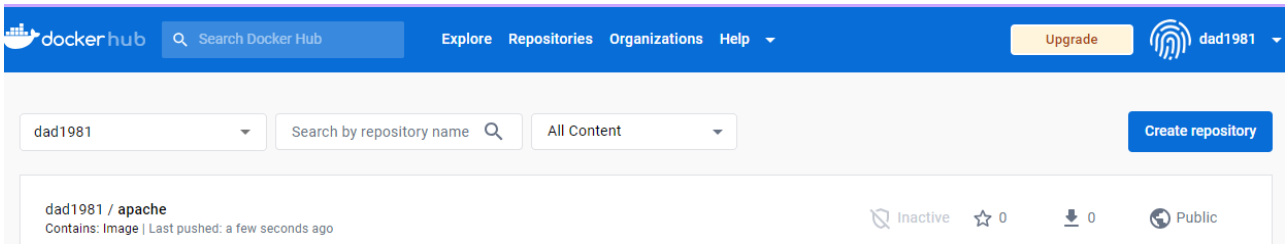


Рисунок 4.26 – Загруженный образ на GitHub

[8] Вместо dad1981 указать **свой** логин на докер-хаб.

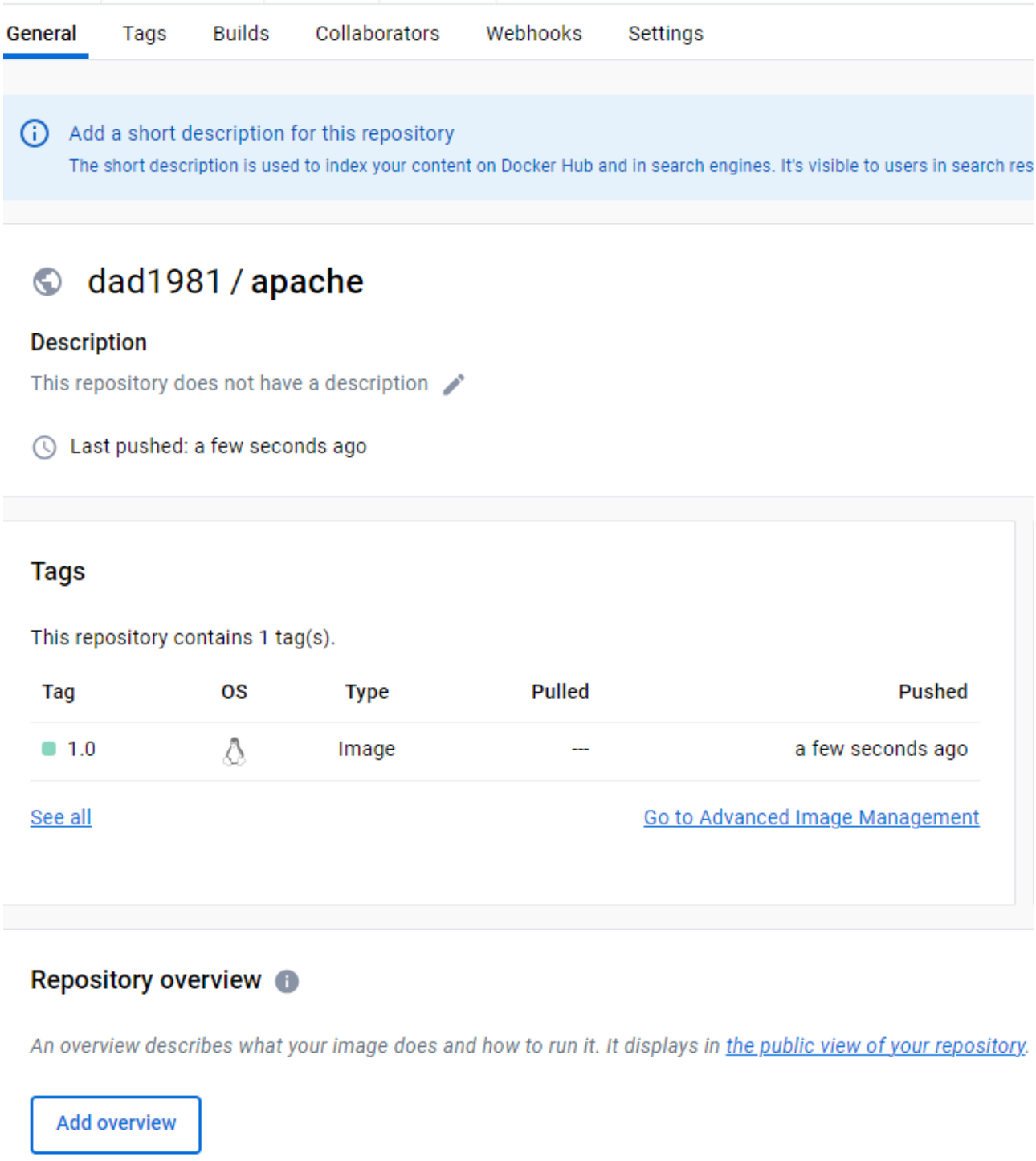


Рисунок 4.27 – Информация об образе на GitHub

Далее создать новую папку, перейти в нее и выполнить:
`git clone https://pa.k-lab.su/dad/dockercompose.git`.

Появится папка *dockercompose*, перейти в нее и найти в ней файл *docker-compose.yaml*. Изменить его в части image: dad1981/apache:1.0, указав тот образ, который был отправлен на Docker-hub, и в части названия контейнера. Сохранить файл.

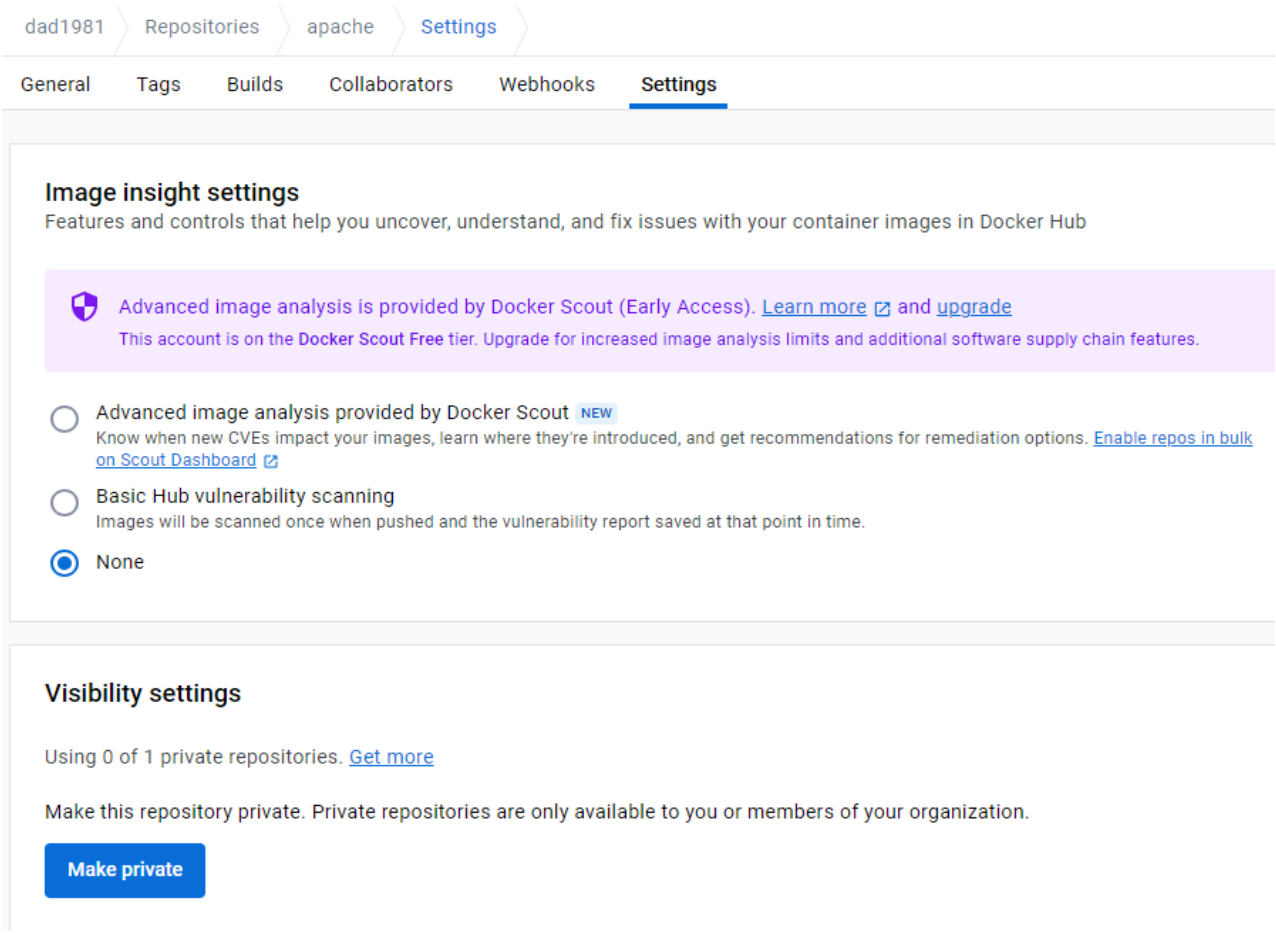


Рисунок 4.28 – Демонстрация публичности репозитория (о чем говорит кнопка Make Private)

Далее, находясь в папке с измененным файлом *docker-compose.yaml*, выполнить: docker compose up –d. Начнется извлечение образа. После успешного создания и запуска контейнера посмотреть запущенный контейнер посредством команды (рис. 4.29):

```
docker ps -a
```

```
CONTAINER ID   IMAGE                 COMMAND                  CREATED
93bb56e2237b   dad1981/apache:1.0    "apache2ctl -D FOREG…"   About a minute ago
```

а

```
STATUS              PORTS                                   NAMES
Up About a minute   0.0.0.0:8080->80/tcp, :::8080->80/tcp   dockercompose-apache_my-1
```

б

Рисунок 4.29 – Результаты запуска контейнера: а – начало; б – окончание

Выполнить проверку, результаты которой будут такими же, как на рис. 4.21.

Остановить контейнер и выполнить удаление неработающих контейнеров:

```
docker system prune -all.
```

4.4.5. Задание

Создать приложение web-сервер в Docker-контейнере, используя *Dockerfile* и *Docker-compose*.

1. С помощью *Dockerfile* создать образ для реализации приложения, созданного при выполнении лабораторной работы 2. Если в качестве средств разработки используется *Python Django*, то в качестве исходного образа можно использовать официальный образ Python версии 3.11 (FROM python:3.11). Создать web-приложение в контейнере *docker*, которое ранее создано при выполнении лабораторной работы 2.

Как пример рассмотрим перенос приложения Django. Для создания образа необходимо перенести в папку с Docker-файлом все необходимые директории проекта и приложения Django. Скопировать их в образ через Dockerfile. Возможно создать текстовый файл с зависимостями, где их перечислить и при установке PIP просто указать файл. Использовать инструкцию COPY. Пример такого Dockerfile и структуры каталогов:

```
# pull official base image
FROM python:3.10-alpine
# install dependencies
RUN pip install --upgrade pip
COPY ./requirements.txt .
RUN pip install -r requirements.txt
# copy project
COPY . .
CMD ["python", "manage.py", "runserver", "0.0.0.0:8007"]
```

Пример файла зависимостей **requirements.txt**:

```
Django
psycopg2-binary
requests
```

Примерная структура каталога (включены директории проекта Django и приложения) показана на рис. 4.30.

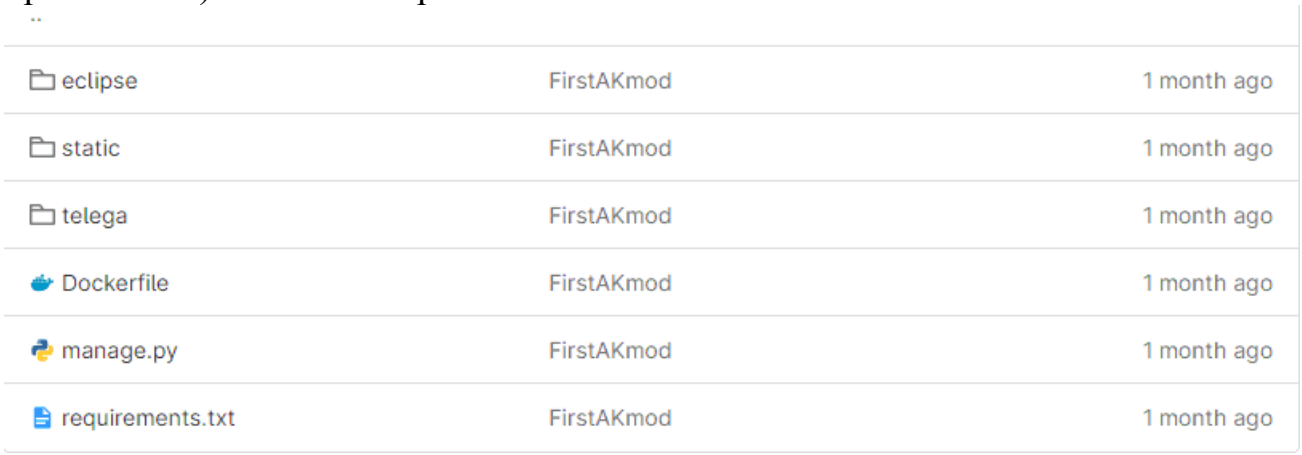


Рисунок 4.30 – Примерная структура каталогов, добавляемых в собираемый образ

2. Загрузить образ на Docker-hub.

3. В ВМ с ОС Debian в каталоге */home/[имя пользователя]* при помощи команд утилиты *git* клонировать удаленный репозиторий, ранее созданный и наполненный в процессе выполнения лабораторной работы 1, в отчете привести скриншот папки клонированного репозитория.

4. В папке с репозиторием создать папку LAB3, в которую выложить отчет, docker-compose.yaml со всеми параметрами использования вашего образа на Docker-hub и запуска контейнера.

5. Выполнить коммит изменений в локальном репозитории, выполнить push в удаленный репозиторий.

6. Запустить контейнер: `docker-compose up -d`. Проверить работоспособность.

7. После выполнения всех работ и получения положительного результата рекомендуется удалить все неработающие контейнеры и образы, с ними связанные, командой `docker system prune --all`.

4.4.6. Содержание отчета

- описание проделанной работы;
- Dockerfile;
- docker-compose.yml;
- код из файла vews.py Django-приложения (при условии его использования);
- скриншот репозитория Docker-hub, отображающий имя пользователя и содержимое репозитория;
- код html-страницы;
- git-ссылка (.git) на репозиторий учебного GitLab с файлом *docker-compose.yml*.

4.4.7. Работа с Git

1. В ранее созданном локальном репозитории LABS создать папку LAB4, в которую выложить отчет и все используемые в процессе работы файлы.

2. Выполнить коммит изменений в локальном репозитории, выполнить push в удаленный репозиторий.

В ЭИОС вместе с отчетом в качестве комментария загрузить ссылку на репозиторий в формате https://[адрес]/[имя пользователя]/labs.git.

4.4.8. Контрольные вопросы

1. Что такое Dockerfile?
2. Как создать и запустить Dockerfile?
3. Каково назначение файла *docker-compose.yaml*?
4. Как работать с образом Docker-hub?

4.5. Лабораторная работа 5

Тема: Тома, сети и переменные окружения Docker.

Цель: Изучение способов хранения данных в контейнерах Docker, освоение принципов и средств работы с томами, сетями и переменными окружения Docker.

4.5.1. Описание основных действий

Как известно, жизненный цикл контейнера предполагает создание контейнера, работу контейнера и его удаление. Все данные, полученные в процессе его работы, сохраняются в дополнительном верхнем слое (writable layer) контейнера. При необходимости пересборки контейнера требуется удаление старого контейнера и создание нового. Таким образом, данные из writable layer контейнера, закончившего свой жизненный цикл, будут потеряны. Чтобы этого не происходило, в Dockere имеется механизм работы с данными. Данные в Docker могут храниться либо временно, либо постоянно. Рассмотрим сначала работу с вре́менными данными.

4.5.2. Вре́менное хранение данных

В контейнерах Docker организовать работу с вре́менными данными можно двумя способами.

1. Простой (стандартный) способ. По умолчанию файлы, создаваемые приложением, работающим в контейнере, сохраняются в слое контейнера, поддерживающем запись. Для того чтобы этот механизм работал, ничего специально настраивать не нужно. Получается очень эффективно. Приложению достаточно просто сохранить данные и продолжить выполнять свои функции. Однако после того как контейнер перестанет существовать, исчезнут и данные, сохраненные таким простым способом.

2. Способ, обеспечивающий высокий уровень производительности. Годится, если нет необходимости в хранении данных дольше, чем существует контейнер. Суть способа состоит в том, чтобы подключить к контейнеру *tmpfs* – временное хранилище информации, которое использует оперативную память хоста. Это позволит ускорить выполнение операций по записи и чтению данных.

Однако часто бывает так, что данные нужно хранить и после того, как контейнер прекратит свое существование. Для этого существуют механизмы постоянного хранения данных.

4.5.3. Постоянное хранение данных

Существуют два способа, позволяющие сделать срок жизни данных больше, чем срок жизни контейнера. Один из способов заключается в

использовании технологии *bind mount*. При таком подходе к контейнеру можно примонтировать, например, реально существующую на диске папку. Работать с данными, хранящимися в такой папке, смогут и процессы, находящиеся за пределами Docker. Процесс монтирования *tmpfs* и технология *bind mount* показаны на рис. 4.31.

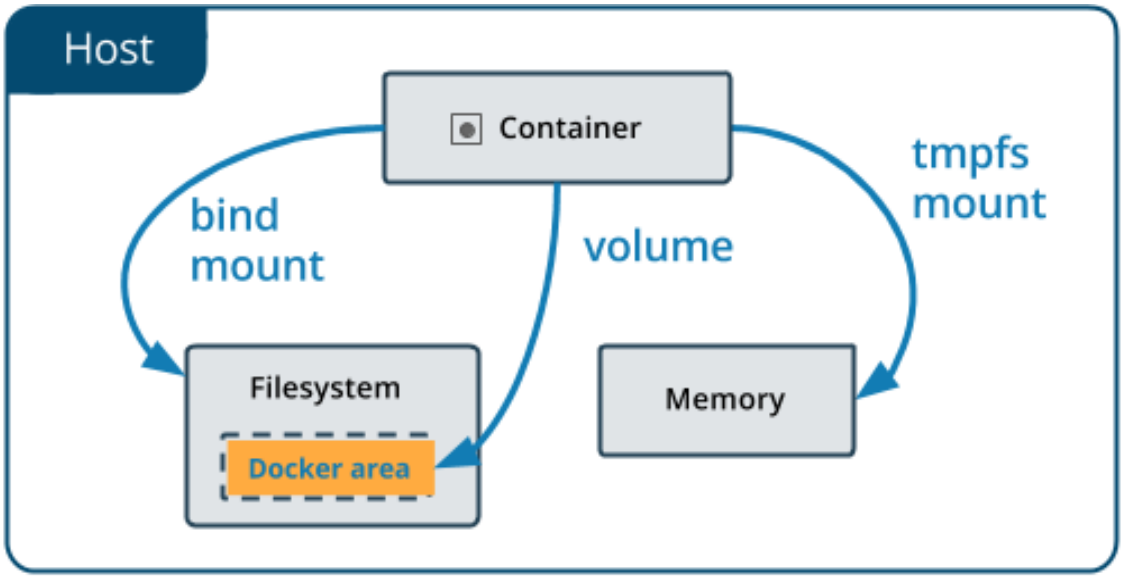


Рисунок 4.31 – Процесс монтирования tmpfs и технология bind mount

4.5.4. Тома Docker

Том в Docker – это файловая система, которая расположена на хост-машине за пределами контейнеров. За создание и управление томами отвечает Docker. Основные свойства томов Docker:

- представляют собой средства для постоянного хранения информации;
- самостоятельны и отделены от контейнеров;
- могут совместно использоваться разные контейнеры;
- позволяют организовать эффективное чтение и запись данных;
- могут быть размещены на ресурсах удаленного облачного провайдера;
- могут быть зашифрованы;
- им могут быть присвоены имена;
- контейнер может организовать заблаговременное наполнение тома данными;
- удобны для тестирования.

Создание томов

Тома можно создавать средствами Docker или с помощью запросов к Docker Engine API (https://docs.docker.com/engine/api/, https://ru.linux-console.net/?p=7763#gsc.tab=0).

Вот инструкция, которая позволяет создать том при запуске контейнера: `VOLUME /my_volume`.

При этом Docker после создания контейнера создаст том, содержащий данные, которые уже имеются в указанном месте. Важно, что если создается том с использованием Dockerfile, то также необходимо указать точку монтирования тома.

Создавать тома в Dockerfile можно, используя формат JSON. Кроме того, тома можно создавать средствами командной строки во время работы контейнера.

Работа с томами из командной строки

Создать самостоятельный том:

```
docker volume create —-name my_volume
```

Просмотреть список томов Docker:

```
docker volume ls
```

Исследовать конкретный том можно так:

```
docker volume inspect my_volume
```

Удалить том:

```
docker volume rm my_volume
```

Удалить все тома, которые не используются контейнерами (перед удалением томов Docker запросит подтверждение):

```
docker volume prune
```

Если том связан с каким-либо контейнером, такой том нельзя удалить до тех пор, пока не удален соответствующий контейнер. При этом даже если контейнер удален, Docker не всегда это «понимает». Если контейнера нет, а Docker не дает удалить том, можно воспользоваться командой:

```
docker system prune
```

Эта команда предназначена для очистки ресурсов Docker. После выполнения этой команды должна появиться возможность удалить тома, статус которых до этого определялся неправильно.

Полезные команды при работе с томами Docker

- `docker volume create`
- `docker volume ls`
- `docker volume inspect`

- `docker volume rm`
- `docker volume prune`

Вот список часто используемых параметров для `--mount`, применимых в команде вида `docker run --mount my_options my_image`:

- `type=volume`
- `source=volume_name`
- `destination=/path/in/container`
- `readonly`

Работа с томами с помощью docker-compose

Удобнее использовать тома с помощью файлов docker-compose. Например, данный файл docker-compose примонтирует внутрь контейнера папку из файловой системы хоста:

```
services:
  apache_my:
    image: dad1981/apache:1.0
    volumes:
      - type: bind
        source: /html1
        target: /var/www/html
    ports:
      - 8080:80
```

где:
`source: /html1` – папка на хосте (должна быть создана);
`- target: /var/www/html` – точка монтирования внутри контейнера. Это каталог для сайта web-сервера Apache по умолчанию.

Пример docker-compose, использующего `volume`:

```
services:
  apache_my:
    image: dad1981/apache:1.0
    volumes:
      - /html1:/var/www/html
    ports:
      - 8080:80
```

где до двоеточия – папка на хост-машине (если ее нет, она будет создана), после – сопоставленная папка в контейнере.

Пример. Создать папку /html, в которую поместить файл *index.html* следующего содержания:

```
<H1>This is an index file of host</H1>
```

Для начала создать файл *docker-compose.yaml* следующего содержания:

```
services:
  apache_my:
    image: dad1981/apache:1.0
    volumes:
      - type: bind
        source: /html1
        target: /var/www/html
    ports:
      - 8080:80
```

Запуск контейнера: `docker-compose up -d` (`-d` – обозначает запуск контейнера в фоновом режиме). Проверка должна привести к корректному отображению сообщения, как показано на рис. 4.32.

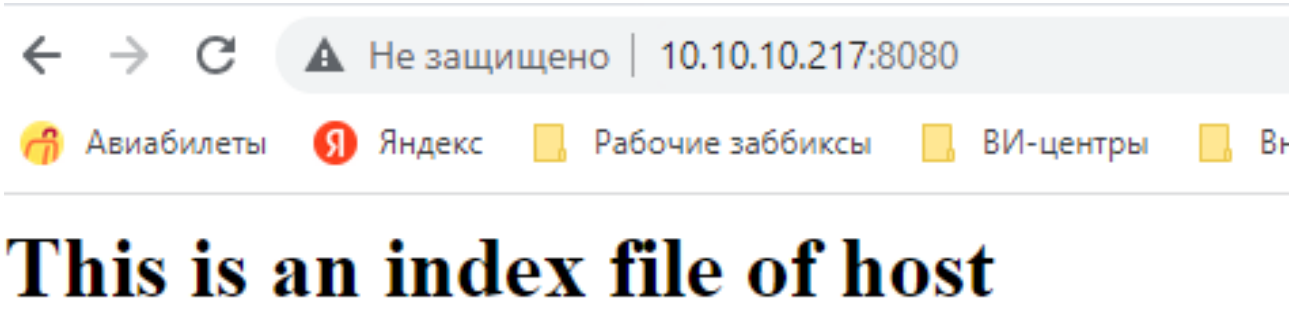


Рисунок 4.32 – Проверка корректности запуска контейнера

4.5.5. Сети в контейнерах Docker

Сеть Docker в основном используется для установления связи между контейнерами Docker и внешним миром через хост-машину. Это своего рода коммуникационный канал, через который все изолированные контейнеры взаимодействуют друг с другом в различных ситуациях для выполнения необходимых действий.

Все перечисленные в этом подразделе команды протестированы с правами root в Ubuntu.

Для управления сетевыми операциями, такими как создание новой сети, подключение контейнера к сети, отключение контейнера от сети, перечисление доступных сетей, удаление сетей и т. д., используется команда:

```
# docker network
```

Пример выполнения команды показан на рис. 4.33.

```
root@CPDockerTEST:/home/ubuntu# docker network

Usage:  docker network COMMAND

Manage networks

Commands:
  connect     Connect a container to a network
  create      Create a network
  disconnect  Disconnect a container from a network
  inspect     Display detailed information on one or more networks
  ls          List networks
  prune       Remove all unused networks
  rm          Remove one or more networks

Run 'docker network COMMAND --help' for more information on a command.
root@CPDockerTEST:/home/ubuntu#
```

Рисунок 4.33 – Пример выполнения команды `network`

4.5.6. Сетевые драйверы в Docker

Чтобы получить список всех сетей, запустить команду:

```
# docker network ls
```

Результат выполнения команды показан на рис. 4.34.

```
root@CPDockerTEST:/home/ubuntu# docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
2898a81657c5        bridge              bridge              local
94b2d88e4b68        host                host                local
e8467d9c80b1        none                null                local
root@CPDockerTEST:/home/ubuntu#
```

Рисунок 4.34 – Получение списка всех сетей

Bridge network. При запуске Docker автоматически создается сеть типа «мост по умолчанию». Недавно запущенные контейнеры будут автоматически подключаться к нему. Можно создавать пользовательские настраиваемые мостовые сети. Пользовательские мостовые сети превосходят сетевые мосты по умолчанию в том плане, что можно задать самостоятельную адресацию, выделив, например, сеть с большей маской, соответственно, саму сеть сделать короче (с меньшим количеством адресов).

Host network удаляет сетевую изоляцию между контейнером и хостом Docker и напрямую использует сеть хоста. Если запустить контейнер, который привязывается к порту 80, и при этом использовать хост-сеть, то приложение

контейнера будет доступно через порт 80 по IP-адресу хоста. Это значит, что больше нельзя запускать несколько веб-серверов на хосте, на том же порте, так как порт теперь является общим для всех контейнеров в сети хоста.

None network. В сети такого типа контейнеры не подключены ни к одной сети и не имеют доступа к внешней сети или другим контейнерам. Эта сеть используется, когда необходимо полностью отключить сетевой стек в контейнере.

Overlay network создает внутреннюю частную сеть, которая охватывает все узлы, участвующие в кластере *swarm*. Оверлейные сети облегчают обмен данными между сервисом *Docker Swarm* и автономным контейнером или между двумя автономными контейнерами на разных демонах Docker.

Macvlan network. Некоторые приложения, особенно устаревшие приложения, отслеживающие сетевой трафик, ожидают прямого подключения к физической сети. В такой ситуации следует использовать сетевой драйвер *Macvlan* для назначения MAC-адреса виртуальному сетевому интерфейсу каждого контейнера, что делает его физическим сетевым интерфейсом, напрямую подключенным к физической сети.

Для того, чтобы работать с сетью Docker, необходимы драйверы. Имеется несколько драйверов по умолчанию, которые обеспечивают основной функционал по работе с сетью:

bridge: сетевой драйвер по умолчанию. Указываемый драйвер определяет тип создаваемой сети. Мостовые сети обычно используются, когда приложения работают в автономных контейнерах, которые должны взаимодействовать.

host: для автономных контейнеров, удаляет сетевую изоляцию между контейнером и Docker-хостом. Хост доступен только для служб Swarm в Docker версии 17.06 и выше.

overlay: overlay-сети соединяют несколько демонов Docker вместе и позволяют сервисам Swarm связываться друг с другом. Также можно использовать overlay-сети для облегчения связи между сервисом Swarm и автономным контейнером или между двумя автономными контейнерами на разных демонах Docker. Эта стратегия устраняет необходимость выполнять маршрутизацию на уровне ОС между этими контейнерами.

macvlan: сети Macvlan позволяют назначать MAC-адрес контейнеру, делая его физическим устройством в сети. Демон Docker направляет трафик в контейнеры по их MAC-адресам. Использование драйвера macvlan иногда является лучшим выбором при работе с устаревшими приложениями, а также приложениям, которые ожидают прямого подключения к физической сети, а не маршрутизации через сетевой стек хоста Docker.

none: для данного контейнера будут отключены все сети. Обычно используется в сочетании с пользовательским сетевым драйвером. Ни один не доступен для swarm-услуг.

bridge подходят, когда требуется несколько контейнеров для связи на одном хосте Docker.

host подходят, когда сетевой стек не должен быть изолирован от хоста Docker, но надо, чтобы другие аспекты контейнера были изолированы.

overlay подходят, когда нужны контейнеры, работающие на разных хостах Docker, или в случае нескольких приложений, работающих вместе с сервисами **Swarm**.

macvlan подходят, когда нужны контейнеры, чтобы они выглядели как физические хосты в сети, каждый из которых имеет уникальный MAC-адрес.

Использование драйверов **macvlan, none, host, overlay** предлагается к самостоятельному изучению. В дальнейшем при запуске контейнеров на одном хосте рекомендуется использовать сеть типа **bridge**.

Сети можно создавать непосредственно с помощью CLI интерфейса Docker. Информацию об этом можно изучить с помощью команды:

```
docker network create –help
```

Посмотреть список созданных сетей можно командой:

```
docker network ls
```

Однако гораздо удобнее для создания сети использовать Docker-compose. В предыдущих лабораторных работах при использовании Docker-compose указывались порты для соединения порта контейнера с портом хоста. По умолчанию создавалась сеть с драйвером *bridge* и названием вида *[название контейнера]-default* и адресацией 172.[17-255].0.0/16, где первый адрес – адрес самого *bridge* на хосте. Кроме того, для обеспечения возможности выхода трафика из контейнера во «внешний мир» при установке самого Docker создается сетевой интерфейс, обычно имеющий название *docker*0 и адрес 172.16.0.1/16.

4.5.7. Создание сети с использованием Docker-compose

Сети в Docker-compose-файле создаются директивой *network*.

Пример: запустить два контейнера – сервер приложений *tomcat* и СУБД *mysql*:

```
services:
  mysql:
    container_name: mysql
    image: mysql:latest
    restart: always
    environment:
      - MYSQL_ROOT_PASSWORD=root
    ports:
     - "3306:3306"
    networks:
      vpcbr:
        ipv4_address: 10.5.0.5

  apigw-tomcat:
    container_name: apigw-tomcat
```

```
    image: apache-tomcat:latest
    ports:
     - "8080:8080"
     - "8009:8009"
    networks:
      vpcbr:
        ipv4_address: 10.5.0.6
    depends_on:
     - mysql

networks:
  vpcbr:
    driver: bridge
    ipam:
     config:
       - subnet: 10.5.0.0/24
         gateway: 10.5.0.1
```

В данном примере указано, что оба контейнера будут подключены в сеть *vpcbr* и каждому контейнеру задан IP-адрес. В следующем блоке эта сеть описана:

```
networks:
  vpcbr:
    driver: bridge
    ipam:
     config:
       - subnet: 10.5.0.0/16
         gateway: 10.5.0.1
```

Здесь указано, что адрес 10.5.0.1 – это адрес самого интерфейса *bridge* на хосте, по данному адресу можно получить доступ к сервисам, развернутым на самой хост-машине.

4.5.8. Использование переменных окружения Docker

Docker позволяет устанавливать переменные окружения как во время сборки Docker-образов, так и во время запуска Docker-контейнеров (при запуске также можно переопределять переменные, указанные во время сборки).

При добавлении переменной окружения к экземпляру Docker-образа значение этой переменной должно отличаться в зависимости от того, где собирается образ. Каждый раз редактировать Dockerfile, меняя это значение, и пересобирать образ крайне неудобно. Также может потребоваться заменить значение переменной во время запуска Docker-контейнера из собранного образа.

Рассмотрим пример добавления в Dockerfile двух новых инструкций:

```
FROM alpine:3.7
ARG APP_ENV=production
ENV APP_ENV="${APP_ENV}"
```

Вместо APP_ENV можно указать RAILS_ENV, NODE_ENV или любую другую переменную окружения, значение которой требуется установить во время сборки Docker-образа.

Названия переменных, которые используются в инструкциях ARG и ENV, не обязательно должны совпадать, однако для удобства следует делать их одинаковыми. В приведенном выше примере при запуске контейнера переменная окружения (APP_ENV) по умолчанию будет иметь значение, установленное в инструкции ARG на этапе сборки Docker-образа (production). Убедимся в этом:

```
# Обычная сборка docker-образа (build), запуск контейнера (run) и получение значения переменной:

docker build -t my_test_app .
docker run --rm my_test_app env | grep APP_ENV
> APP_ENV=production
```

Сборка Docker-образа с переопределенным значением переменной с помощью параметра --build-arg, запуск контейнера и получение значения переменной:

```
docker build -t my_test_app --build-arg APP_ENV=staging .
docker run --rm my_test_app env | grep APP_ENV
> APP_ENV=staging
```

Переопределение значения переменной окружения во время запуска контейнера выглядит так:

```
docker run --rm -e APP_ENV=development my_test_app env | grep APP_ENV
> APP_ENV=development
```

Если используется утилита Docker-compose, то конфигурационный файл *docker-compose.yml* может выглядеть так:

```
services:
  my_test_app:
    build:
      context: "."
      args:
        APP_ENV: "helloworld"
```

Теперь если выполнить команду Docker-compose `up --build` и проверить переменные окружения в Docker-контейнере, то можно увидеть `APP_ENV=helloworld`. Как и ожидалось, значение переменной соответствует тому, которое было установлено в конфигурационном файле.

4.5.9. Задание

Модифицировать разрабатываемое приложение следующим образом:

1. Выполнить логирование http-подключений (время подключения, IP-адрес клиента).
2. Обеспечить запись логов в текстовый файл. Путь к файлу задать через переменную окружения, созданную с использованием Docker-compose.
3. Собрать образ для создания контейнера с приложением и загрузить (push) его на Docker-hub.
4. При помощи Docker-compose выполнить следующее:
   - файл лога из задания 2 расположить в примонтированную (*bind*) к контейнеру папку с хост-системы;
   - создать переменную окружения, указывающую на директорию внутри контейнера, примонтированную с помощью *bind* из предыдущего пункта;
   - шаблоны html или PHP-файлы приложения расположить в томе Docker типа *volume,* примонтированном с файловой системы хоста.
5. Создать сеть с названием – *фамилия студента в латинской транскрипции (например, Petrov)*. Контейнер подключить к созданной сети. Задать адресацию: `10.20.[номер варианта].0/24`.
6. В созданном ранее локальном репозитории LABS на ВМ создать папку LAB4, в которую загрузить файл *docker-compose.yaml* и папки с логами и шаблонами из условий задания.
7. Выполнить коммит изменений в локальном репозитории, выполнить push в удаленный репозиторий.

4.5.10. Содержание отчета

- краткое пошаговое описание проделанной работы;
- листинг файла docker-compose.yaml;
- скриншот репозитория Docker-hub, отображающий имя пользователя и содержимое репозитория;
- Git-ссылка (.git) на репозиторий учебного GitLab с docker-compose.yaml и папками.

4.5.11. Работа с Git

1. В ранее созданном локальном репозитории LABS создать папку LAB5, в которую выложить отчет и все используемые в процессе работы файлы.

2. Выполнить коммит изменений в локальном репозитории, выполнить push в удаленный репозиторий.

В ЭИОС вместе с отчетом в качестве комментария загрузить ссылку на репозиторий в формате https://[адрес]/[имя пользователя]/labs.git.

4.5.12. Контрольные вопросы

1. Что такое переменные окружения Docker?
2. Как работать с томами Docker из командной строки?
3. Как построить сеть в контейнерах Docker и какие это дает преимущества?
4. Как создать сеть с использованием Docker-compose?
5. Для чего предназначена команда `# docker network`?

4.6. Лабораторная работа 6

Тема: Использование системы оркестрации k3s.

Цель: Понимание принципов работы системы оркестрации Kubernetes на примере k3s.

4.6.1. Общие положения

k3s – это облегченная дистрибуция Kubernetes, разработанная компанией Rancher Labs (сейчас часть SUSE). Она предназначена для быстрой и простой установки и запуска кластеров Kubernetes, особенно в средах с ограниченными ресурсами, таких как IoT устройства, устройства edge-вычислений, а также для разработки и тестирования. K3s включает в себя все необходимые компоненты Kubernetes, но имеет меньший размер и более простую архитектуру, что делает его удобным для развертывания в условиях, где ресурсы ограничены.

4.6.2. Создание образа контейнеров

Установка k3s крайне проста: имеется установочный скрипт, который можно скачать и запустить следующей командой:

```
curl -sfL https://get.k3s.io | sh -
```

Кроме того, рекомендуется установить систему мониторинга для Kubernetes – k9s. Ознакомиться с установкой предлагается самостоятельно.

После установки ПО создадим приложение, состоящие из одного пода, включающего два контейнера – контейнер с web-сервером nginx, работающий в качестве обратного прокси, а также контейнер с простым web-приложением на языке GO, выводящим слово «Привет» и случайное трехзначное число. Образы контейнеров были собраны с помощью Dockerfile. Данная процедура описана в

лабораторной работе 4. После установки k3s (или, при желании, k9s) создается папка, в которой создаются следующие файлы:

- `deployment.yml` – основной файл, описывающий развертывание;
- `pvc.yml` – persistent volume claim – запрос на постоянный том;
- `pv.yml` – описание тома;
- `service.yml` – описание сервиса.

Постоянный том (persistent volume, PV) предоставляет возможности для длительного хранения данных в кластерах Kubernetes. Данные, хранящиеся на PV, не теряются при сбое отдельного контейнера или пода целиком.

В рабочих нагрузках (workloads) нельзя использовать PV напрямую. Необходимо дополнительно создать Persistent Volume Claim (PVC), который позволяет запросить постоянный том с нужными параметрами и затем использовать его в рабочих нагрузках.

Содержимое файла pvc.yml:

```
#Версия API
apiVersion: v1
#Тип объекта
kind: PersistentVolumeClaim
#Метаданные - название volume
metadata:
  name: ngn-pv
#Описание параметров
spec:
  storageClassName: manual
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

Содержимое файла pv.yml:

```
apiVersion: v1
kind: PersistentVolume
metadata:
    name: ngn
    labels:
       type: local
spec:
   storageClassName: manual
   capacity:
      storage: 1Gi
  accessModes:
      - ReadWriteOnce
#Локальная папка хоста, используемая в качестве PV
  hostPath:
      path: "/opt/ngn"
```

Содержимое файла service.yml:

```
apiVersion: v1
kind: Service
```

```
# Метадата сервиса.
metadata:
  name: nginx-service
spec:
  # Селектор подов,
  # которым будет пускаться трафик.
  # Трафик может идти в любой под,
  # который матчит данному селектору.
  selector:
    app: nginx-pod
  # Порты для проксирования соединений.
  ports:
  # Порт, который служба прослушивает внутри
  - port: 80
    #Порт на хосте (сервере). Допускается использование портов по
номерам выше 30000
    nodePort: 30036
    # Порт пода, куда будет идти трафик дальше.
    targetPort: 80
  type: NodePort
```

Содержимое файла deployment.yml

```
apiVersion: apps/v1
kind: Deployment
# метаданные самого деплоймента.
metadata:
  name: nginx-dep
spec:
  # Количество реплик пода – в однонодном варианте не имеет смысла
держать более 1
  replicas: 1
  # Селектор, который выбирает
  # какие поды принадлежат этому деплойменту.
  selector:
    matchLabels:
      app: nginx-pod
  # Шаблон пода,
  # который будет использоваться при
  # маштабировании.
  template:
    # Метаданные пода.
    # тут обычно помещаются лейблы,
    # с помощью которых деплоймент идентифицирует
    # свои поды.
    metadata:
      labels:
        app: nginx-pod
    spec:
      #Указываем какой использовать Persistent Volume
      volumes:
        - name: ngn
          persistentVolumeClaim:
            claimName: ngn-pv
```

```
      # Масссив контейнеров
      containers:
        # Название контейнера внутри пода (контейнер nginx).
      - name: nginx-app
        # Образ контейнера (в данном примере образ в частном
репозитории Docker.
        image: locregistry.ddadem.ru/nginxuds:1.0
        # Указываем какой использовать PV и в какую папку внутри
контейнера монтировать, в данной папке файлы настройки nginx
        volumeMounts:
          - mountPath: "/etc/nginx/sites-enabled"
            name: ngn
        # Ресурсы требуемые для работы приложения.
        resources:
          # Минимальное количество ресурсов,
          # которое кластер гарантированно предоставит приложению.
          # Также данные значения используются для того,
          # чтобы выяснить, на какой ноде запускать приложение.
          requests:
            memory: "50Mi"
            cpu: "30m"
          # Максимально возможные значения приложения.
          # Если приложение выйдет за лимиты,
          # то кубер убьет приложение.
          limits:
            memory: "128Mi"
            cpu: "100m"
        # Порты контейнера.
        ports:
        - containerPort: 80
          protocol: TCP
      # Контейнер web-приложение на GO
      - name: nginx-goapp
        # Образ контейнера.
        image: locregistry.ddadem.ru/gotest1:1.0
        resources:
          requests:
            memory: "50Mi"
            cpu: "30m"
          limits:
            memory: "128Mi"
            cpu: "100m"
        ports:
        - containerPort: 8090
          protocol: TCP
```

Следует обратить внимание, что в данном примере используются образы контейнеров, расположенные в частном репозитории образов Docker, что видно по строчке:

```
image: locregistry.ddadem.ru/gotest1:1.0
```

Для использования публичных образов репозитория docker-hub можно указать:

```
image: dad1981/gotest1:1.0
```

Если образ непубличный, тогда используйте создание секрета:

```
kubectl create secret docker-registry regcred --docker-username=<your-name> --docker-password=<your-pword> --docker-email=<your-email> -n <your-namespace>
```

4.6.3. Создание пространства имен

Создаем namespace:

```
kubectl create namespace nginx-ns
```

Далее применим наше развертывание:

```
kubectl --namespace nginx-ns apply -f <путь к папке с указанными выше файлами>
```

Вывод команды:

```
deployment.apps/nginx-dep created
persistentvolumeclaim/ngn-pv created
persistentvolume/ngn created
service/nginx-service created
```

При этом в k9s отобразятся все этапы создания, как показано на рис. 4.35.

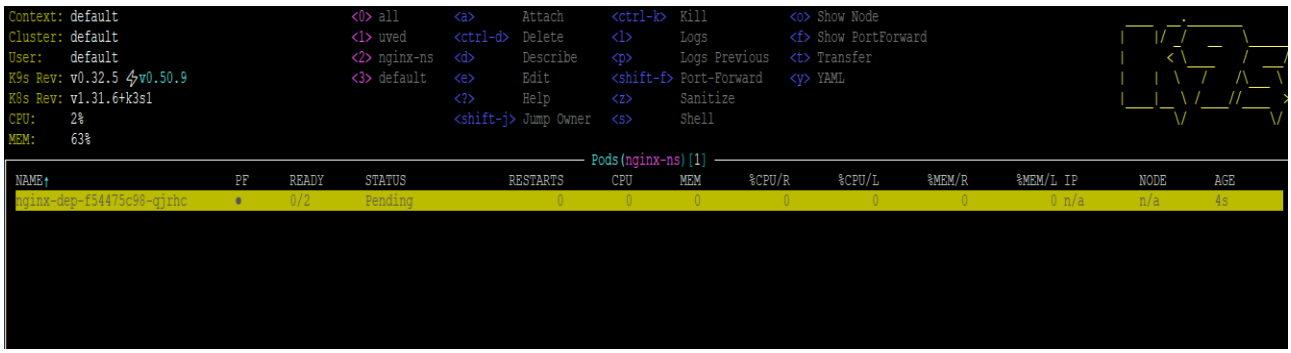


Рисунок 4.35 – Один из этапов создания пода

4.6.4. Создание пода

Созданный под показан на рис. 4.36, в результате чего под создан и запущен. Если далее на поде нажать Enter, то отобразятся контейнеры в поде (рис. 4.37). Выход из k9s – Ctrl+C.

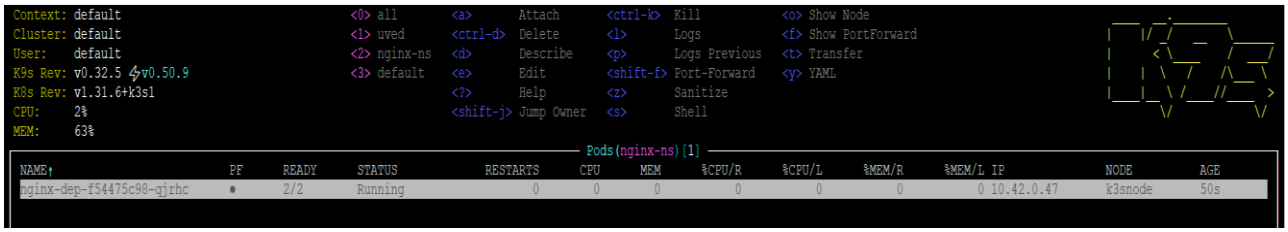


Рисунок 4.36 – Создание пода

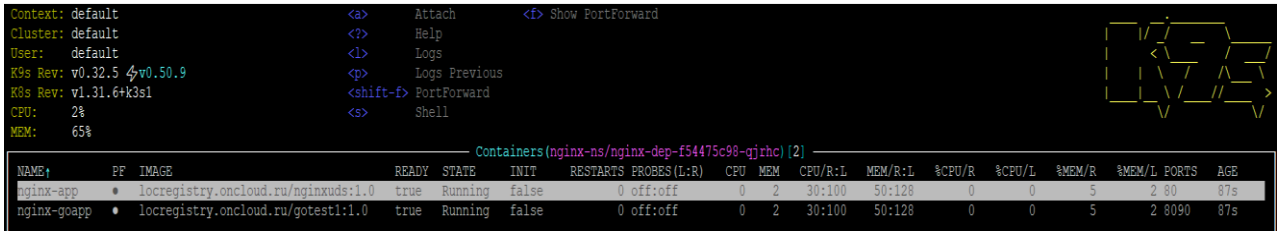


Рисунок 4.37 – Контейнеры в поде

Для проверки из браузера набрать http://IP-адрес:30036, в результате чего в браузере появится соответствующее сообщение (рис. 4.38).

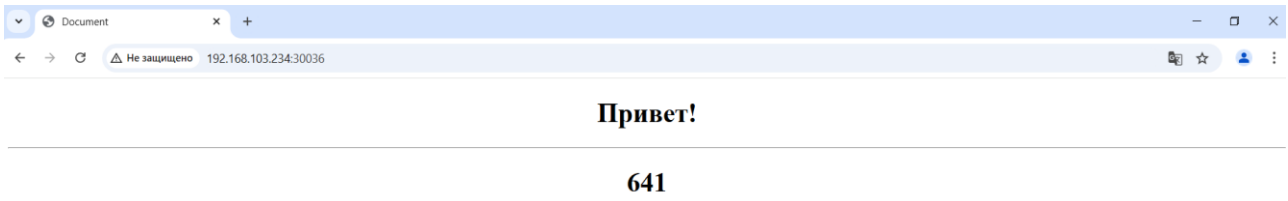


Рисунок 4.38 – Результат проверки запуска контейнера из пода

Примечание: для публикации вашего приложения «извне» не очень удобно и красиво оставлять такой адрес и порт. Для публикации существует множество сценариев, таких как использование обратных прокси – балансировщиков нагрузки (traefik, haproxy), которые реализуют функционал frontend, могут находиться за пределами кластера на выделенных виртуальных или физических серверах, реализовывать функционал SSL/TLS терминации и т. д. Кроме того, в Kubernetes существует специальная сущность, реализующая вышеуказанный функционал, «под капотом» использующая вышеописанное ПО (и не только) – ingress controller. Разобраться с ingress controller предлагается самостоятельно.

Также рекомендуется к прочтению статья [52], которая простым языком описывает сущности Kubernetes.

4.6.5. Задание

Установить k9s, k3s. Развернуть приложение из предыдущих лабораторных работ минимум в одном поде с двумя контейнерами либо в

нескольких подах. В названии подов, namespaces и контейнеров использовать фамилию студента в латинской транскрипции, например: ivanov-ns, ivanov-pod, Ivanov-cont-subd и т. д. Использовать PV/PVC для постоянного хранения данных.

4.6.6. Содержание отчета

- описание проделанной работы;
- скриншоты исполнения команд;
- скриншоты k9s.

4.6.7. Работа с Git

1. В ранее созданном локальном репозитории LABS создать папку LAB6, в которую выложить отчет и все используемые в процессе работы файлы.
2. Выполнить коммит изменений в локальном репозитории, выполнить push в удаленный репозиторий.

В ЭИОС вместе с отчетом в качестве комментария загрузить ссылку на репозиторий в формате https://[адрес]/[имя пользователя]/labs.git.

4.6.8. Контрольные вопросы

1. Для чего предназначена система оркестрации?
2. Какие средства оркестрации используются в разработке?
3. Что такое под и как с ним работать?
4. Как развернуть контейнеры в поде?
5. Какие сущности Kubernetes используются в разработке?

4.7. Лабораторная работа 7

Тема: Применение методологии CI/CD.

Цель: Изучение технологий автоматизированной сборки, упаковки и тестирования приложения, освоение средств автоматизации развертывания в разных окружениях.

4.7.1. Теоретические положения

CI/CD – одна из практик DevOps, подразумевающая непрерывную интеграцию и доставку. Этот набор принципов предназначен для повышения удобства, частоты и надежности развертывания изменений программного обеспечения или продукта. CI/CD относится к agile-практикам и позволяет разработчикам уделять внимание реализации бизнес-требований, качеству кода и безопасности продукта.

Основные цели CI/CD:

- обеспечение последовательного и автоматизированного способа сборки, упаковки и тестирования продуктов или приложений;
- автоматизация развертывания в разных окружениях;
- сведение к минимуму ошибок и проблем.

4.7.2. Принципы CI/CD

Для CI/CD существуют четыре руководящих принципа:

1. Разделение ответственности. Каждый из участников процесса делит ответственность за те или иные этапы жизненного цикла продукта. Проектируется бизнес-логистика, внедряются сквозные функции, проводятся приемочные тесты и организуется логистика кода.

2. Снижение рисков. Каждая команда, участвующая в разработке продукта, стремится к снижению рисков – контролируется корректность бизнес-логистики, проверяется пользовательский опыт, улучшается хранение и обработка данных и прочее.

3. Сокращение цикла обратной связи. Разработчик и клиент должны стремиться к увеличению скорости внесения изменений и согласования правок. Сборку и тестирование кода можно автоматизировать. А для ситуаций, когда требуется участие человека, можно минимизировать число информационных посредников.

4. Реализация среды. У разработчиков должно быть общее рабочее пространство с основной и вспомогательными ветками для контроля версий и качества, приемлемости, отказоустойчивости и других критериев.

4.7.3. Этапы CI/CD

Методология CI/CD подразумевает разделение процесса разработки на семь этапов:

1. Написание кода. Разработчики пишут код своего модуля и проводят тестирование в ручном режиме. После этого результат работы соединяется в главной ветке с текущей версией проекта. После того, как в главной ветке публикуются все коды модулей, начинается второй этап.

2. Сборка. Выбранная система контроля версий инициирует автоматическую сборку и последующее тестирование проекта. Триггеры для активации сборки могут быть настроены самостоятельно. Для автоматизации сборки применяется Jenkins или другой инструмент.

3. Ручное тестирование. После проверки CI-системой работоспособности тестовой версии код передается для ручного исследования.

4. Релиз. После ручного тестирования в сборку вносятся исправления. Следом проходит релиз версии кода для клиентов.

5. Развертывание. На этом этапе текущая (рабочая) версия кода размещается на production-серверах разработчика. Клиент может взаимодействовать с программой и изучать ее функции.

6. Поддержка и мониторинг. Продукт начинает использоваться конечными пользователями. При этом разработчики продолжают его поддерживать и проводят анализ пользовательского опыта.

7. Планирование. Исходя из пользовательского опыта, разрабатывается новый функционал и готовится план доработок. После этого разработчик начинает написание кода – и цикл замыкается.

4.7.4. Преимущества и недостатки CI/CD

Метод обеспечивает оперативность вывода нового функционала продукта (работа с запросами клиентов). Как правило, это считанные дни или недели. В то же время при классическом подходе к разработке клиентского софта это может занять год.

Кроме того, команда разработки получает пул альтернатив кода, что оптимизирует затраты ресурсов на решение задачи (за счет автоматизации первичного тестирования функционала).

Качество продукта повышается за счет параллельного тестирования функциональных блоков будущей системы. Узкие места и критические моменты фиксируются и отрабатываются еще на ранних стадиях цикла.

Тем не менее, руководители проектов ошибочно принимают методологию как панацею и стремятся внедрить ее во все свои разработки. При недостатке опыта это приводит к усложнению работ по IT-продуктам компании.

Также заслуживает внимания и организация взаимодействия между проектными группами, поскольку CI/CD сильно завязан на человеческий фактор. Инженеры, scrum-специалисты, аналитики, dev-группы и другие функциональные единицы должны работать в единой экосистеме с адекватным руководством и проектным управлением.

4.7.5. Инструменты для CI/CD

Разработчики ПО используют различные инструменты для автоматизации тестирования и доставки кода своих проектов конечным пользователям.

GitLab. Среда позволяет управлять репозиториями проекта, документировать функциональность и результаты доработок и тестов, а также отслеживать ошибки и работать с CI/CD pipeline.

Docker. Система автоматического развертывания проектов. Поддерживает контейнеризацию и позволяет упаковать проект вместе со всем окружением и зависимостями в контейнер, который можно перенести на Linux-систему.

Travis-CI. Инструмент подключается к репозиториям в GitHub при минимуме настроек. Решение облачное и не требует локальной установки. Кроме того, оно бесплатно для open-source проектов.

Circle-CI. Продукт также использует бесшовную интеграцию с GitHub. Предлагается веб-интерфейс для отслеживания версий сборок и ведения задач.

Также решение поддерживает отправку оповещений по почте, slack и другим каналам связи.

Jenkins. Довольно популярный инструмент в DevOps среде. Заслужил свою репутацию благодаря работе с различными плагинами, позволяющими гибко настраивать CI/CD процессы под требования разработки конкретного продукта.

TeamCity. В бесплатном режиме позволяет работать только с 3 агентами сборки, техническая поддержка предоставляется по подписке.

PHP Censor. CI-сервер для автоматизации сборки PHP-проектов, работающий с репозиториями GitLab, GitHub, Mercurial и др. Для тестирования используются библиотеки Atoum, PHP Spec, Behat и др. Проект хорошо документирован, но предполагает самостоятельную настройку и хостинг.

Rex. Предназначен для автоматизации CI-процессов в дата-центрах. Функционирует на Perl-скриптах.

Open Build Service (OBS). Предназначен для автоматизации CI/CD в разработке дистрибутивов приложений.

Buildbot. CI-система, позволяющая автоматизировать сборку и тестирование приложений. Поддерживает современные VCS и позволяет гибко настраивать сборку за счет Python-компонентов.

4.7.6. Применение методологии с использованием Gitlab

Основой для применения методологии CI/CD в GitLab служат Pipeline и Gitlab Runner. Pipeline (конвейер) – это процесс автоматизации (последовательность шагов), который используют для сборки, запуска тестов и развертывания программного продукта на протяжении всего жизненного цикла разработки. Определяется в файле gitlab-ci.yml в корне проекта. Описывается в формате yaml. Может состоять из (рис. 4.39):

- jobs, описывающих что нужно делать;
- stages – способ группировки job, плюс задает последовательность выполнения jobs.

Каждый проект определяет только один конвейер GitLab CI/CD. Конвейер проекта – это план для серии шагов, которые будут применены к файлу проекта. Выполнение этих шагов технически не является конвейером и называется «запуском конвейера» или «экземпляром конвейера».

GitLab Runner – приложение с открытым исходным кодом, которое выполняет задания конвейерной обработки GitLab CI/CD по инструкциям из специального файла .gitlab-ci.yml. Оно позволяет запускать автоматизированные сборки, являясь как бы агентом GitLab внутри целевой или тестовой системы.

Установка и настройка GitLab Runner отличается в зависимости от типа ОС и фактически состоит из двух задач:

1. Установка самого GitlabRunner на целевой системе, его настройка (GitlabRunner работает от собственного пользователя, поэтому в Linux-системах в некоторых случаях приходится давать этому пользователю

root полномочия через sudoers, однако в большинстве случаев для деплоя и запуска контейнеров достаточно внести пользователя в группу docker). На целевой системе работает как агент Gitlab, в простейшем случае запрашивая наличие новых заданий по http/https.

2. Регистрация GitLab Runner в целевом проекте. При этом GitLab выдает для runner[9] специальный ключ (токен), который runner на целевой системе надо указать.

Подробное описание установки и настройки GitLab Runner можно посмотреть на https://www.dmosk.ru/miniinstruktions.php?mini=gitlab-runner-web.

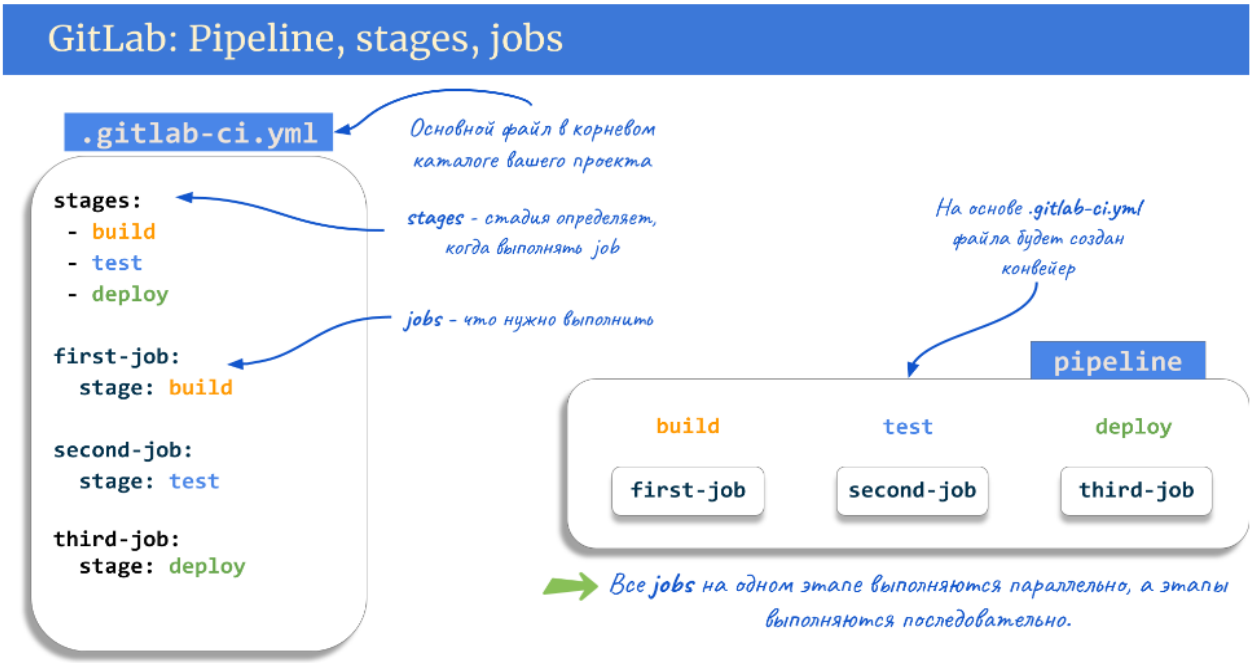


Рисунок 4.39 – Структура файла gitlab-ci.yml

4.7.7. Пример использования CI/CD

Рассмотрим небольшой упрощенный пример CI/CD

Представленный ниже docker-compose-файл описывает сервис, состоящий из трех контейнеров, реализующих функционал наполнения телеграмм-канала с объявлениями и web-приложение для модерации сообщений, отправленных через телеграмм-бот. Данные контейнеры реализуют функционал:

- - web-приложения для модерации;
- - телеграмм-бота для подачи объявлений.

```
- docker-compose файл
- СУБД

services:
```

[9] Системное ПО на сервере для взаимодействия с GitLab

```
  botik:
    build: tgdockak/
    container_name: bot_altay_kray
    restart: always
    environment:
      - NAMEBD=${NAMEBD}
      - USERBD=${USERBD}
      - PASSWORDBD=${PASSWORDBD}
      - HOSTBD=${HOSTBD}
    volumes:
      - /sambot1:/sambot1:rw
    depends_on:
      - subd
      - moder

  moder:
    build: altaiecl/
    container_name: moder_app_altay_kray
    restart: always
    environment:
      - MAINURL=${MAINURL}
      - NAMEBD=${NAMEBD}
      - USERBD=${USERBD}
      - PASSWORDBD=${PASSWORDBD}
      - HOSTBD=${HOSTBD}
      - ADMIN=${ADMIN}
      - EMAIL=${EMAIL}
      - PASSWORD=${PASSWORD}
    volumes:
      - /logsob1:/logsob1:rw
    ports:
      - 8007:8007
    depends_on:
      - subd

  subd:
    image: postgres:15
    container_name: subd
    environment:
      - PGDATA=/var/lib/postgresql/data/pgdb
      - POSTGRES_USER=${USERBD}
      - POSTGRES_PASSWORD=${PASSWORDBD}
      - POSTGRES_DB=${NAMEBD}
    volumes:
      - /opt/data:/var/lib/postgresql/data/:z
    restart: unless-stopped
```

На GitLab создан проект, содержащий исходный код приложения и файлы Docker (Dockerfile и docker-compose).

Установим и зарегистрируем 2 runner (для стенда разработки runner с тегом devrunner, для продуктивного стенда – runner с тегом deployrunner). Далее в корне проекта создадим файл `.gitlab-ci.yml` следующего содержания:

```
#Определим шаги (test - для тестовой среды, deploy для продуктивной)
stages:
 - test
 - deploy

#Описание каждого шага
test:
  stage: test
  # Тэг для runner
  tags:
     - devrunner
 #команды, выполняемые на сервере
  script: |
     docker compose -f docker-compose.yml up --build --force-recreate -d
#Правила запуска (описано далее)
  rules:
    - if: '$CI_COMMIT_BRANCH == "dev"'
      changes:
        - altaiecl/**/*
        - tgdockak/**/*
        - .env
        - docker-compose.yml
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event" && $CI_MERGE_REQUEST_TARGET_BRANCH_NAME == "dev"'
      changes:
        - altaiecl/**/*
        - tgdockak/**/*
        - .env
        - docker-compose.yml

deploy:
  stage: deploy
  tags:
     - deployrunner
  script: |
     docker compose -f docker-compose.yml up --build --force-recreate -d
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      changes:
        - altaiecl/**/*
        - tgdockak/**/*
        - .env
        - docker-compose.yml
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event" && $CI_MERGE_REQUEST_TARGET_BRANCH_NAME == "main"'
      changes:
        - altaiecl/**/*
        - tgdockak/**/*
        - .env
        - docker-compose.yml
```

Поясним правила (rules):

- `if: '$CI_COMMIT_BRANCH == "main"'` – означает «запускать при коммите в указанную ветку».
- `if: '$CI_PIPELINE_SOURCE == "merge_request_event" & $CI_MERGE_REQUEST_TARGET_BRANCH_NAME == "main"'` – означает «запускать при запросах на слияние в указанные ветки».
- `Changes` – означает «в дополнение к первому условию применить только при изменении указанных файлов в указанных каталогах»).

Далее в GitLab в нашем проекте создадим новую ветку `dev` из исходной main (рис. 4.40 – 4.41).

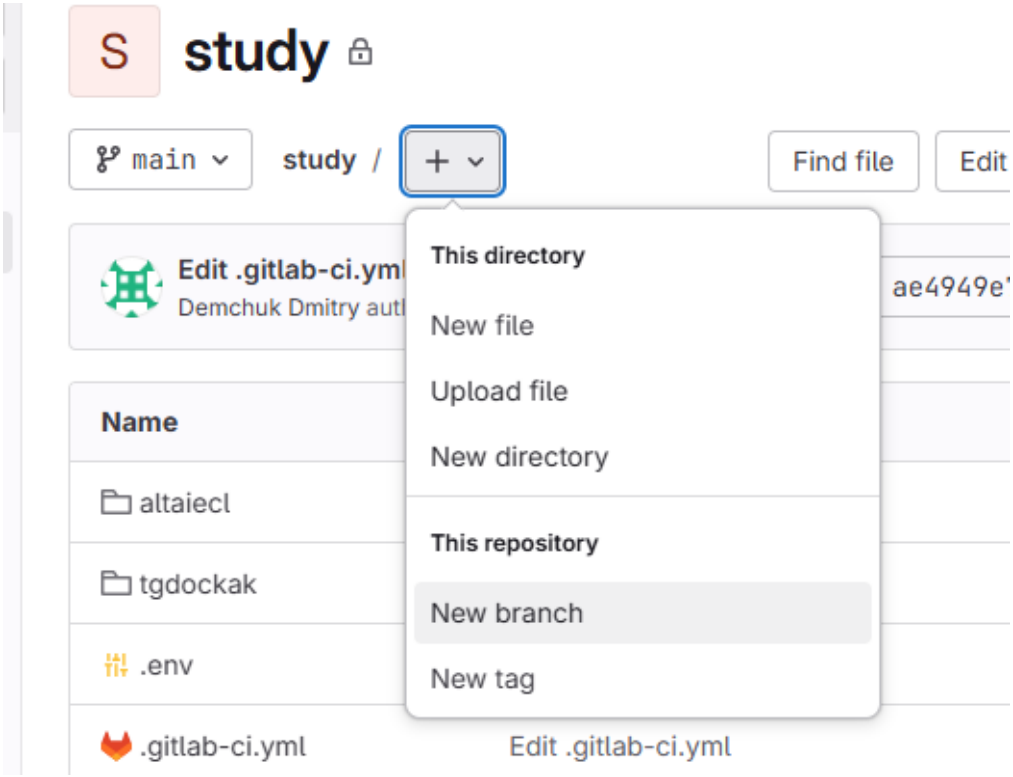


Рисунок 4.40 – Операция создания новой ветки в репозитории

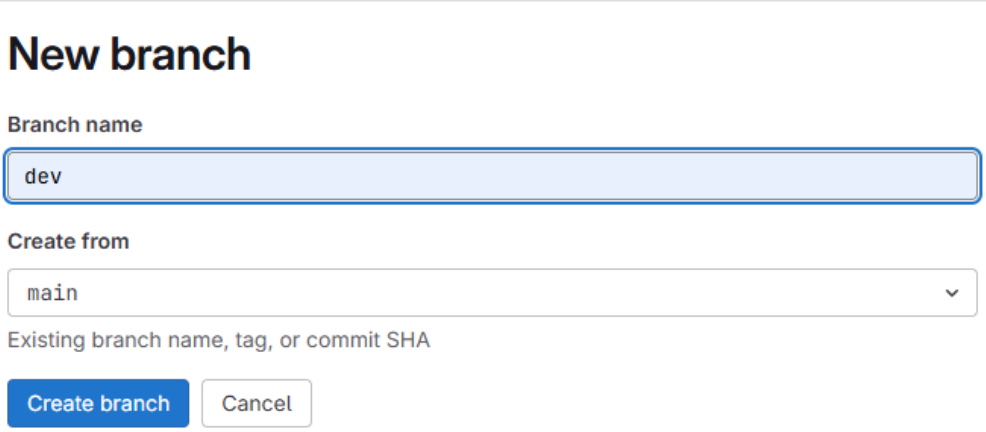


Рисунок 4.41 – Ввод основных параметров новой ветки в репозитории

Переключение между ветками осуществляется выбором (рис. 4.42):

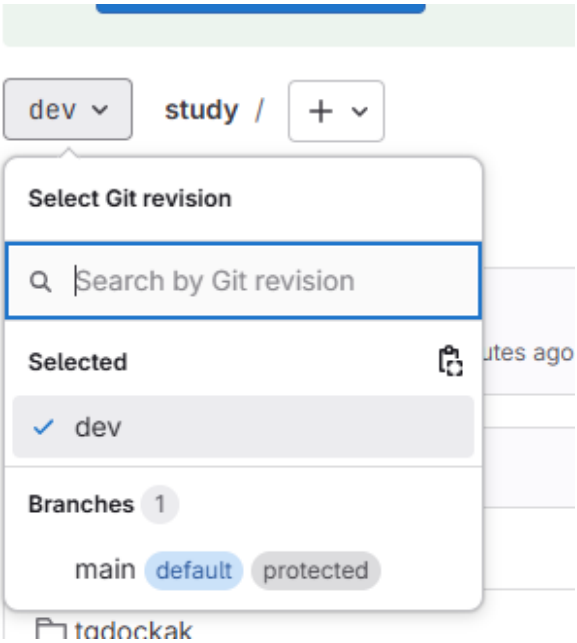


Рисунок 4.42 – Переключение между ветками

Далее в среде разработки переключимся в ветку dev и изменим файл шаблона формы авторизации:

```
<html>
 <head>
  <title>Авторизация</title>
</head>
<body>
<style>
  #avform{
    background:#f1f1f1;
        width:400px;
        height:250px;
        position:absolute;
        top:50%;
        left:50%;
        margin:-200px 0 0 -200px;
  }
</style>
{% block content %}
<div id="avform">
<h1 align="center">Авторизуйтесь</h1><br>
<form align="center" action="" method="post">
  {% csrf_token %}
 {{ form.as_p}}
  <button type="submit" name="button">Вход</button></br>
</form>
</div>
{% endblock %}
```

```
</body>
</html>
```

Изменим `<h1 align="center">Авторизуйтесь</h1><br>` на `<h1 align="center">Требуется авторизация</h1><br>`. Сохраним изменения, выполним коммит в локальный репозиторий, далее отправим изменения в orign-репозотрий. Далее в GitLab перейдем в Build/Pipelines и увидим запущенное задание CI/CD (рис. 4.43). Успешный результат выполнения показан на рис. 4.44. Если нажать мышкой на статус, то можно увидеть, какой шаг был выполнен (рис. 4.45).

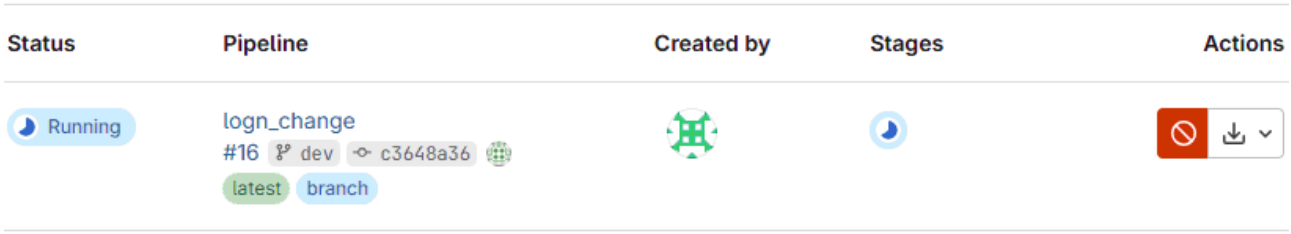


Рисунок 4.43 – Запущенное задание CI/CD

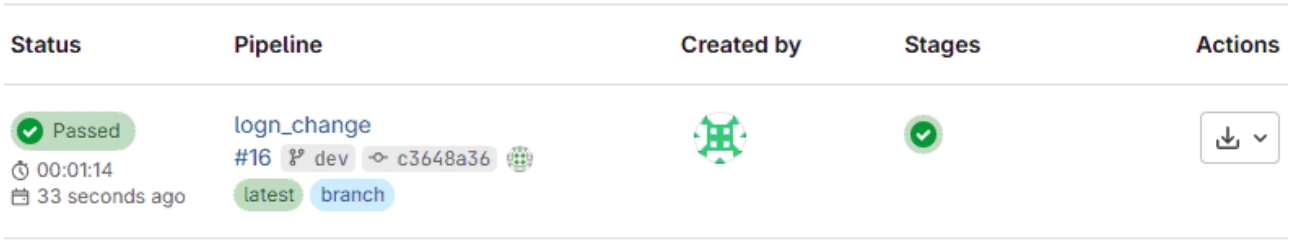


Рисунок 4.44 – Успешный результат выполнения задания

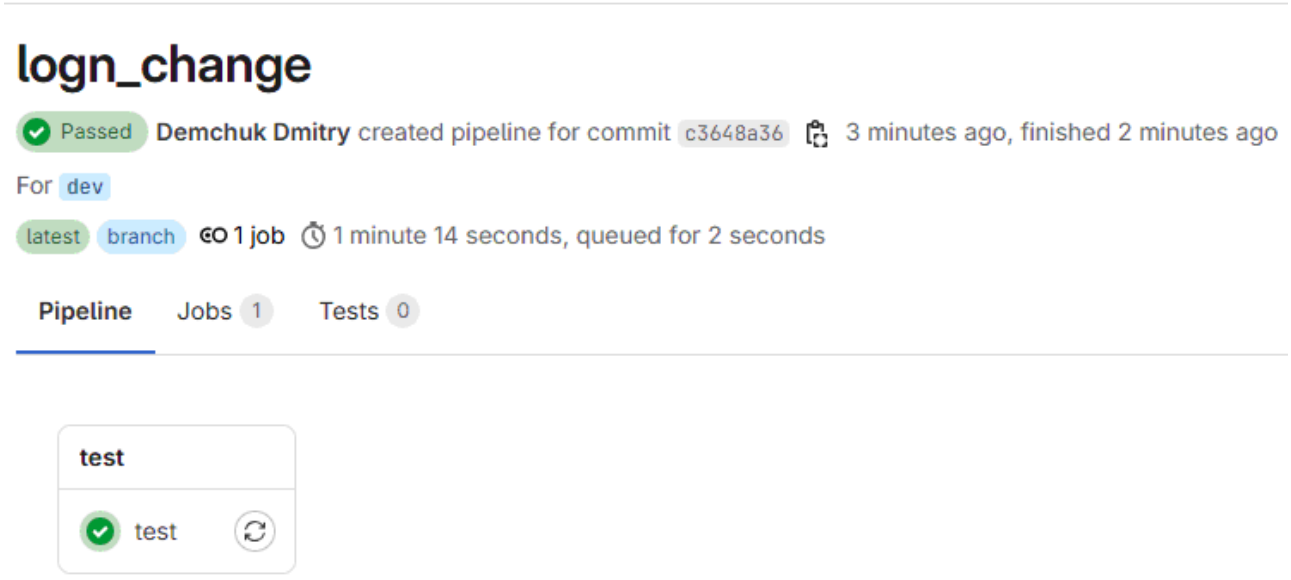


Рисунок 4.45 – Просмотр статуса выполнения задания

Далее, если нажать на название шага, то можно увидеть вывод stdout либо stderr в случае ошибок (рис. 4.46). Проверка запущенных контейнеров на сервере тестового контура показана на рис. 4.47. Проверка из браузера показана на рис. 4.48.

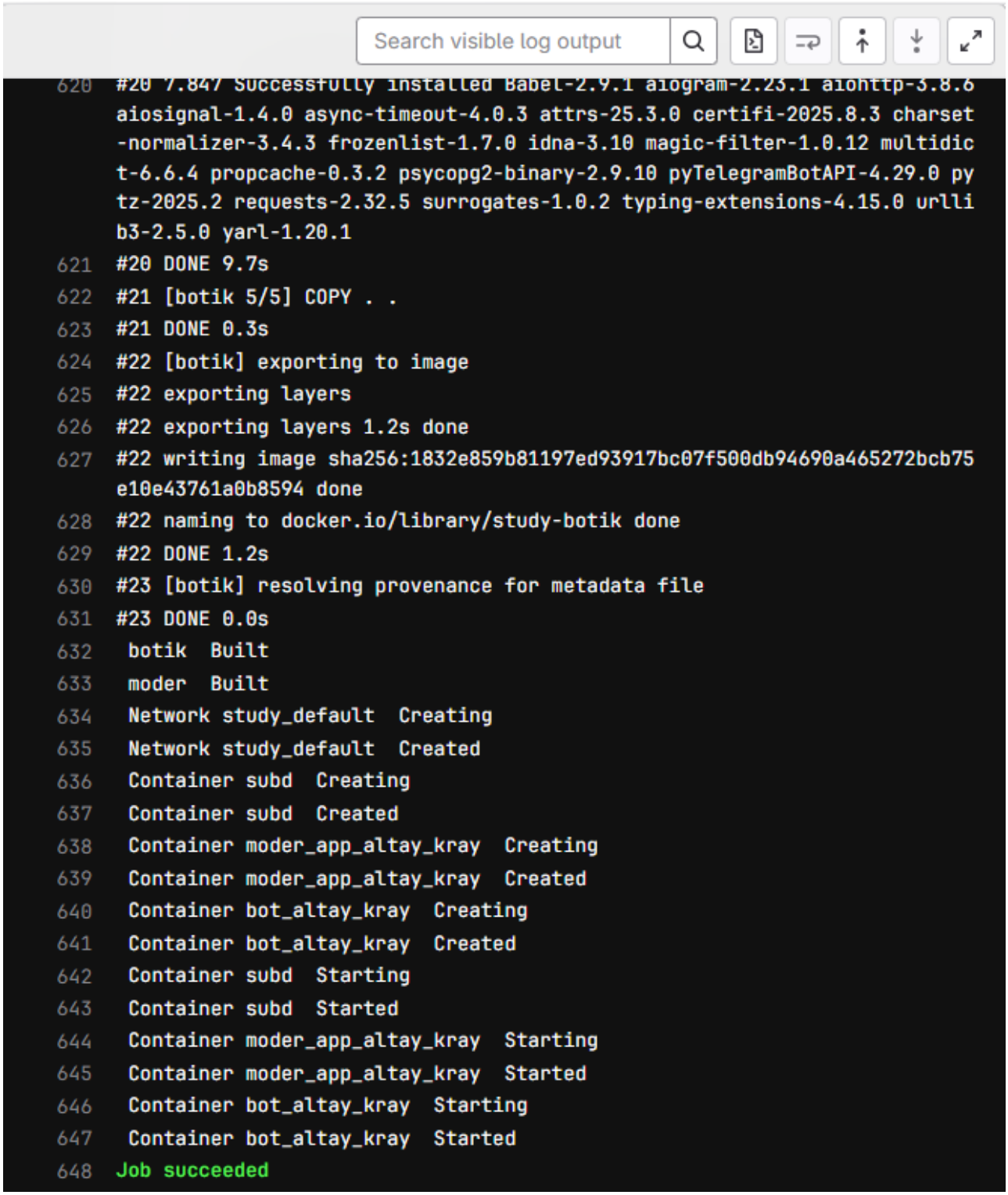


Рисунок 4.46 – Вывод стандартного потока вывода при выполнении задания

```
root@docker:/booot/study# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS
89d17e93775a   study-botik    "python3 aiobot.py"      4 minutes ago    Up 4 minut
008d25e2b6fe   study-moder    "/entrypoint.sh"         4 minutes ago    Up 4 minut
11c7b83172fd   postgres:15    "docker-entrypoint.s…"   4 minutes ago    Up 4 minut
```

Рисунок 4.47 – Проверка запущенных контейнеров из командной строки

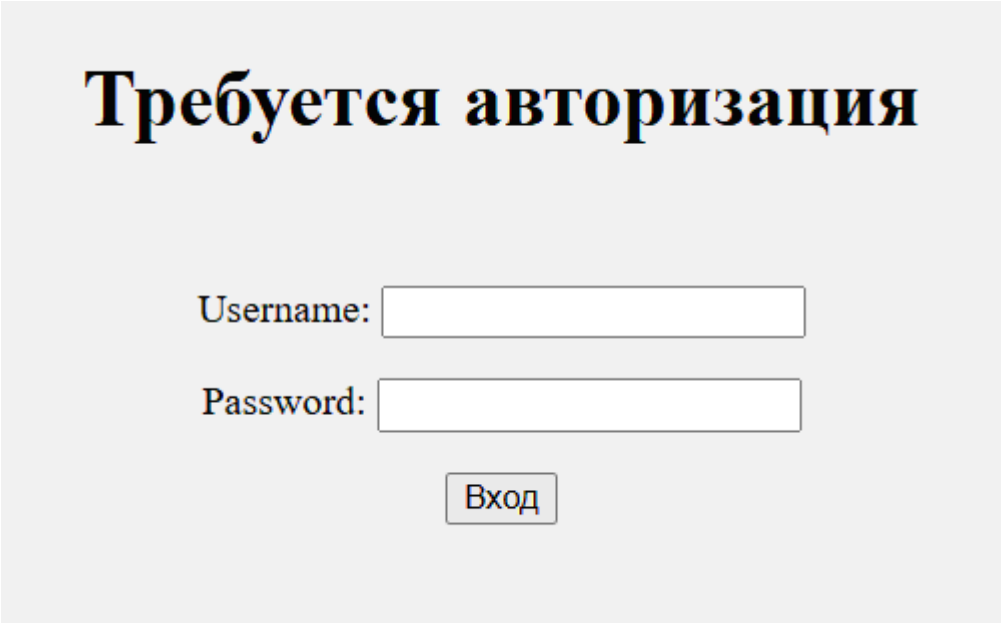


Рисунок 4.48 – Проверка запущенных контейнеров из браузера

Далее внесенные изменения проверяются командой тестировщиков. После того как все тесты пройдены, в GitLab необходимо создать запрос на слияние веток и слить изменения с веткой main. После этого произойдет запуск изменений уже на шаге build на продуктивный стенд, используя runner с тегом deployrunner.

4.7.8. Задание

1. Установить на целевой системе GitlabRunner, зарегистрировать его на GitLab.
2. Организовать доставку приложения до целевой системы и тестирование его работоспособности, используя CI/CD Pipeline.
3. В соответствии с индивидуальным заданием реализовать внедрение изменений с помощью CI/CD.

4.7.9. Содержание отчета

- скриншоты выполнения задания;
- скриншоты репозитория, зарегистрированного runner;
- содержимое .gitlab-ci.yml;
- скриншоты выполнения PipeLine.

4.7.10. Работа с Git

1. В ранее созданном локальном репозитории LABS создать папку LAB7, в которую выложить отчет и все используемые в процессе работы файлы.
2. Выполнить коммит изменений в локальном репозитории, выполнить push в удаленный репозиторий.

В ЭИОС вместе с отчетом в качестве комментария загрузить ссылку на репозиторий в формате https://[адрес]/[имя пользователя]/labs.git.

4.7.11. Контрольные вопросы

1. Что такое CI/CD?
2. Что такое Pipeline?
3. Как используется и строится файл docker-compose.yml?
4. Как запустить задание CI/CD?
5. Как проверить запущенные контейнеры?

5. Курсовое проектирование

5.1. Общие сведения

Курсовая работа (КР) выполняется в процессе изучения основного курса по DevOps-инженерии либо на следующем этапе и посвящается разработке информационной системы (ИС) с микросервисной архитектурой с использованием современных технологий проектирования и разработки ИС.

Выполнение КР может осуществляться на основе проектов, выполненных на предыдущих этапах обучения, а также базироваться на реальных производственных проектах информатизации/цифровизации. Целесообразно согласовать тему КР с темой будущей магистерской диссертации и обсудить ее особенности с выбранным научным руководителем.

Курсовая работа оформляется в виде пояснительной записки (ПЗ), сдается на проверку в последние две недели семестра и защищается студентом в соответствии с графиком, утвержденным руководителем.

Руководитель КР осуществляет следующие функции:

– согласование темы КР и выдача студенту задания на КР;

– предоставление помощи студенту относительно конкретизации содержания КР и разработки календарного плана ее выполнения;

– консультации и научно-методическое руководство работой студента в процессе выполнения КР;

– систематический контроль за ходом выполнения КР;

– предоставление помощи студенту в сборе основного и дополнительного материала для выполнения КР и оформления ПЗ;

– проверка завершенной КР и пояснительной записки;

– подготовка студента к защите КР перед комиссией.

Работа с настоящим методическим материалом в идеале предполагает последовательное его изучение «от корки до корки» с дальнейшим обращением к соответствующим частям подраздела 5.5, а также к подразделу 5.6 в процессе проектирования ИС, сверяясь при этом с графиком в подразделе 5.3.

Для студентов, по каким-либо причинам не имеющим достаточного времени на добросовестное изучение методических указаний, рекомендуется в обязательном порядке ознакомиться с подразделами 5.3, 5.5 и 5.8, чтобы правильно оценить масштаб деятельности. Дальнейшая работа над проектом с целью минимизации риска несоответствия работы установленным требованиям должна выверяться по содержанию разделов согласно требованиям к содержанию отчетов о выполнении лабораторных работ (см. раздел 4).

5.2. Цель и задачи выполнения курсовой работы

Курсовая работа представляет собой завершенную проектную разработку в профессиональной сфере для решения актуальной задачи цифровой экономики, в которой:

− сформулирована актуальность и место решаемой задачи информатизации в определенной предметной области;

− анализируется литература и информация, полученная с помощью глобальных сетей, по функционированию подобных систем и технологий в данной области или в смежных предметных областях;

− исследуются подходы, математические методы, математические модели, алгоритмы, программы, технические и инструментальные средства разработки приложений с современной архитектурой применительно к созданию информационного и программного обеспечения ИС;

− определяются и конкретно описываются выбранные студентом современные технологии для решения задачи;

− разрабатывается база данных для информационного обеспечения решения задачи при помощи современной СУБД;

− разрабатывается программное приложение на одном или нескольких языках программирования при помощи современных средств разработки и отладки ПО;

− осуществляется версионный контроль разрабатываемого приложения ИС при помощи современных распределенных средств управления версиями;

− осуществляется тестирование выполненных разработок с применением специальных методов и современных средств тестирования и технологий CI/CD.

Целью выполнения КР является практическое закрепление теоретической части этого курса, а именно углубление знаний по программированию и коллективной разработке приложений с применением современных технологий и методологии DevOps, а также формирование умений и навыков разработки ПО и приложений на его основе.

В результате выполнения курсовой работы студенты должны приобрести практические навыки:

− организации программного процесса коллективной разработки ИС;

− выбора современных технологий для коллективной разработки ИС;

− управления требованиями к ИС, постановки задачи на создание ИС с микросервисной архитектурой;

− постановки задачи на создание ИС;

− развертывание рабочей среды для коллективной разработки ИС;

− разработки и тестирования ИС с использованием современных технологий CI/CD;

− сопровождения ИС в условиях реализации версионного контроля;

− оформления технической документации к ИС и программному приложению.

5.3. Содержание и этапы выполнения курсовой работы

Курсовая работа предполагает решение конкретной практической задачи информатизации и содержит разработку рабочих проектных решений,

связанных с созданием программного приложения с микросервисной архитектурой.

Задание на КР:

В соответствии с согласованной индивидуальной темой реализовать ИС с микросервисной архитектурой – логически разделить ИС на выполняемые задачи и для каждой задачи реализовать отдельный контейнер (или группу контейнеров) как законченное приложение. Обмен данными между микросервисами реализовать через WEB-API (запрос/ответ).

В качестве средства контейнеризации использовать Docker, допускается использование системы оркестрации.

Содержание пояснительной записки к курсовой работе фактически представляет собой рабочий проект приложения на индивидуально выбранную тему. Наиболее приемлемым вариантом будет реализация программного приложения в контексте выполнения магистерской диссертации. Также возможно выполнение КР по новой теме, предложенной студентом и согласованной с руководителем. По согласованию с руководителем возможна разработка и написание комплексных курсовых работ командой из нескольких студентов.

Типовая тематика курсового проектирования по DevOps-инеженерии представлена в подразделе 5.4.

В процессе выполнения КР студент обязан разработать программное приложение в соответствии с индивидуально поставленной задачей, для чего должен:

- сформулировать функциональные требования к ИС и представить их в формальном виде;
- осуществить обоснованный выбор средств разработки приложения: СУБД, платформы для web-разработки, языковых средств, средств управления программным процессом, в т.ч. конфигурированием и тестированием, и пр.
- построить функциональные модели для решения задач информатизации и реализации отдельных микросервисов;
- разработать диаграммы реализации (Deployment Diagrams UML) программного приложения;
- выполнить установку и настройку ОС Linux для дальнейшего развертывания рабочей среды;
- выбрать систему контроля версий и на ее базе создать репозиторий для проекта;
- развернуть и настроить среду разработки программного приложения коллективным способом;
- развернуть Docker Debian и сформировать контейнер для разработки приложения с микросервисной архитектурой;
- разработать программные модули и интерфейсную часть приложения;
- выполнить тестирование приложения и описать полученные результаты;

– представить решения по развертыванию и дальнейшему сопровождению приложения;

– оформить пояснительную записку и защитить КР в соответствии с действующими стандартами.

Выполнение курсовой работы должно соответствовать утвержденному календарному графику и включать следующие этапы (таблица 5.1).

Таблица 5.1 – Календарный график выполнения курсовой работы

Номер этапа	Наименование этапа	Неделя выполнения
1.	Выбор темы и согласование задания на курсовую работу	1 – 3
2.	Изучение информационных процессов в данной предметной области: описание существующей схемы управления объектом и ее недостатков, составление схемы инфопотоков, анализ существующих компьютерных разработок по выбранной теме	3 – 4
3.	Постановка задачи на создание приложения: формализация требований ко всем частям приложения и системе в целом, планирование ресурсов и итераций командного проекта	5 – 6
4.	Изучение функциональной части и построение функциональных моделей, диаграмм поведения и реализации UML, иллюстрирующих решение задачи информатизации и реализацию автоматизированных функций приложения	6 – 7
5.	Создание репозитория для командного проекта, организация версионного контроля компонентов приложения. Выбор средств разработки программного приложения коллективным способом	7 – 8
6.	Программная реализация приложения. Описание микросервисов приложения и их взаимодействия между собой посредством API	9 – 10
7.	Тестирование приложения Unit-тестами, системное и исследовательское тестирование	11 – 12
8.	Разработка решений по развертыванию и дальнейшему сопровождению отдельных микросервисов и приложения в целом	13 – 14
9.	Оформление пояснительной записки к курсовой работе	14 – 15
10.	Сдача на проверку и защита курсовой работы	16 – 17

Более детальное описание структуры пояснительной записки к курсовой работе представлено в подразделе 5.5.

5.4. Тематика курсового проектирования

В соответствии с квалификационной характеристикой направлений подготовки цифровой экономики возможны следующие основные направления тематики магистерских курсовых работ:

1. Разработка социальных и экономических информационных систем, подсистем и модулей, обеспечивающих обработку информации по комплексу задач бизнес-процессов и функций управления ресурсами в различных сферах деятельности.
2. Разработка информационных управляющих систем (подсистем, модулей) для управления различными экономическими объектами.
3. Разработка систем (подсистем) компьютерного мониторинга, контроля и анализа состояния различных организационных объектов и их среды.
4. Разработка корпоративных информационных систем для информатизации бизнес-процессов на территориально распределенных объектах.
5. Разработка облачных технологий и сервисов для распределенной обработки данных или распределенного управления организационными объектами.
6. Разработка цифровых сервисов для цифровизации отдельных процессов и задач в разных областях деятельности.
7. Разработка компонентов цифровых платформ и цифровых сервисов на основе Data Mining для организации деятельности в разных областях деятельности.
8. Разработка систем (подсистем) информационной поддержки принятия решения для менеджеров различного уровня.
9. Разработка интеллектуальных систем для решения задач планирования, прогнозирования, поддержки принятия решений.
10. Разработка систем интеллектуального анализа больших данных (Data Mining) для решения задач планирования, прогнозирования, поддержки принятия решений.
11. Проектирование информатизации предприятий разных сфер на основе существующих систем электронного бизнеса (ERP, CRM, ECM, HRM, SCM).
12. Разработка информационно-поисковых и экспертных систем обработки информации для принятия решений в разных сферах в условиях риска и неопределенности.
13. Разработка учебных информационных систем для тестирования и проверки знаний, умений с использованием средств искусственного интеллекта.
14. Построение экспертных систем поддержки принятия решений в разных областях применения.

15. Разработка информационно-аналитических систем для анализа слабо структурированных проблемных ситуаций в организационных средах с использованием Data Science и средств искусственного интеллекта.

16. Исследование и адаптация различных объектов и процессов с применением статистического имитационного моделирования.

17. Создание и исследование информационных технологий управления в организационных системах.

18. Разработка системы риск-менеджмента в социальных и экономических средах.

19. Модели и алгоритмы обработки экспертных знаний в производственных, социально-технических и социально-экономических системах, а также обеспечения их технической, экономической и информационной устойчивости.

5.5. Требования к структуре курсовой работы

Результаты выполнения курсовой работы оформляются каждым студентом в виде пояснительной записки. ПЗ содержит весь проектный материал по созданию ИС, представленный в виде текста, расчетных формул, таблиц, рисунков и др. Типовая структура ПЗ по курсовому проектированию следующая:

- титульный лист типовой формы (см. приложение А);
- задание на курсовое проектирование на типовом бланке (см. приложение Б);
- реферат;
- содержание;
- введение;
- основная часть, состоящая из нескольких разделов;
- заключение;
- перечень ссылок;
- приложения.

Задание на КР размещается сразу после титульного листа и является документом, который определяет объем и порядок выполнения работы, регламентирует исходные данные, функциональные требования к ИС и основные результаты проектирования. Задание утверждается руководителем КР.

Реферат предназначен для ознакомления с проектом. Он должен быть коротким, информативным и содержать сведения, которые позволяют представить сущность КР. Реферат должен содержать:

- библиографическое описание КР;
- текст реферата;
- перечень ключевых слов.

Библиографическое описание содержит сведения об объеме ПЗ, количество иллюстраций, таблиц, приложений, а также количество источников в перечне ссылок. Это описание выполняется согласно требованиям

действующих стандартов по библиотечному и издательскому делу и имеет такой вид:

Курсовая работа: ___ стр., ___ рис., ___ табл., ___ прил., ___ ист.

При этом сведения приводятся с учетом всех приложений.

Текст реферата должен отображать информацию, представленную в ПЗ и, как правило, в определенной последовательности:

- объект разработки или исследования;
- цель работы;
- методы и средства разработки;
- результаты работы;
- основные технико-эксплуатационные, конструктивные и технологические характеристики;
- значимость работы и выводы.

Объем реферата не более 850 знаков. Реферат располагается на одной странице формата А4.

Ключевые слова, существенные для раскрытия сути КР, размещают после текста реферата. Перечень ключевых слов содержит от 5 до 15 слов (словосочетаний), напечатанных прописными буквами в именительном падеже в строку через запятые.

Лист реферата выполняется с основной надписью.

Содержание располагают после реферата, начиная с новой страницы.

Содержание включает:

- введение;
- последовательно перечисленные названия всех разделов, подразделов;
- заключение;
- перечень использованных источников;
- последовательно перечисленные приложения с заголовками.

В содержании указываются номера страниц, на которых расположено начало соответствующего материала.

Нумерация страниц в ПЗ сквозная, начинается с титульного листа, но номера страниц проставляются, начиная с содержания.

Перечень условных сокращений и аббревиатур располагают на отдельной странице. В перечне в виде списка перечисляются обозначения и сокращение, использованные в тексте ПО, а через короткое тире по правую сторону приводится их расшифровка.

Введение пояснительной записки должно содержать оценку современного состояния и актуальности решаемой задачи, краткое описание используемых методов и средств решения задачи, цель работы и область ее применения.

Введение располагается с новой страницы.

Основная часть пояснительной записки должна включать следующие разделы:

1. Характеристика объекта информатизации.
2. Техническое задание на создание приложения.
3. Функциональная структура приложения.

4. Формирование среды разработки приложения.
5. Программная реализация приложения.
6. Тестирование приложения.
7. Развертывание приложения.

Примерный объем пояснительной записки – 30–50 страниц без учета приложений.

В разделе 1 дается краткая характеристика предметной области. Приводится подробная информация о структуре объекта информатизации, выполняемых функциях и решаемых задачах. Особо выделяется бизнес-процесс, подлежащий информатизации (процесс информатизации), дается его определение, формулируется цель (процесса, а не информатизации), приводятся правила организации и возможные ограничения. Описание должно давать представление о масштабе деятельности объекта, численности персонала, занятого в бизнес-процессе, характере взаимодействия с другими подразделами и задачами.

В данном разделе необходимо разместить иллюстрацию, характеризующую объект и процесс информатизации. Это может быть какая-либо модель, формально представляющая объект, функционально-структурная схема, организационная диаграмма и т. п.

В разделе 2 приводится назначение разрабатываемого продукта (ИС) и цели его создания. При описании назначения указывают вид деятельности, которая автоматизируется (расчет, управление, диагностика, планирование, прогнозирование, проектирование и т. п.) и перечень объектов информатизации, на которых предполагается ее использовать. При описании цели создания продукта приводят наименование и необходимые значения технических, технологических, производственно-экономических или других показателей объекта информатизации, которые должны быть достигнуты в результате создания ИС, и указывают критерии оценки достижения указанных целей.

Указывается перечень задач и автоматизируемых функций ИС, их назначение, основные характеристики. Приводятся требования к режимам функционирования ИС в целом и по отдельным задачам (функциям), разбиваются на несколько пунктов в зависимости от количества задач (функций), например:

1. Требования к задаче (или функции) “...”.
2. Требования к задаче (или функции) “...”.
3. Требования к задаче (или функции) “...”.

В каждом пункте указывается полное название задачи, функции или отдельной задачи. Надо помнить, что задача ИС – это функция или часть функции ИС, являющаяся формализованной совокупностью действий, выполнение которых приводит к результату заданного вида. Поэтому в качестве задач надо выбирать такие, для которых можно четко сформулировать результат. Такими задачами, например, могут быть: задача введения входных данных, задача формирования статистической отчетности, задача диагностики состояния объекта, задача анализа данных и принятия решений и др.

Кроме того, указывают нефункциональные требования к ИС (не менее пяти), отражающие ее качественные характеристики согласно ГОСТ Р ИСО/МЭК 9126-93.

Также предъявляются требования к обеспечивающим подсистемам ИС:

− требования к организации данных, которые должны храниться в ИС, включая основные сущности БД: уместно представить диаграмму классов UML или логическую модель данных;

− требования к составу, области применения (ограничение) и средствам использования в системе математических методов и моделей, типичных алгоритмов и алгоритмов, которые подлежат разработке;

− требования к программному обеспечению (ПО), в том числе к операционной среде, к инструментальным средствам разработки ИС и управлению ее жизненным циклом, к составу и функциям специального ПО, подлежащего разработке;

− требования к техническому обеспечению, а именно к конструктивным и эксплуатационным характеристикам средств ТО ИС, к конфигурациям компьютеров, серверов и клиентских устройств.

При формулировании требований к обеспечивающим подсистемам необходимо учитывать, что эти требования обусловлены спецификой задач, которые решаются в ИС, и требованиями к другим частям ИС.

В разделе 3 приводятся проектные решения по функциональной структуре ИС согласно требованиям к задачам (функциям) ИС, приведенным в разделе 2. Приводится краткое (не более 100 слов) описание процесса функционирования объекта информатизации в условиях функционирования ИС. Составляется описание процессов выполнения каждой задачи (функции). Выделяются существенные структурные элементы и действия, определяются связи между ними. Уместно построить функционально-структурную схему ИС в виде диаграмм поведения UML. Представить структурную схему микросервисной архитектуры ИС по принципу: «одна задача – один сервис».

В разделе 4 отражается обоснованный выбор программного средства для разработки специального ПО. Это должен быть какой-либо фреймворк для создания web-приложения или web-сервер с PHP. Отражается установка выбранных СУБД и фреймворка для создания сервисов web-приложения, установка и настройка Docker Debian.

Также отражается создание репозитория проекта ИС для осуществления версионного контроля проекта (может быть использована любая система контроля версий: Microsoft Visual SourceSafe, IBM ClearCase, CVS, Subversion, Perforce, Git, Mercurial, Bazaar, Darcs и др.; рекомендуемая система управления версиями – Git), создание инициализирующего коммита, размещение в репозитории созданного проекта приложения.

Все шаги документируются при помощи иллюстраций и кратких комментариев к ним.

В разделах 5 и 6 рассматривается первая половина этапов цикла DevOps – CD: «Написание кода», «Сборка», «Тестирование».

В разделе 5 отражается процесс рабочего проектирования (реализации) ИС: создание БД, образа для реализации приложения, разработка и контейнеризация web-приложения посредством Docker с обеспечением логирования http-подключений (время подключения, IP-адрес клиента), размещение текущей версии проекта в удаленном репозитории Git. Примерная структура раздела:

- основные файлы приложения (при использовании Django – urls.py, models.py, settings.py);
- код, реализующий бизнес-логику приложения;
- Docker-файл;
- средства web-интерфейса (функция index, шаблон страницы);
- файл docker-compose.yaml.

Код, реализующий бизнес-логику приложения, если он достаточно объемный, может быть вынесен в приложение.

Пример реализации простейшего web-приложения с микросервисной архитектурой и его краткого описания представлен в приложении В.

Все приведенные компоненты описываются. Целесообразно привести структуру приложения в виде диаграммы компонентов UML, пример которой показан на рис. В.6.

Таким образом реализуется первый этап цикла DevOps части CD: «Написание кода».

В разделе 6 реализуются второй и третий этапы DevOps части CD: «Сборка» и «Тестирование». Необходимо отразить настройку и работу автоматической сборки приложения на базе выбранной системы контроля версий и последующее тестирование проекта. Описать настроенные триггеры для активации сборки. Представить листинги unit-тестов и результаты их выполнения, текстовые описания и результаты прогона других тестов в соответствии с возможностями используемых средств автоматизированного тестирования. Также необходимо доказательно и лаконично изложить доводы по поводу того, что ИС соответствует функциональным требованиям, которые сформулированы в ТЗ (раздел 2).

В разделе 7 необходимо представить и описать процедуру доставки приложения ИС до целевой системы и тестирование его работоспособности, используя CI/CD Pipeline. Рассмотреть вторую половину этапов цикла DevOps – CD: «Развертывание», «Поддержка и мониторинг», «Планирование» в условиях потенциального или реального использования ИС конечными пользователями. Для этапа «Планирование» представить проект нового функционала и план доработок для захода на следующий цикл DevOps CI/CD.

В целом текст основной части ПЗ должен давать полное представление о всех этапах жизненного цикла ИС, а разделы 5–7 должны полностью отражать все этапы цикла DevOps CI/CD. При этом множественные рисунки и копии экрана, не несущие основной информационной нагрузки, целесообразно располагать в приложениях к ПЗ.

Заключение (краткие выводы студента по курсовой работе) должно содержать краткое описание сущности решенной задачи, содержать оценку

работоспособности разработанной ИС и правильности полученных результатов, а также перспективы развития ИС.

Текст заключения может разделяться на пункты.

Перечень использованных источников должен содержать список использованной литературы, оформленный согласно требованиям ГОСТ.

Приложения размещают после основной части ПЗ. В них подаются разные материалы (таблицы, рисунки, схемы, формы документов, листинги программ и др.), которые являются необходимым дополнением основного материала, но не могут быть последовательно размещены в основной части ПЗ из-за большого объема или способа изложения. Приложения должны содержать: заполненные макеты входных документов, структуру БД ИС, листинги исходных текстов программного приложения ИС, распечатки сформированных отчетов, материалы, полученные при тестировании, а также по данным контрольного примера. Приложения обозначаются буквами, начиная с А, Б и т. д.

Текст пояснительной записки выполняется в редакторе MS Word 2003 и выше в печатном виде на стандартных листах формата А4 (210*290 мм) с соблюдением полей: верхнее и нижнее по 20 мм, левое – 25 мм, правое – 15 мм. Шрифт для набора текста – Times New Roman, размер – 14 пт, интервал – полуторный. Страницы ПЗ должны быть заполнены полностью. Исключение составляет последняя страница раздела. В конце остальных страниц допускается не более 2–3 пустых интервалов, свободное место необходимо дополнять текстом, масштабировать рисунки и таблицы и т. д.

Графический материал: схема данных БД, диаграммы UML и BPWin, структура программного приложения, экранные формы, укрупненные блок-схемы алгоритмов, диаграммы для демонстрации результатов тестирования помещаются по тексту в виде рисунков MS Word.

Более детальные требования к оформлению ПЗ представлены в подразделе 5.6.

5.6. Требования к оформлению пояснительной записки

Пояснительная записка должна быть выполнен печатным способом с использованием компьютера и принтера на одной стороне листа белой бумаги формата А4 через полтора интервала. Для создания текста ПЗ рекомендуется использовать текстовый редактор MS Word. Шрифт на протяжении всего документа должен быть одинаковый: Times New Roman 14-го размера, за исключением оформления иллюстраций, таблиц и формул, в которых допускается использовать шрифт меньшего размера.

Текст ПЗ следует печатать, соблюдая следующие размеры полей: слева – 25 мм, справа – 15 мм, сверху и снизу – 20 мм.

При оформлении текста работы следует использовать абзацный отступ, который должен составлять 15-17 мм от левого поля документа.

Вне зависимости от способа выполнения ПЗ качество напечатанного текста и оформления иллюстраций, таблиц, распечаток с ПЭВМ должно удовлетворять требованию их четкого воспроизведения.

Наименования структурных элементов ПЗ «Реферат», «Содержание», «Обозначения и сокращения», «Введение», «Заключение», «Список использованных источников» служат заголовками структурных элементов ПЗ.

Основную часть ПЗ следует делить на разделы, подразделы и пункты. Разделы, подразделы и пункты следует нумеровать арабскими цифрами и записывать с абзацного отступа. Разделы должны иметь порядковую нумерацию в пределах всего текста, за исключением приложений. Каждый раздел работы должен начинаться с новой страницы.

Разделы, подразделы должны иметь заголовки. Пункты, как правило, заголовков не имеют. Заголовки должны четко и кратко отражать содержание разделов, подразделов. Заголовки разделов, подразделов и пунктов следует печатать с абзацного отступа с прописной буквы без точки в конце, не подчеркивая. Если заголовок состоит из двух предложений, их разделяют точкой.

Нумерация страниц ПЗ. Страницы ПЗ следует нумеровать арабскими цифрами, соблюдая сквозную нумерацию по всему тексту ПЗ. Номер страницы проставляют в центре нижней части листа без точки. Титульный лист включают в общую нумерацию страниц. Номер страницы на титульном листе не проставляют. Иллюстрации и таблицы, расположенные на отдельных листах, включают в общую нумерацию страниц ПЗ.

Нумерация разделов, подразделов, пунктов, подпунктов ПЗ. Разделы ПЗ должны иметь порядковые номера в пределах всего документа, обозначенные арабскими цифрами без точки и записанные с абзацного отступа. Подразделы должны иметь нумерацию в пределах каждого раздела. Номер подраздела состоит из номеров раздела и подраздела, разделенных точкой. В конце номера подраздела точка не ставится. Разделы, как и подразделы, могут состоять из одного или нескольких пунктов. Если документ не имеет подразделов, то нумерация пунктов в нем должна быть в пределах каждого раздела, и номер пункта должен состоять из номеров раздела и пункта, разделенных точкой. В конце номера пункта точка не ставится.

Иллюстрации. Иллюстрации (чертежи, графики, схемы, компьютерные распечатки, диаграммы, фотоснимки) следует располагать в ПЗ непосредственно после текста, в котором они упоминаются впервые, или на следующей странице. Иллюстрации могут быть в компьютерном исполнении, в том числе и цветные. На все иллюстрации должны быть даны ссылки в ПЗ.

Иллюстрации, за исключением иллюстраций в приложениях, следует нумеровать арабскими цифрами сквозной нумерацией. Если рисунок один, то он обозначается «Рисунок 1». Слово «Рисунок» и его наименование располагают посередине строки. Допускается нумеровать иллюстрации в пределах раздела. В этом случае номер иллюстрации состоит из номера раздела и порядкового номера иллюстрации, разделенных точкой. Например, Рисунок 1.1. Иллюстрации при необходимости могут иметь наименование и

пояснительные данные (подрисуночный текст). Слово «Рисунок» и наименование помещают после пояснительных данных и располагают следующим образом: Рисунок 1.1 – Функциональная схема. Иллюстрации каждого приложения обозначают отдельной нумерацией арабскими цифрами с добавлением перед цифрой обозначения приложения. Например, Рисунок А.3. При ссылках на иллюстрации следует писать «... в соответствии с рисунком 2» при сквозной нумерации и «... в соответствии с рисунком 1.2» при нумерации в пределах раздела.

Таблицы применяют для лучшей наглядности и удобства сравнения показателей. Название таблицы при его наличии должно отражать ее содержание, быть точным, кратким. Название таблицы следует помещать над таблицей слева, без абзацного отступа в одну строку с ее номером через тире.

При переносе части таблицы название помещают только над первой частью таблицы, нижнюю горизонтальную черту, ограничивающую таблицу, не проводят.

Таблицу следует располагать в ПЗ непосредственно после текста, в котором она упоминается впервые, или на следующей странице. На все таблицы должны быть ссылки в ПЗ. При ссылке следует писать слово «таблица» с указанием ее номера.

Таблицу с большим количеством строк допускается переносить на другой лист (страницу). При переносе части таблицы на другой лист (страницу) слово «Таблица» и ее номер указывают один раз справа над первой частью таблицы, над другими частями пишут слово «Продолжение» и указывают номер таблицы, например: «Продолжение таблицы 1». При переносе таблицы на другой лист (страницу) заголовок помещают только над ее первой частью.

Таблицу с большим количеством граф допускается делить на части и помещать одну часть под другой в пределах одной страницы. Если строки и графы таблицы выходят за формат страницы, то в первом случае в каждой части таблицы повторяется головка, во втором случае – боковик.

Если повторяющийся в разных строках графы таблицы текст состоит из одного слова, то его после первого написания допускается заменять кавычками; если из двух и более слов, то при первом повторении его заменяют словами «То же», а далее – кавычками. Ставить кавычки вместо повторяющихся цифр, марок, знаков, математических и химических символов не допускается. Если цифровые или иные данные в какой-либо строке таблицы не приводят, то в ней ставят прочерк.

Таблицы, за исключением таблиц приложений, следует нумеровать арабскими цифрами сквозной нумерацией. Допускается нумеровать таблицы в пределах раздела. В этом случае номер таблицы состоит из номера раздела и порядкового номера таблицы, разделенных точкой. Таблицы каждого приложения обозначают отдельной нумерацией арабскими цифрами с добавлением перед цифрой обозначения приложения. Если в документе одна таблица, то она должна быть обозначена «Таблица 1» или «Таблица В.1», если она приведена в приложении В.

Заголовки граф и строк таблицы следует писать с прописной буквы в единственном числе, а подзаголовки граф – со строчной буквы, если они составляют одно предложение с заголовком, или с прописной буквы, если они имеют самостоятельное значение. В конце заголовков и подзаголовков таблиц точки не ставят. Таблицы слева, справа и снизу, как правило, ограничивают линиями. Допускается применять размер шрифта в таблице меньший, чем в тексте. Горизонтальные и вертикальные линии, разграничивающие строки таблицы, допускается не проводить, если их отсутствие не затрудняет пользование таблицей. Заголовки граф, как правило, записывают параллельно строкам таблицы. При необходимости допускается перпендикулярное расположение заголовков граф. Головка таблицы должна быть отделена линией от остальной части таблицы.

Формулы и уравнения. Уравнения и формулы следует выделять из текста в отдельную строку. Выше и ниже каждой формулы или уравнения должно быть оставлено не менее одной свободной строки. Если уравнение не умещается в одну строку, то оно должно быть перенесено после знака равенства (=) или после знаков плюс (+), минус (-), умножения (х), деления (:) или других математических знаков, причем знак в начале следующей строки повторяют. При переносе формулы на знаке, символизирующем операцию умножения, применяют знак “Х”.

Пояснение значений символов и числовых коэффициентов следует приводить непосредственно под формулой в той же последовательности, в которой они даны в формуле.

Формулы в ПЗ следует нумеровать порядковой нумерацией в пределах всего ПЗ арабскими цифрами в круглых скобках в крайнем правом положении на строке. Одну формулу обозначают – (1). Формулы, помещаемые в приложениях, должны нумероваться отдельной нумерацией арабскими цифрами в пределах каждого приложения с добавлением перед каждой цифрой обозначения приложения, например формула (В.1). Ссылки в тексте на порядковые номера формул дают в скобках. Пример – ... в формуле (1). Допускается нумерация формул в пределах раздела. В этом случае номер формулы состоит из номера раздела и порядкового номера формулы, разделенных точкой, например (3.1).

Ссылки. В ПЗ допускаются ссылки на данный документ, стандарты, технические условия и другие документы при условии, что они полностью и однозначно определяют соответствующие требования и не вызывают затруднений в пользовании документом.

Ссылаться следует на документ в целом или его разделы и приложения. Ссылки на подразделы, пункты, таблицы и иллюстрации не допускаются, за исключением подразделов, пунктов, таблиц и иллюстраций данного документа.

При ссылках на стандарты и технические условия указывают только их обозначение, при этом допускается не указывать год их утверждения при условии полного описания стандарта в списке использованных источников в соответствии с ГОСТ 7.1. Ссылки на использованные источники следует приводить в квадратных скобках.

Перечень обозначений и сокращений. Перечень должен располагаться столбцом. Слева в алфавитном порядке приводят сокращения, условные обозначения, символы, единицы физических величин и термины, справа – их детальную расшифровку.

Список использованных источников. Сведения об источниках следует располагать в порядке появления ссылок на источники в тексте ПЗ и нумеровать арабскими цифрами без точки и печатать с абзацного отступа.

Приложения оформляют как продолжение данного документа на последующих его листах или выпускают в виде самостоятельного документа. В тексте документа на все приложения должны быть даны ссылки. Приложения располагают в порядке ссылок на них в тексте документа, за исключением справочного приложения «Библиография», которое располагают последним.

Каждое приложение следует начинать с новой страницы с указанием наверху посередине страницы слова «Приложение», его обозначения и степени. Приложение должно иметь заголовок, который записывают симметрично относительно текста с прописной буквы отдельной строкой. Приложения обозначают заглавными буквами русского алфавита, начиная с А, за исключением букв Е, З, Й, О, Ч, Ь, Ы, Ъ. После слова «Приложение» следует буква, обозначающая его последовательность. Допускается обозначение приложений буквами латинского алфавита, за исключением букв I и O. В случае полного использования букв русского и латинского алфавитов допускается обозначать приложения арабскими цифрами. Если в документе одно приложение, оно обозначается «Приложение А».

Текст каждого приложения при необходимости может быть разделен на разделы, подразделы, пункты, подпункты, которые нумеруют в пределах каждого приложения. Перед номером ставится обозначение этого приложения. Приложения должны иметь общую с остальной частью документа сквозную нумерацию страниц. При необходимости такое приложение может иметь «Содержание». Приложениям или частям, выпущенным в виде самостоятельного документа, обозначение присваивают как части документа с указанием в коде документа ее порядкового номера.

5.7. Требования к демонстрационным материалам

Традиционным способом представления на защите КР результатов, полученных при ее выполнении, является демонстрация компьютерным способом слайдов презентации, выполненной посредством инструмента MS Power Point либо его аналогов, таких как Open Office Impress, Prezi, Keynote, Slides, Slidebean, Google Slides и др.

Содержание презентации должно быть примерно следующим:

- титульный слайд, содержащий информацию об учебном заведении и кафедре, о виде и названии работы, а также полные данные об исполнителе и руководителе работы;
- суть решаемой в КР задачи, ее актуальность и значимость, характеристика объекта информатизации (1-2 слайда);

− цель работы и конкретизирующие ее задачи, объект и предмет исследования при выполнении КР (1 слайд);

− систематизированные результаты анализа предметной области (характеристика процесса информатизации) и подобных существующих решений задачи (2-3 слайда);

− математическая постановка задачи (1-2 слайда);

− основные проектные (информационные и математические модели, структурные и функциональные схемы, методы и средства разработки, стек используемых технологий) и рабочие (фактические результаты применения используемых технологий) решения (3-5 слайдов);

− основные результаты тестирования полученного решения (2-4 слайда);

− сведения об апробации полученных результатов: научные публикации, участие в конференциях (при наличии) (1-2 слайда);

− сведения об использовании полученных результатов на производстве (при наличии) (1-2 слайда);

− выводы по работе (1-2 слайда);

− заключительный слайд.

Общепринятыми требованиями к слайдам презентации являются следующие:

− презентация должна быть выполнена в сдержанно-элегантном строгом стиле;

− каждый слайд должен быть критичен, все лишнее, избыточное, некритичное подлежит удалению из презентации;

− каждый выдвигаемый на слайде тезис должен быть обоснован;

− каждый слайд должен быть озаглавлен;

− содержание слайдов должно быть сбалансированным в части использования текстовой и графической информации;

− все обозначения, используемые в демонстрируемых математических выражениях, должны быть расшифрованы на этом же слайде;

− текстовый материал необходимо выполнять шрифтами с одинаковой толщиной линий, например, Arial или Verdana размером не менее 24 пунктов (использование шрифта меньшего размера допускается в исключительных случаях, например, на сложных графических объектах, в таблицах или в примечаниях к слайду);

− размер шрифта на графических объектах и в таблицах не должен быть меньше 18 пунктов;

− шрифт заголовка слайда должен быть заметно больше шрифта текстовых блоков этого слайда;

− в случае если на слайд выносится только текстовая информация, ее следует структурировать в формате нумерованного или маркированного списка, стремясь при этом к минимизации объема текста и отражая самую суть;

− все слайды, за исключением первого, должны быть пронумерованы.

Таким образом, учитывая вариативность объема по некоторым пунктам в зависимости от специфики решаемой задачи, общий объем презентации в норме может составлять от 10 до 20 слайдов. Важно понимать, что объем

демонстрируемого студентом материала должен соответствовать регламенту выступления. При оценивании презентации по этому параметру следует учитывать, что каждый слайд необходимо демонстрировать столько времени, сколько нужно слушателям для неспешного ознакомления с его содержанием.

Также следует учесть, что между количеством слайдов презентации и продолжительностью доклада связь нелинейная, которая определяется спецификой решаемой задачи, масштабом проведенного исследования и мастерством публичного выступления докладчика. Поэтому отладку презентации перед защитой целесообразно осуществлять посредством многократных тренировок доклада на время.

5.8. Порядок сдачи на проверку и защиты курсовой работы

Защита курсовой работы проводится с целью:

– определения уровня владения студентом универсальными, общепрофессиональными и профессиональными компетенциями в соответствии с требованиями образовательного стандарта по соответствующему направлению подготовки;

– предоставления студенту возможности тренировки в публичном выступлении и демонстрации полученных результатов;

– получения объективной оценки работы.

Защиты КР проводятся в соответствии с календарным графиком (см. таблицу 5.1) в последние 2 недели учебного семестра. Очередность прохождения защиты КР студентами и конкретные даты защит устанавливаются руководителем с учетом предпочтений студентов и оформляются в виде графика защиты КР. График защиты КР подписывает руководитель КР и заведующий кафедрой. График защиты доводится до сведения студентов не позже, чем за 20 дней до конца зачетной недели.

Защита проводится комиссией, в состав которой, кроме руководителя, могут входить другие члены профессорско-преподавательского состава кафедры.

Не менее чем за 3 дня до даты защиты, определяемой согласно графику защиты (см. выше), студент должен сдать полностью подготовленную ПЗ на проверку. ПЗ представляется в электронном виде и загружается в ЭИОС.

В процессе защиты студент должен в течение 5–7 минут кратко раскрыть актуальность, цель и задачи курсовой работы, содержание работы, используемый модельно-методический аппарат, используемые технологии DevOps-инженерии и т. д., изложить полученные результаты и выводы, после чего ответить на вопросы членов комиссии.

Представляется целесообразным подготовка конспекта доклада, однако студент должен свободно излагать содержание работы, не пользуясь постоянно подготовленным докладом. Выступление должно быть иллюстрировано таблицами, графиками, схемами и т. п. Демонстрационный материал должен

быть представлен методом проекции. При ответах на вопросы студент имеет право пользоваться пояснительной запиской.

По результатам защиты комиссия принимает решение об оценке курсовой работы. Оценка КР при ее защите основывается на следующих критериях:

– соответствие работы утвержденной теме;

– степень решения поставленных задач, обоснованность, значимость и полнота сделанных выводов и предложений;

– качество выступления и ответов на поставленные вопросы при защите КР.

Оценка по результатам защиты выставляется по четырехбалльной шкале: «отлично», «хорошо», «удовлетворительно», «неудовлетворительно».

Оценка **«отлично»** выставляется, если:

– работа выполнена в соответствии с заданием, содержание и оформление работы в полной мере соответствует заявленной теме и требованиям методических указаний;

– выступление на защите структурировано, раскрыты актуальность темы, цель и задачи работы, структура объекта информатизации, логика представленных выводов;

– длительность выступления соответствует регламенту;

– ответы на вопросы членов комиссии логичны, раскрывают сущность вопроса, показывают самостоятельность и глубину изучения проблемы;

– применение технологий DevOps-инженерии как в самой работе, так и во время выступления широко;

– эффективная работа студента в соответствии с календарным графиком (см. таблицу 5.1) с положительной динамикой в течение семестра.

Оценка **«хорошо»** выставляется, если:

– работа выполнена в соответствии с заданием, содержание и оформление работы в полной мере соответствует заявленной теме и требованиям методических указаний;

– выступление на защите структурировано, допускаются одна-две неточности при раскрытии актуальности темы, целей и задач работы, объекта информатизации, допускается погрешность в логике выведения одного из наиболее значимых выводов, которая устраняется в ходе дополнительных уточняющих вопросов;

– длительность выступления в основном соответствует регламенту;

– в ответах на вопросы членов комиссии допущено нарушение логики, но в целом раскрыта сущность вопроса, показывающая самостоятельность и глубину изучения проблемы;

– применение технологий DevOps-инженерии как в самой работе, так и во время выступления ограничено;

– работа студента не в полной мере соответствует календарному графику (см. таблицу 5.1).

Оценка **«удовлетворительно»** выставляется, если:

– работа выполнена в соответствии с заданием, но содержание и оформление работы не в полной мере соответствует теме и требованиям методических указаний;

– выступление на защите структурировано, допускаются неточности при раскрытии актуальности темы, целей и задач работы, объекта информатизации, допущена грубая погрешность в логике выведения одного из наиболее значимых выводов, которая при указании на нее устраняется с трудом;

– длительность выступления не соответствует регламенту;

– ответы на вопросы членов комиссии не раскрывают до конца сущности вопросов, показывают недостаточную самостоятельность и глубину изучения проблемы;

– применение технологий DevOps-инженерии как в самой работе, так и во время защиты недостаточно;

– работа студента в основном не соответствует календарному графику (см. таблицу 5.1), большая часть работы выполнена в последние 2 недели;

– в ходе защиты студент демонстрирует понимание содержания допущенных им ошибок.

Оценка **«неудовлетворительно»** выставляется, если:

– работа выполнена с нарушением задания, содержание и оформление ПЗ не отвечает требованиям методических указаний;

– выступление на защите не структурировано, недостаточно раскрываются актуальность темы, цели и задачи работы, объект информатизации, допускаются грубые погрешности в логике выведения нескольких из наиболее значимых выводов, которые при указании на них не устраняются;

– длительность выступления не соответствует регламенту;

– ответы на вопросы членов комиссии не раскрывают сущности вопроса, показывают отсутствие самостоятельности и глубины изучения проблемы студентом;

– технологии DevOps-инженерии не применялись ни в работе, ни при выступлении;

– работа студента полностью не соответствует календарному графику (см. таблицу 5.1), материал ПЗ руководителю представлен впервые на защите;

– в ходе процедуры защиты студент демонстрирует непонимание содержания допущенных им ошибок.

В случае неявки студента на защиту работы по уважительной причине студенту назначается защита в другое время, но не позже даты окончания зачетной недели.

Заключение

Данное учебное пособие отражает DevOps-процесс создания и использования информационных систем. Рассмотренные в пособии технологии DevOps-инженерии являются классическими и широко используемыми в современной практике производства ПО. Проработка данных технологий формирует у студентов представление о базовых принципах непрерывной разработки и развертывания ИС с использованием современных инструментальных средств.

Материал учебного пособия сформирован на основе учебных курсов «Современные технологии разработки ПО» и «Методология и технология проектирования ИС», реализуемых в СибГУТИ, а также «Автоматизация программирования» и «Коллективная разработка приложений», реализуемых в КубГУ, читаемых авторами на протяжении последних лет. Также при подготовке учебного пособия использован опыт реализации в СибГУТИ программы дополнительного профессионального образования «DevOps-инженерия». Сочетание теоретической базы по указанным дисциплинам и программе переподготовки с опытом авторов в применении данных знаний в учебном процессе в разных образовательных организациях позволило создать учебное пособие, ориентирующее студентов на реализацию полного жизненного цикла ИС, включая качественную реализацию ее обеспечивающих компонентов, конфигурирование в единый хорошо документированный программный продукт и внедрение в производственную сферу в непрерывном режиме.

Реальный производственный опыт авторов в области создания и эксплуатации информационных и программных систем в соответствии с методологией DevOps делает пособие достаточно практичным инструментом для начинающего DevOps-инженера, коим в современном мире перманентно развивающихся цифровых технологий могут и должны считать себя выпускники вузов, обучающиеся по направлениям подготовки цифровой экономики.

Список рекомендуемой литературы

Список основной литературы:

1. Методология и технология разработки программных систем: методы и модели программной инженерии: учебное пособие / А.Н. Полетайкин, Н.Ю. Добровольская; Министерство науки и высшего образования Российской Федерации, Кубанский государственный университет. − Краснодар: Кубанский гос. ун-т, 2025. − 229 с. – URL: https://sibsutis.ru/upload/0f5/Software_engineering_1.pdf.
2. Митра Р., Надареишвили И. Микросервисы. От архитектуры до релиза. – СПб.: Питер, 2023. – 336 с.: ил. – (Серия «Бестселлеры O’Reilly»).
3. Архитектурные решения информационных систем : учебник для вузов / А. И. Водяхо, Л. С. Выговский, В. А. Дубенецкий, В. В. Цехановский. – 3-е изд., стер. – Санкт-Петербург : Лань, 2022. – 356 с. – ISBN 978-5-507-44710-7. – Текст : электронный // Лань : электронно-библиотечная система. – URL: https://e.lanbook.com/book/254624 (дата обращения: 14.06.2023). – Режим доступа: для авториз. пользователей.
4. Лаврищева, Е. М. Программная инженерия и технологии программирования сложных систем : учебник для вузов / Е. М. Лаврищева. – 2-е изд., испр. и доп. – Москва : Издательство Юрайт, 2023. – 432 с. – (Высшее образование). – Текст : электронный // Образовательная платформа Юрайт [сайт]. – URL: https://urait.ru/bcode/513067 (дата обращения: 26.02.2025).
5. Доррер, Г. А. Методология программной инженерии : учебное пособие / Г. А. Доррер. – Красноярск : Сибирский государственный университет науки и технологий имени академика М.Ф. Решетнева, 2021. – 190 с.
6. Меле, А. Django 2 в примерах / А. Меле ; перевод Д. В. Плотникова. – Москва : ДМК Пресс, 2019. – 408 с. – ISBN 978-5-97060-746-6. – Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт]. – URL: https://www.iprbookshop.ru/126199.html. – Режим доступа: для авторизир. пользователей.
7. Меле А. Django 4 в примерах / пер. с англ. А. В. Логунова. – М.: ДМК Пресс, 2023. – 800 с.
8. Шениг, Г. -Ю. PostgreSQL 11. Мастерство разработки / Г. -Ю. Шениг ; перевод А. А. Слинкин. – Москва : ДМК Пресс, 2019. – 352 с. – ISBN 978-5-97060-671-1. – Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт]. – URL: https://www.iprbookshop.ru/125100.html. – Режим доступа: для авторизир. пользователей.
9. Ригс, С. Администрирование PostgreSQL 9. Книга рецептов / С. Ригс, Х. Кросинг ; перевод Е. В. Самохвалов. – Москва : ДМК Пресс, 2018. – 364 с. – ISBN 978-5-97060-609-4. – Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт]. – URL: https://www.iprbookshop.ru/124979.html. – Режим доступа: для авторизир. пользователей.

10. Златопольский, Д. М. Основы программирования на языке Python / Д. М. Златопольский. – 2-е изд. – Москва : ДМК Пресс, 2018. – 396 с. – ISBN 978-5-97060-641-4. – Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт]. – URL: https://www.iprbookshop.ru/124998.html (дата обращения: 18.10.2022). – Режим доступа: для авторизир. пользователей.

11. Карякин, М. И. Технологии программирования и компьютерный практикум на языке Python : учебное пособие / М. И. Карякин, К. А. Ватульян, Р. М. Мнухин. – Ростов-на-Дону, Таганрог : Издательство ЮФУ, 2022. – 241 с. – ISBN 978-5-9275-4108-9. – Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт]. – URL: https://www.iprbookshop.ru/125718.html (дата обращения: 09.11.2022). – Режим доступа: для авторизир. пользователей.

12. Херинг, М. DevOps для современного предприятия / М. Херинг ; перевод М. А. Райтман. – Москва : ДМК Пресс, 2020. – 232 с. – ISBN 978-5-97060-836-4. – Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт]. – URL: https://www.iprbookshop.ru/126198.html (дата обращения: 26.02.2025). – Режим доступа: для авторизир. пользователей (Лицензия: до 31.12.2029).

13. Ленц, М. Python: непрерывная интеграция и доставка / М. Ленц ; перевод А. Е. Мамонов, Д. А. Беликов. – Москва : ДМК Пресс, 2020. – 168 с. – ISBN 978-5-97060-797-8. – Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт]. – URL: https://www.iprbookshop.ru/126206.html (дата обращения: 30.11.2022). – Режим доступа: для авторизир. пользователей.

14. Курносов, М. Г. Анализ и организация функционирования вычислительных систем / М. Г. Курносов, Д. М. Берлизов. – Новосибирск: Автограф, 2020. – 54 с. – ISBN 978-5- 907221-23-6. – Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт]. – URL: https://www.iprbookshop.ru/102116.html (дата обращения: 30.11.2024).

15. Ефимов А.В. Разработка высоконагруженных сервисов распределенной обработки и хранения информации : учебное пособие / А. В. Ефимов, А. В. Гонцова ; Сибирский государственный университет телекоммуникаций и информатики. – Новосибирск : СибГУТИ, 2020. – 83 с. : ил. – Загл. с титул. экрана. – Электрон. версия без печ. публикации. – URL: http://ellib.sibsutis.ru/ellib/2021/975_Efimov_A.V._Razrabotka_vysokonagruzhennykh_servisov_.pdf. – Режим доступа: по паролю. – Библиогр.: с. 77-78. – : ~Б. ц. – Текст : электронный.

Список дополнительной литературы:

16. Скрынник, О. В. DevOps для ИТ-менеджеров: концентрированное структурированное изложение передовых идей / О. В. Скрынник. – 2-е изд. – Москва : ДМК Пресс, 2019. – 126 с. – ISBN 978-5-97060-692-6. – Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт]. – URL: https://www.iprbookshop.ru/124535.html (дата обращения: 26.02.2025). – Режим доступа: для авторизир. пользователей (Лицензия: до 31.12.2029).

17. Грувер, Г. Запуск и масштабирование DevOps на предприятии / Г. Грувер ; перевод Д. Денисов, А. Липова, А. Мукосеев. – Москва : ДМК Пресс, 2018. – 80 с. – ISBN 978-5-97060-704-6. – Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт]. – URL: https://www.iprbookshop.ru/124549.html (дата обращения: 26.02.2025). – Режим доступа: для авторизир. пользователей (Лицензия: до 31.12.2029).

18. Джин Ким, Патрик Дебуа, Джон Уиллис и Джез Хамбл. Руководство по DevOps. Пер. с англ.: И. Лейко, И. Васильев. – Изд-во «Манн, Иванов и Фербер», 2018. – 512 с.

19. Джез Хамбл, Дэвид Фарли. Непрерывное развертывание ПО: Автоматизация процессов сборки, тестирования и внедрения новых версий программ. Пер. с англ.: А. Сысонюк. – Изд-во «Вильямс». 2016, 432 с.

20. Методы программирования : учебно-методическое пособие / авторы В. В. Подколзин, А. Н. Полетайкин, Е. П. Лукащик [и др.] ; Министерство науки и высшего образования Российской Федерации, Кубанский государственный университет. – Краснодар : Кубанский государственный университет, 2020. – 174 с.

21. Кафка Д., Мурти Д. Нигам С. Микросервисная архитектура. Реализация и проектирование. Пер. с англ. – М.: ДМК-Пресс, 2017. – 432 с.

22. Фаулер М., Палмер Р., Льюис Джеймс. Микросервисы. Паттерны разработки и лучшие практики. Пер. с англ. – М.: ДМК-Пресс, 2018. – 510 с.

23. МакКроски Р. Микросервисы. Создание расширяемых и отказоустойчивых приложений. Пер. с англ. – М.: ДМК-Пресс, 2018. – 460 с.

24. Саммервиль Дж., Жерарди Р. Микросервисы в действии. Пер. с англ. – СПб.: Питер, 2018. – 304 с.

25. Ганри Ш. Микросервисы. Надежные и масштабируемые системы. Пер. с англ. – М.: ООО «ДМК Пресс», 2019. – 340 с.

26. Баджуска Д., Москето Р. Микросервисы изнутри. Основы, инструменты и практические рекомендации. Пер. с англ. – М.: ДМК-Пресс, 2020. – 480 с.

27. Полетайкин, А. Н. Социальные и экономические информационные системы. Законы функционирования и принципы построения [Электронный ресурс] : учебное пособие / А. Н. Полетайкин. – Электрон. текстовые данные. – Новосибирск : Сибирский государственный университет телекоммуникаций и информатики, 2016. – 241 с. – 2227-8397. – Режим доступа: http://www.iprbookshop.ru/54800.html.

28. Дерябкин, В. П. Проектирование информационных систем по методологии UML с использованием Qt-технологии программирования [Электронный ресурс] : учебное пособие / В. П. Дерябкин, В. В. Козлов. – Электрон. текстовые данные. – Самара : Самарский государственный технический университет, ЭБС АСВ, 2017. – 156 с. – 2227-8397. – Режим доступа: http://www.iprbookshop.ru/83601.html.

29. Носова, Л. С. Case-технологии и язык UML [Электронный ресурс] : учебно-методическое пособие / Л. С. Носова. – 2-е изд. – Электрон. текстовые данные. – Челябинск, Саратов : Южно-Уральский институт управления и

экономики, Ай Пи Эр Медиа, 2019. – 67 с. – ISBN 978-5-4486-0670-0. – Режим доступа: http://www.iprbookshop.ru/81479.html.

30. Мостовой Я.А. Управление программными проектами [Электронный ресурс]: учебное пособие / Мостовой Я.А. – Электрон. текстовые данные. – Самара: Поволжский государственный университет телекоммуникаций и информатики, 2016. – 103 с. – Режим доступа: http://www.iprbookshop.ru/71894.html. – ЭБС «IPRbooks».

31. Влацкая И.В. Проектирование и реализация прикладного программного обеспечения [Электронный ресурс]: учебное пособие / Влацкая И.В., Заельская Н.А., Надточий Н.С. – Электрон. текстовые данные. – Оренбург: Оренбургский государственный университет, ЭБС АСВ, 2015. – 119 с. – Режим доступа: http://www.iprbookshop.ru/54145.html. – ЭБС «IPRbooks».

32. Ершова, Н. Ю. Организация вычислительных систем : учебное пособие / Н. Ю. Ершова, А. В. Соловьев. – 3-е изд. – Москва : Интернет-Университет Информационных Технологий (ИНТУИТ), Ай Пи Ар Медиа, 2021. – 221 с. – ISBN 978-5-4497-0904-2. – Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт]. – URL: https://www.iprbookshop.ru/102024.html.

33. Чекмарев, Ю. В. Вычислительные системы, сети и телекоммуникации / Ю. В. Чекмарев. – 2-е изд. – Саратов : Профобразование, 2019. – 184 с. – ISBN 978-5-4488-0071-9. – Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт]. – URL: https://www.iprbookshop.ru/87989.html.

34. Лиманова, Н. И. Архитектура вычислительных систем и компьютерных сетей : учебное пособие / Н. И. Лиманова. – Самара : Поволжский государственный университет телекоммуникаций и информатики, 2017. – 197 с. – Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт]. – URL: https://www.iprbookshop.ru/75368.html.

35. Галас В.П. Вычислительные системы, сети и телекоммуникации. Часть 1. Вычислительные системы [Электронный ресурс]: электронный учебник / Галас В.П. – Электрон. текстовые данные. – Владимир: Владимирский государственный университет им. А.Г. и Н.Г. Столетовых, 2016. – 232 с. – Текст : электронный // Цифровой образовательный ресурс IPR SMART: [сайт]. – URL: https://www.iprbookshop.ru/57363.

36. Хорошевский, В. Г. Архитектура вычислительных систем [Текст] : учеб. пособие / В. Г. Хорошевский. – 2-е изд., перераб. и доп. – Москва : Изд-во МГТУ им. Н. Э. Баумана, 2008. – 519 с. : ил. – Библиогр.: с. 519. – ISBN 978-5-7038-3175-5.

37. Дружинин Д.В. Высокопроизводительные вычисления и облачные технологии : учебное пособие / Дружинин Д.В. – Томск : Издательство Томского государственного университета, 2020. – 93 с. – ISBN 978-5-94621-921-1. – Текст : электронный // IPRSMART : [сайт]. – URL: https://www.iprbookshop.ru/116813.html.

38. Малявко А.А. Суперкомпьютеры и системы. Построение вычислительных кластеров : учебное пособие / Малявко А.А., Менжулин С.А. –

Новосибирск: Новосибирский государственный технический университет, 2018. – 96 с. ISBN 978-5-7782-3633-2. – Текст: электронный // IPR SMART: [сайт]. – URL: https://www.iprbookshop.ru/91542.html

39. Губарев В.В. Введение в облачные вычисления и технологии: учебное пособие / Губарев В.В., Савульчик С.А., Чистяков Н.А. – Электрон. текстовые данные. – Новосибирск: Новосибирский государственный технический университет, 2013. – 48 с. – ISBN 978-5-907221-23-6. – Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт]. – URL: https://www.iprbookshop.ru/44905.

40. Клементьев И.П. Введение в облачные вычисления: учебное пособие для СПО / Клементьев И.П., Устинов В.А. – Саратов : Профобразование, 2019. – 298 с. – ISBN 978-5-4488-0359-8. – Текст : электронный // IPR SMART : [сайт]. – URL: https://www.iprbookshop.ru/86193.html.

41. Зиангирова Л.Ф. Технологии облачных вычислений : учебное пособие / Зиангирова Л.Ф. – Саратов : Вузовское образование, 2016. – 300 с. – Текст : электронный // IPR SMART : [сайт]. – URL: https://www.iprbookshop.ru/41948.html.

42. Алексеев, В. А. Паттерны проектирования программных систем: методические указания к проведению лабораторных работ по курсу «Архитектура программных систем» / В. А. Алексеев. – Электрон. текстовые данные. – Липецк : Липецкий государственный технический университет, ЭБС АСВ, 2016. – 33 с. – ISBN 978-5-94839-537-1. – Текст: электронный // Цифровой образовательный ресурс IPR SMART: [сайт]. – URL: https://www.iprbookshop.ru/74412.

43. Джефф Форсье. Django - Разработка веб-приложений на Python. – 2009.

44. Адриан Головатый. Django - Подробное руководство, 2-е издание. – 2010.

45. Django makes it easier to build better web apps more quickly and with less code. [Электронный ресурс]: Режим доступа: https://www.djangoproject.com.

46. Почему вам стоит выбрать фреймворк Django для своего следующего проекта. [Электронный ресурс] Режим доступа: https://tproger.ru/articles/pochemu-vam-stoit-vybrat-frejmvork-django-dlja-svoego-sledujushhego-proekta/ (дата обращения 24.08.2025).

47. Аутентификация и авторизация в проекте с микросервисной архитектурой: стратегии, практический пример. [Электронный ресурс] Режим доступа: https://habr.com/ru/companies/spectr/articles/715290/ (дата обращения 13.09.2024).

48. Best Practices to Secure Microservices with Spring Security. [Электронный ресурс] Режим доступа: https://www.geeksforgeeks.org/best-practices-to-secure-microservices-with-spring-security/ (дата обращения 13.09.2024).

49. Архитектура и стратегии аутентификации. [Электронный ресурс] Режим доступа: https://www.researchgate.net/figure/Authentication-architecture-and-strategies_fig1_317213681 (дата обращения 24.09.2024).

50. Справочник security-архитектора: обзор подходов к реализации аутентификации и авторизации в микросервисных системах. [Электронный ресурс] Режим доступа: https://habr.com/ru/companies/huawei/articles/527098/?mobile=yes (дата обращения 24.09.2024).

51. Как тестировать микросервисы [Электронный ресурс] Режим доступа: https://testengineer.ru/how-to-test-microservices/ (дата обращения 24.08.2025).

52. Внутреннее устройство Kubernetes-кластера простым языком. [Электронный ресурс] Режим доступа: https://habr.com/ru/companies/flant/articles/583660/ (дата обращения 14.08.2025).

Приложение А.
Образец титульного листа пояснительной записки к КР

<Профильное министерство, полное название>
(<Профильное министерство, сокращенное название>)

<Вуз, полное название>
(<Вуз, сокращенное название>)

Кафедра _____

П О Я С Н И Т Е Л Ь Н А Я З А П И С К А

к курсовой работе по дисциплине «DevOps-инженерия»

на тему: _____

_____

_____

Исполнитель
студент гр. _____ _____ _____
(группа) (подпись) (ФИО)

Руководитель _____ _____
(подпись) (должность, ФИО)

Оценка _____ Дата защиты _____

<Город>,
20__

Приложение Б.
Образец бланка задания на КР

<Профильное министерство, полное название>
(<Профильное министерство, сокращенное название>)

<Вуз, полное название>
(<Вуз, сокращенное название>)

Кафедра _____

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ
по дисциплине «DevOps-инженерия»
студенту _____ группы _____

1. Тема курсовой работы: _____

_____

2. Перечень исходных материалов для курсового проектирования: _____

_____

3. Основные функции приложения: _____

_____

_____

4. Используемые инструментальные средства: _____

_____

5. Основные результаты работы приложения: _____

_____

6. Функционал приложения, обеспечиваемого контейнерами Docker:

   1. _____
   2. _____
   3. _____

7. Срок сдачи проекта на проверку и защиты: с ___.12.20__ по ___.12.20__ г.

Руководитель _____ _____
(подпись) (должность, ФИО)

Приложение В.
Пример реализации простейшего web-приложения с микросервисной архитектурой

Имеется сайт агентства по поиску вакансий, в функционал которого входит вывод данных об имеющихся вакансиях и средней предлагаемой зарплате по всем вакансиям (рис. В.1).

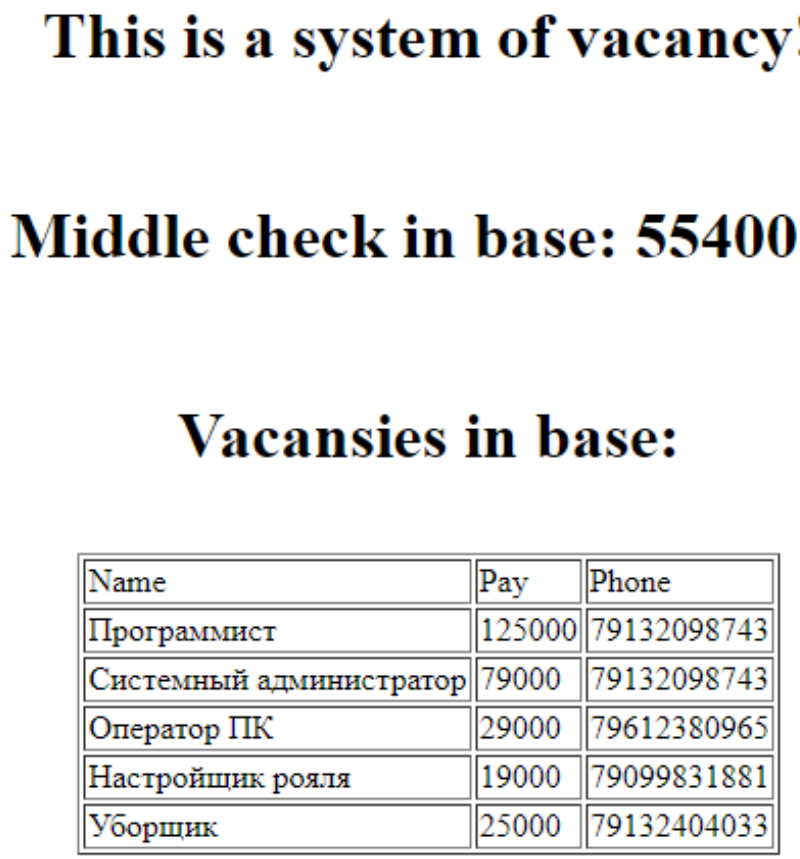


Рисунок В.1 – Внешний вид исправного сайта агентства по поиску вакансий

Предполагается, что за функционал сайта отвечает отдельное приложение, а функционал, обеспечивающий ведение информации о вакансиях, также реализуется в виде отдельного приложения. Взаимодействие приложения сайта с приложением вакансий обеспечивается путем web-запросов.

В случае остановки приложения, обеспечивающего взаимодействие с вакансиями, сайт станет неисправен. Однако он сохранит работоспособность, поскольку основное приложение сайта продолжит работать, но информация о вакансиях будет недоступна (см. рис. В.2).

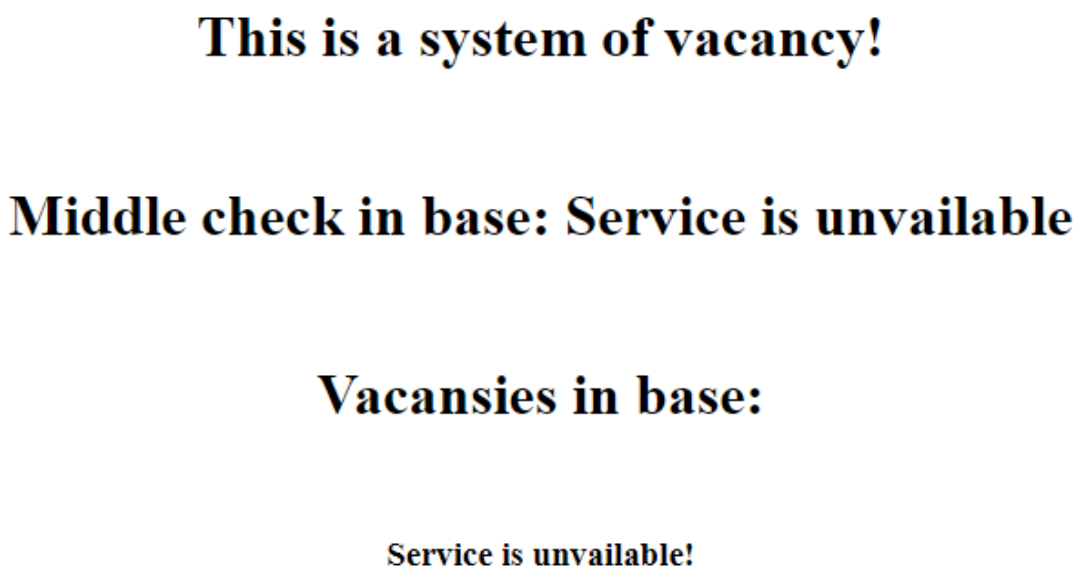


Рисунок В.2 – Работа неисправного сайта в условиях остановки приложения, обеспечивающего взаимодействие с вакансиями

Приведем пример реализации приложения вакансий. Создадим Django-приложение (см. лабораторную работу 2).

Пример файла urls.py:

```
from django.contrib import admin
from django.urls import path, include, re_path
from . import views

urlpatterns = [
    path(r'req', views.req, name='req'),
    path(r'req1', views.req1, name='req1'),
]
```

В данном файле имеются две функции обработки: `req` и `req1`, вызываемые по `url /req` или `req1`. Приведем эти функции.

```
def req(request):
   a={}
   b=vacancy.objects.all()
   j=0
   k=0
   for i in b:
     j=j+i.zarpl
     k+=1
   a['sz']=j/k
   otv=json.dumps(a)
```

```
    return HttpResponse(otv)

def req1(request):
    a={}
    a['vac']=[]
    b=vacancy.objects.all()
    for i in b:
        el={}
        el['id'], el['name'],   el['zarpl'], el['tel'] =
i.pk, i.name, i.zarpl, i.tel
        a['vac'].append(el)
    otv=json.dumps(a)
    return HttpResponse(otv)
```

Как можно видеть, данные функции выполняют чтение вакансий из БД, упаковывают результат в словарь, переводят в формат json и возвращают HTTP-ответ в виде json-строки.

База данных состоит из одной таблицы, описание которой приведено в models.py:

```
class vacancy(models.Model):
    name = models.CharField(max_length=200)
    zarpl = models.IntegerField()
    tel = models.CharField(max_length=20)
```

Далее папку с приложением и проектом необходимо скопировать в образ контейнера Docker. Пример Docker-файла:

```
# Забираем официальный базовый образ Python
FROM python:3.10-alpine
# Установка необходимых библиотек, перечисленных в файле
requirements.txt
RUN pip install --upgrade pip
COPY ./requirements.txt .
RUN pip install -r requirements.txt
#  Копирование папок проекта и приложения в корень
контейнера (Docker файл в этой же директории), а так же
задание переменных окружения
COPY . .
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
#Команда запуска
CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

Файл requirements.txt:

```
Django
psycopg2-binary
requests
```

Далее покажем пример приложения самого сайта:

```
Urls.py
from django.contrib import admin
from django.urls import path, include, re_path
from . import views

urlpatterns = [
    path(r'', views.index, name='index'),
]
```

Функция обработки **index**:

```
def index(request):
   try:
       res = requests.get('http://vacan:8000/req')
       load=json.loads(str(res.text))
   except:
       load={}
       load['sz']='Service is unvailable'

   try:
       res1 = requests.get('http://vacan:8000/req1')
       load1=json.loads(str(res1.text))
   except:
          load1={}
          load1['vac']='NONE'
   l1={}
   l1['param1']=load
   l1['param2']=load1

   return render(request, 'index.html',l1)
```

Данная функция делает запрос к приложению вакансий, получает json, парсит его и отправляет в шаблон.

ВНИМАНИЕ! Полужирным выделено название контейнера vacan с приложением вакансий. Это должно быть ИМЯ КОНТЕЙНЕРА DOCKER данного приложения, так как внутри Docker-сети возможно взаимодействие именно по именам контейнера и можно абстрагироваться от IP-адресов.

Шаблон:

```
<center><H1>  This  is  a  system  of  vacancy!
</H1></center></br>
<center><H1>  Middle  check  in  base:  {{param1.sz}}
</H1></center></br>
<center><H1> Vacansies in base: </br> </H1></center></br>
{% if param2.vac == 'NONE' %}
 <center><H3>    Service    is    unvailable!    </br>
</H3></center></br>
{% else %}
<center>
<table border="1">
 <tr><td>Name</td><td>Pay</td><td>Phone</td></tr>
{% for i in param2.vac %}
 <tr><td>{{  i.name  }}</td><td>{{  i.zarpl  }}</td><td>{{
i.tel }}</td></tr>
{% endfor %}
</table>
</center>
{% endif %}
```

Создание образа Docker и файла Docker в данном случае не отличается от приложения вакансий. Приведем пример файла docker-compose:

```
version: '3.7'

services:
#Контейнер вакансий
  vacan:
#указывает , что все данные для построения образа
находится в поддиректории imgvacan/
    build: imgvacan/
# задает имя контейнера
    container_name: vacan
    restart: always
#Аналогично собирается образ сайта
  main:
    build: imgmain/
    container_name: main
    restart: always
    ports:
      - 8001:8001
```

Следует обратить внимание на то, что для контейнера **vacan** порт не пробрасывается на хост-систему, потому что взаимодействие осуществляется внутри сети Docker. Для контейнера с сайтом порт пробрасывается на хост-систему.

Для запуска сборки образа и старта контейнера необходимо, находясь в директории с файлом **docker-compose.yml**, запустить команду:

```
docker-compose up -d
```

Начнется сборка, процесс которой отобразится так, как показано на рис. В.3.

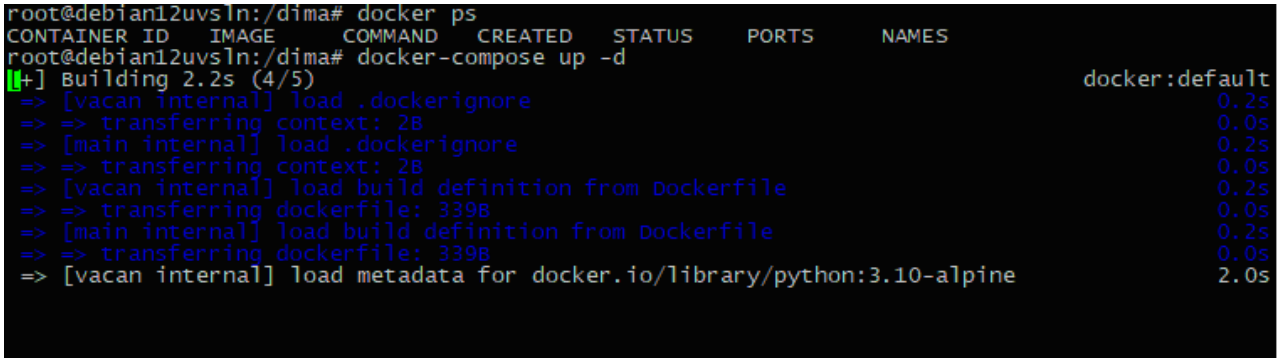


Рисунок В.3 – Процесс сборки приложения с МСА

В случае успешной сборки запустятся контейнеры (см. рис. В.4).



Рисунок В.4 – Результат успешной сборки и запуск контейнеров

Командой docker ps можно посмотреть запущенные контейнеры (рис. В.5).

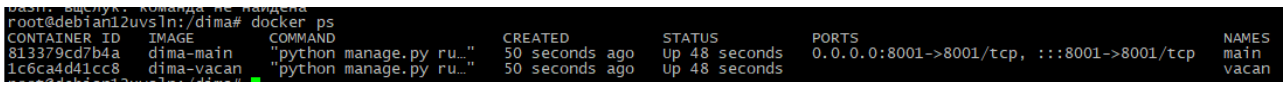


Рисунок В.5 – Результат выполнения команды docker ps

Проверка работы приложения показана на рис. В.1.
Данное приложение имеется на Gitlab: http://pa.k-lab.su/dad/mca.git.

Здесь представлен простейший пример микросервисной архитектуры, физическая реализация которой показана на рис. В.6. Приложение **Main** на основе данных, полученных через собственный web-интерфейс посредством функции **Requests** инициирует запросы к приложению **Vacan** через Web API. Последний рассылает сообщения функциям Req и Req1, реализованные на базе приложения **Vacan**, которые, обращаясь к БД **Details**, вычисляют требуемые результаты и возвращают их обратно в **Main**. Приложение **Main** обладает собственной БД **Mainbase**, в которую может записать, например, параметры пользовательского запроса и полученный от **Vacan** результат.

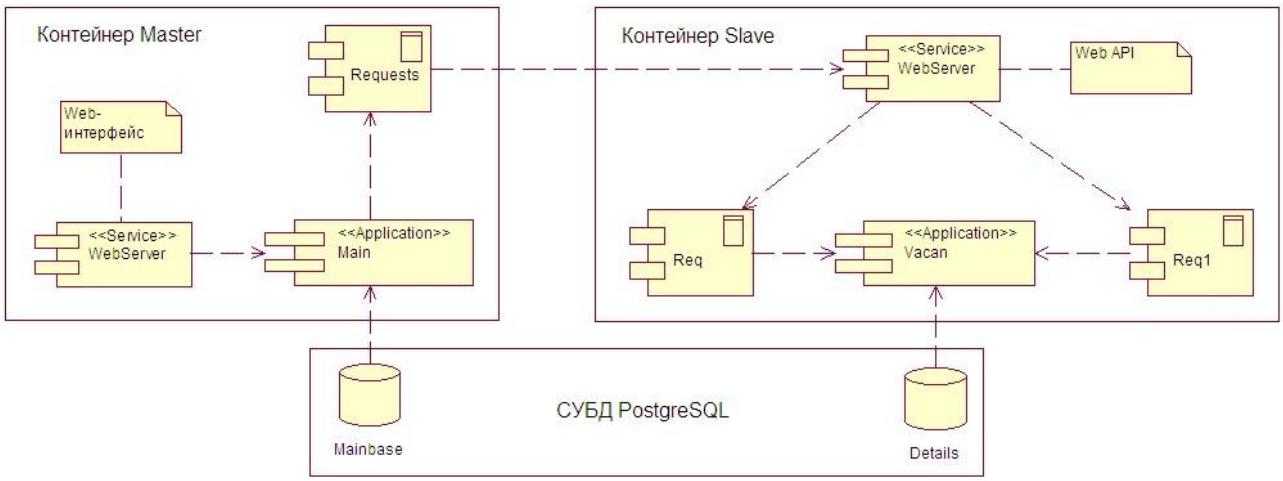


Рисунок В.6 – Диаграмма компонентов UML МСА-приложения

В более сложных системах строятся кластеры хостов, устанавливаются системы определения сервисов и т. д. Далее можно вынести в отдельное приложение (сервис), например, работу с базой соискателей, систему генерации отчетов и пр. Web-запросы можно реализовать более сложными способами, передавая, например, параметры фильтрации и т. д. Несмотря на то, что в данном примере оба приложения (сервиса) реализованы с помощью Django, это совсем не обязательно. Сервисы могут быть реализованы с помощью любых инструментов, главное согласовать форматы web-запросов и ответов на запросы.

Учебное издание

Полетайкин Алексей Николаевич
Демчук Дмитрий Анатольевич

DevOps-инженерия

Корректор *А.Р. Шерстнёва*

Электронное издание.
Рекомендовано к публикации
Редакционно-издательским советом СибГУТИ,
протокол № 3 от 30.09.2025 г.